

Important Information

Thank you for choosing Freenove products!

Getting Started

First, please read the **Read Me First.pdf** document in the unzipped folder you created.

If you have not yet downloaded the zip file, associated with this kit, please do so now and unzip it.

Get Support and Offer Input

Freenove provides free and responsive product and technical support, including but not limited to:

- Product quality issues
- Product use and build issues
- Questions regarding the technology employed in our products for learning and education
- Your input and opinions are always welcome
- We also encourage your ideas and suggestions for new products and product improvements

For any of the above, you may send us an email to:

support@freenove.com

Safety and Precautions

Please follow the following safety precautions when using or storing this product:

- Keep this product out of the reach of children under 6 years old.
- This product should be **used only when there is adult supervision present** as young children lack necessary judgment regarding safety and the consequences of product misuse.
- This product contains small parts and parts, which are sharp. This product contains electrically conductive parts. **Use caution with electrically conductive parts near or around power supplies, batteries and powered (live) circuits.**
- When the product is turned ON, activated or tested, some parts will move or rotate. **To avoid injuries to hands and fingers keep them away from any moving parts!**
- It is possible that an improperly connected or shorted circuit may cause overheating. **Should this happen, immediately disconnect the power supply or remove the batteries and do not touch anything until it cools down!** When everything is safe and cool, review the product tutorial to identify the cause.
- Only operate the product in accordance with the instructions and guidelines of this tutorial, otherwise parts may be damaged or you could be injured.
- Store the product in a cool dry place and avoid exposing the product to direct sunlight.
- After use, always turn the power OFF and remove or unplug the batteries before storing.

Any concerns? ✉ support@freenove.com

About Freenove

Freenove provides open source electronic products and services worldwide.

Freenove is committed to assist customers in their education of robotics, programming and electronic circuits so that they may transform their creative ideas into prototypes and new and innovative products. To this end, our services include but are not limited to:

- Educational and Entertaining Project Kits for Robots, Smart Cars and Drones
- Educational Kits to Learn Robotic Software Systems for Arduino, Raspberry Pi and micro: bit
- Electronic Component Assortments, Electronic Modules and Specialized Tools
- **Product Development and Customization Services**

You can find more about Freenove and get our latest news and updates through our website:

<http://www.freenove.com>

sale@freenove.com

Copyright

All the files, materials and instructional guides provided are released under [Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License](https://creativecommons.org/licenses/by-nc-sa/3.0/). A copy of this license can be found in the folder containing the Tutorial and software files associated with this product.



This means you can use these resource in your own derived works, in part or completely but **NOT for the intent or purpose of commercial use.**

Freenove brand and logo are copyright of Freenove Creative Technology Co., Ltd. and cannot be used without written permission.



Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

Contents

Important Information.....	1
Contents.....	1
Preface.....	1
ESP8266.....	2
CH340	5
Programming Software	15
Environment Configuration	18
Chapter 0 LED	26
Project 0.1 Blink	26
Chapter 1 LED	32
Project 1.1 Blink	32
Chapter 2 Button & LED	39
Project 2.1 Button & LED	39
Project 2.2 MINI table lamp	44
Chapter 3 LED Bar	47
Project 3.1 Flowing Light	47
Chapter 4 Analog & PWM	52
Project 4.1 Breathing LED.....	52
Project 4.2 Meteor Flowing Light.....	60
Chapter 5 RGB LED.....	64
Project 5.1 Random Color Light.....	64
Project 5.2 Gradient Color Light.....	69
Chapter 6 Buzzer.....	71
Project 6.1 Doorbell.....	71
Project 6.2 Alertor	77
Chapter 7 Serial Communication.....	80
Project 7.1 Serial Print.....	80
Project 7.2 Serial Read and Write	86
Chapter 8 ADC Converter	89

Project 8.1 Read the Voltage of Potentiometer.....	89
Chapter 9 Potentiometer & LED	95
Project 9.1 Soft Light.....	95
Project 9.2 Color Light.....	98
Chapter 10 Photoresistor & LED	102
Project 10.1 NightLamp	102
Chapter 11 Thermistor	107
Project 11.1 Thermometer	107
Chapter 12 WiFi Working Modes	113
Project 12.1 Station mode.....	113
Project 12.2 AP mode	118
Project 12.3 AP+Station mode	123
Chapter 13 TCP/IP	127
Project 13.1 As Client.....	127
Project 13.2 As Server.....	141
Chapter 14 Smart home	147
Project 14.1 Control_LED_through_Web	147
What's next?	155
End of the Tutorial	155

Preface

ESP8266 is a micro control unit with integrated Wi-Fi launched by Espressif, which features strong properties and integrates rich peripherals. It can be designed and studied as an ordinary Single Chip Microcontroller (SCM) chip, or connected to the Internet and used as an Internet of Things device.

ESP8266 can be developed using the Arduino platform, which will definitely make it easier for people who have learned Arduino to master. Moreover, the code of ESP8266 is completely open-source, so beginners can quickly learn how to develop and design IOT smart household products including smart curtains, fans, lamps and clocks.

Generally, ESP8266 projects consist of code and circuits. Don't worry even if you've never learned code and circuits, because we will gradually introduce the basic knowledge of C programming language and electronic circuits, from easy to difficult. Our products contain all the electronic components and modules needed to complete these projects. It's especially suitable for beginners.

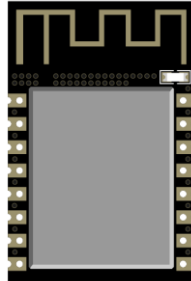
We divide each project into four parts, namely Component List, Component Knowledge, Circuit and Code. Component List helps you to prepare material for the experiment more quickly. Component Knowledge allows you to quickly understand new electronic modules or components, while Circuit helps you understand the operating principle of the circuit. And Code allows you to easily master the use of ESP8266 and accessory kit. After finishing all the projects in this tutorial, you can also use these components and modules to make products such as smart household, smart cars and robots to transform your creative ideas into prototypes and new and innovative products.

In addition, if you have any difficulties or questions with this tutorial or toolkit, feel free to ask for our quick and free technical support through support@freenove.com

ESP8266

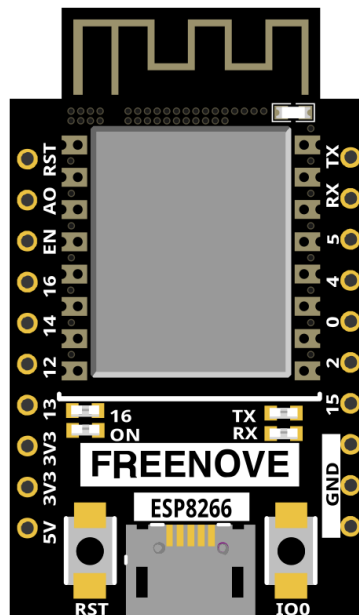
ESP8266 has PCB on-board antenna. The PCB on-board antenna is an integrated antenna in the chip module itself, so it is convenient to carry and design.

PCB on-board antenna

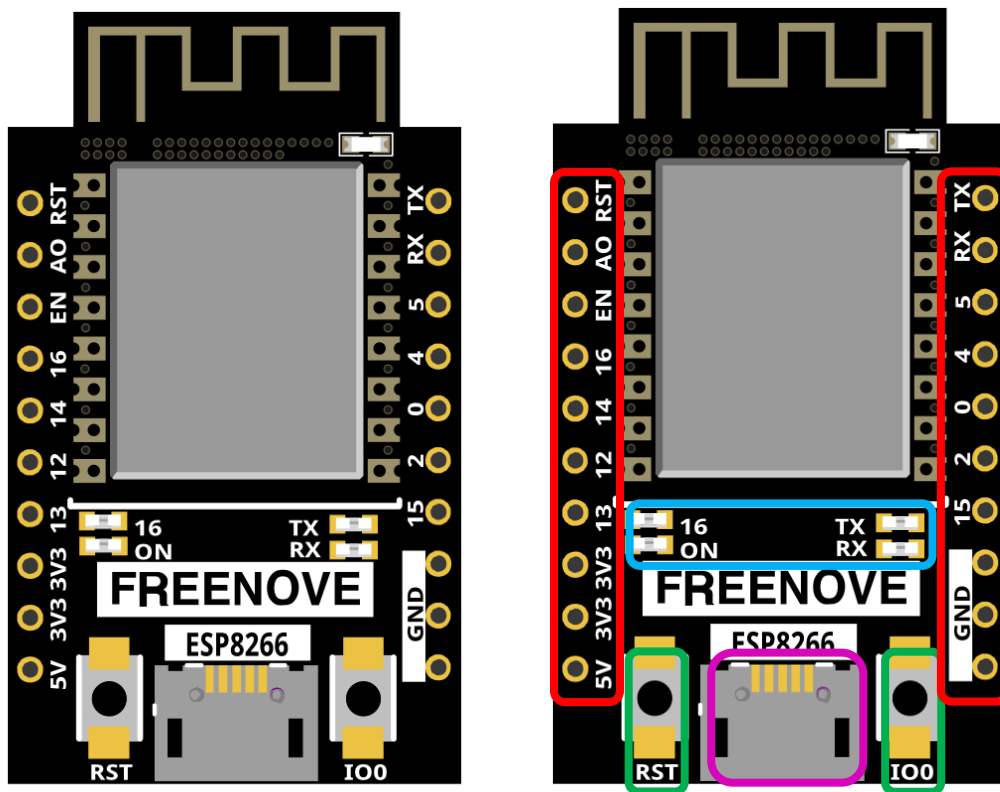


In this tutorial, the ESP8266 development board is designed based on the PCB on-board antenna-packaged ESP8266 module. The following tutorials will be based on the ESP8266 development board.

ESP8266 development board



The hardware interfaces of ESP8266 are distributed as follows:



Compare the left and right images. We've boxed off the resources on the ESP8266 in different colors to facilitate your understanding of the ESP8266 development board.

Box color	Corresponding resources introduction
	GPIO pin
	LED indicator
	Reset button, Boot mode selection button
	USB port

NO.	Pin Name	Functional Description
1	RST	Reset Pin, Active Low
2	ADC	AD conversion, Input voltage range 0~3.3V, the value range is 0~1024.
3	EN	Chip Enabled Pin, Active High
4	IO16	Connect with RST pin to wake up Deep Slee
5	IO14	GPIO14; HSPI_CLK
6	IO12	GPIO12; HSPI_MISO
7	IO13	GPIO13; HSPI_MOSI; UART0_CTS
8	VCC	Module power supply pin, Voltage 3.0V ~ 3.6V
9	GND	GND
10	IO15	GPIO15; MTDO; HSPICS; UART0
11	IO2	GPIO2; UART1_TXD
12	IO0	GPIO0; UART1_RXD
13	IO4	GPIO4
14	IO5	GPIO5; IR_R
15	RXD	UART0_RXD; GPIO3
16	TXD	UART0_TXD; GPIO1

Description of the ESP8266 series module boot mode:

Mode	CH_PD(EN)	RST	GPIO15	GPIO0	GPIO2	TXD0
Download mode	high	high	low	low	high	high
Running mode	high	high	low	high	high	high

Notes: Some of the pins inside the module have been pulled or pulled down.

For more information, please visit:

If you want to learn more about this, you can read the following files:

[“Freenove_Basic_Starter_Kit_for_ESP8266/Datasheet/esp-12s_datasheet_en.pdf”](#)

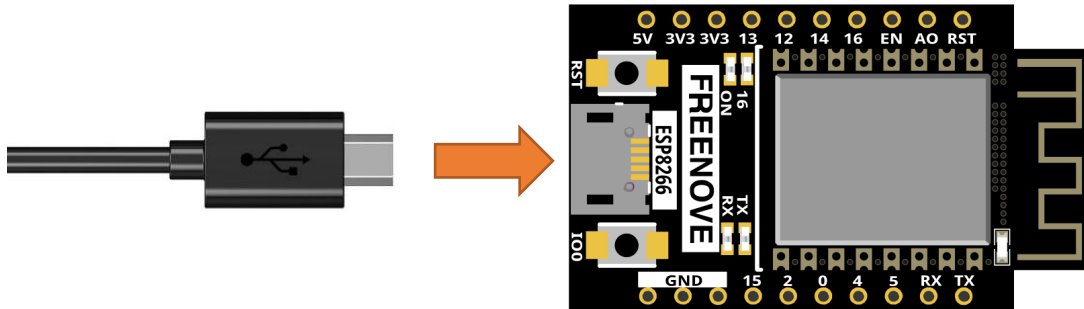
CH340

ESP8266 uses CH340 to download codes. So before using it, we need to install CH340 driver in our computers.

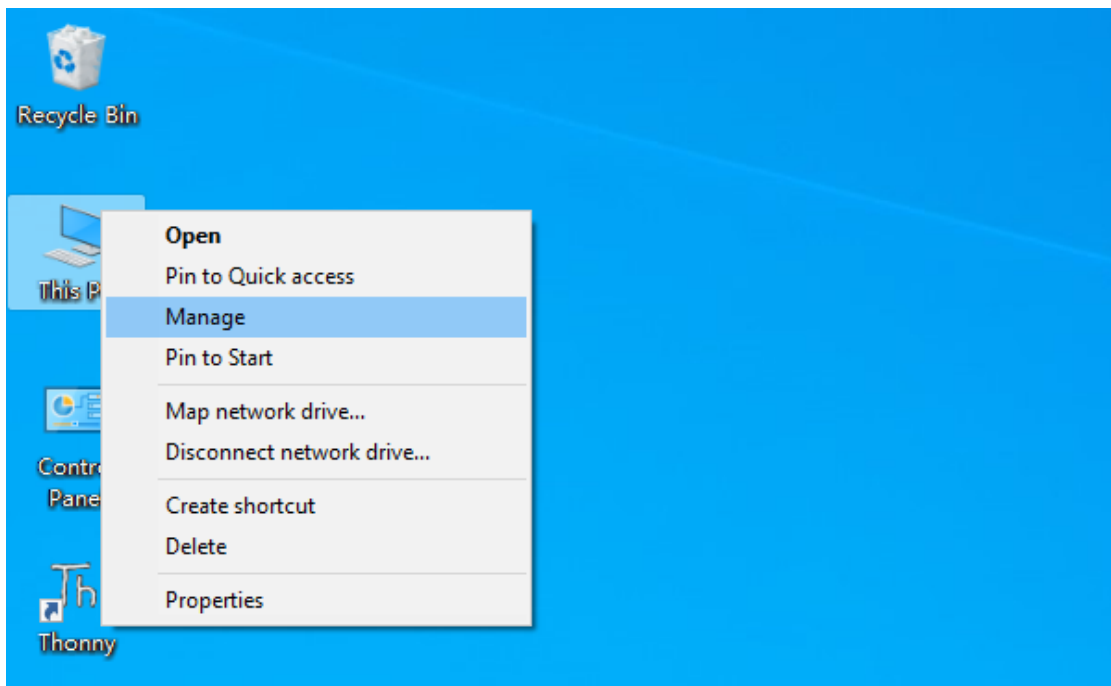
Windows

Check whether CH340 has been installed

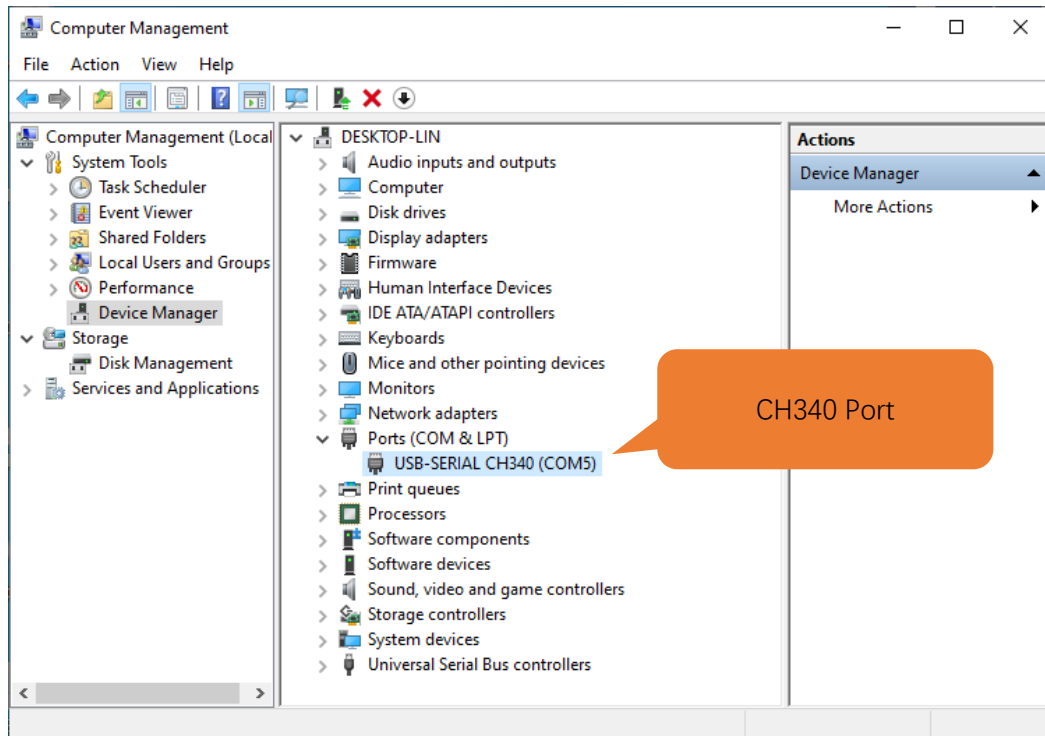
1. Connect your computer and ESP8266 with a USB cable.



2. Turn to the main interface of your computer, select "This PC" and right-click to select "Manage".



- Click “Device Manager”. If your computer has installed CH340, you can see “USB-SERIAL CH340 (COMx)”. And you can click [here](#) to move to the next step.



Installing CH340

- First, download CH340 driver, click <http://www.wch-ic.com/search?q=CH340&t=downloads> to download the appropriate one based on your operating system.

index / search / search CH340

All (14)

Downloads (7)

Products (4)

Application (2)

Video (1)

News (0)

keyword CH340

Downloads(7)

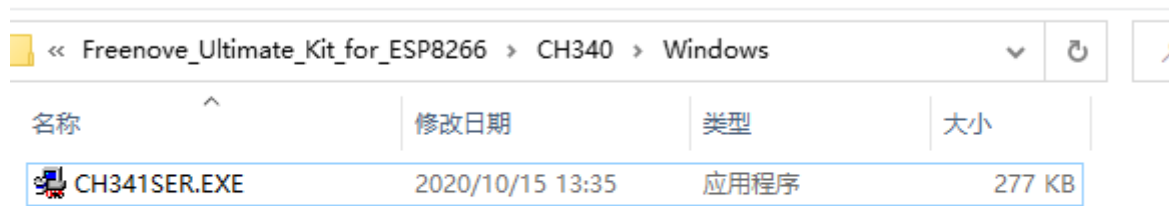
file category	file content	version	upload time
Driver&Tools			
CH341SER.EXE	CH340/CH341 USB to serial port Windows driver, supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-18
CH341SER.ZIP	CH340/CH341 USB to serial port Windows driver, includes DLL dynamic library and non-standard baud rate settings and other instructions. Supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-05
CH341SER_ANDROID...	CH340/CH341 USB to serial port Android free drive application library, for Android OS 3.1 and above version which supports USB Host mode already, no need to load Android kernel driver, no root privileges. Contains apk, lib library, file (Host Driver), App Demo Example (USB to UART Demo)	1.6	2019-04-19
CH341SER_LINUX...	CH340/CH341 USB to serial port LINUX driver	1.5	2018-03-18
CH341SER_MAC.ZIP	CH340/CH341 USB to serial port MAC OS driver	1.5	2018-07-05
Others			
PRODUCT_GUIDE.P...	Electronic selection of product selection manual, please refer to related product technical manual for more technical information.	1.4	2018-12-29
InstallNoteOn64...	Instructions for the driver after 18 years of August cannot be installed under some 64-bit WIN7 (English)	1.0	2019-01-10

If you would not like to download the installation package, you can open

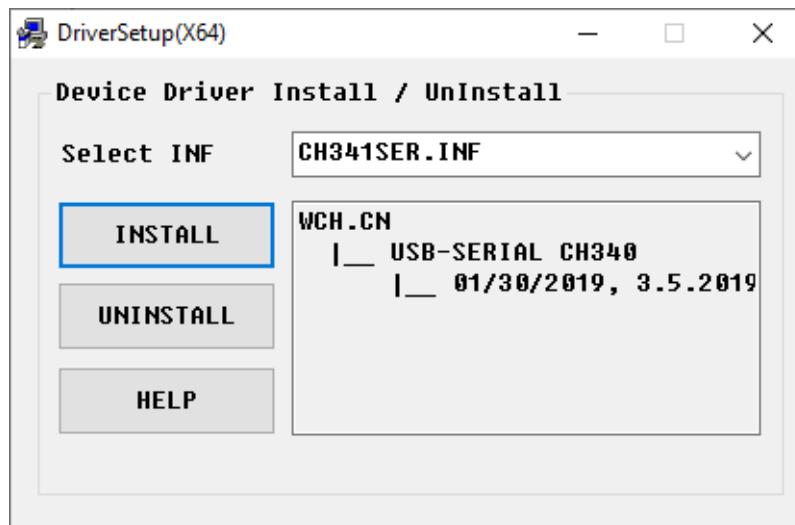
“**Freenove_Basic_Starter_Kit_for_ESP8266/CH340**”, we have prepared the installation package.

Name	Date modified	Type	Size
Linux	8/14/2020 5:24 PM	File folder	
MAC	8/14/2020 5:23 PM	File folder	
Windows	8/14/2020 5:23 PM	File folder	

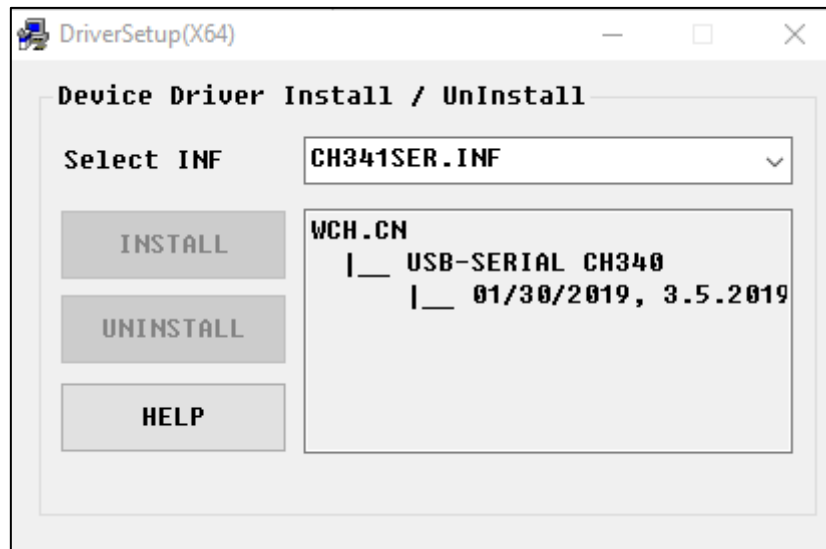
2. Open the folder "Freenove_Basic_Starter_Kit_for_ESP8266/CH340/Windows/"



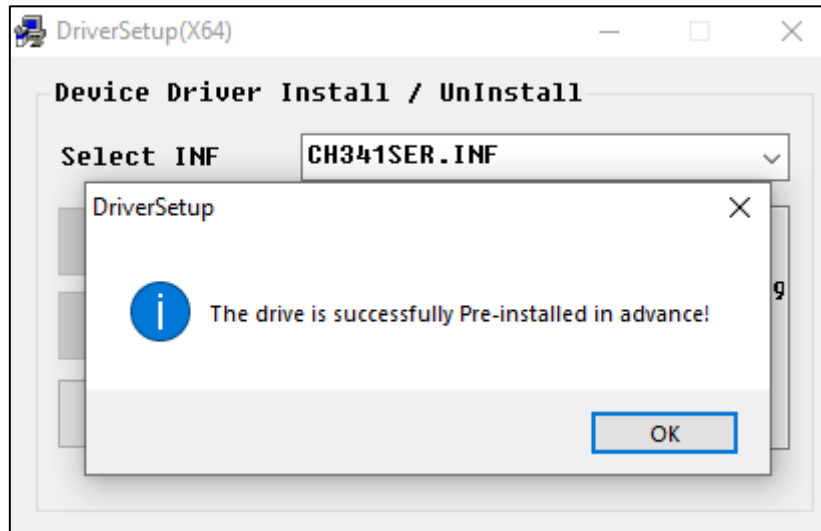
3. Double click "CH341SER.EXE".



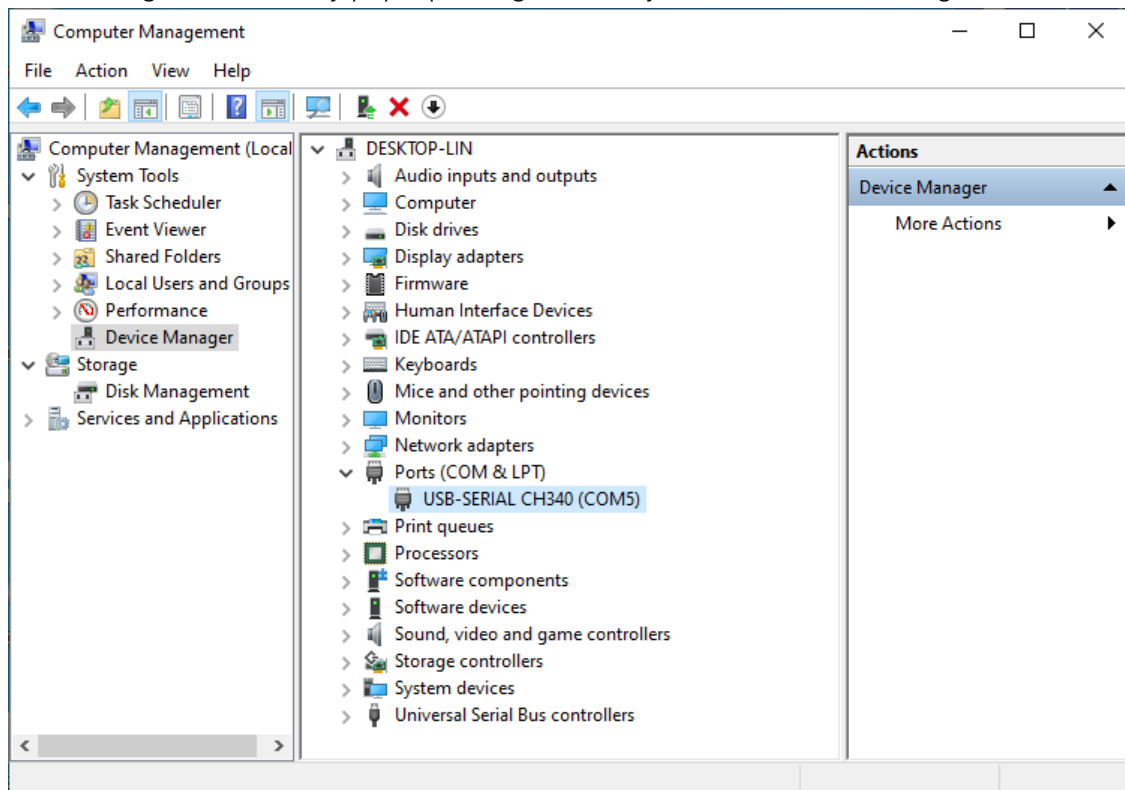
- Click "INSTALL" and wait for the installation to complete.



- Install successfully. Close all interfaces.



6. When ESP8266 is connected to computer, select "This PC", right-click to select "Manage" and click "Device Manager" in the newly pop-up dialog box, and you can see the following interface.



7. So far, CH340 has been installed successfully. Close all dialog boxes.

MAC

First, download CH340 driver, click <http://www.wch-ic.com/search?q=CH340&t=downloads> to download the appropriate one based on your operating system.

The screenshot shows the WCH website's search results for the keyword 'ch340'. The left sidebar lists categories: All (14), Downloads (7), Products (4), Application (2), Video (1), and News (0). The main content area shows a table of downloads under the 'Downloads (7)' section.

file category	file content	version	upload time
Driver&Tools			
CH341SER.EXE	CH340/CH341 USB to serial port Windows driver, supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-18
CH341SER.ZIP	CH340/CH341 USB to serial port Windows driver, includes DLL dynamic library and non-standard baud rate settings and other instructions. Supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-05
CH341SER_ANDROID...	CH340/CH341 USB to serial port Android free drive application library, for Android OS 3.1 and above version which supports USB Host mode already, no need to load Android kernel driver, no root privileges. Contains apk, lib library file (Java Driver), App Demo Example, AT Demo SDK).	1.6	2019-04-19
CH341SER_LINUX...	CH340/CH341 USB to serial port LINUX driver	1.5	2018-03-18
CH341SER_MAC.ZI...	CH340/CH341 USB to serial port MAC OS driver	1.5	2018-07-05

Orange callout boxes highlight the following files:

- Windows**: CH341SER.EXE
- Linux**: CH341SER_LINUX...
- MAC**: CH341SER_MAC.ZI...

If you would not like to download the installation package, you can open

“**Freenove_Basic_Starter_Kit_for_ESP8266/CH340**”, we have prepared the installation package.

Second, open the folder “**Freenove_Basic_Starter_Kit_for_ESP8266/CH340/MAC/**”

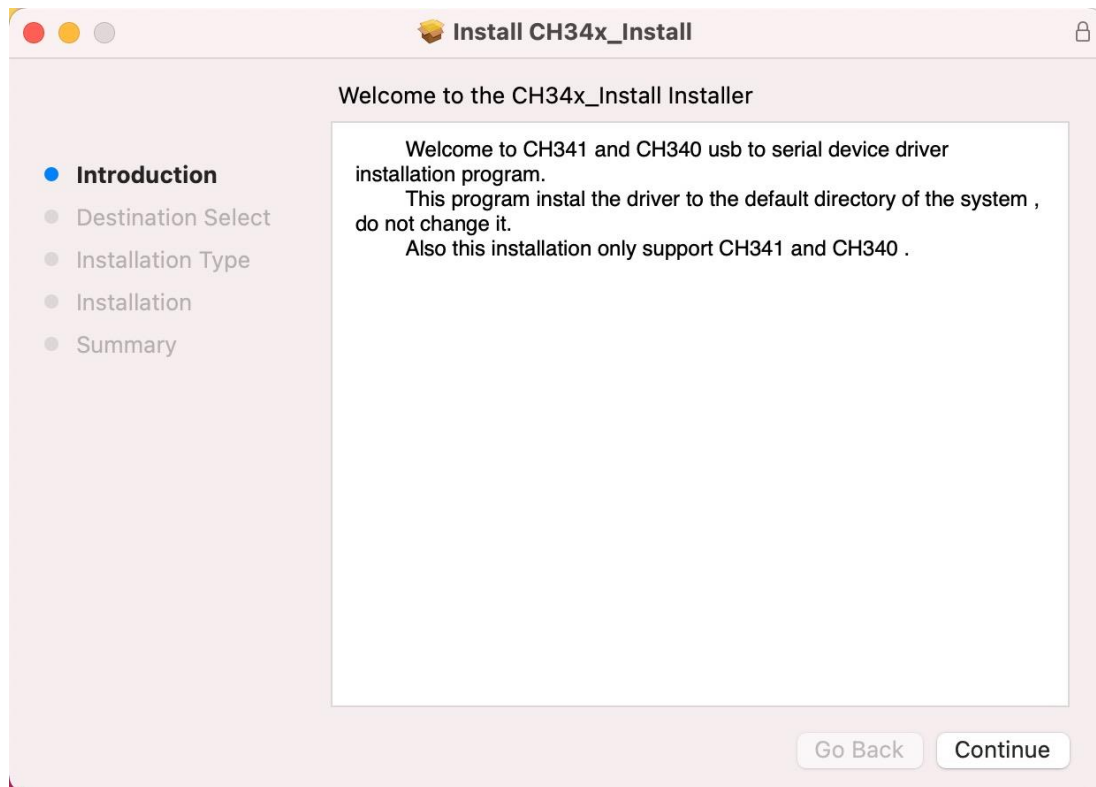
The screenshot shows a macOS Finder window titled 'MAC'. The left sidebar shows the 'Favorites' section with options like AirDrop, Recents, Applications, Desktop, Documents, and Downloads. The main pane displays a list of files:

Name	Modified	Size	Kind
CH34x_Install_V1.5.pkg	Dec 8, 2016 at 2:00 PM	26 KB	Installer package
ReadMe.pdf	Dec 8, 2016 at 4:09 PM	146 KB	Adobe...ocument

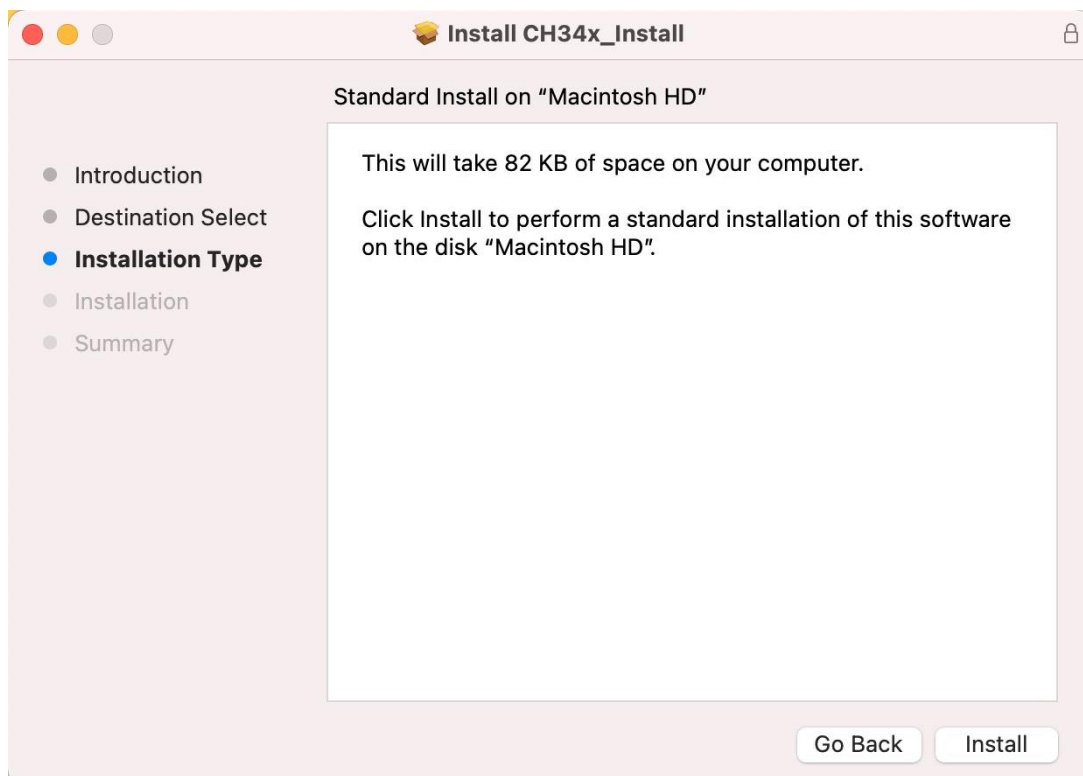
An orange callout box points to the 'CH34x_Install_V1.5.pkg' file with the text 'Run it.'

Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

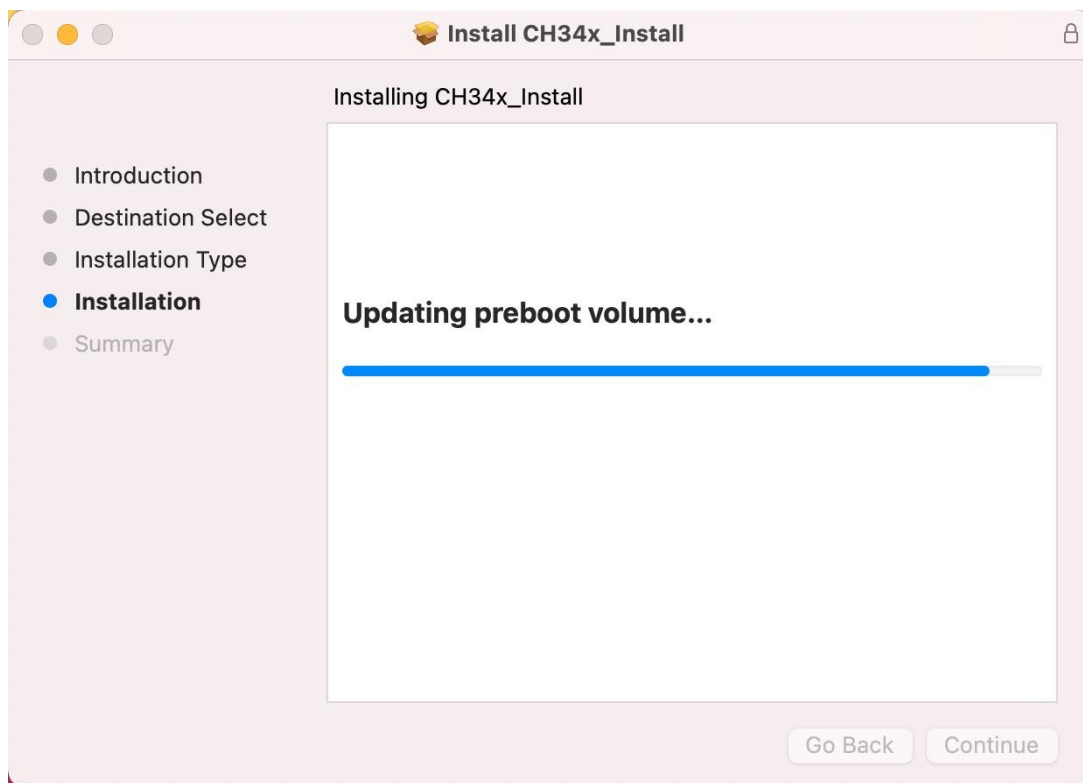
Third, click Continue.



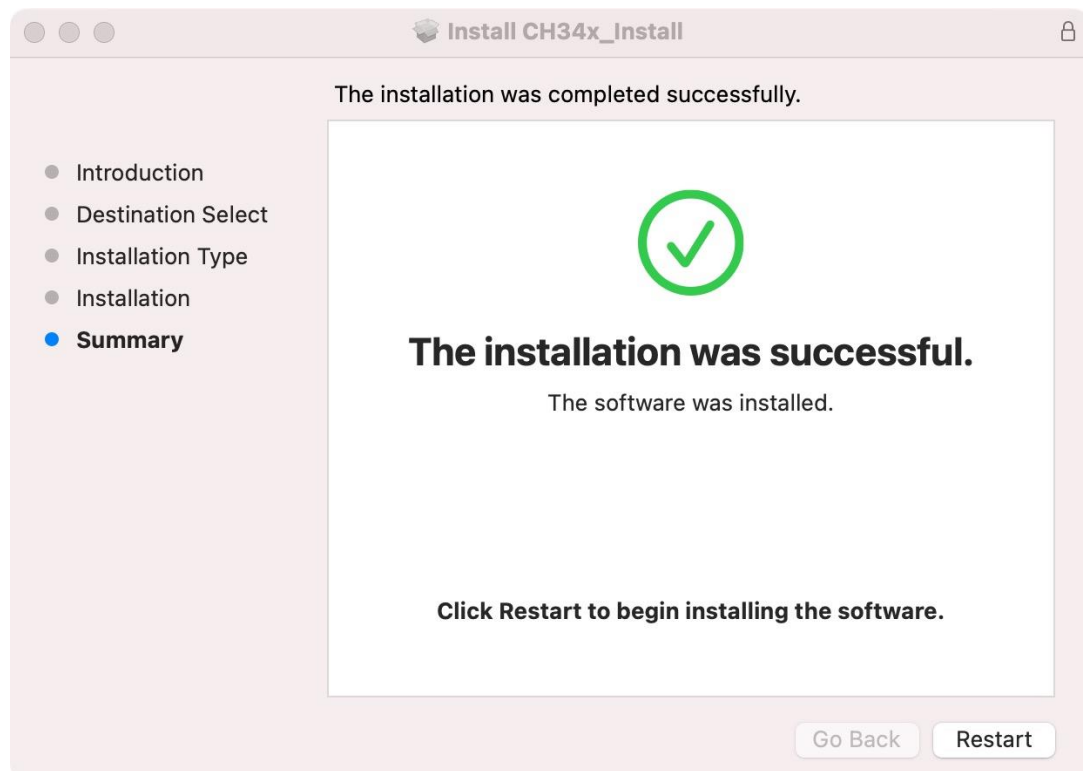
Fourth, click Install.



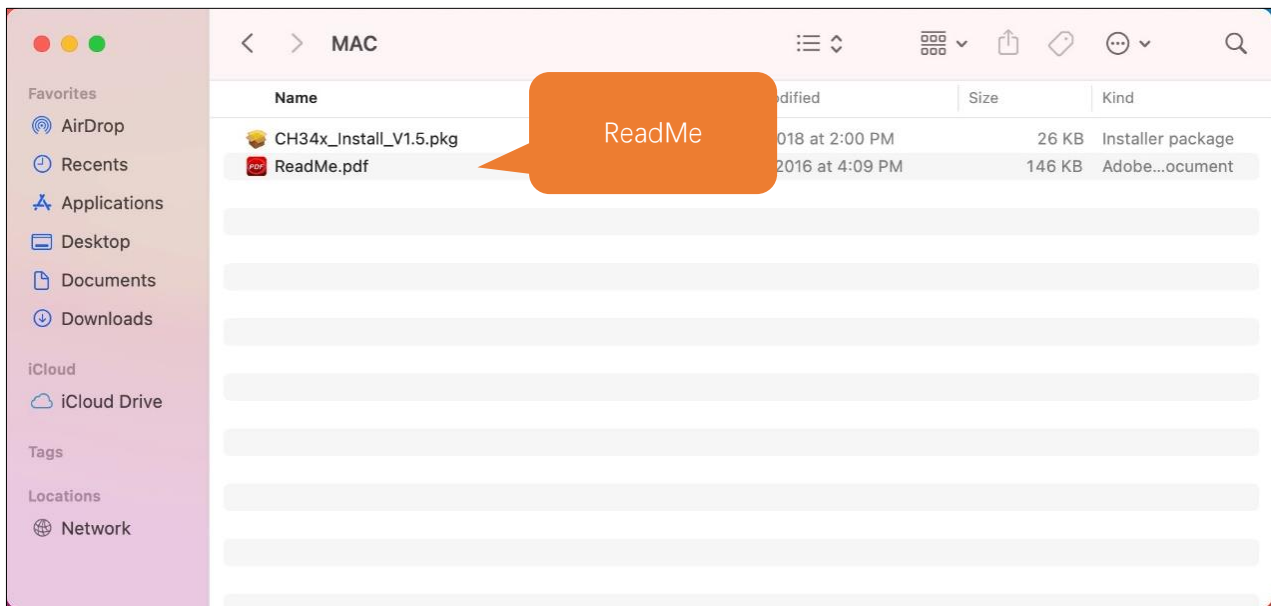
Then, waiting Finish.



Finally, restart your PC.



If you still haven't installed the CH340 by following the steps above, you can view readme.pdf to install it.

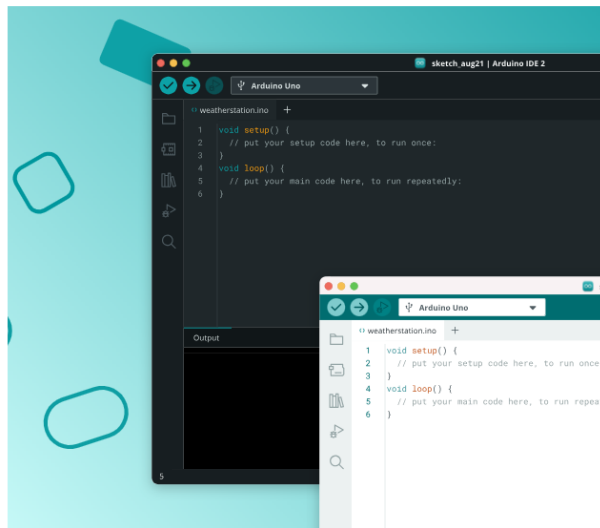


Programming Software

Arduino Software (IDE) is used to write and upload the code for Arduino Board.

First, install Arduino Software (IDE): visit <https://www.arduino.cc/en/software/>

Bring Your Projects to Life with Arduino Software



Arduino IDE 2.3.6

Release notes

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit)

DOWNLOAD



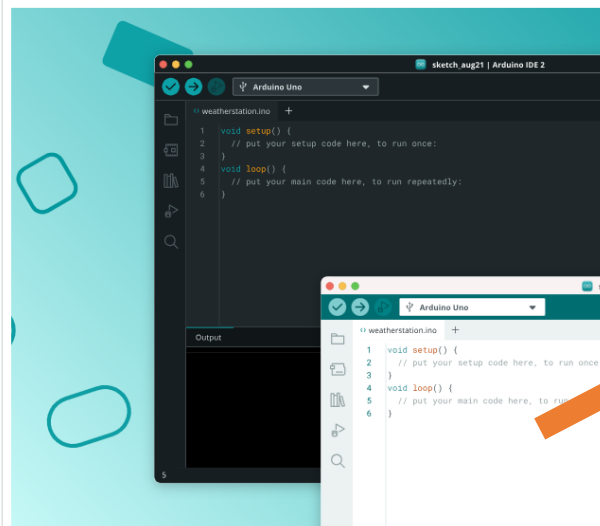
Nightly Builds

Download a preview of the incoming release with the most updated features and bugfixes.

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

Select and download corresponding installer based on your operating system. If you are a Windows user, please select the "Windows" to download and install the driver correctly.

Bring Your Projects to Life with Arduino Software



Arduino IDE 2.3.6

Release notes

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit)

DOWNLOAD

Windows Win 10 or newer (64-bit)

Windows MSI installer

Windows ZIP file

Linux Appliance (64-bit X86-64)

Linux ZIP file (64-bit X86-64)

macOS Intel 10.15 Catalina or newer (64-bit)

macOS Apple Silicon 11 Big Sur or newer (64-bit)



Legacy IDE (1.8.19)

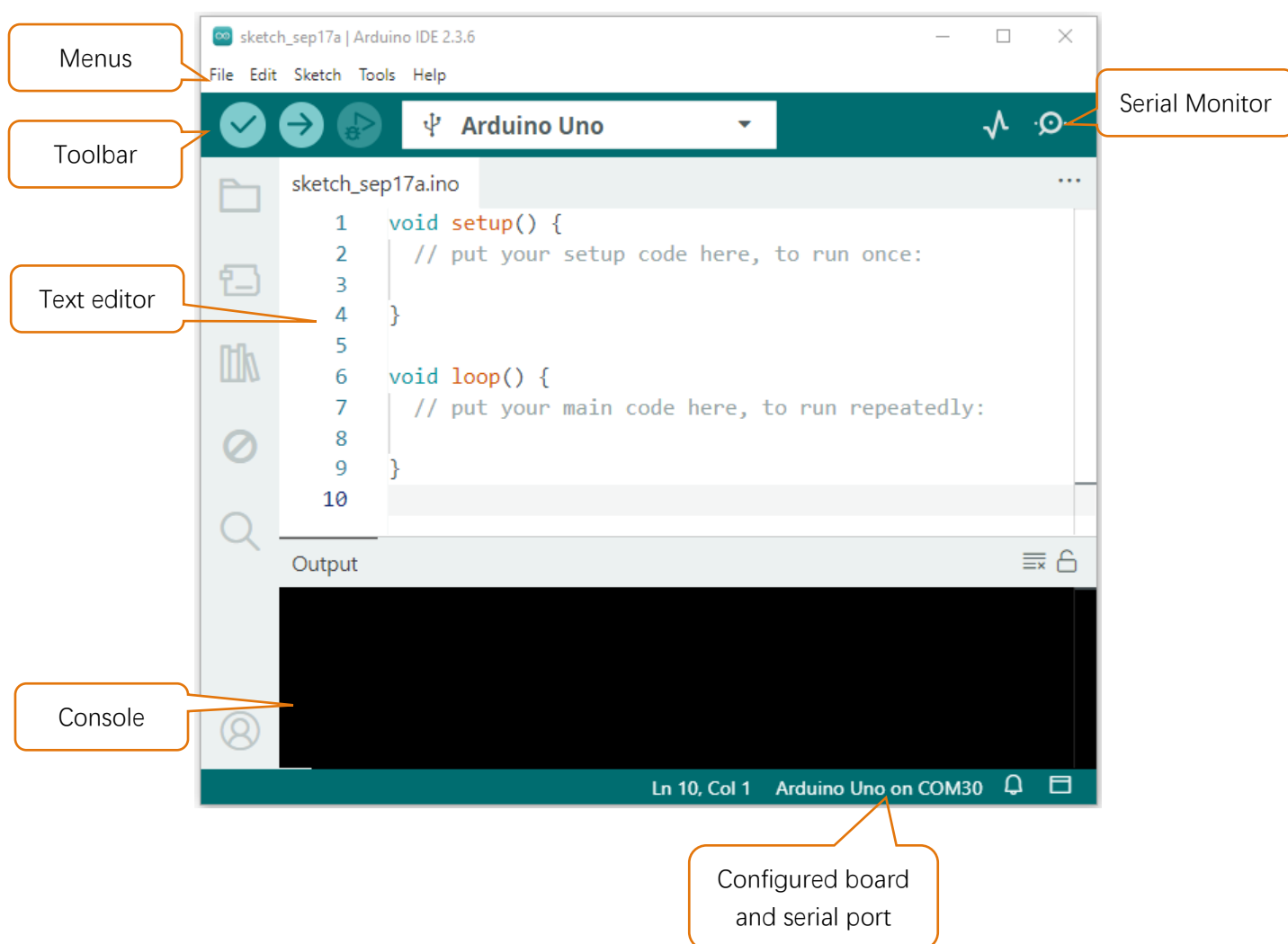
Download a legacy version of the Arduino IDE.

Any concerns? support@freenove.com

After the downloading completes, run the installer. For Windows users, there may pop up an installation dialog box of driver during the installation process. When it is popped up, please allow the installation. After installation is completed, an shortcut will be generated in the desktop.



Run it. The interface of the software is as follows:

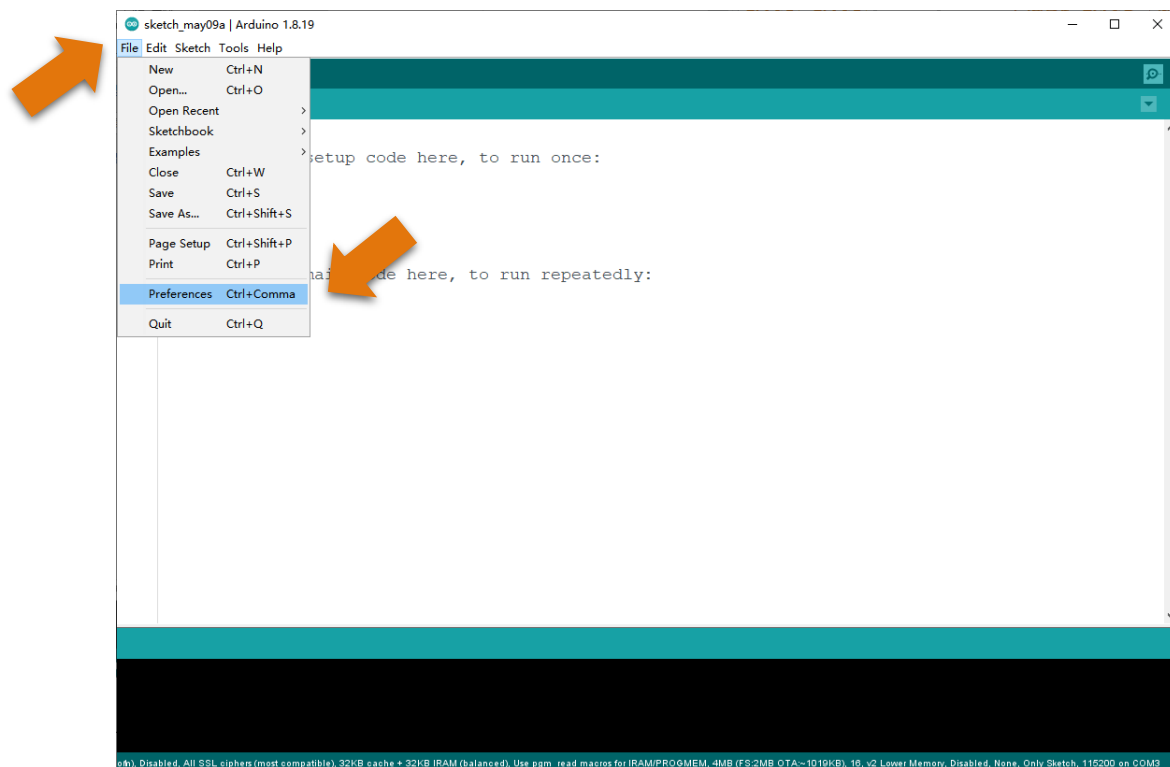


Programs written with Arduino IDE are called sketches. These sketches are written in a text editor and are saved with the file extension.ino. The editor has features for cutting/pasting and for searching/replacing text. The console displays text output by the Arduino IDE, including complete error messages and other information. The bottom right-hand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, open the serial monitor, and access the serial plotter.

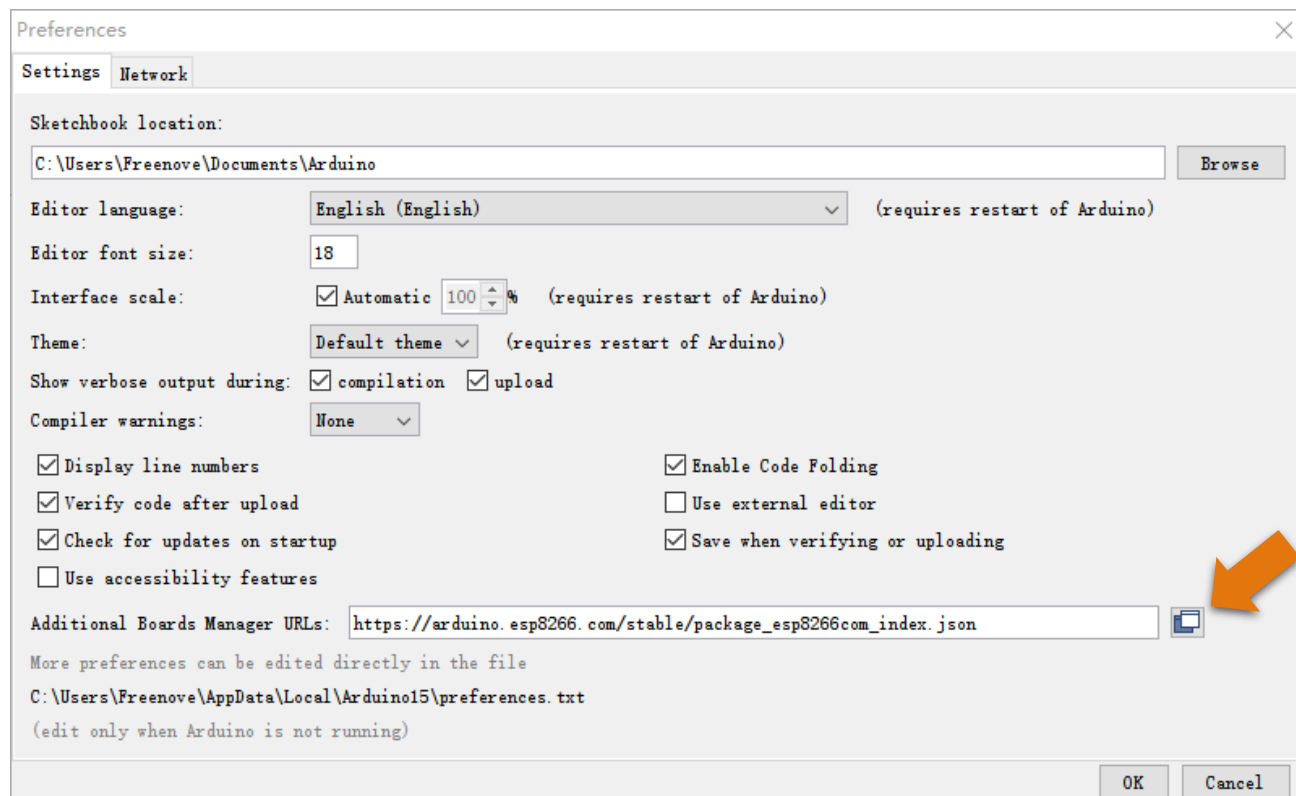
	Verify Checks your code for errors compiling it.
	Upload Compiles your code and uploads it to the configured board.
	Debug Troubleshoot code errors and monitor program running status.
	Serial Plotter Real-time plotting of serial port data charts.
	Serial Monitor Used for debugging and communication between devices and computers.

Environment Configuration

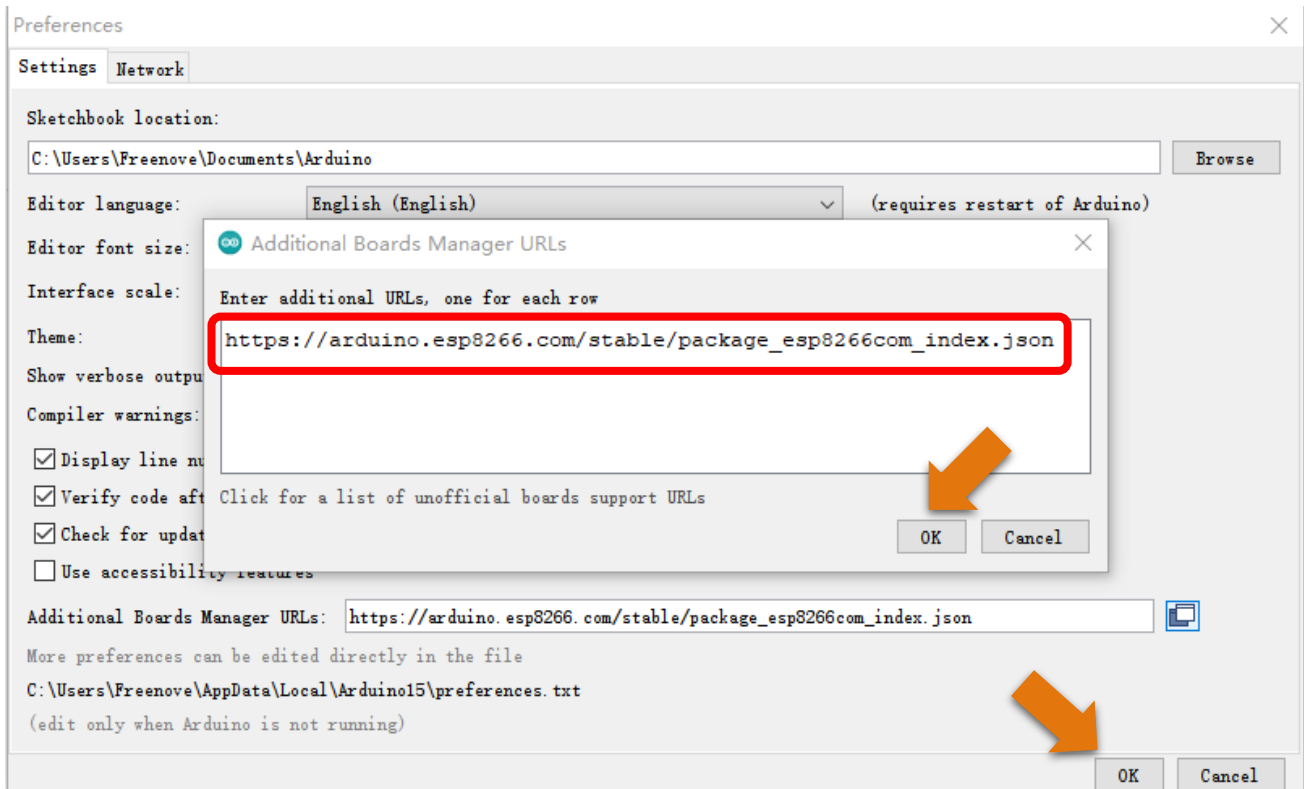
First, open the software platform arduino, and then click File in Menus and select Preferences.



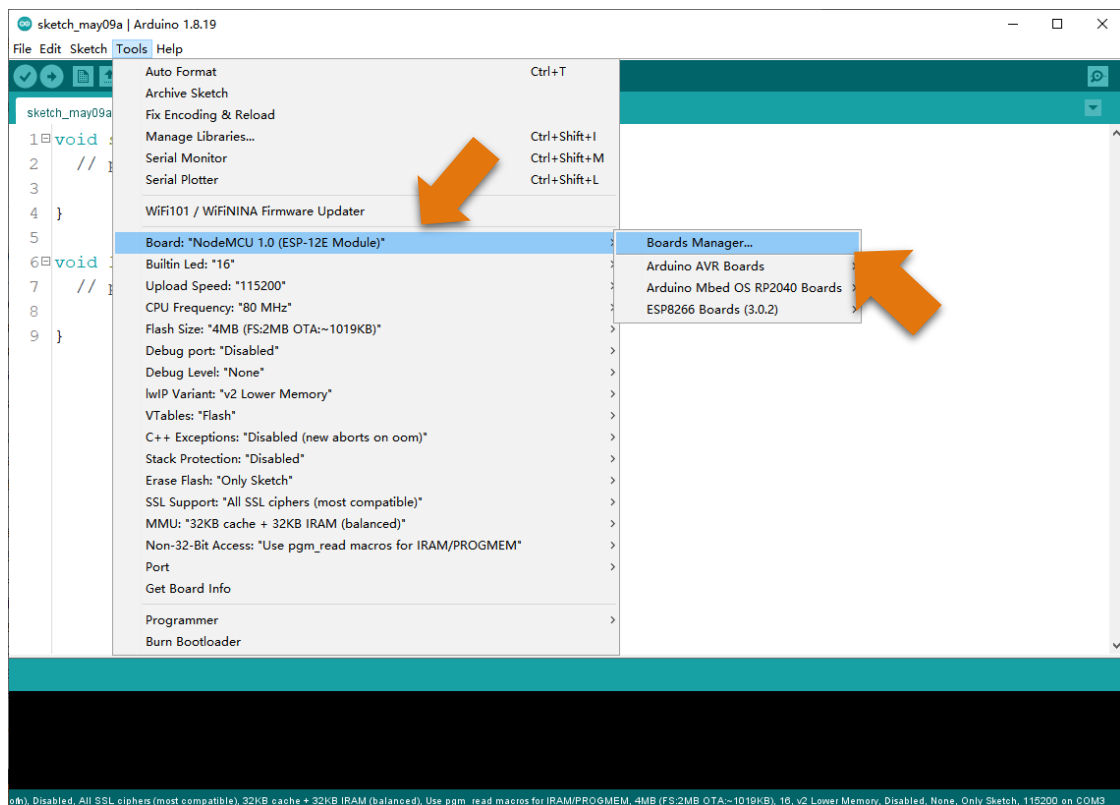
Second, click on the symbol behind "Additional Boards Manager URLs"



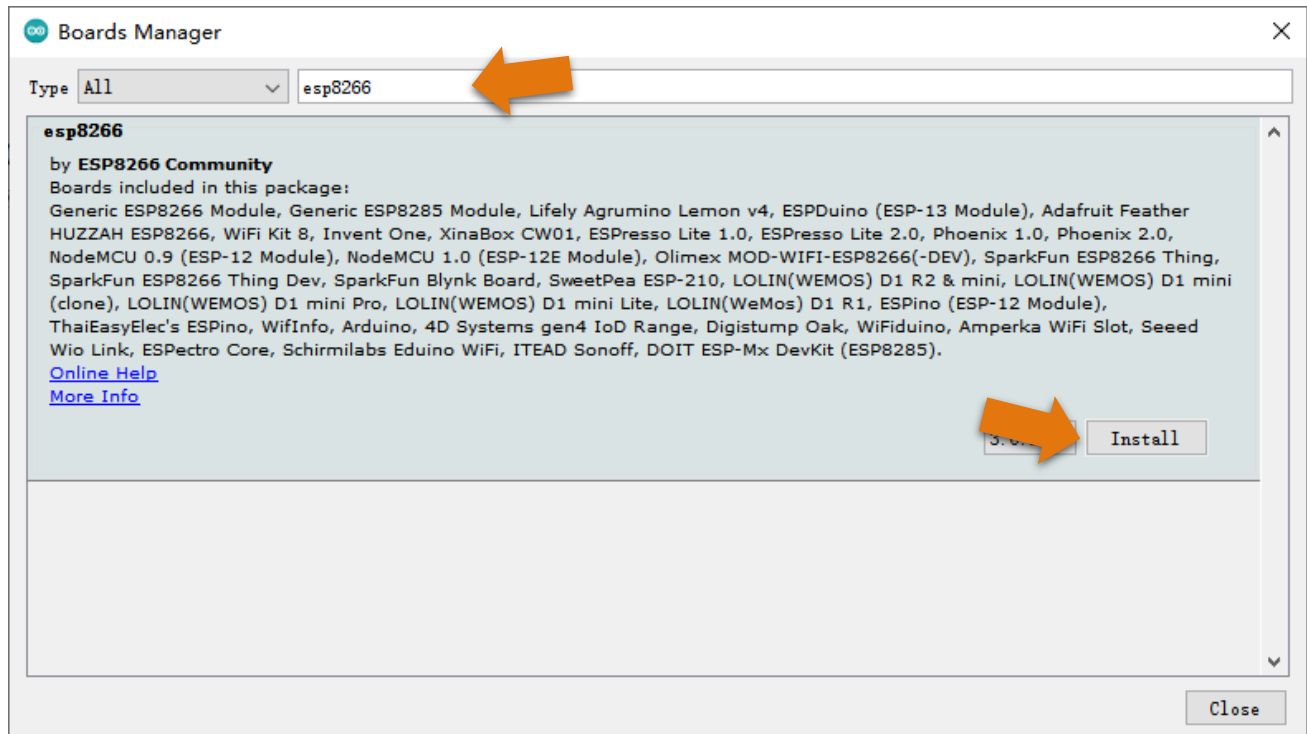
Third, fill in https://arduino.esp8266.com/stable/package_esp8266com_index.json in the new window, click OK, and click OK on the Preferences window again.



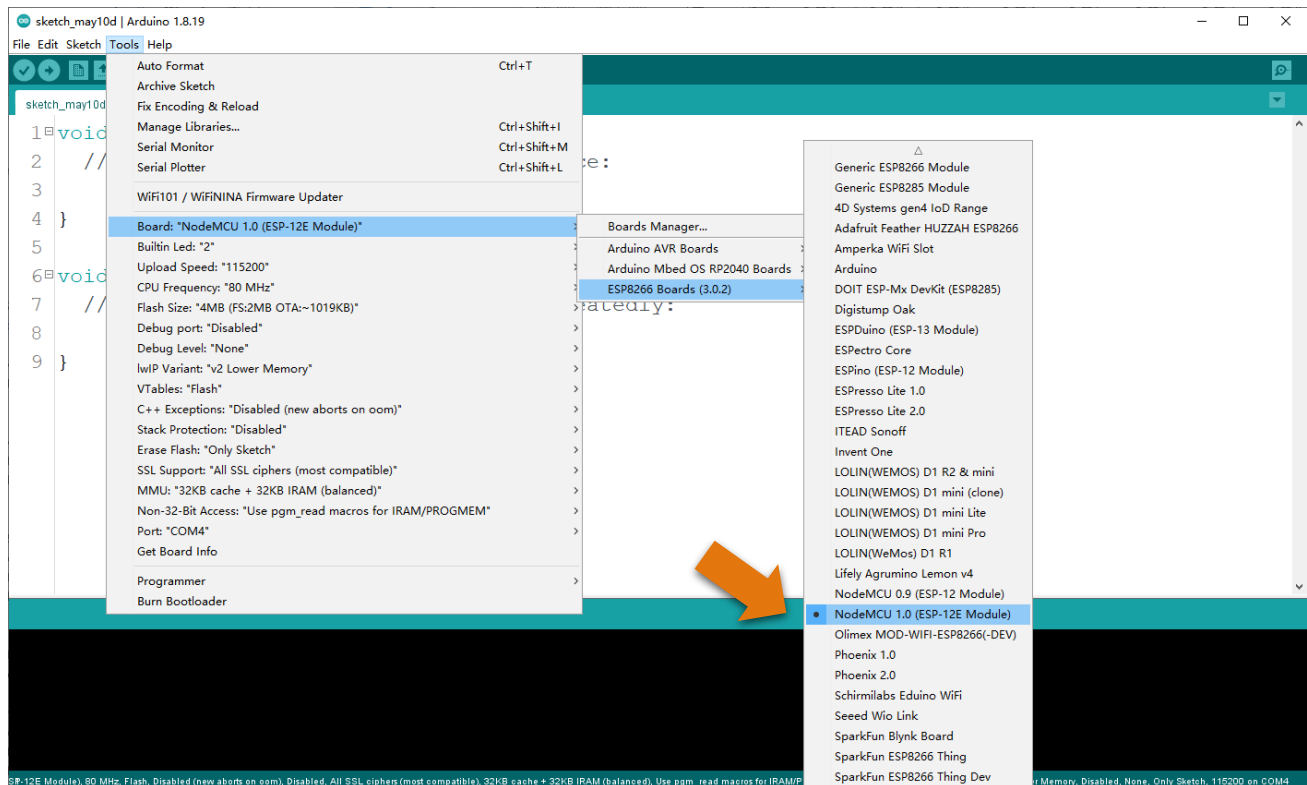
Fourth, click Tools in Menu, select Board:"ArduinoUno", and then select "Boards Manager".



Fifth, input "esp8266" in the window below, and press Enter. click "Install" to install.

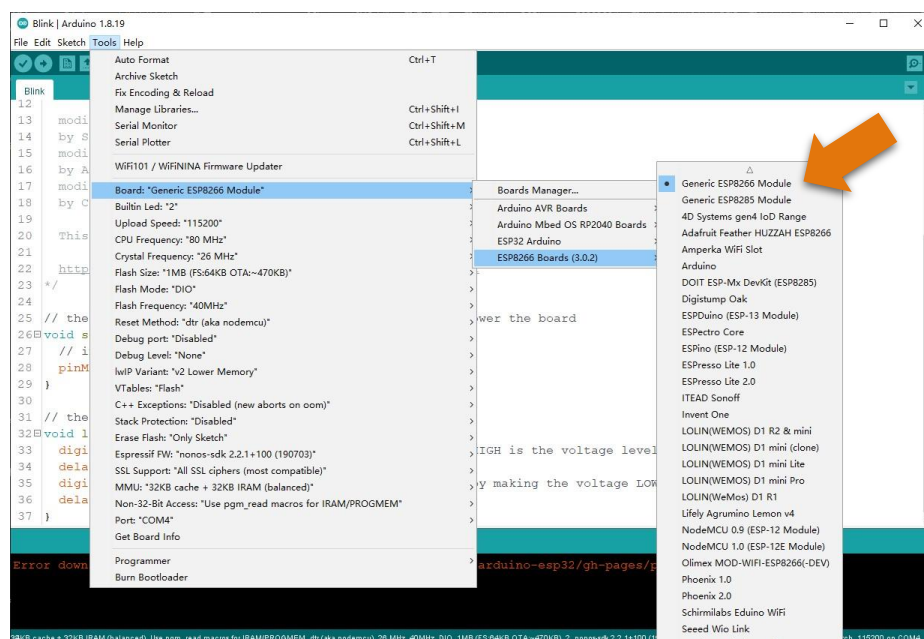


When finishing installation, click Tools in the Menus again and select Board: "NodeMCU 1.0(ESP-12E Module)", and then you can see information of ESP8266 click "NodeMCU 1.0(ESP-12E Module)" so that the ESP8266 programming development environment is configured.

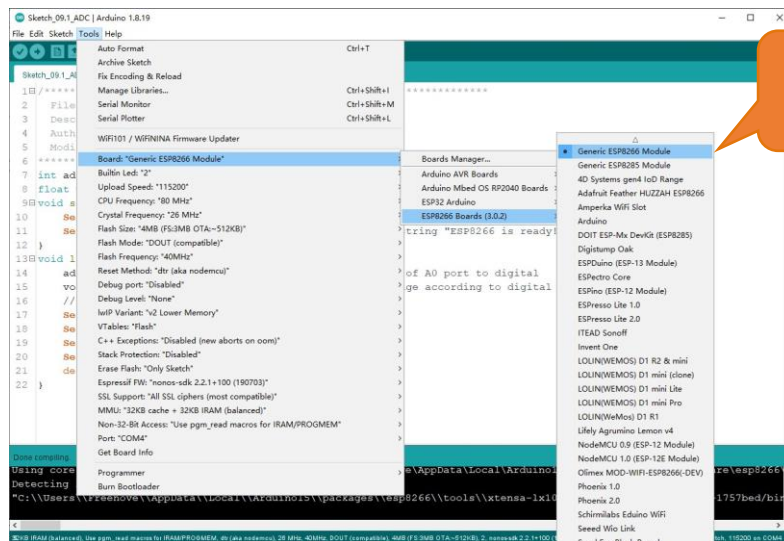


In our tutorial, we chose "NodeMCU 1.0(ESP-12E Module)" as the development board Module. This choice will facilitate learning and understanding of ESP8266. Of course, you can choose "Generic ESP8266 Module". Select "Generic ESP8266 Module" to apply to all Generic ESP8266 modules. Of course, this setup will have some more common configuration.

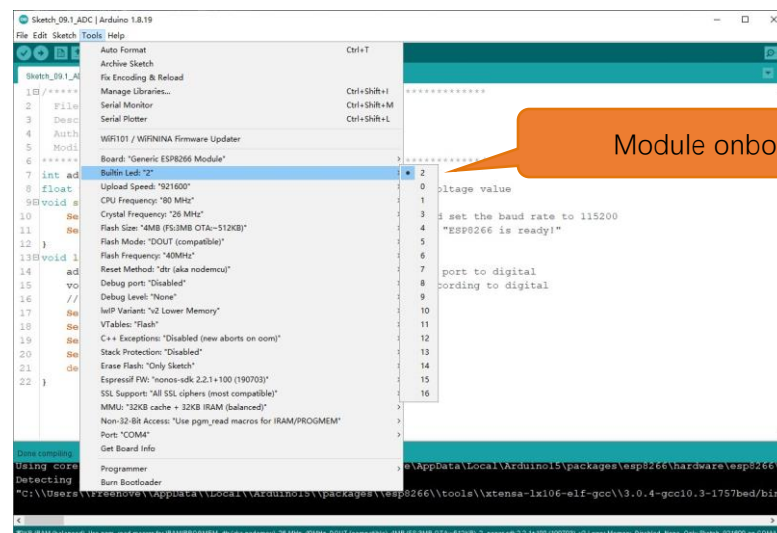
When you select "Generic ESP8266 Module", the interface is as follows:



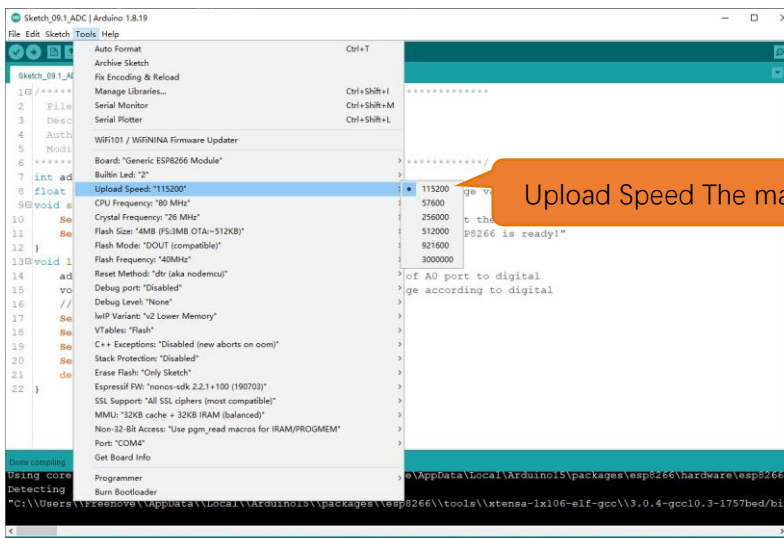
Select the module type. Here you can choose the appropriate module based on your requirements.



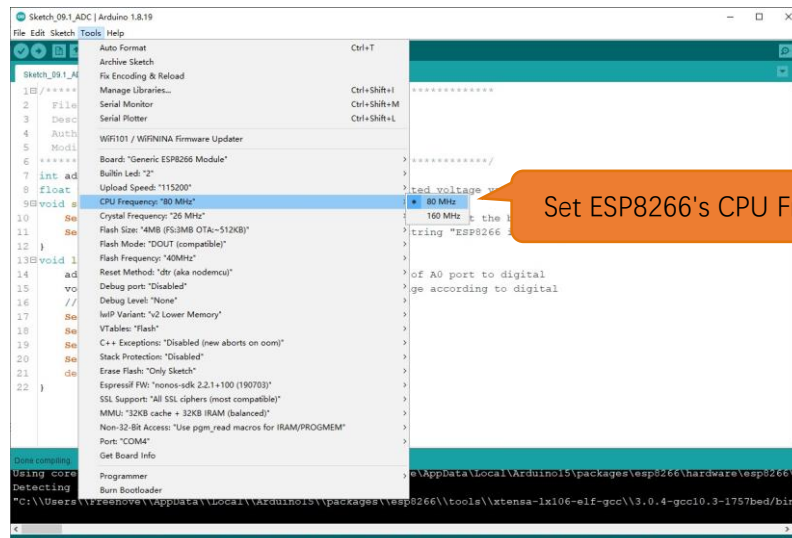
Module Onboard LED, in our ESP8266 development board, has an onboard LED of 2.



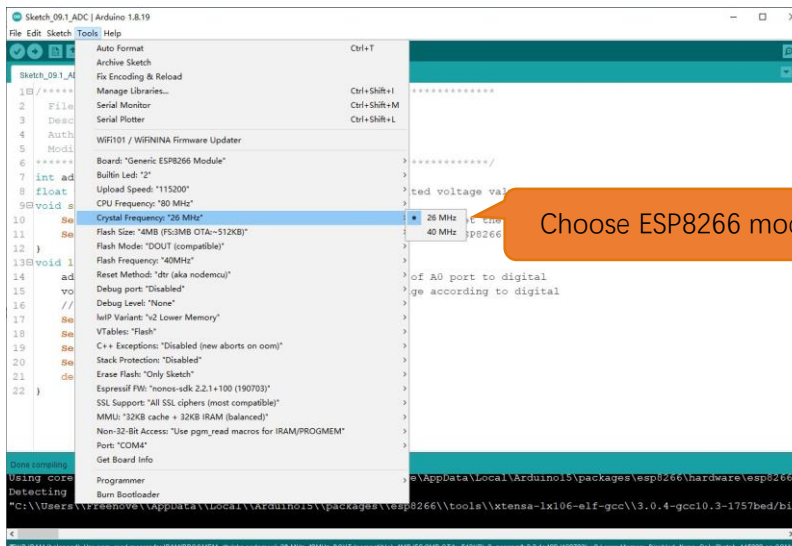
Upload Speed The maximum value is 921600. By default, Upload Speed is 115200. You can choose according to your needs.



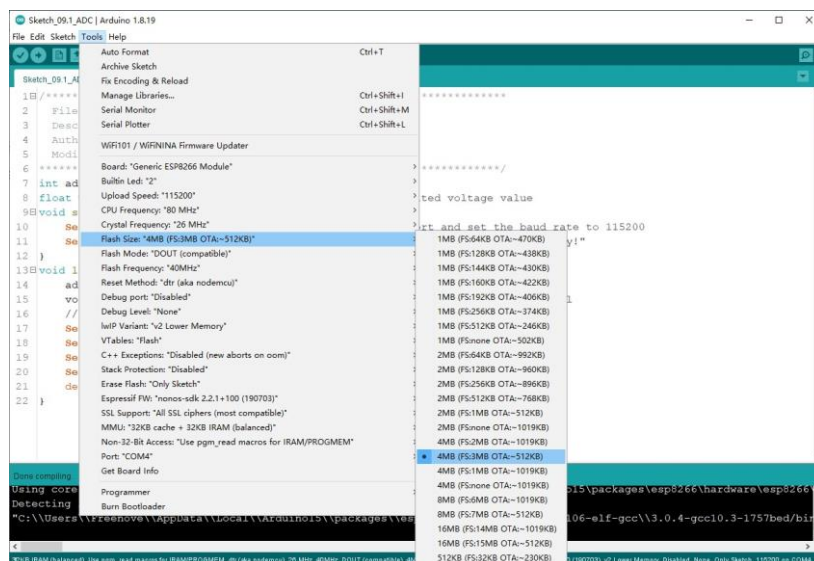
ESP8266's CPU frequency standard is 80MHz, which can be changed to 160MHz.



Most ESP8266 modules use 26 MHz crystals, but some have other values.



Choose the appropriate Flash size based on your ESP8266 module type. In our ESP8266 development board, we chose 4MB (FS: 3MB OTA: ~512KB).



Here we need to select Flash mode. On our ESP8266 development board, choose "DIO" mode or "DOUT" mode for better compatibility. If the ESP8266 module is abnormal, check whether the ESP8266 module works in the two modes.

Flash works in DOUT, DIO, QOUT, and QIO modes.

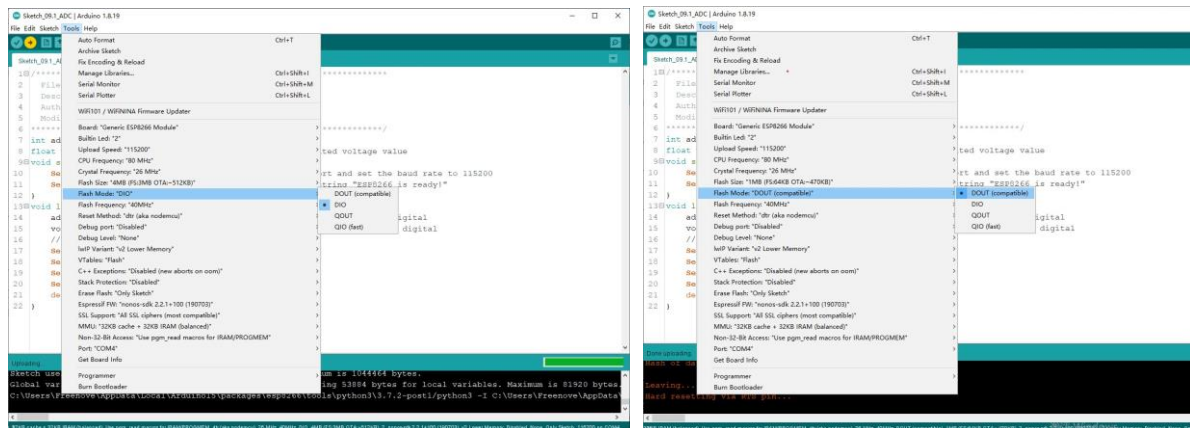
1. DOUT: Address is input in 1-line mode and data is output in 2-line mode.

2. DIO: Address is input in 2-line mode and data is output in 2-line mode.

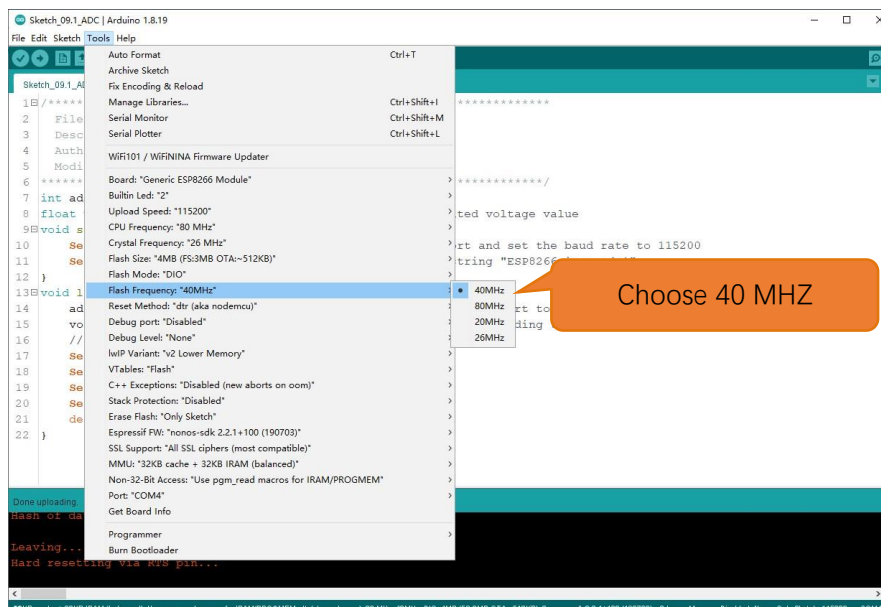
3. QOUT: Address is input in 1-line mode and data is output in 4-line mode.

4. QIO: Address is input in 4-line mode and data is output in 4-line mode.

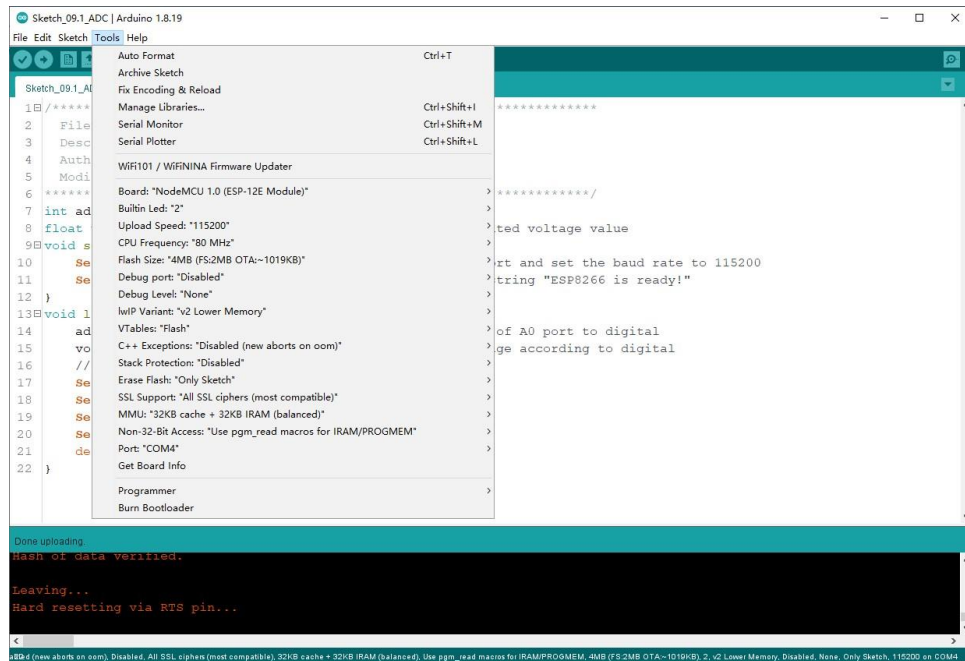
If you need to use the QIO mode, ensure that the Flash supports the QIO mode.



The flash chip connected to most chips operates at 40MHz clock speed, but you can try a lower value if the device fails to boot. The highest flash clock speed of 80MHz will provide the best performance, but can cause crashes if the flash or board design cannot achieve this speed.



If you select NodeMCU 1.0(ESP-12E Module), the following interface is displayed:



Here, you can see that this is similar to "Generic ESP8266 Module" in that the omitted parts are configured with default values. If you have problems working through this tutorial, try using the "Generic ESP8266 Module" configuration.

If you need any support, please feel free to contact us via: support@freenove.com

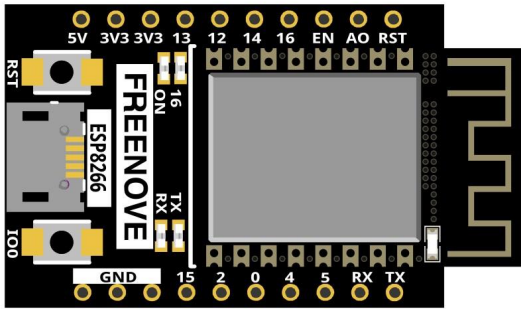
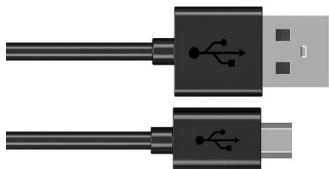
Chapter 0 LED

This chapter is the Start Point in the journey to build and explore ESP8266 electronic projects. We will start with simple “Blink” project.

Project 0.1 Blink

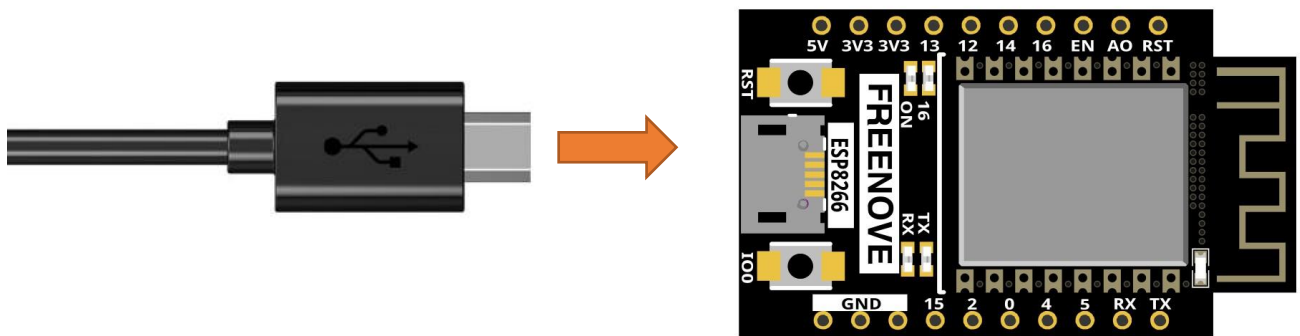
In this project, we will use ESP8266 to control blinking a common LED.

Component List

ESP8266 x1 	USB cable 
--	---

Power

ESP8266 needs 5v power supply. In this tutorial, we need connect ESP8266 development board to computer via USB cable to power it and program it. We can also use other 5v power source to power it.



In the following projects, we only use USB cable to power ESP8266 development board by default.

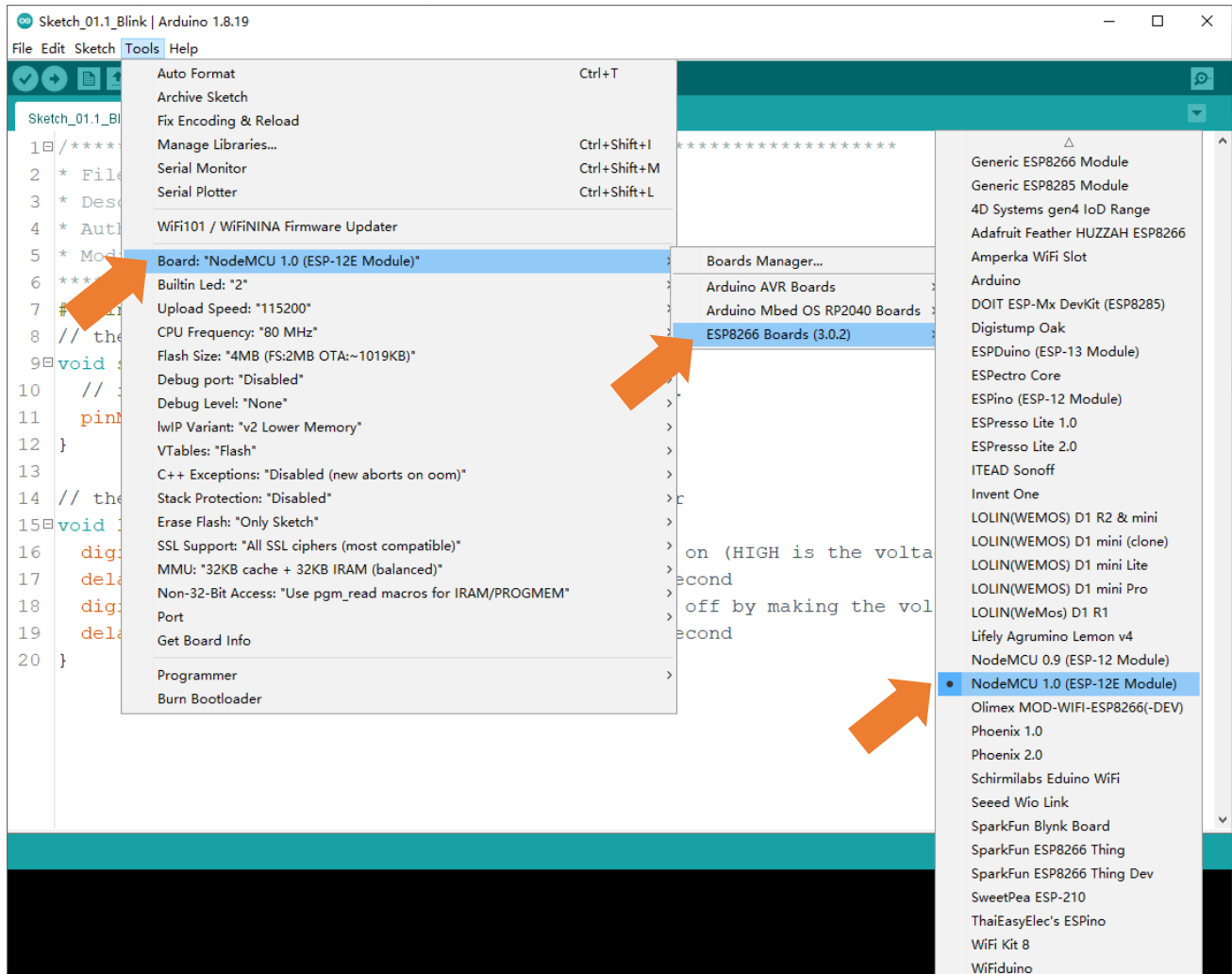
Sketch

According to the circuit, when the GPIO2 of ESP8266 output level is high, the LED turns ON. Conversely, when the GPIO2 ESP8266 output level is low, the LED turns OFF. Therefore, we can let GPIO2 circularly output high and low level to make the LED blink.

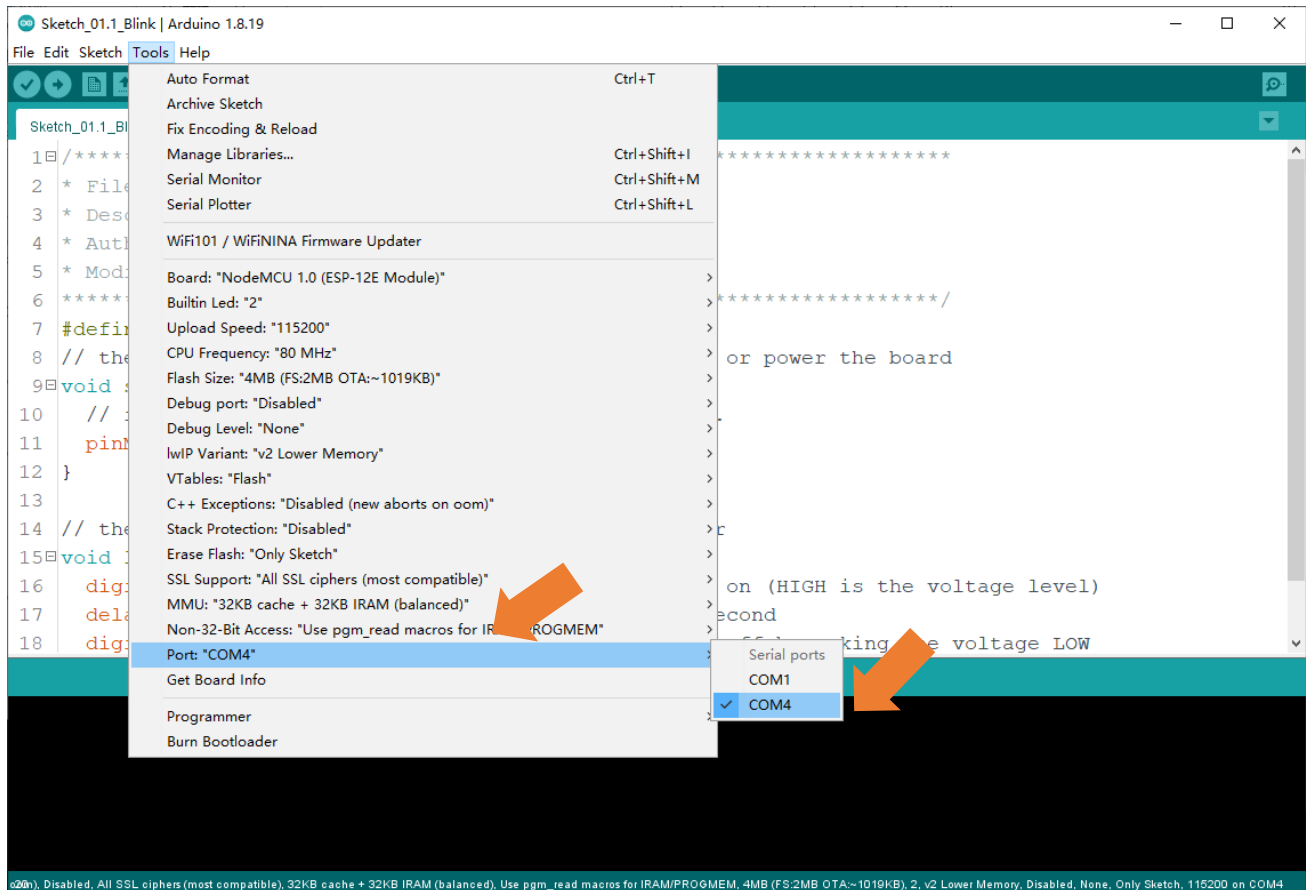
Upload the following Sketch:

Freenove_Basic_Starter_Kit_for_ESP8266\C\Sketches\Sketch_01.1_Blink

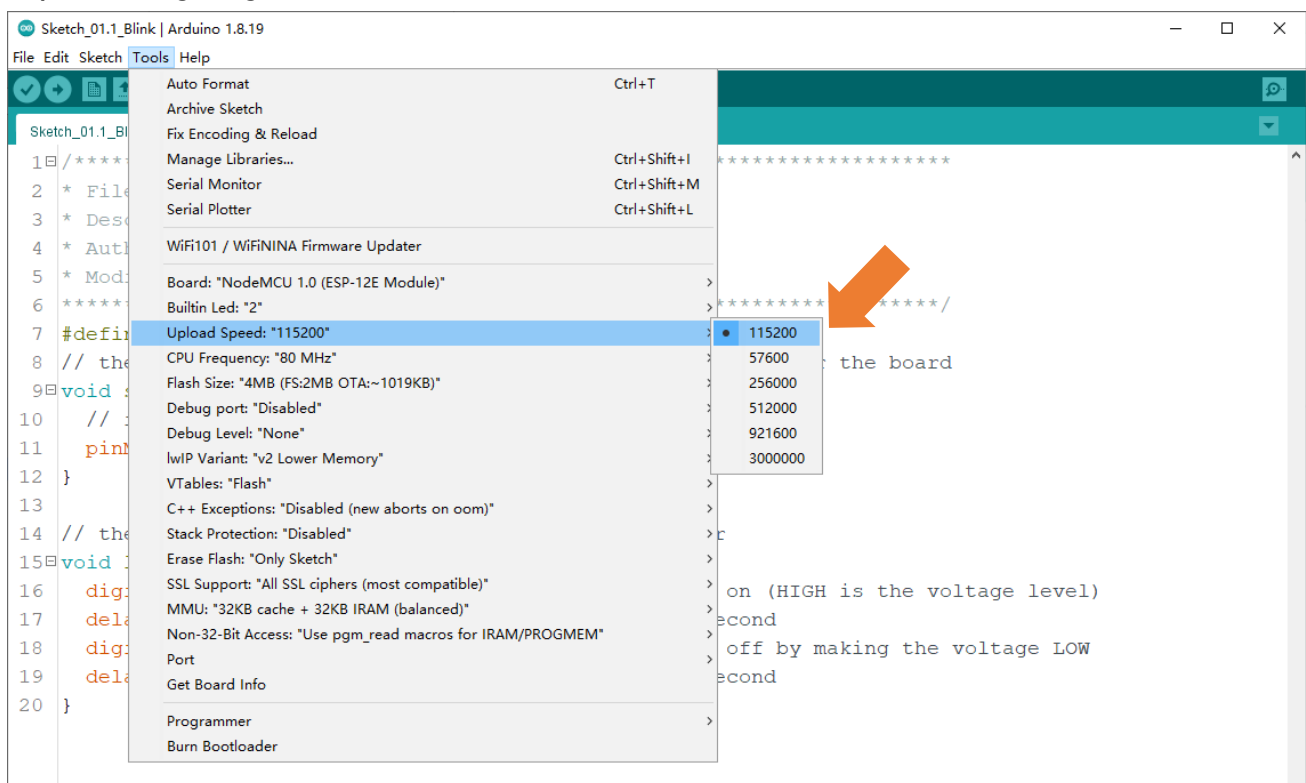
Before uploading the code, click "Tools", "Board" and select " **NodeMCU 1.0 (ESP-12E Module)** ".



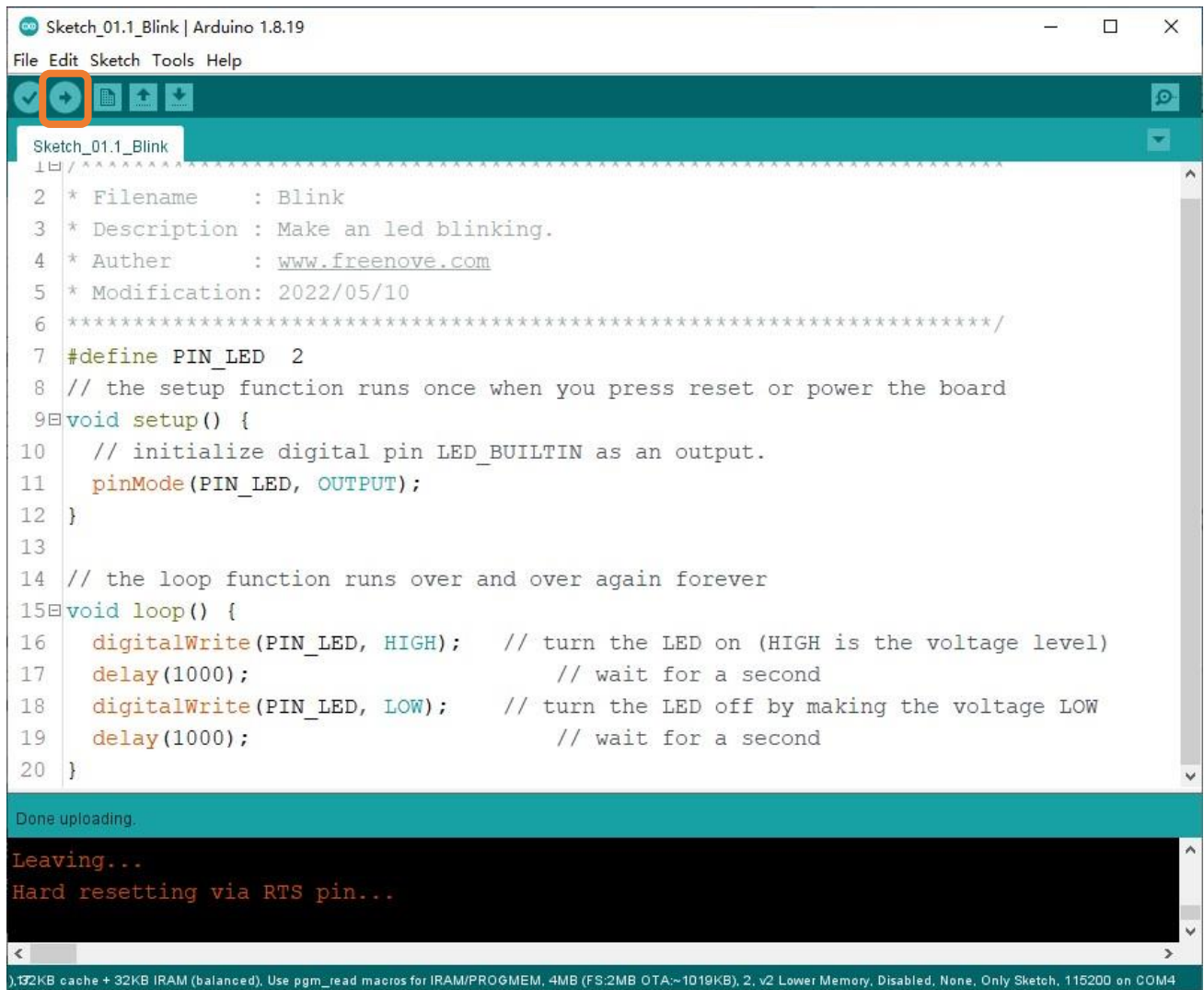
Select the serial port.



Note: For macOS users, if the uploading fails, please set the baud rate to 115200 before clicking “Upload Using Programmer”.



Sketch_01.1_Blink



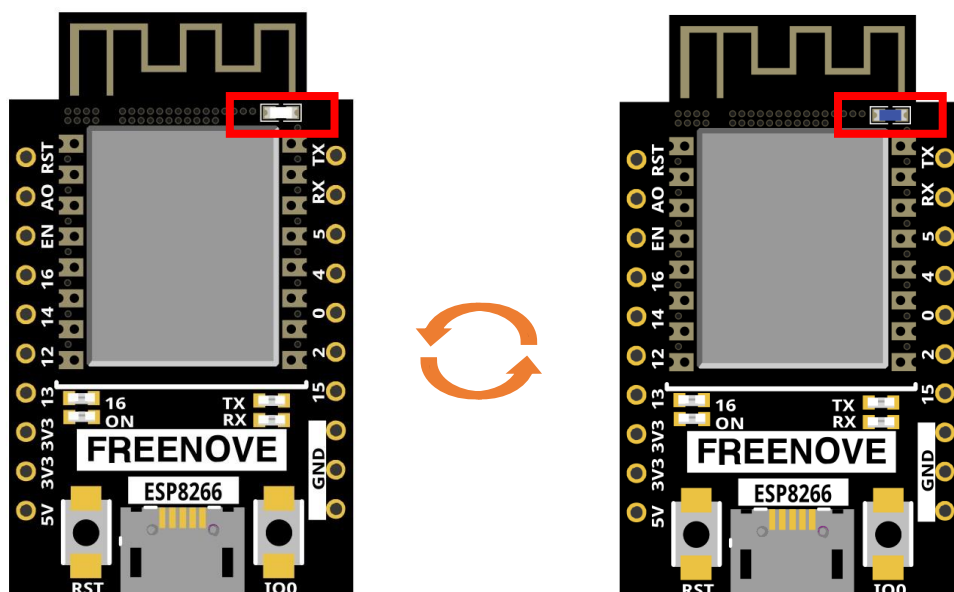
The screenshot shows the Arduino IDE interface with the 'Sketch_01.1_Blink' sketch loaded. The 'Upload' button (a green arrow) is highlighted with a red box. The code is as follows:

```
Sketch_01.1_Blink | Arduino 1.8.19
File Edit Sketch Tools Help

Sketch_01.1_Blink
1 //
2 * Filename      : Blink
3 * Description   : Make an led blinking.
4 * Author       : www.freenove.com
5 * Modification: 2022/05/10
6 *****/
7 #define PIN_LED 2
8 // the setup function runs once when you press reset or power the board
9 void setup() {
10   // initialize digital pin LED_BUILTIN as an output.
11   pinMode(PIN_LED, OUTPUT);
12 }
13
14 // the loop function runs over and over again forever
15 void loop() {
16   digitalWrite(PIN_LED, HIGH); // turn the LED on (HIGH is the voltage level)
17   delay(1000);                 // wait for a second
18   digitalWrite(PIN_LED, LOW);  // turn the LED off by making the voltage LOW
19   delay(1000);                 // wait for a second
20 }
```

Below the code editor, the status bar shows 'Done uploading.' and 'Leaving... Hard resetting via RTS pin...'. At the bottom, a status bar provides technical details: '132KB cache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA~1019KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4'.

Click "Upload", Download the code to ESP8266 and your LED in the circuit starts Blink.



If you have any concerns, please contact us via: support@freenove.com

Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

The following is the program code:

```

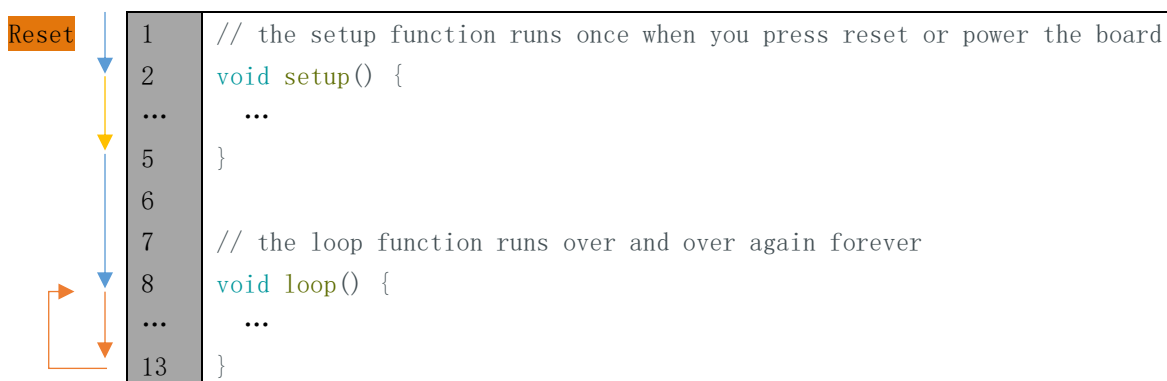
1  #define PIN_LED 2
2  // the setup function runs once when you press reset or power the board
3  void setup() {
4      // initialize digital pin LED_BUILTIN as an output.
5      pinMode(PIN_LED, OUTPUT);
6  }
7
8  // the loop function runs over and over again forever
9  void loop() {
10     digitalWrite(PIN_LED, HIGH); // turn the LED on (HIGH is the voltage level)
11     delay(1000);                 // wait for a second
12     digitalWrite(PIN_LED, LOW);  // turn the LED off by making the voltage LOW
13     delay(1000);                 // wait for a second
14 }

```

The Arduino IDE code usually contains two basic functions: void setup() and void loop().

After the board is reset, the setup() function will be executed firstly, and then the loop() function.

setup() function is generally used to write code to initialize the hardware. And loop() function is used to write code to achieve certain functions. loop() function is executed repeatedly. When the execution reaches the end of loop(), it will jump to the beginning of loop() to run again.



Reset

Reset operation will lead the code to be executed from the beginning. Switching on the power, finishing uploading the code and pressing the reset button will trigger reset operation.

In the circuit, ESP8266's GPIO2 is connected to the LED, so the LED pin is defined as 2.

```
1 #define PIN_LED 2
```

This means that after this line of code, all PIN_LED will be treated as 2.

In the setup () function, first, we set the PIN_LED as output mode, which can make the port output high level or low level.

```

4 // initialize digital pin PIN_LED as an output.
5 pinMode(PIN_LED, OUTPUT);

```

Then, in the loop () function, set the PIN_LED to output high level to make LED light up.

```
10 digitalWrite(PIN_LED, HIGH); // turn the LED on (HIGH is the voltage level)
```

Wait for 1000ms, that is 1s. Delay () function is used to make control board wait for a moment before executing the next statement. The parameter indicates the number of milliseconds to wait for.

11	<code>delay(1000);</code>	<code>// wait for a second</code>
----	---------------------------	-----------------------------------

Then set the PIN_LED to output low level, and LED light off. One second later, the execution of loop () function will be completed.

12	<code>digitalWrite(PIN_LED, LOW);</code>	<code>// turn the LED off by making the voltage LOW</code>
13	<code>delay(1000);</code>	<code>// wait for a second</code>

The loop() function is constantly being executed, so LED will keep blinking.

Reference

`void pinMode(int pin, int mode);`

Configures the specified pin to behave either as an input or an output.

Parameters

pin: the pin number to set the mode of.

mode: INPUT, OUTPUT, INPUT_PULLDOWN, or INPUT_PULLUP.

`void digitalWrite (int pin, int value);`

Writes the value HIGH or LOW (1 or 0) to the given pin which must have been previously set as an output.

For more related functions, please refer to <https://www.arduino.cc/reference/en/>

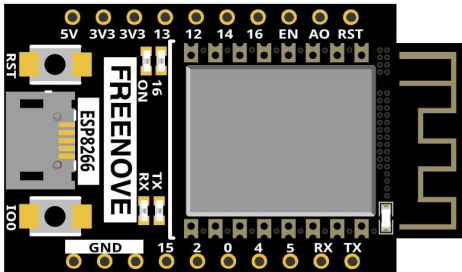
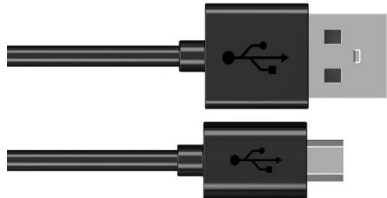
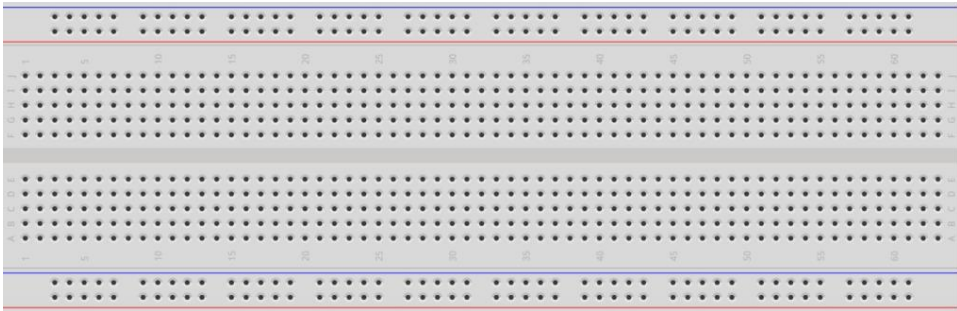
Chapter 1 LED

This chapter is the Start Point in the journey to build and explore ESP8266 electronic projects. We will start with simple “Blink” project.

Project 1.1 Blink

In this project, we will use ESP8266 to control blinking a common LED.

Component List

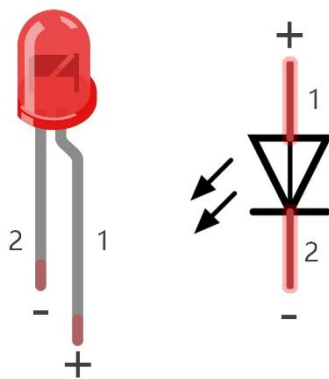
ESP8266 x1 	USB cable 	
Breadboard x1 		
LED x1 	Resistor 220Ω x1 	Jumper wire M/M x3 

Component knowledge

LED

A LED is a type of diode. All diodes only work if current is flowing in the correct direction and have two poles. A LED will only work (light up) if the longer pin (+) of LED is connected to the positive output from a power source and the shorter pin is connected to the negative (-). Negative output is also referred to as Ground (GND). This type of component is known as “diodes” (think One-Way Street).

All common 2 lead diodes are the same in this respect. Diodes work only if the voltage of its positive electrode is higher than its negative electrode and there is a narrow range of operating voltage for most all common diodes of 1.9 and 3.4V. If you use much more than 3.3V the LED will be damaged and burn out.



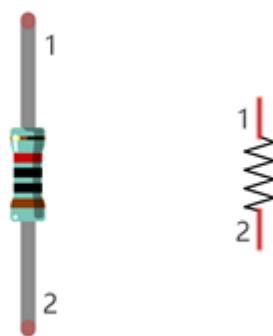
LED	Voltage	Maximum current	Recommended current
Red	1.9-2.2V	20mA	10mA
Green	2.9-3.4V	10mA	5mA
Blue	2.9-3.4V	10mA	5mA
Volt ampere characteristics conform to diode			

Note: LEDs cannot be directly connected to a power supply, which usually ends in a damaged component. A resistor with a specified resistance value must be connected in series to the LED you plan to use.

Resistor

Resistors use Ohms (Ω) as the unit of measurement of their resistance (R). $1M\Omega=1000k\Omega$, $1k\Omega=1000\Omega$.

A resistor is a passive electrical component that limits or regulates the flow of current in an electronic circuit. On the left, we see a physical representation of a resistor, and the right is the symbol used to represent the presence of a resistor in a circuit diagram or schematic.

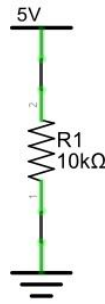


The bands of color on a resistor is a shorthand code used to identify its resistance value. For more details of resistor color codes, please refer to the appendix of this tutorial.

With a fixed voltage, there will be less current output with greater resistance added to the circuit. The relationship between Current, Voltage and Resistance can be expressed by this formula: $I=V/R$ known as Ohm's Law where I = Current, V = Voltage and R = Resistance. Knowing the values of any two of these allows you to solve the value of the third.

In the following diagram, the current through R1 is: $I=U/R=5V/10k\Omega=0.0005A=0.5mA$.

Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

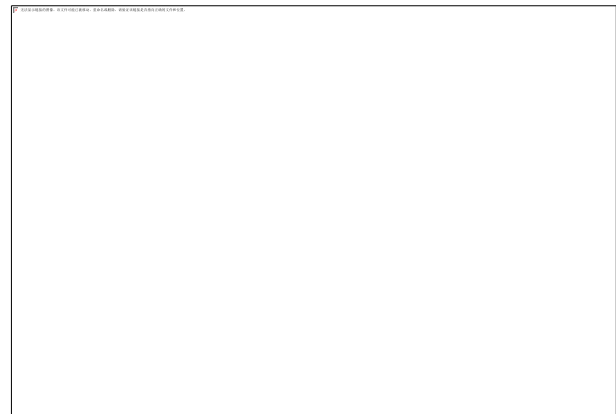
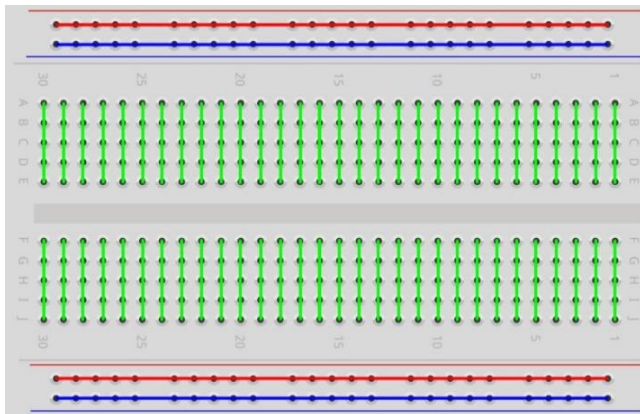


WARNING: Never connect the two poles of a power supply with anything of low resistance value (i.e. a metal object or bare wire) this is a Short and results in high current that may damage the power supply and electronic components.

Note: Unlike LEDs and diodes, resistors have no poles and are non-polar (it does not matter which direction you insert them into a circuit, it will work the same)

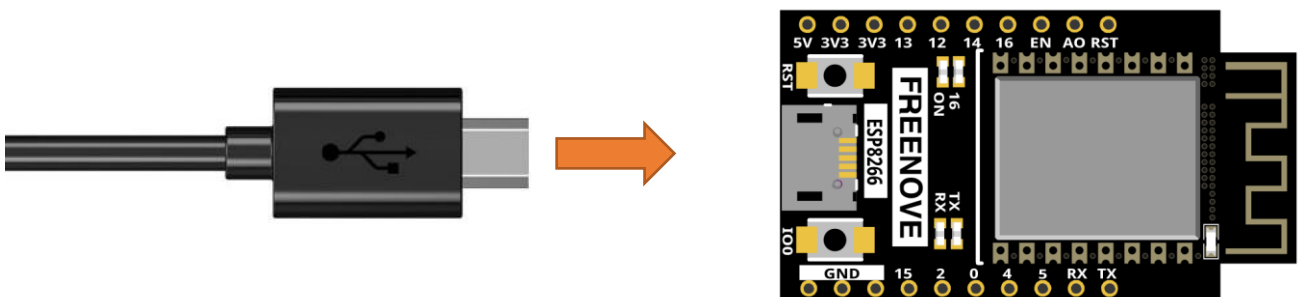
Breadboard

Here we have a small breadboard as an example of how the rows of holes (sockets) are electrically attached. The left picture shows the way to connect pins. The right picture shows the practical internal structure.



Power

ESP8266 needs 5v power supply. In this tutorial, we need connect ESP8266 to computer via USB cable to power it and program it. We can also use other 5v power source to power it.



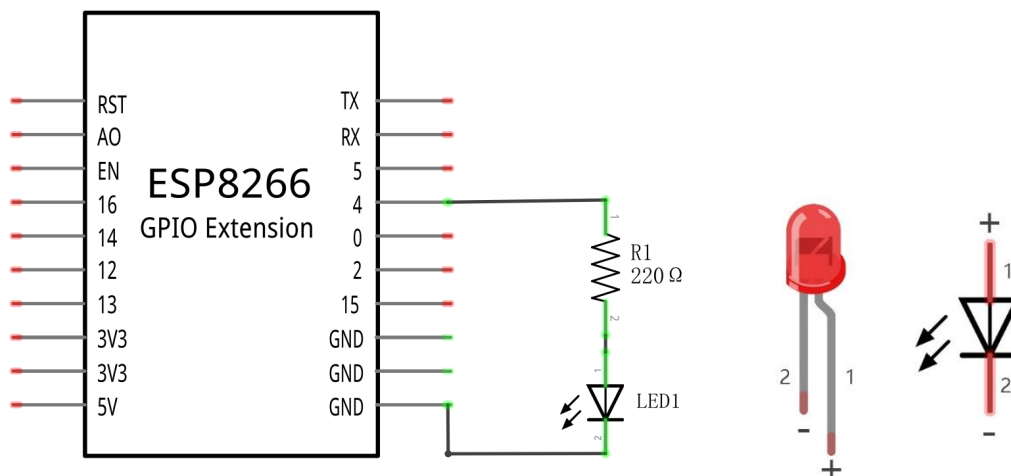
In the following projects, we only use USB cable to power ESP8266 by default.

Circuit

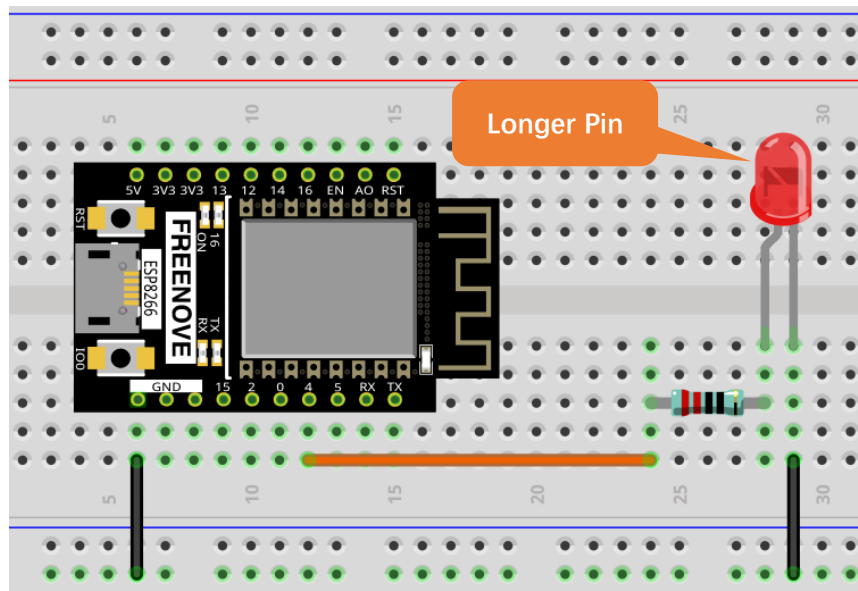
First, disconnect all power from the ESP8266. Then build the circuit according to the circuit and hardware diagrams. After the circuit is built and verified correct, connect the PC to ESP8266.

CAUTION: Avoid any possible short circuits (especially connecting 5V or GND, 3.3V and GND)! WARNING: A short circuit can cause high current in your circuit, generate excessive component heat and cause permanent damage to your hardware!

Schematic diagram



Hardware connection. **If you need any support, please contact us via: support@freenove.com**



Sketch

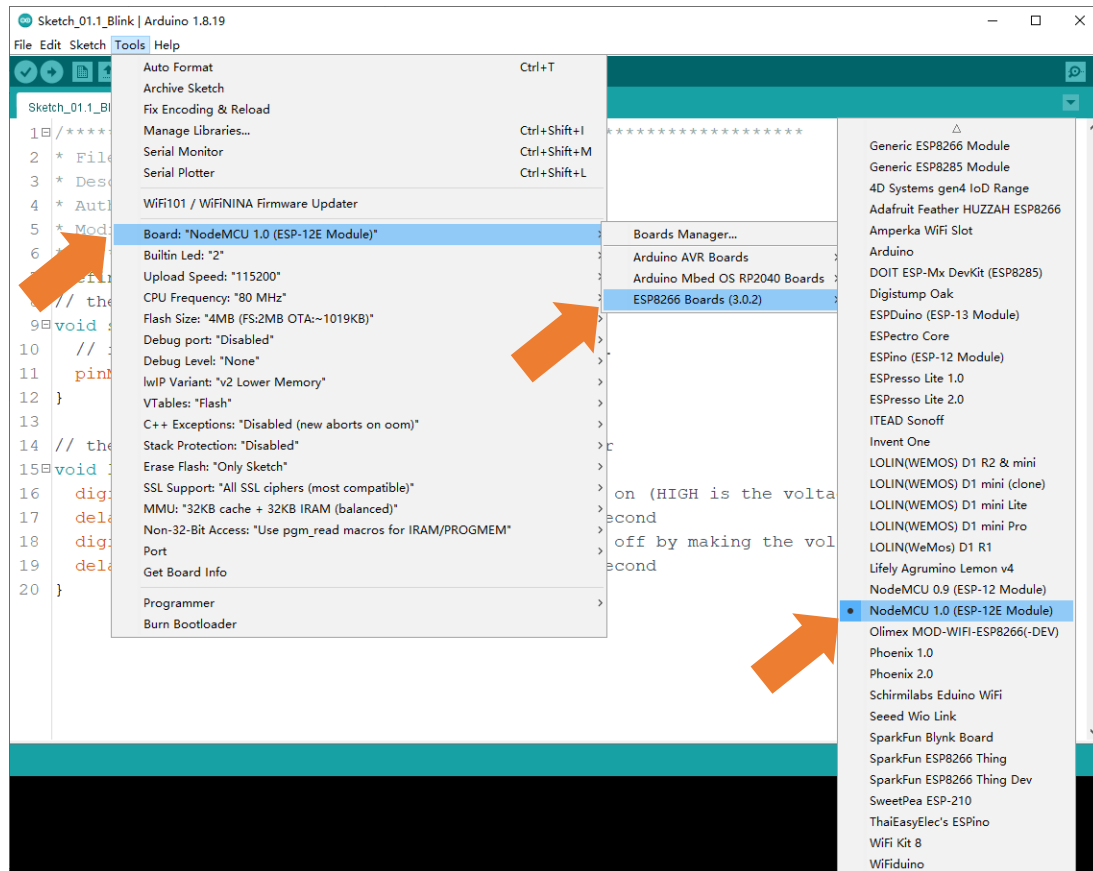
According to the circuit, when the GPIO4 of ESP8266 output level is high, the LED turns ON. Conversely, when the GPIO4 ESP8266 output level is low, the LED turns OFF. Therefore, we can let GPIO4 circularly output high and low level to make the LED blink.

Any concerns? ✉ support@freenove.com

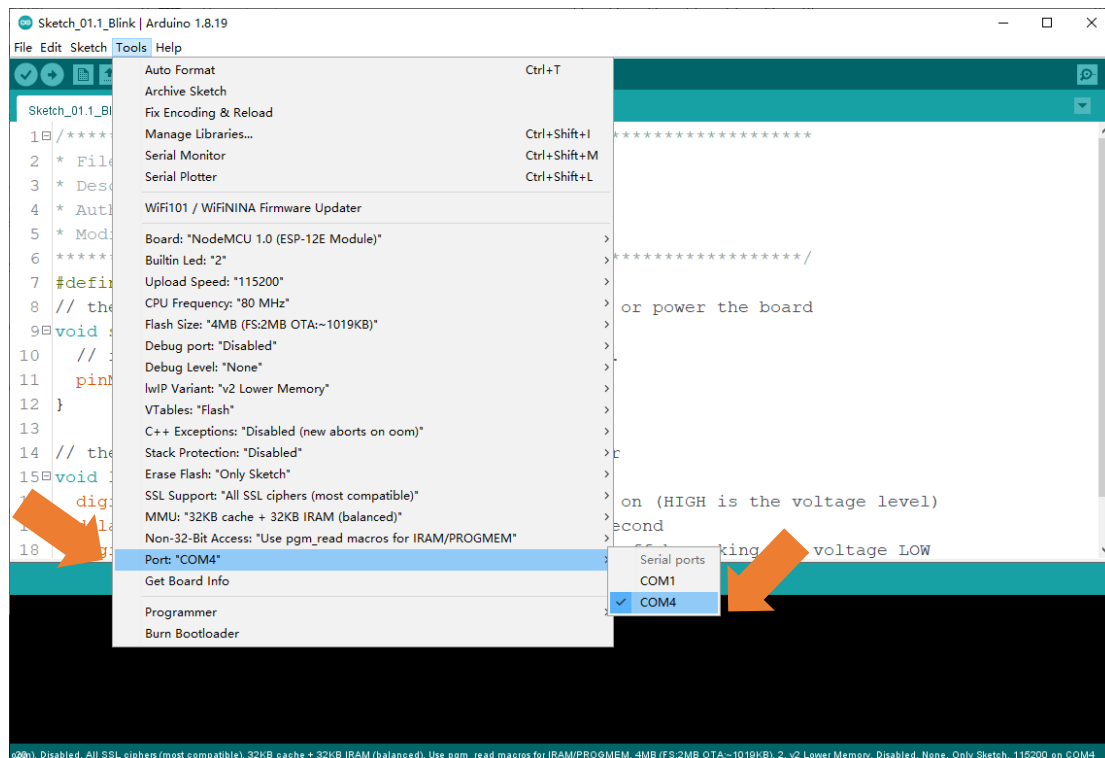
Upload the following Sketch:

Freenove_Basic_Starter_Kit_for_ESP8266\C\Sketches\Sketch_01.1_Blink2

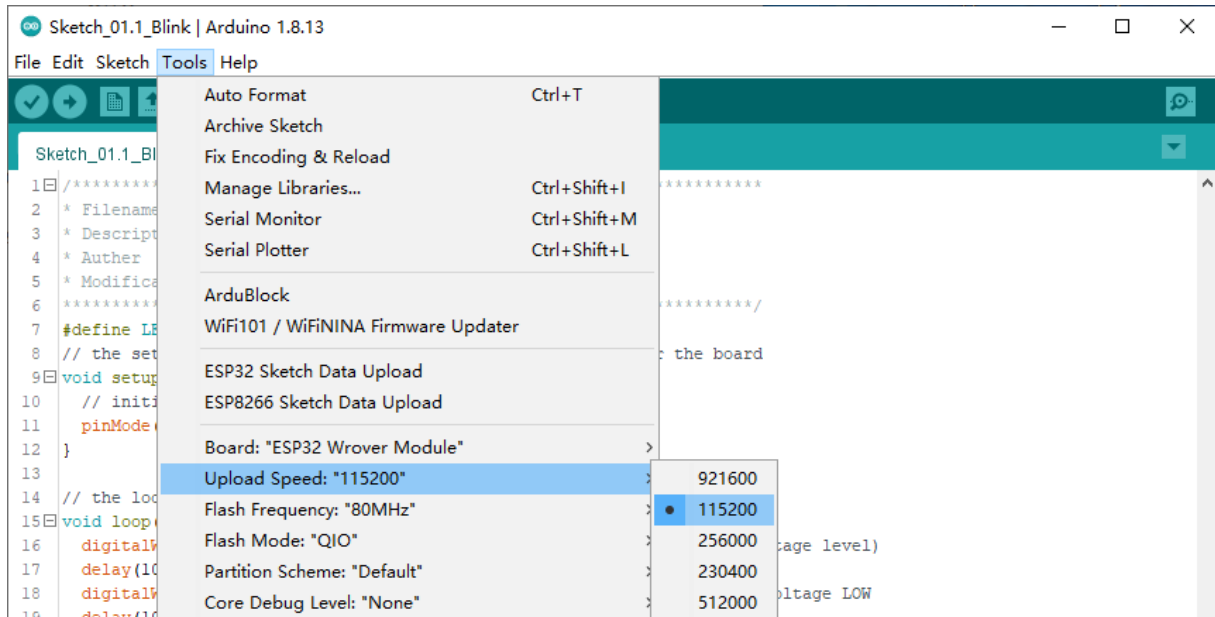
Before uploading the code, click "Tools", "Board" and select "NodeMCU 1.0 (ESP-12E Module)".



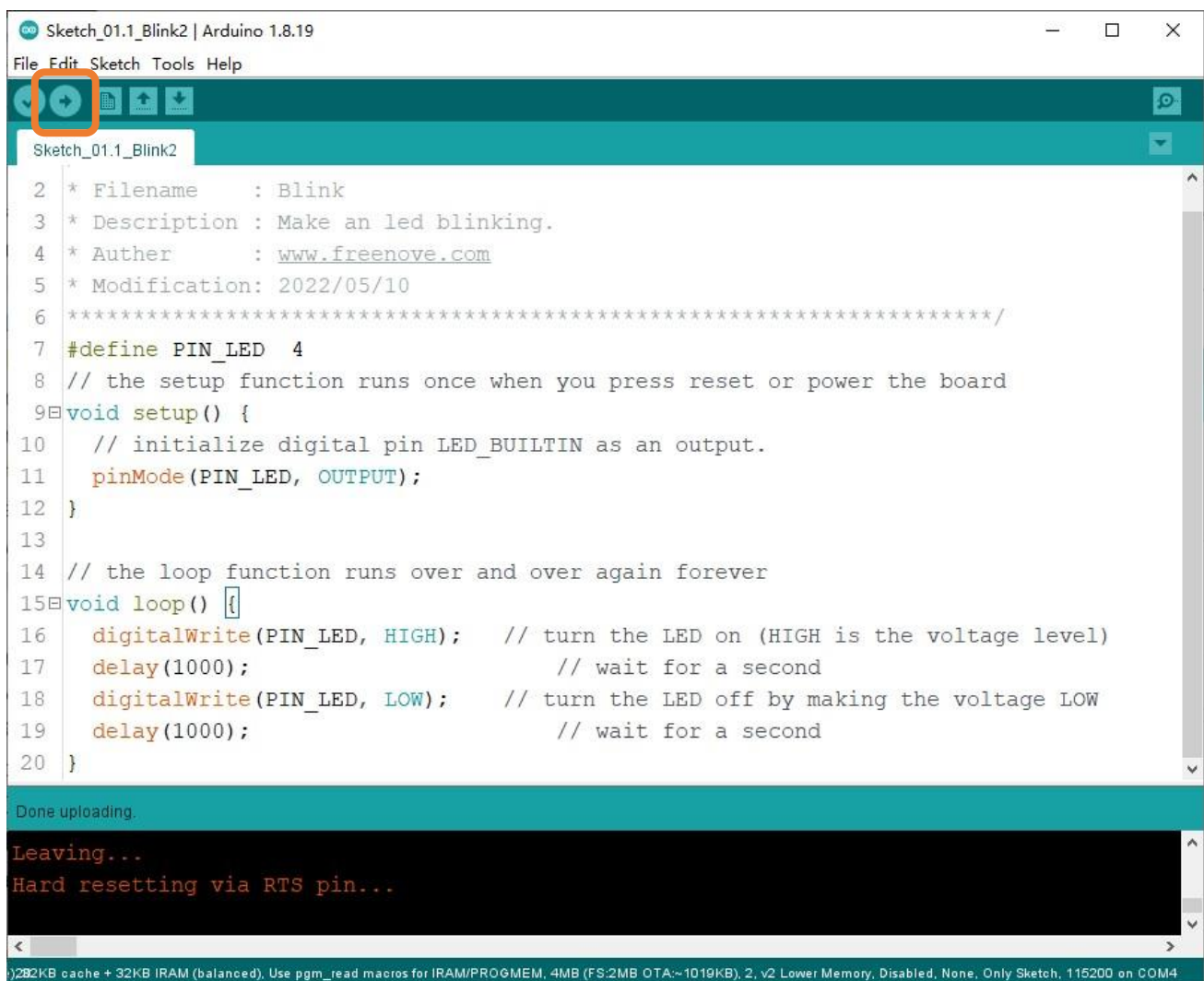
Select the serial port.



Note: For macOS users, if the uploading fails, please set the baud rate to 115200 before clicking “Upload Using Programmer”.

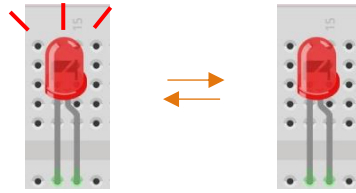


Sketch_01.1_Blink2



Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

Click "Upload", Download the code to ESP8266 and your LED in the circuit starts Blink.



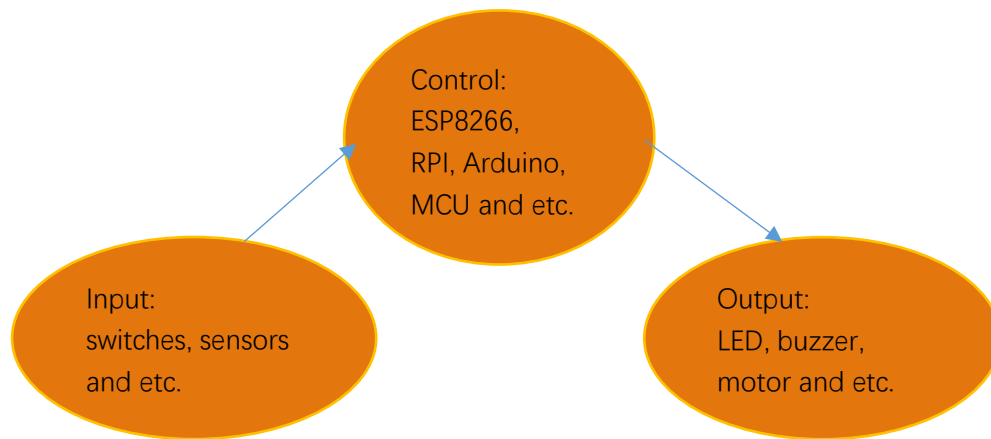
If you have any concerns, please contact us via: support@freenove.com

The following is the program code:

```
1  #define PIN_LED 4
2  // the setup function runs once when you press reset or power the board
3  void setup() {
4      // initialize digital pin LED_BUILTIN as an output.
5      pinMode(PIN_LED, OUTPUT);
6  }
7
8  // the loop function runs over and over again forever
9  void loop() {
10     digitalWrite(PIN_LED, HIGH); // turn the LED on (HIGH is the voltage level)
11     delay(1000);                 // wait for a second
12     digitalWrite(PIN_LED, LOW);  // turn the LED off by making the voltage LOW
13     delay(1000);                 // wait for a second
14 }
```

Chapter 2 Button & LED

Usually, there are three essential parts in a complete automatic control device: INPUT, OUTPUT, and CONTROL. In last section, the LED module was the output part and ESP8266 was the control part. In practical applications, we not only make LEDs flash, but also make a device sense the surrounding environment, receive instructions and then take the appropriate action such as LEDs light up, make a buzzer turn ON and so on.

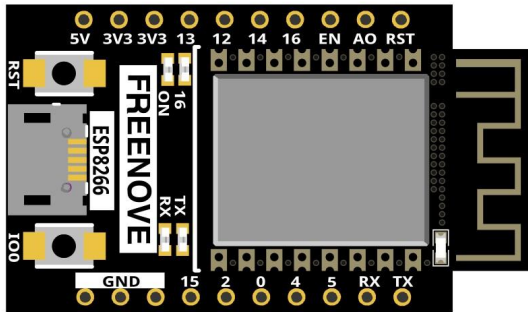
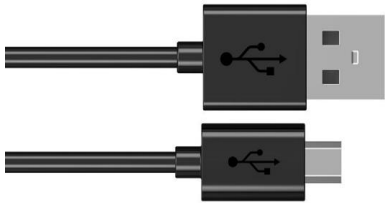
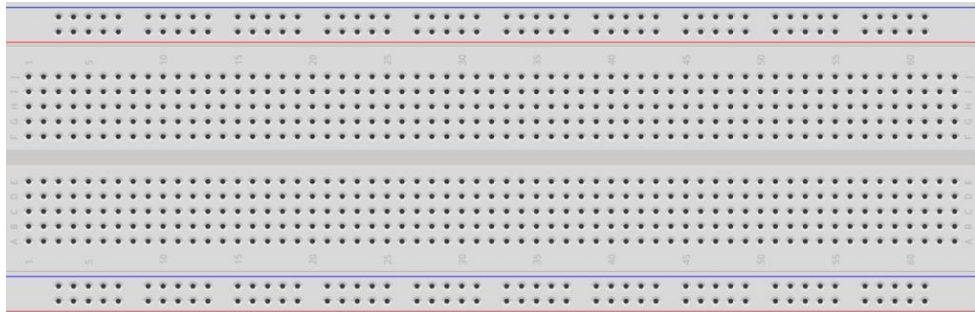


Next, we will build a simple control system to control a LED through a push button switch.

Project 2.1 Button & LED

In the project, we will control the LED state through a Push Button Switch. When the button is pressed, our LED will turn ON, and when it is released, the LED will turn OFF.

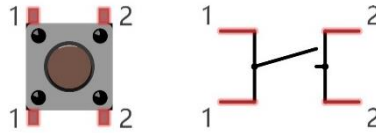
Component List

ESP8266 x1 	USB cable 			
Breadboard x1 				
Jumper wire M/M x6 	LED x1 	Resistor 220Ω x1 	Resistor 10kΩ x2 	Push button x1 

Component knowledge

Push button

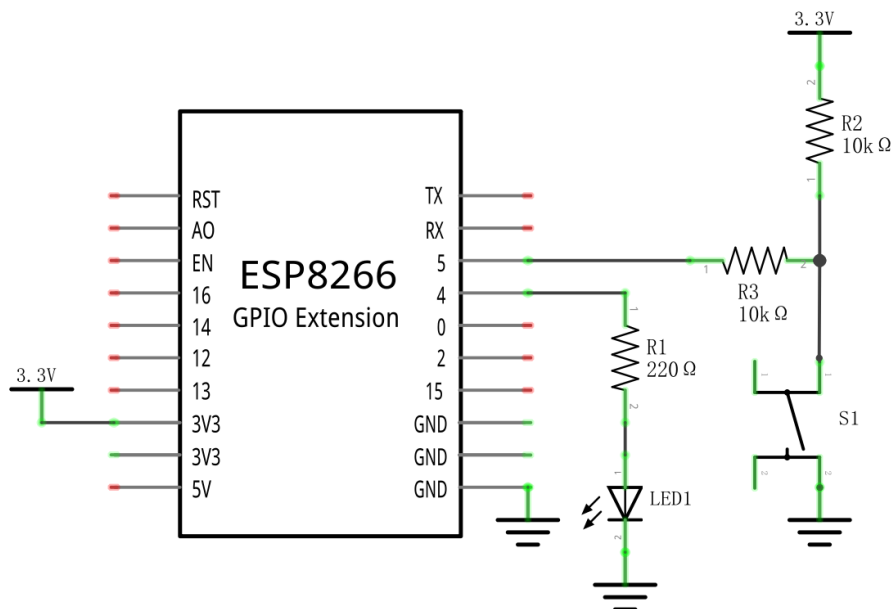
This type of push button switch has 4 pins (2 Pole Switch). Two pins on the left are connected, and both left and right sides are the same per the illustration:



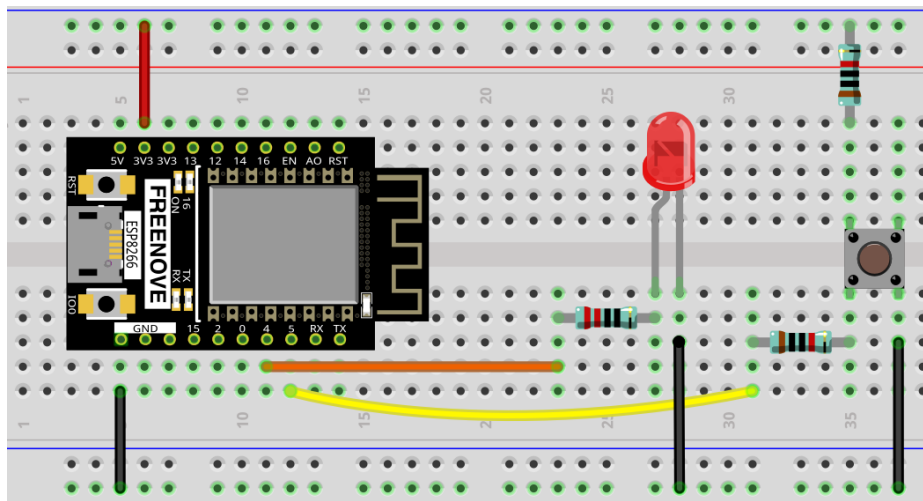
When the button on the switch is pressed, the circuit is completed (your project is powered ON).

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

Sketch

This project is designed for learning how to use push button switch to control a LED. We first need to read the state of switch, and then determine whether to turn the LED ON in accordance to the state of the switch. Upload following sketch:

Freenove_Basic_Starter_Kit_for_ESP8266\C\Sketches\Sketch_02.1_ButtonAndLed

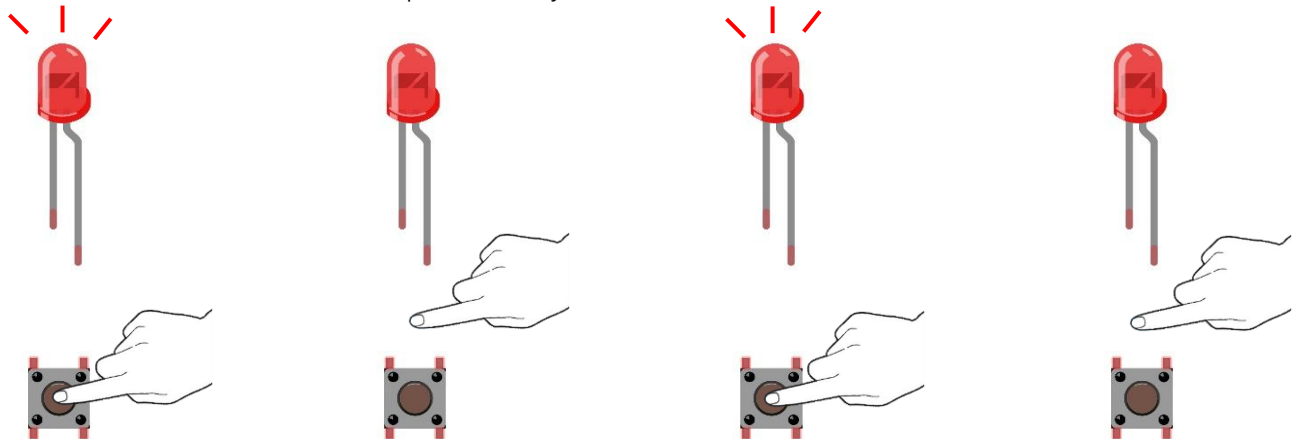
Sketch_02.1_ButtonAndLed

```

1 //*****
2  Filename      : ButtonAndLed
3  Description   : Control led by button.
4  Auther       : www.freenove.com
5  Modification  : 2022/05/10
6  *****/
7  #define PIN_LED      4
8  #define PIN_BUTTON 5
9  // the setup function runs once when you press reset or power the board
10 void setup() {
11    // initialize digital pin PIN_LED as an output.
12    pinMode(PIN_LED, OUTPUT);
13    pinMode(PIN_BUTTON, INPUT);
14 }
15
16 // the loop function runs over and over again forever
17 void loop() {
18    if (digitalRead(PIN_BUTTON) == LOW) {
19        digitalWrite(PIN_LED, HIGH);
20    }else{
21        digitalWrite(PIN_LED, LOW);
22    }
23 }
  
```

Done uploading.
Leaving...
Hard resetting via RTS pin...

Download the code to ESP8266, then press the key, the LED turns ON, release the switch, the LED turns OFF.



If you have any concerns, please contact us via: support@freenove.com

Any concerns? ✉ support@freenove.com

The following is the program code:

```
1  #define PIN_LED    4
2  #define PIN_BUTTON 5
3  // the setup function runs once when you press reset or power the board
4  void setup() {
5      // initialize digital pin PIN_LED as an output.
6      pinMode(PIN_LED, OUTPUT);
7      pinMode(PIN_BUTTON, INPUT);
8  }
9
10 // the loop function runs over and over again forever
11 void loop() {
12     if (digitalRead(PIN_BUTTON) == LOW) {
13         digitalWrite(PIN_LED, HIGH);
14     }else{
15         digitalWrite(PIN_LED, LOW);
16     }
17 }
```

In the circuit connection, LED and button are connected with GPIO4 and GPIO5 respectively, so define PIN_LED and PIN_BUTTON as 4 and 5 respectively.

```
1  #define PIN_LED    4
2  #define PIN_BUTTON 5
```

In the while cycle of main function, use digitalRead(PIN_BUTTON) to determine the state of button. When the button is pressed, the function returns low level, the result of "if" is true, and then turn on LED. Otherwise, turn off LED.

```
11 void loop() {
12     if (digitalRead(PIN_BUTTON) == LOW) {
13         digitalWrite(PIN_LED, HIGH);
14     }else{
15         digitalWrite(PIN_LED, LOW);
16     }
17 }
```

Reference

int digitalRead (int pin);

This function returns the value read at the given pin. It will be "HIGH" or "LOW"(1 or 0) depending on the logic level at the pin.

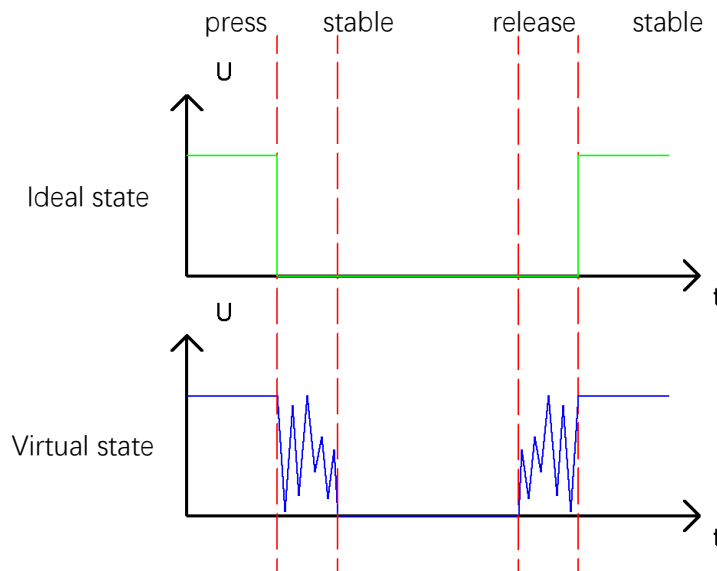
Project 2.2 MINI table lamp

We will also use a push button switch, LED and ESP8266 to make a MINI table lamp but this will function differently: Press the button, the LED will turn ON, and pressing the button again, the LED turns OFF. The ON switch action is no longer momentary (like a door bell) but remains ON without needing to continually press on the Button Switch.

First, let us learn something about the push button switch.

Debounce for Push Button

The moment when a push button switch is pressed, it will not change from one state to another state immediately. Due to tiny mechanical vibrations, there will be a short period of continuous buffeting before it completely reaches another state too fast for humans to detect but not for computer microcontrollers. The same is true when the push button switch is released. This unwanted phenomenon is known as “bounce”.



Therefore, if we can directly detect the state of the push button switch, there are multiple pressing and releasing actions in one pressing cycle. This buffeting will mislead the high-speed operation of the microcontroller to cause many false decisions. Therefore, we need to eliminate the impact of buffeting. Our solution: to judge the state of the button multiple times. Only when the button state is stable (consistent) over a period of time, can it indicate that the button is actually in the ON state (being pressed).

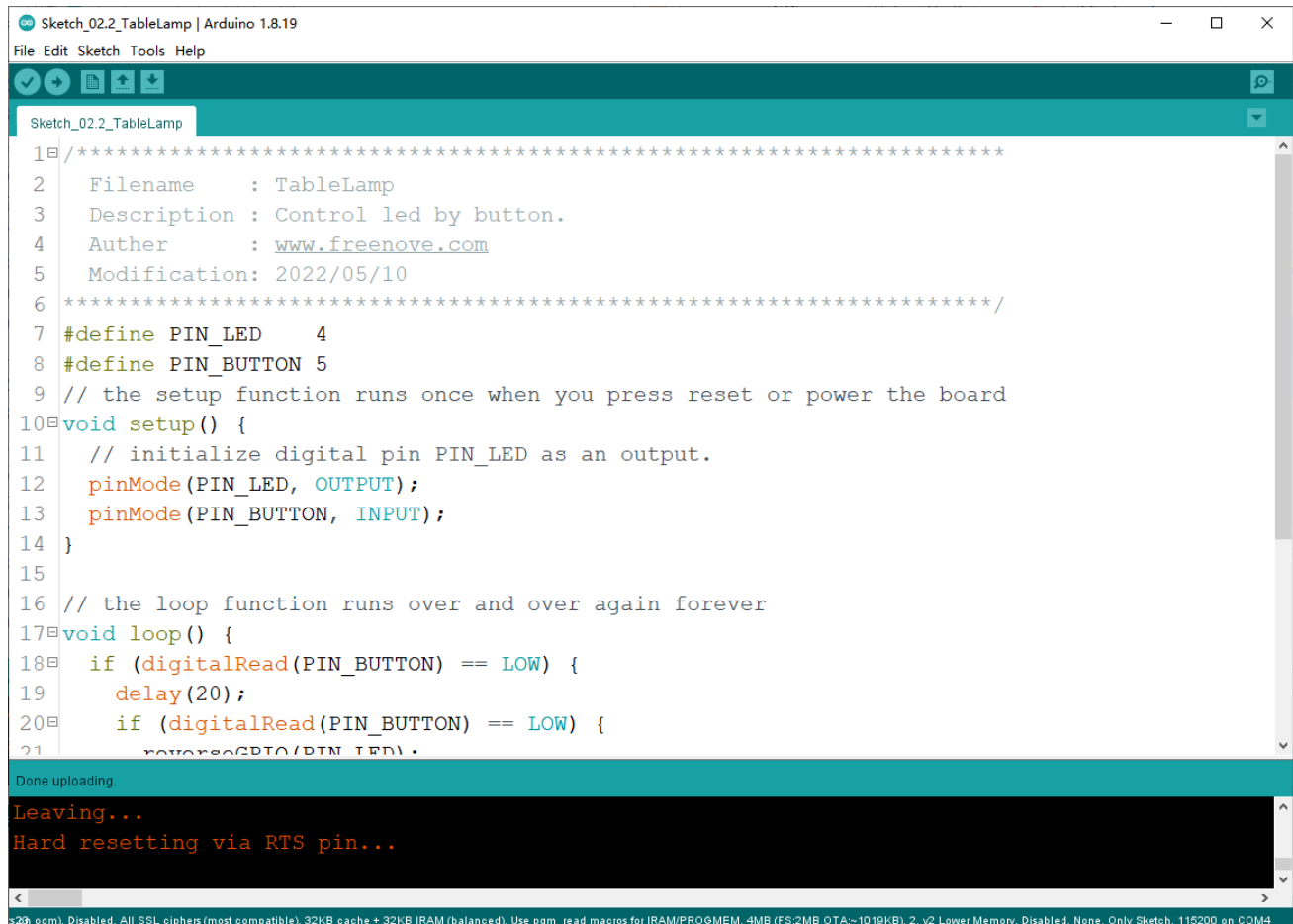
This project needs the same components and circuits as we used in the previous section.

Sketch

Upload following sketch:

Freenove_Basic_Starter_Kit_for_ESP8266\C\Sketches\Sketch_02.2_Tablelamp

Sketch_02.2_Tablelamp



```
Sketch_02.2_TableLamp | Arduino 1.8.19
File Edit Sketch Tools Help

Sketch_02.2_TableLamp

1 /*****
2  Filename      : TableLamp
3  Description   : Control led by button.
4  Auther       : www.freenove.com
5  Modification: 2022/05/10
6  *****/
7 #define PIN_LED      4
8 #define PIN_BUTTON 5
9 // the setup function runs once when you press reset or power the board
10 void setup() {
11     // initialize digital pin PIN_LED as an output.
12     pinMode(PIN_LED, OUTPUT);
13     pinMode(PIN_BUTTON, INPUT);
14 }
15
16 // the loop function runs over and over again forever
17 void loop() {
18     if (digitalRead(PIN_BUTTON) == LOW) {
19         delay(20);
20         if (digitalRead(PIN_BUTTON) == LOW) {
21             digitalWrite(PIN_LED, !digitalRead(PIN_LED));
22         }
23     }
24 }
```

Done uploading.
Leaving...
Hard resetting via RTS pin...

529 com), Disabled, All SSL ciphers (most compatible), 32KB cache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA~1019KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4

Download the code to the ESP8266, press the button, the LED turns ON, and press the button again, the LED turns OFF.

If you have any concerns, please contact us via: support@freenove.com

Any concerns? ✉ support@freenove.com

The following is the program code:

```

1  #define PIN_LED    4
2  #define PIN_BUTTON 5
3  // the setup function runs once when you press reset or power the board
4  void setup() {
5      // initialize digital pin PIN_LED as an output.
6      pinMode(PIN_LED, OUTPUT);
7      pinMode(PIN_BUTTON, INPUT);
8  }
9
10 // the loop function runs over and over again forever
11 void loop() {
12     if (digitalRead(PIN_BUTTON) == LOW) {
13         delay(20);
14         if (digitalRead(PIN_BUTTON) == LOW) {
15             reverseGPIO(PIN_LED);
16         }
17         while (digitalRead(PIN_BUTTON) == LOW);
18         delay(20);
19         while (digitalRead(PIN_BUTTON) == LOW);
20     }
21 }
22
23 void reverseGPIO(int pin) {
24     digitalWrite(pin, ! digitalRead(pin));
25 }

```

When judging the push button state, if it is detected as "pressed down", wait for a certain time to detect again to eliminate the effect of bounce. When confirmed, flip the LED on and off. Then it starts to wait for the pressed button to be released, and waits for a certain time to eliminate the effect of bounce after it is released.

```

12     if (digitalRead(PIN_BUTTON) == LOW) {
13         delay(20);
14         if (digitalRead(PIN_BUTTON) == LOW) {
15             reverseGPIO(PIN_LED);
16         }
17         while (digitalRead(PIN_BUTTON) == LOW);
18         delay(20);
19         while (digitalRead(PIN_BUTTON) == LOW);
20     }

```

The subfunction reverseGPIO() means reading the state value of the specified pin, taking the value back and writing it to the pin again to achieve the function of flipping the output state of the pin.

```

23 void reverseGPIO(int pin) {
24     digitalWrite(pin, ! digitalRead(pin));
25 }

```

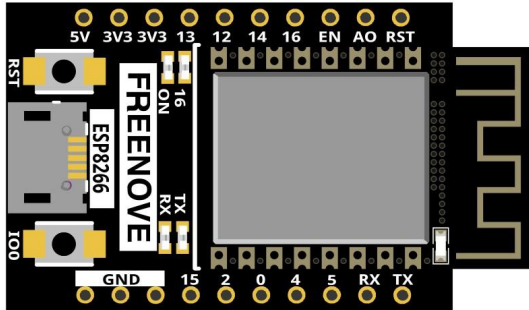
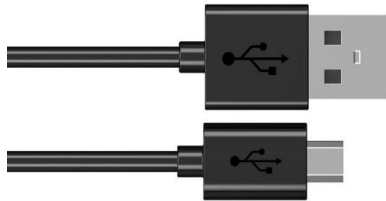
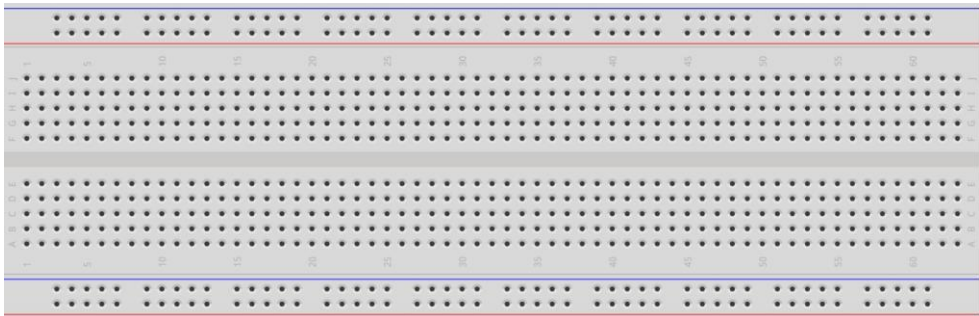
Chapter 3 LED Bar

We have learned how to control a LED blinking, next we will learn how to control a number of LEDs.

Project 3.1 Flowing Light

In this project, we use a number of LEDs to make a flowing light.

Component List

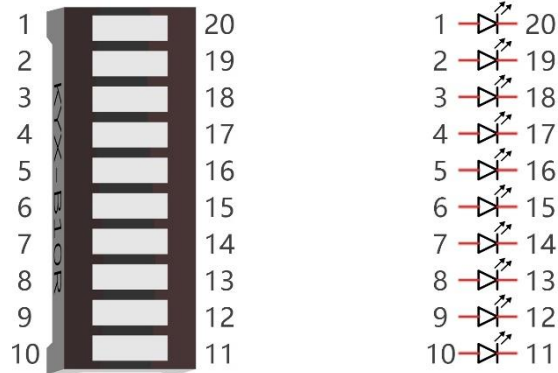
ESP8266 x1 	USB cable 	
Breadboard x1 		
Jumper wire M/M x11 	LED bar graph x1 	Resistor 220Ω x10 

Component knowledge

Let's learn about the basic features of these components to use and understand them better.

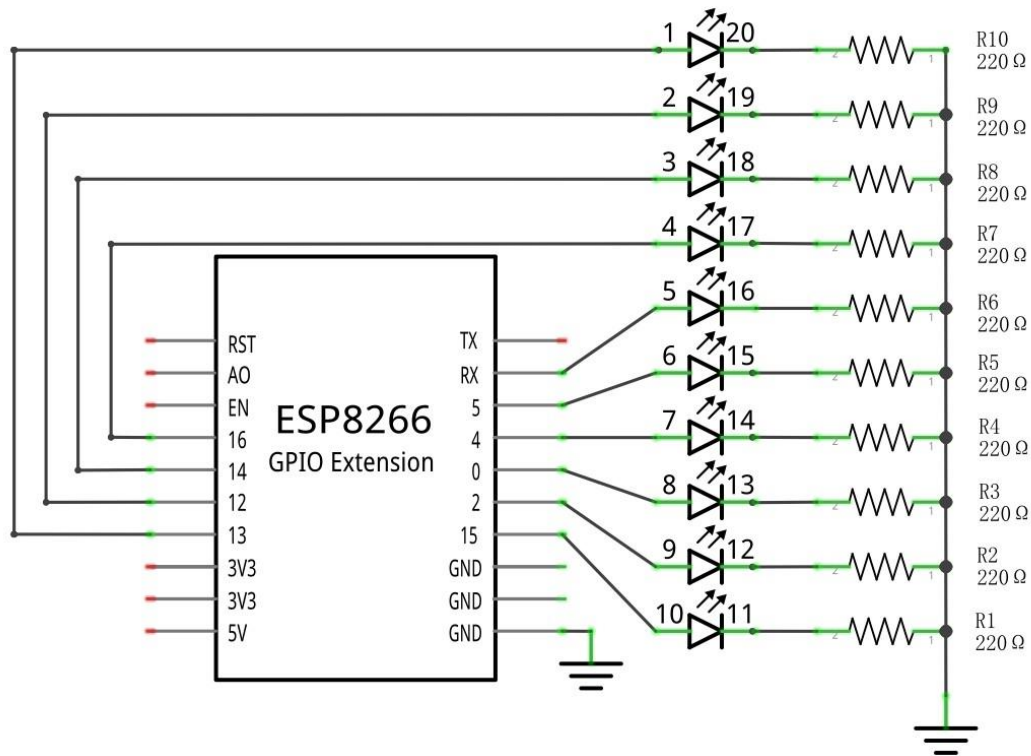
LED bar

A LED bar graph has 10 LEDs integrated into one compact component. The two rows of pins at its bottom are paired to identify each LED like the single LED used earlier.

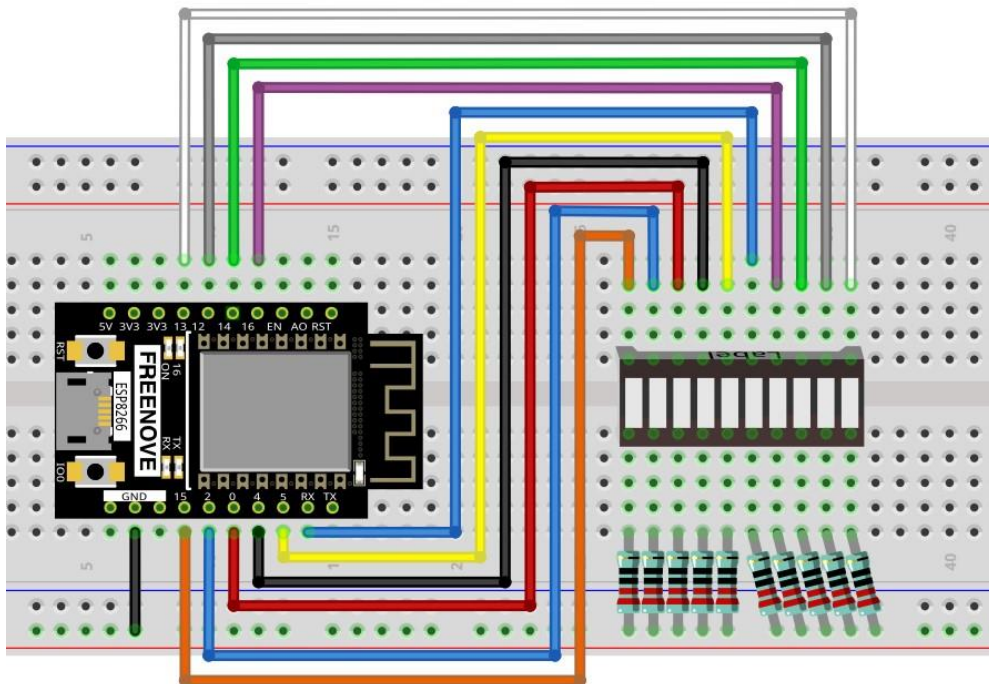


Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



If LED bar does not work, try to rotate it for 180°. The label is random.

Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

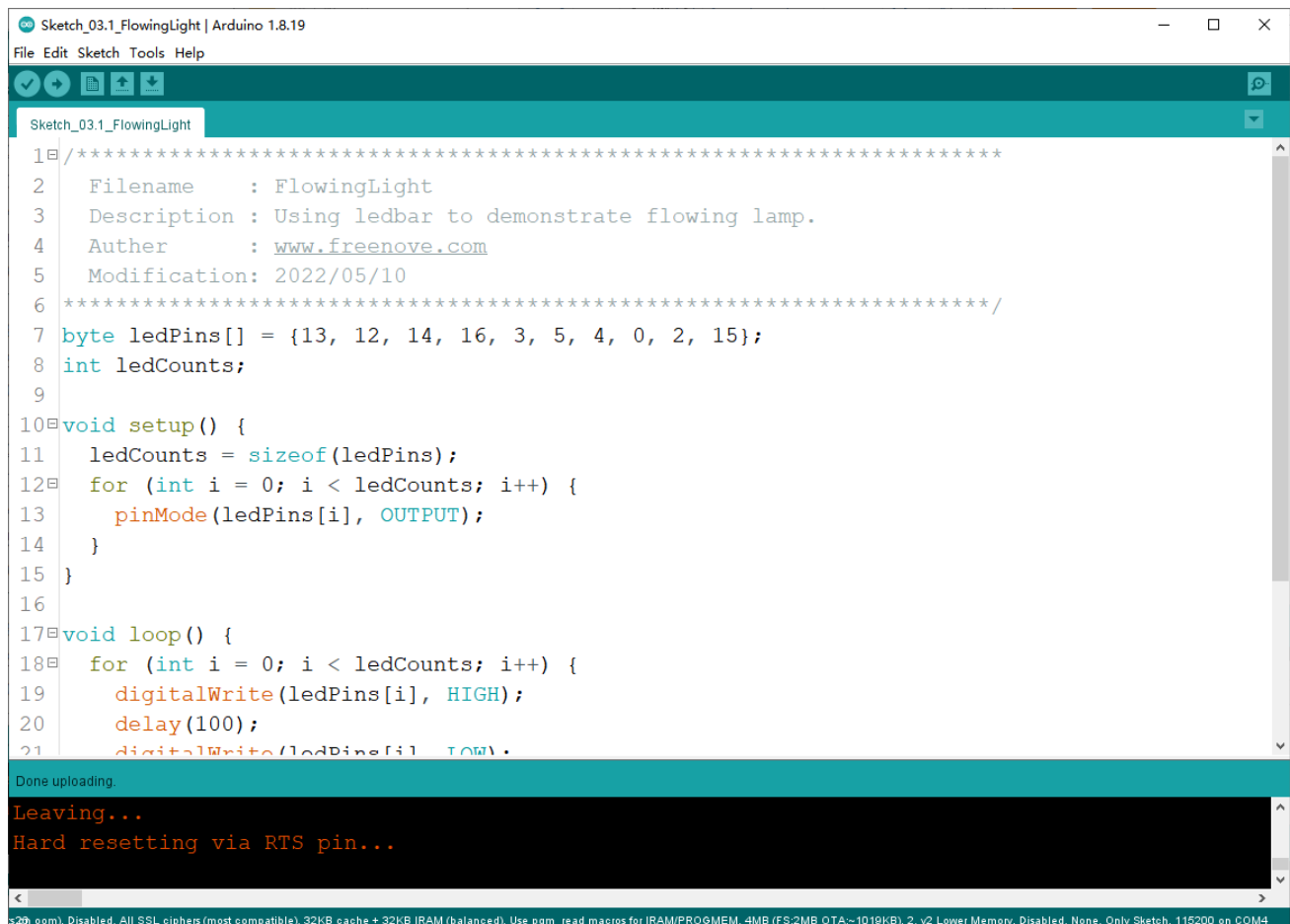
Sketch

This project is designed to make a flowing water lamp. Which are these actions: First turn LED #1 ON, then turn it OFF. Then turn LED #2 ON, and then turn it OFF... and repeat the same to all 10 LEDs until the last LED is turns OFF. This process is repeated to achieve the “movements” of flowing water.

Upload following sketch:

Freenove_Basic_Starter_Kit_for_ESP8266\C\Sketches\Sketch_03.1_FlowingLight

Sketch_03.1_FlowingLight



```

1 //*****
2   Filename      : FlowingLight
3   Description   : Using ledbar to demonstrate flowing lamp.
4   Author        : www.freenove.com
5   Modification  : 2022/05/10
6   *****/
7 byte ledPins[] = {13, 12, 14, 16, 3, 5, 4, 0, 2, 15};
8 int ledCounts;
9
10 void setup() {
11     ledCounts = sizeof(ledPins);
12     for (int i = 0; i < ledCounts; i++) {
13         pinMode(ledPins[i], OUTPUT);
14     }
15 }
16
17 void loop() {
18     for (int i = 0; i < ledCounts; i++) {
19         digitalWrite(ledPins[i], HIGH);
20         delay(100);
21         digitalWrite(ledPins[i], LOW);

```

Done uploading.
Leaving...
Hard resetting via RTS pin...

Download the code to ESP8266 and LED bar graph will light up from left to right and from right to left.



If you have any concerns, please contact us via: support@freenove.com

Any concerns? support@freenove.com

The following is the program code:

```

1  byte ledPins[] = {13, 12, 14, 16, 3, 5, 4, 0, 2, 15};
2  int ledCounts;
3
4  void setup() {
5      ledCounts = sizeof(ledPins);
6      for (int i = 0; i < ledCounts; i++) {
7          pinMode(ledPins[i], OUTPUT);
8      }
9  }
10
11 void loop() {
12     for (int i = 0; i < ledCounts; i++) {
13         digitalWrite(ledPins[i], HIGH);
14         delay(100);
15         digitalWrite(ledPins[i], LOW);
16     }
17     for (int i = ledCounts - 1; i > -1; i--) {
18         digitalWrite(ledPins[i], HIGH);
19         delay(100);
20         digitalWrite(ledPins[i], LOW);
21     }
22 }
```

Use an array to define 10 GPIO ports connected to LED bar graph for easier operation.

```

1  byte ledPins[] = {13, 12, 14, 16, 3, 5, 4, 0, 2, 15};
```

In setup(), use sizeof() to get the number of array, which is the number of LEDs, then configure the GPIO port to output mode.

```

5      ledCounts = sizeof(ledPins);
6      for (int i = 0; i < ledCounts; i++) {
7          pinMode(ledPins[i], OUTPUT);
8      }
```

Then, in loop(), use two “for” loop to realize flowing water light from left to right and from right to left.

```

12     for (int i = 0; i < ledCounts; i++) {
13         digitalWrite(ledPins[i], HIGH);
14         delay(100);
15         digitalWrite(ledPins[i], LOW);
16     }
17     for (int i = ledCounts - 1; i > -1; i--) {
18         digitalWrite(ledPins[i], HIGH);
19         delay(100);
20         digitalWrite(ledPins[i], LOW);
21     }
```

Chapter 4 Analog & PWM

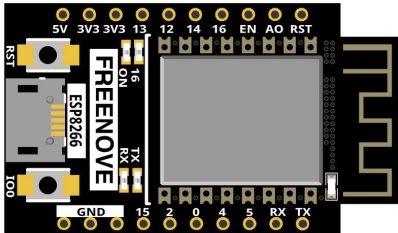
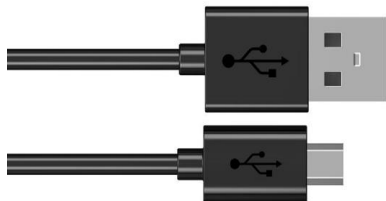
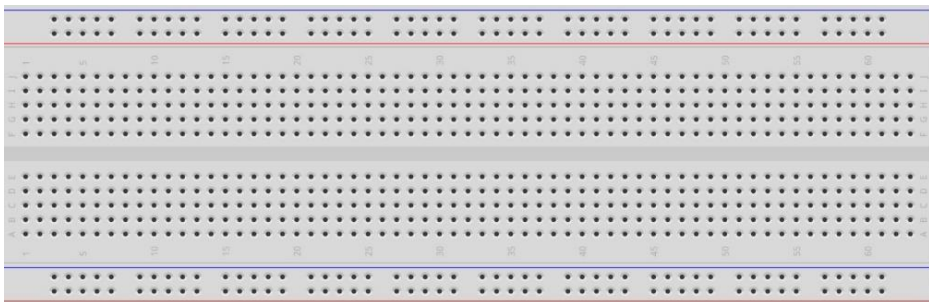
In previous study, we have known that one button has two states: pressed and released, and LED has light-on/off state, then how to enter a middle state? How to output an intermediate state to let LED "semi bright"? That's what we're going to learn.

First, let's learn how to control the brightness of a LED.

Project 4.1 Breathing LED

Breathing light, that is, LED is turned from off to on gradually, and gradually from on to off, just like "breathing". So, how to control the brightness of a LED? We will use PWM to achieve this target.

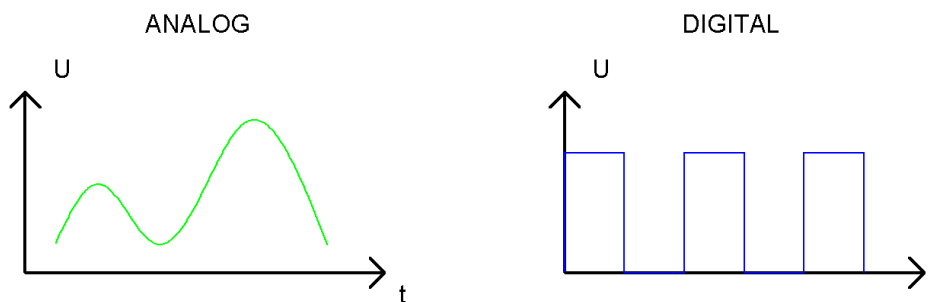
Component List

ESP8266 x1		USB cable	
			
Breadboard x1			
			
LED x1	Resistor 220Ω x1	Jumper wire M/M x3	
			

Related knowledge

Analog & Digital

An analog signal is a continuous signal in both time and value. On the contrary, a digital signal or discrete-time signal is a time series consisting of a sequence of quantities. Most signals in life are analog signals. A familiar example of an analog signal would be how the temperature throughout the day is continuously changing and could not suddenly change instantaneously from 0°C to 10°C. However, digital signals can instantaneously change in value. This change is expressed in numbers as 1 and 0 (the basis of binary code). Their differences can more easily be seen when compared when graphed as below.



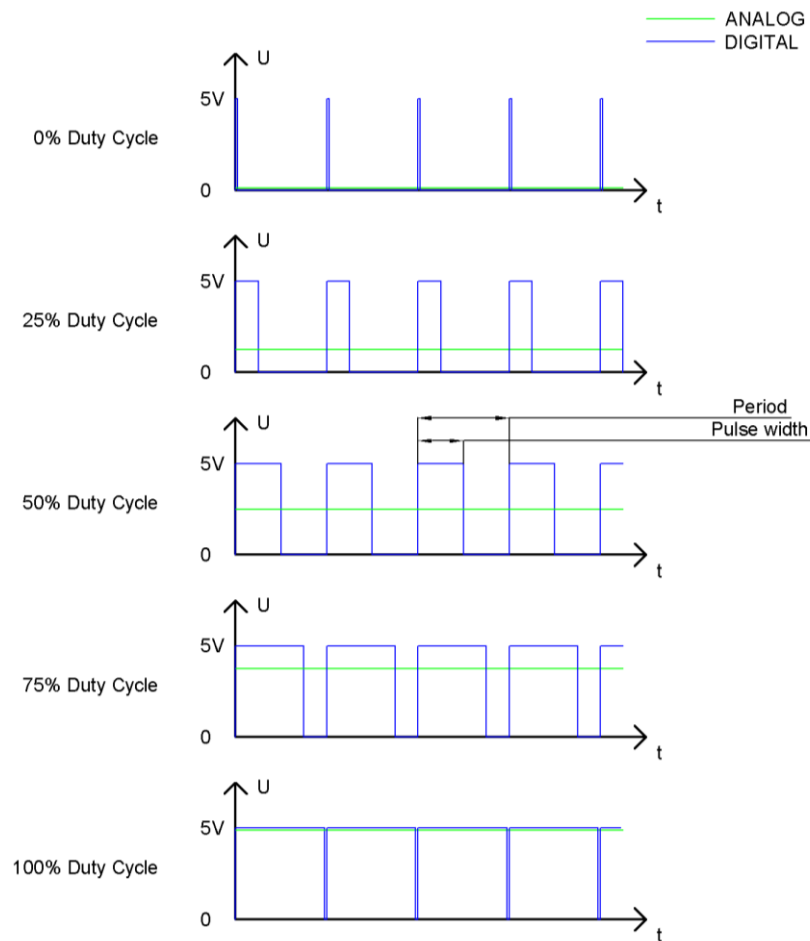
In practical application, we often use binary as the digital signal, that is a series of 0's and 1's. Since a binary signal only has two values (0 or 1), it has great stability and reliability. Lastly, both analog and digital signals can be converted into the other.

PWM

PWM, Pulse-Width Modulation, is a very effective method for using digital signals to control analog circuits. Common processors cannot directly output analog signals. PWM technology makes it very convenient to achieve this conversion (translation of digital to analog signals)

PWM technology uses digital pins to send certain frequencies of square waves, that is, the output of high levels and low levels, which alternately last for a while. The total time for each set of high levels and low levels is generally fixed, which is called the period (Note: the reciprocal of the period is frequency). The time of high level outputs are generally called "pulse width", and the duty cycle is the percentage of the ratio of pulse duration, or pulse width (PW) to the total period (T) of the waveform.

The longer the outputs of high levels last, the longer the duty cycle and the higher the corresponding voltage in the analog signal will be. The following figures show how the analog signal voltages vary between 0V-5V (high level is 5V) corresponding to the pulse width 0%-100%:



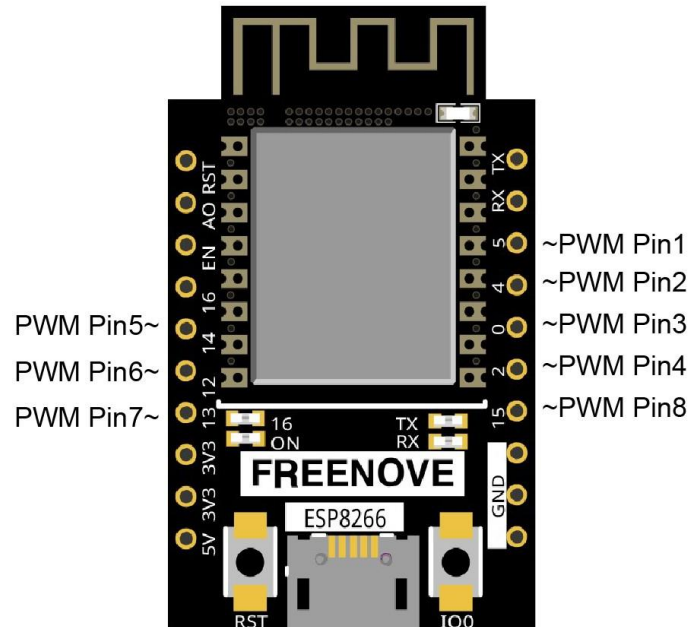
The longer the PWM duty cycle is, the higher the output power will be. Now that we understand this relationship, we can use PWM to control the brightness of a LED or the speed of DC motor and so on.

It is evident from the above that PWM is not real analog, and the effective value of the voltage is equivalent to the corresponding analog. Therefore, we can control the output power of the LED and other output modules to achieve different effects.

ESP8266 and PWM

The ESP8266 PWM controller has 8 independent channels, each of which can independently control frequency, duty cycle, and even accuracy. Unlike traditional PWM pins, the PWM output pins of ESP8266 are configurable and they can be configured to PWM.

The ESP8266 supports PWM pins as follows:

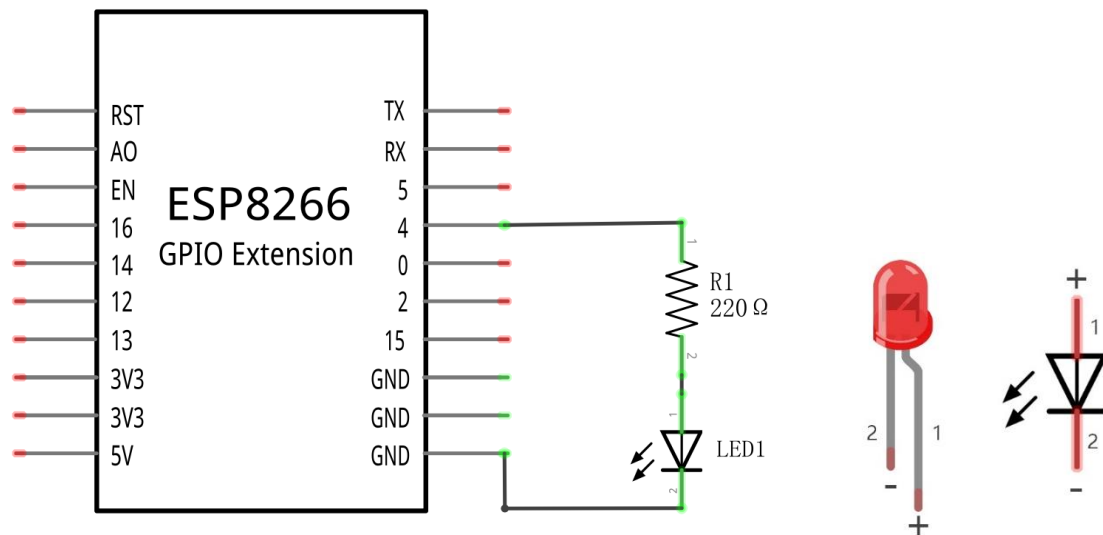


The longer the PWM duty cycle is, the higher the output power will be. Now that we understand this relationship, we can use PWM to control the brightness of an LED or the speed of DC motor and so on.

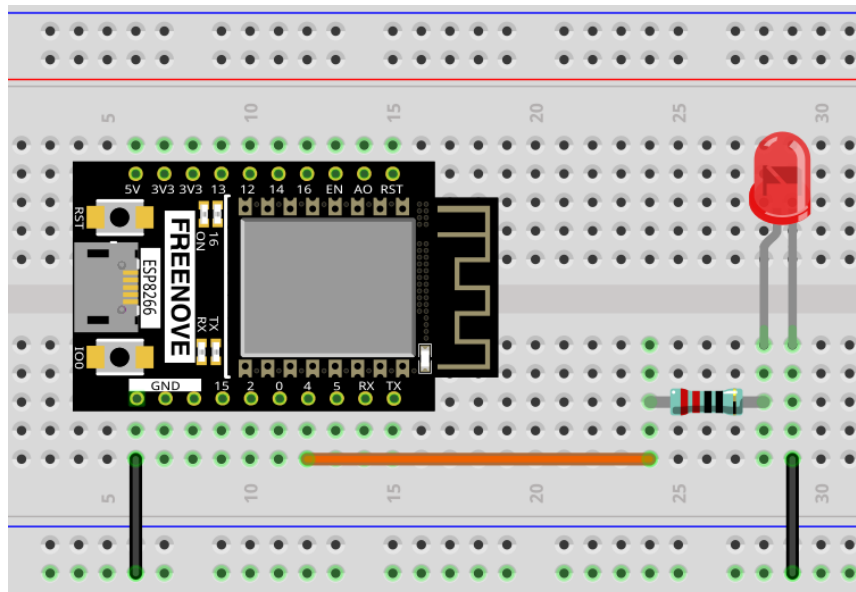
Circuit

This circuit is the same as the one in engineering Blink.

Schematic diagram



Hardware connection. **If you need any support, please contact us via: support@freenove.com**



Sketch

This project is designed to make PWM output GPIO4 with pulse width increasing from 0% to 100%, and then reducing from 100% to 0% gradually.

Upload following sketch:

Freenove_Basic_Starter_Kit_for_ESP8266\C\Sketches\Sketch_04.1_BreathingLight

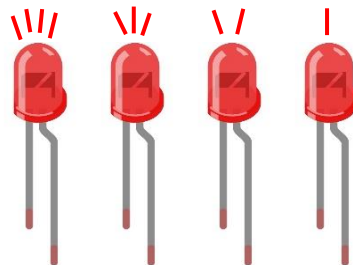
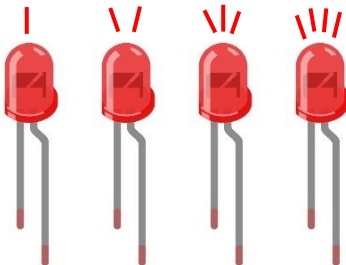
Sketch_04.1_BreathingLight

```
Sketch_04.1_BreathingLight | Arduino 1.8.19
File Edit Sketch Tools Help

Sketch_04.1_BreathingLight

1 //*****
2   Filename      : BreathingLight
3   Description   : Make led light fade in and out, just like breathing.
4   Author       : www.freenove.com
5   Modification  : 2022/05/10
6   *****/
7 int PIN_LED = 4;  // the number of the LED pin
8 void setup() {
9     // initialize the LED pin as an output:
10    pinMode(PIN_LED, OUTPUT);
11 }
12 void loop() {
13     // call breath() cyclically
14     breath(PIN_LED, 6);
15     delay(500);
16     map(0, 0, 255, 0, 1023);
17 }
18 void breath(int PIN_LED, int delayMs) {
19     for (int i = 0; i <= 255; i++) { // "i" change from 0 to 255
20         analogWrite(PIN_LED, i);    // coresponding duty cycle change from 0%-100%
21         delay(delayMs);             // adjust the rate of change of brightness
22     }
23     for (int i = 255; i >= 0; i--) { // "i" change from 255 to 0
24         analogWrite(PIN_LED, i);    // coresponding duty cycle change from 100%-0%
25         delay(delayMs);             // adjust the rate of change of brightness
26     }
27 }
28
29
Done uploading.
Leaving...
Hard resetting via RTS pin...
s29 oom), Disabled, All SSL ciphers (most compatible), 32KB cache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA~1019KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4
```

Download the code to ESP8266, and you'll see that LED is turned from on to off and then from off to on gradually like breathing.



The following is the program code:

```

1  int PIN_LED = 4;    // the number of the LED pin
2  void setup() {
3      // initialize the LED pin as an output:
4      pinMode(PIN_LED, OUTPUT);
5  }
6  void loop() {
7      // call breath() cyclically
8      breath(PIN_LED, 6);
9      delay(500);
10     map(0, 0, 255, 0, 1023);
11 }
12 void breath(int PIN_LED, int delayMs) {
13     for (int i = 0; i <= 255; i++) {    // "i" change from 0 to 255
14         analogWrite(PIN_LED, i);        // coresponding duty cycle change from 0%-100%
15         delay(delayMs);                 // adjust the rate of change of brightness
16     }
17     for (int i = 255; i >= 0; i--) {    // "i" change from 255 to 0
18         analogWrite(PIN_LED, i);        // coresponding duty cycle change from 0%-100%
19         delay(delayMs);                 // adjust the rate of change in brightness
20     }
21 }

```

In the loop(), There are two “for” loops. The first makes the ledPin output PWM from 0% to 100% and the second makes the ledPin output PWM from 100% to 0%. This allows the LED to gradually light and extinguish.

```

13     for (int i = 0; i <= 255; i++) {    // "i" change from 0 to 255
14         analogWrite(PIN_LED, i);        // coresponding duty cycle change from 0%-100%
15         delay(delayMs);                 // adjust the rate of change of brightness
16     }
17     for (int i = 255; i >= 0; i--) {    // "i" change from 255 to 0
18         analogWrite(PIN_LED, i);        // coresponding duty cycle change from 0%-100%
19         delay(delayMs);                 // adjust the rate of change in brightness
20     }

```

You can also adjust the rate of the state change of LED by changing the parameters of the delay() function in the “for” loop.

Reference

analogWrite(pin, value);

Arduino IDE provides the function, analogWrite(pin, value), which can make ports directly output PWM waves. In the function called analogWrite(pin, value), the parameter "pin" specifies the port used to output PWM wave. In ESP8266, The range of value is 0-1023, which represents the duty cycle of 0%-100%. In order to use this function, we need to set the port to output mode.

map(value, fromLow, fromHigh, toLow, toHigh)

This function is used to remap a value, which will return a new value whose percentage in the range of toLow-toHigh is equal to the percentage of "value" in the range of fromLow-fromHigh. For example, 1 is the maximum in the range of 0-1 and the maximum value in the scope of 0-2 is 2, that is, the result value of map (1, 0, 1, 0, 2) is 2.

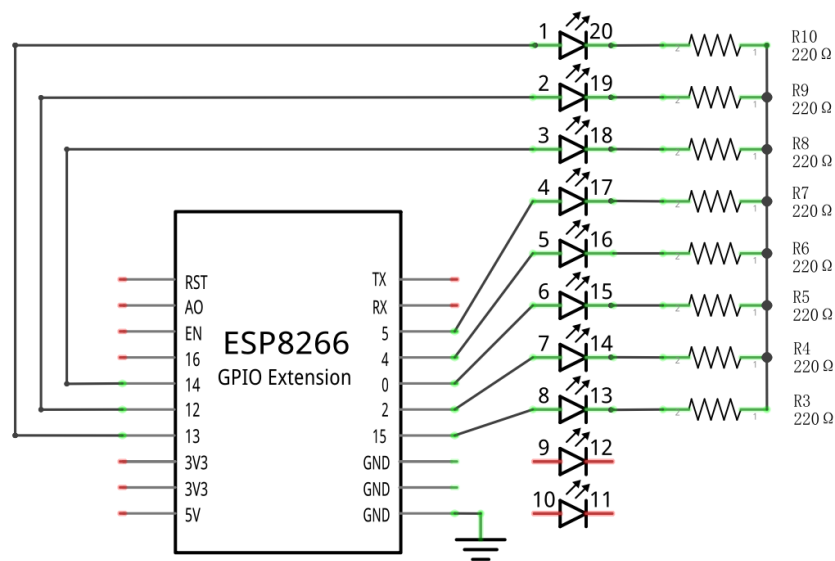
For more related functions, please refer to <https://www.arduino.cc/reference/en/>

Project 4.2 Meteor Flowing Light

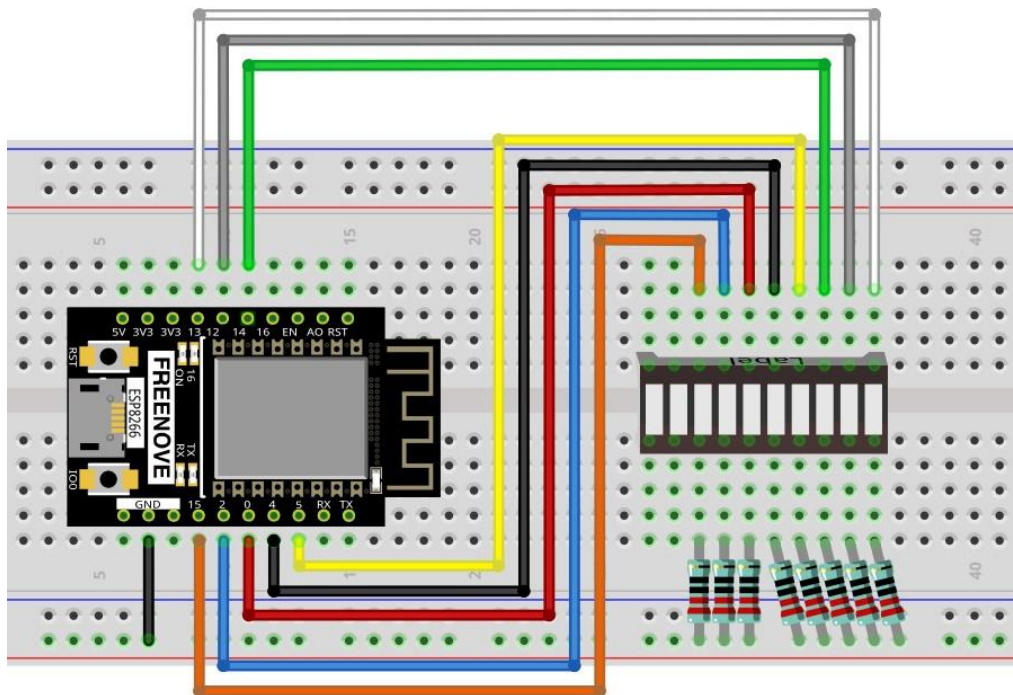
After learning about PWM, we can use it to control LED bar graph and realize a cooler flowing light. The component list, circuit, and hardware are exactly consistent with the project Flowing Light.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



If LED bar does not work, try to rotate it for 180°. The label is random.

Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

Sketch

Meteor flowing light will be implemented with PWM.

Upload following sketch:

Freenove_Basic_Starter_Kit_for_ESP8266\C\Sketches\Sketch_04.2_FlowingLight2

[Sketch_04.2_FlowingLight2](#)

Download the code to ESP8266, and LED bar graph will gradually light up and out from left to right, then light up and out from right to left.

```

Sketch_04.2_FlowingLight2 | Arduino 1.8.19
File Edit Sketch Tools Help

Sketch_04.2_FlowingLight2
1  /*****
2  Filename      : FlowingLight2
3  Description   : More cool flowing light.
4  Auther       : www.freenove.com
5  Modification: 2022/05/10
6  *****/
7  const byte ledPins[] = {13, 12, 14, 5, 4, 0, 2, 15};    //define led pins
8  const int dutys[] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
9  |                 1023, 512, 256, 128, 64, 32, 16, 8, 4, 2,
10 |                 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
11 |                 };    //define the pwm dutys
12  int ledCounts;    //led counts
13  int delayTimes = 50;    //flowing speed ,the smaller, the faster
14  void setup() {
15  ledCounts = sizeof(ledPins);    //get the led counts
16  }
17
18  void loop() {
19  for (int i = 0; i < 20; i++) {    //flowing one side to other side
20  for (int j = 0; j < ledCounts; j++) {
21  analogWrite(ledPins[j], dutys[i + j]);
22  }
}

```

Done uploading.

Leaving...

Hard resetting via RTS pin...

Most compatible, 32KB cache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:3MB OTA~512KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4

The following is the program code:

```

1  const byte ledPins[] = {13, 12, 14, 5, 4, 0, 2, 15};    //define led pins
2  const int dutys[] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3  |                 1023, 512, 256, 128, 64, 32, 16, 8, 4, 2,
4  |                 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
5  |                 };    //define the pwm dutys
6  int ledCounts;    //led counts
7  int delayTimes = 50;    //flowing speed ,the smaller, the faster
8  void setup() {
9  ledCounts = sizeof(ledPins);    //get the led counts
10 }

```

Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

```

11
12 void loop() {
13     for (int i = 0; i < 20; i++) {           //flowing one side to other side
14         for (int j = 0; j < ledCounts; j++) {
15             analogWrite(ledPins[j], dutys[i + j]);
16         }
17         delay(delayTimes);
18     }
19     for (int i = 0; i < 20; i++) {           //flowing one side to other side
20         for (int j = ledCounts - 1; j > -1; j--) {
21             analogWrite(ledPins[j], dutys[i + (ledCounts - 1 - j)]);
22         }
23         delay(delayTimes);
24     }
25 }

```

Defined 10 GPIO and 30 pulse width values.

```

1 const byte ledPins[] = {13, 12, 14, 5, 4, 0, 2, 15}; //define led pins
2 const int dutys[] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
3                     1023, 512, 256, 128, 64, 32, 16, 8, 4, 2,
4                     0, 0, 0, 0, 0, 0, 0, 0, 0, 0
5                     }; //define the pwm dutys

```

In `loop()`, a nested for loop is used to control the pulse width of the PWM, and LED bar graph moves one grid after each 1 is added in the first for loop, gradually changing according to the values in the array `dutys`. As shown in the table below, the value of the second row is the value in the array `dutys`, and the 10 green squares in each row below represent the 10 LEDs on the LED bar graph. Every 1 is added to `i`, the value of the LED bar graph will move to the right by one grid, and when it reaches the end, it will move from the end to the starting point, achieving the desired effect.

0	1	2	3	4	5	7	8	9	1	11	1	1	1	1	1	1	1	1	2	2	2	2	2	2	2	2	2	2	3
									0	0	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0
d	0	0	0	0	0	0	0	0	0	10	5	2	1	6	3	1	8	4	2	0	0	0	0	0	0	0	0	0	0
i										23	1	5	2	4	2	6													
0																													
1																													
2																													
3																													
...																													
1																													
8																													
1																													
9																													
2																													
0																													

In the code, two nested for loops are used to achieve this effect.

```

13   for (int i = 0; i < 20; i++) {           //flowing one side to other side
14       for (int j = 0; j < ledCounts; j++) {
15           ledcWrite(chns[j], dutys[i + j]);
16       }
17       delay(delayTimes);
18   }
19   for (int i = 0; i < 20; i++) {           //flowing from one side to the other
20       for (int j = ledCounts - 1; j > -1; j--) {
21           ledcWrite(chns[j], dutys[i + (ledCounts - 1 - j)]);
22       }
23       delay(delayTimes);
24   }

```

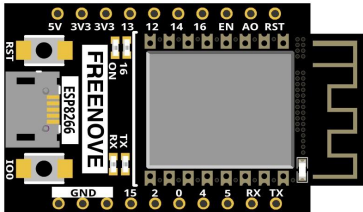
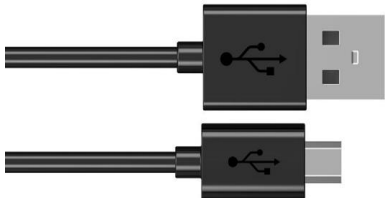
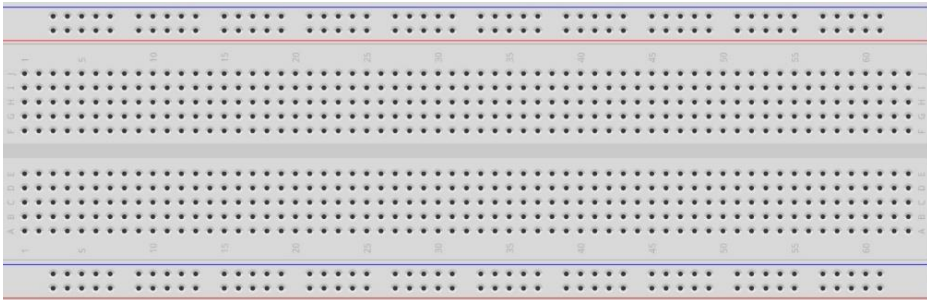
Chapter 5 RGB LED

In this chapter, we will learn how to control a RGB LED. It can emit different colors of light. Next, we will use RGB LED to make a multicolored light.

Project 5.1 Random Color Light

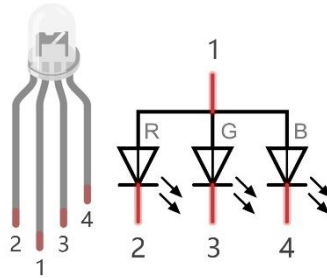
In this project, we will make a multicolored LED. And we can control RGB LED to switch different colors automatically.

Component List

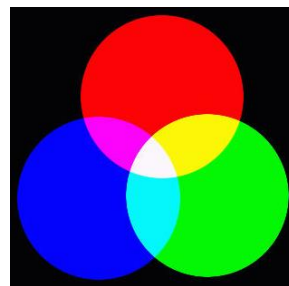
ESP8266 x1	USB cable	
		
Breadboard x1		
		
RGBLED x1	Resistor 220Ω x3	Jumper wire M/M x5
		

Related knowledge

RGB LED has integrated 3 LEDs that can respectively emit red, green and blue light. And it has 4 pins. The long pin (1) is the common port, that is, 3 LED 's positive or negative port. The RGB LED with common positive port and its symbol is shown below. We can make RGB LED emit various colors of light by controlling these 3 LEDs to emit light with different brightness.



Red, green, and blue are known as three primary colors. When you combine these three primary-color lights with different brightness, it can produce almost all kinds of visible lights. Computer screens, single pixel of cell phone screen, neon, and etc. are working under this principle.

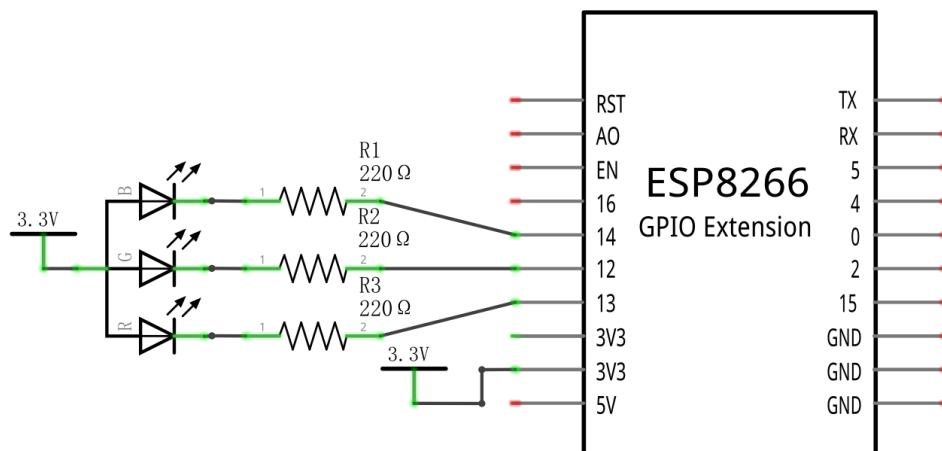


RGB

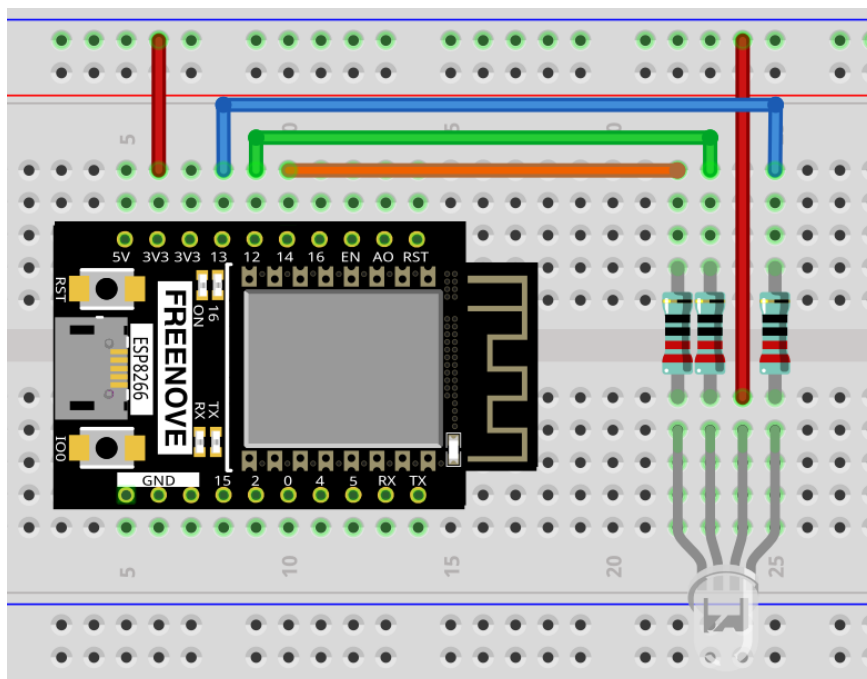
If we use three 8-bit PWMs to control the RGB LED, in theory, we can create $2^8 \times 2^8 \times 2^8 = 16777216$ (16 million) colors through different combinations.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



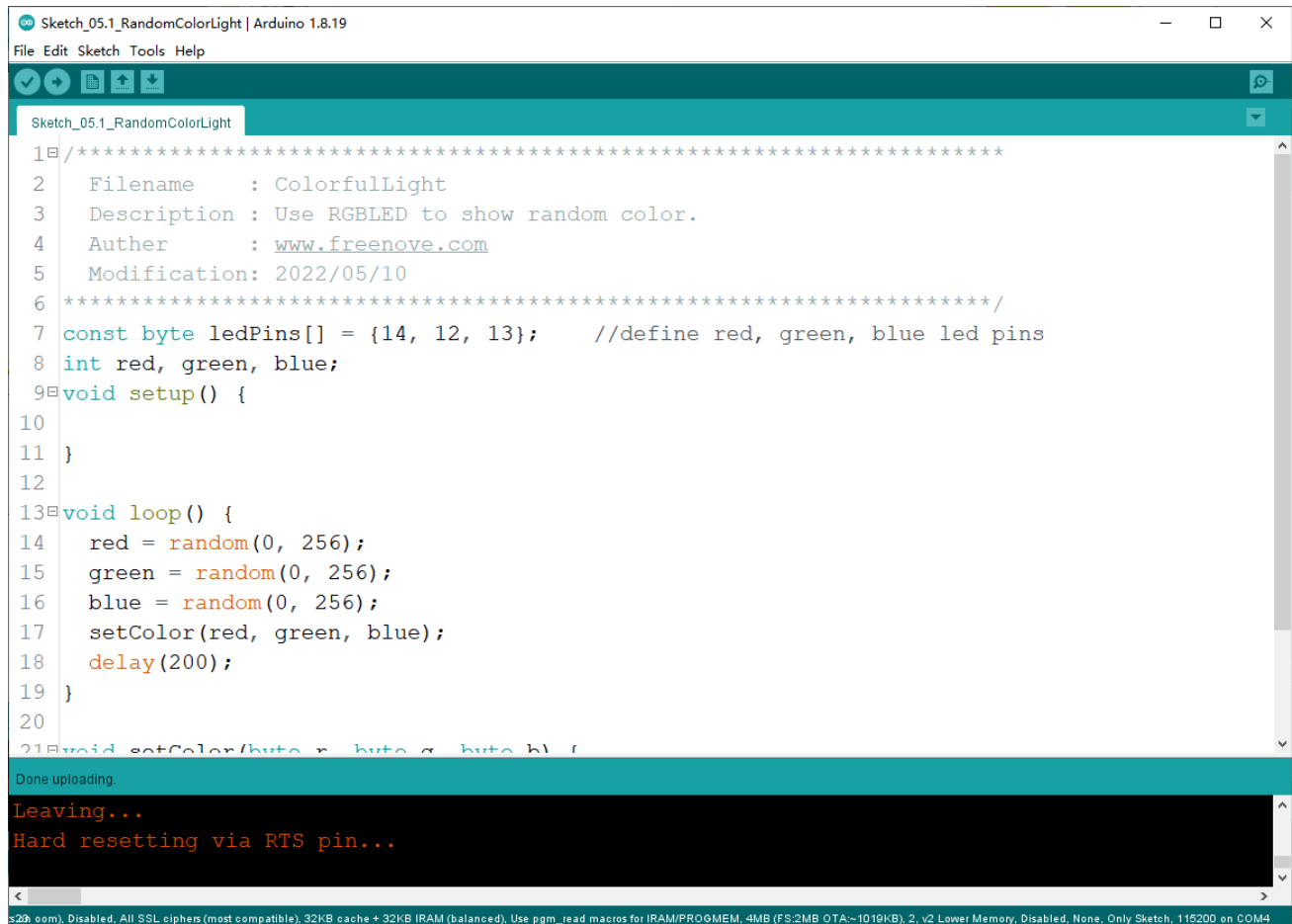
Sketch

We need to create three PWM channels and use random duty cycle to make random RGB LED color.

Upload following sketch:

Freenove_Basic_Starter_Kit_for_ESP8266\C\S sketch_05.1_ColorfulLight

Sketch_05.1_ColorfulLight



```
1 /*****  
2  Filename      : ColorfulLight  
3  Description   : Use RGBLED to show random color.  
4  Author       : www.freenove.com  
5  Modification  : 2022/05/10  
6  *****/  
7  const byte ledPins[] = {14, 12, 13};    //define red, green, blue led pins  
8  int red, green, blue;  
9  void setup() {  
10  
11  }  
12  
13  void loop() {  
14    red = random(0, 256);  
15    green = random(0, 256);  
16    blue = random(0, 256);  
17    setColor(red, green, blue);  
18    delay(200);  
19  }  
20  
21  void setColor(byte r, byte g, byte b) {  
22    digitalWrite(ledPins[0], r);  
23    digitalWrite(ledPins[1], g);  
24    digitalWrite(ledPins[2], b);  
25  }  
26  }  
27  */
```

Done uploading.
Leaving...
Hard resetting via RTS pin...

With the code downloaded to ESP8266, RGB LED begins to display random colors.

If you have any concerns, please contact us via: support@freenove.com

Any concerns? ✉ support@freenove.com

The following is the program code:

```

1  const byte ledPins[] = {14, 12, 13};    //define red, green, blue led pins
2  int red, green, blue;
3  void setup() {
4  }
5
6  void loop() {
7      red = random(0, 256);
8      green = random(0, 256);
9      blue = random(0, 256);
10     setColor(red, green, blue);
11     delay(200);
12 }
13
14 void setColor(byte r, byte g, byte b) {
15     analogWrite(ledPins[0], 255 - r); //Common anode LED, low level to turn on the led.
16     analogWrite(ledPins[1], 255 - g);
17     analogWrite(ledPins[2], 255 - b);
18 }

```

Define RGB LED pins

```

1  const byte ledPins[] = {14, 12, 13};    //define red, green, blue led pins

```

In setColor(), this function controls the output color of RGB LED by the given color value. Because the circuit uses a common anode, the LED lights up when the GPIO outputs low power. Therefore, in PWM, low level is the active level, so 255 minus the given value is necessary.

```

14 void setColor(byte r, byte g, byte b) {
15     analogWrite(chns[0], 255 - r); //Common anode LED, low level to turn on the led.
16     analogWrite(chns[1], 255 - g);
17     analogWrite(chns[2], 255 - b);
18 }

```

In loop(), get three random Numbers and set them as color values.

```

7      red = random(0, 256);
8      green = random(0, 256);
9      blue = random(0, 256);
10     setColor(red, green, blue);
11     delay(200);

```

The related function of software PWM can be described as follows:

long random(min, max);

This function will return a random number(min --- max-1).

Project 5.2 Gradient Color Light

In the previous project, we have mastered the usage of RGB LED, but the random display of colors is rather stiff. This project will realize a fashionable light with soft color changes.

Component list and the circuit are exactly the same as the random color light.

Using a color model, the color changes from 0 to 255 as shown below.



In this code, the color model will be implemented and RGB LED will change colors along the model.

Sketch

Upload following sketch:

Freenove_Basic_Starter_Kit_for_ESP8266\C\Sketch_05.2_SoftColorfulLight

Sketch_05.2_SoftColorfulLight

```

Sketch_05.2_GradientColorLight | Arduino 1.8.19
File Edit Sketch Tools Help
Sketch_05.2_GradientColorLight
1 /*****
2  Filename      : SoftColorfulLight
3  Description   : Colorful light with gradually changing color.
4  Author       : www.freenove.com
5  Modification: 2022/05/10
6  *****/
7  const byte ledPins[] = {14, 12, 13};    //define led pins
8  void setup() {
9  }
10 void loop() {
11   for (int i = 0; i < 256; i++) {
12     setColor(wheel(i));
13     delay(20);
14   }
15 }
16 void setColor(long rgb) {
17   analogWrite(ledPins[0], 255 - (rgb >> 16) & 0xFF);
18   analogWrite(ledPins[1], 255 - (rgb >> 8) & 0xFF);
19   analogWrite(ledPins[2], 255 - (rgb >> 0) & 0xFF);
20 }
21
Done uploading.
Leaving...
Hard resetting via RTS pin...
s2h.com), Disabled, All SSL ciphers (most compatible), 32KB cache+32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA~1019KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4

```

Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

The following is the program code:

```

1  const byte ledPins[] = {14, 12, 13};    //define led pins
2  void setup() {
3  }
4  void loop() {
5      for (int i = 0; i < 256; i++) {
6          setColor(wheel(i));
7          delay(20);
8      }
9  }
10 void setColor(long rgb) {
11     analogWrite(ledPins[0], 255 - (rgb >> 16) & 0xFF);
12     analogWrite(ledPins[1], 255 - (rgb >> 8) & 0xFF);
13     analogWrite(ledPins[2], 255 - (rgb >> 0) & 0xFF);
14 }
15
16 long wheel(int pos) {
17     long WheelPos = pos % 0xff;
18     if (WheelPos < 85) {
19         return ((255 - WheelPos * 3) << 16) | ((WheelPos * 3) << 8);
20     } else if (WheelPos < 170) {
21         WheelPos -= 85;
22         return (((255 - WheelPos * 3) << 8) | (WheelPos * 3));
23     } else {
24         WheelPos -= 170;
25         return ((WheelPos * 3) << 16 | (255 - WheelPos * 3));
26     }
27 }

```

In `setColor()`, a variable represents the value of RGB, and a hexadecimal representation of color is a common representation, such as `0xAABBCC`, where `AA` represents the red value, `BB` represents the green value, and `CC` represents the blue value. The use of a variable can make the transmission of parameters more convenient, in the split, only a simple operation can take out the value of each color channel

```

10 void setColor(long rgb) {
11     analogWrite(ledPins[0], 255 - (rgb >> 16) & 0xFF);
12     analogWrite(ledPins[1], 255 - (rgb >> 8) & 0xFF);
13     analogWrite(ledPins[2], 255 - (rgb >> 0) & 0xFF);
14 }

```

The `wheel()` function is the color selection method for the color model introduced earlier. The **pos** parameter ranges from 0 to 255 and outputs a color value in hexadecimal.

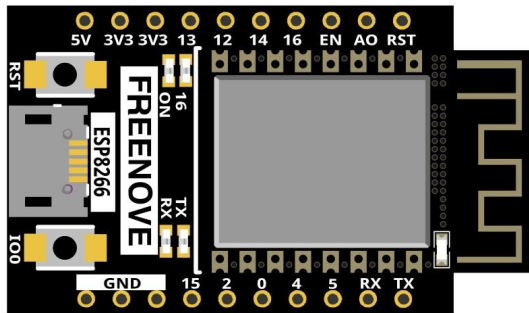
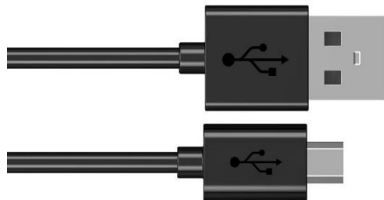
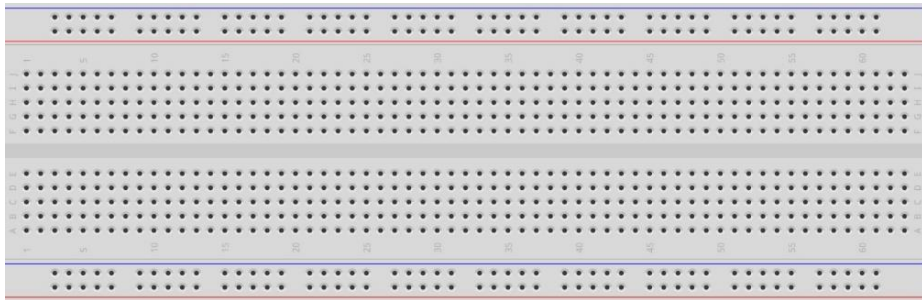
Chapter 6 Buzzer

In this chapter, we will learn about buzzers that can make sounds.

Project 6.1 Doorbell

We will make this kind of doorbell: when the button is pressed, the buzzer sounds; and when the button is released, the buzzer stops sounding.

Component List

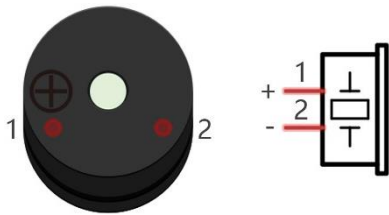
ESP8266 x1		USB cable		
				
Breadboard x1				
				
Jumper wire M/M x9				
				
NPN transistorx1 (S8050)	Active buzzer x1	Push button x1	Resistor 1kΩ x1	Resistor 10kΩ x2
				

Component knowledge

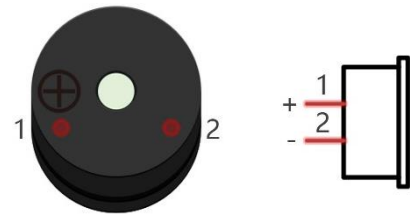
Buzzer

Buzzer is a sounding component, which is widely used in electronic devices such as calculator, electronic warning clock and alarm. Buzzer has two types: active and passive. Active buzzer has oscillator inside, which will sound as long as it is supplied with power. Passive buzzer requires external oscillator signal (generally use PWM with different frequency) to make a sound.

Active buzzer



Passive buzzer



Active buzzer is easy to use. Generally, it can only make a specific frequency of sound. Passive buzzer requires an external circuit to make a sound, but it can be controlled to make a sound with different frequency. The resonant frequency of the passive buzzer is 2kHz, which means the passive buzzer is loudest when its resonant frequency is 2kHz.

Next, we will use an active buzzer to make a doorbell and a passive buzzer to make an alarm.

How to identify active and passive buzzer?

1. Usually, there is a label on the surface of active buzzer covering the vocal hole, but this is not an absolute judgment method.
2. Active buzzers are more complex than passive buzzers in their manufacture. There are many circuits and crystal oscillator elements inside active buzzers; all of this is usually protected with a waterproof coating (and a housing) exposing only its pins from the underside. On the other hand, passive buzzers do not have protective coatings on their underside. From the pin holes viewing of a passive buzzer, you can see the circuit board, coils, and a permanent magnet (all or any combination of these components depending on the model).

Active buzzer



Passive buzzer



Transistor

Because the buzzer requires such large current that GPIO of ESP8266 output capability cannot meet the requirement, a transistor of NPN type is needed here to amplify the current.

Transistor, the full name: semiconductor transistor, is a semiconductor device that controls current. Transistor can be used to amplify weak signal, or works as a switch. It has three electrodes(PINs): base (b), collector (c) and emitter (e). When there is current passing between "be", "ce" will allow several-fold current (transistor magnification) pass, at this point, transistor works in the amplifying area. When current between "be" exceeds a certain value, "ce" will not allow current to increase any longer, at this point, transistor works in the saturation area. Transistor has two types as shown below: PNP and NPN,

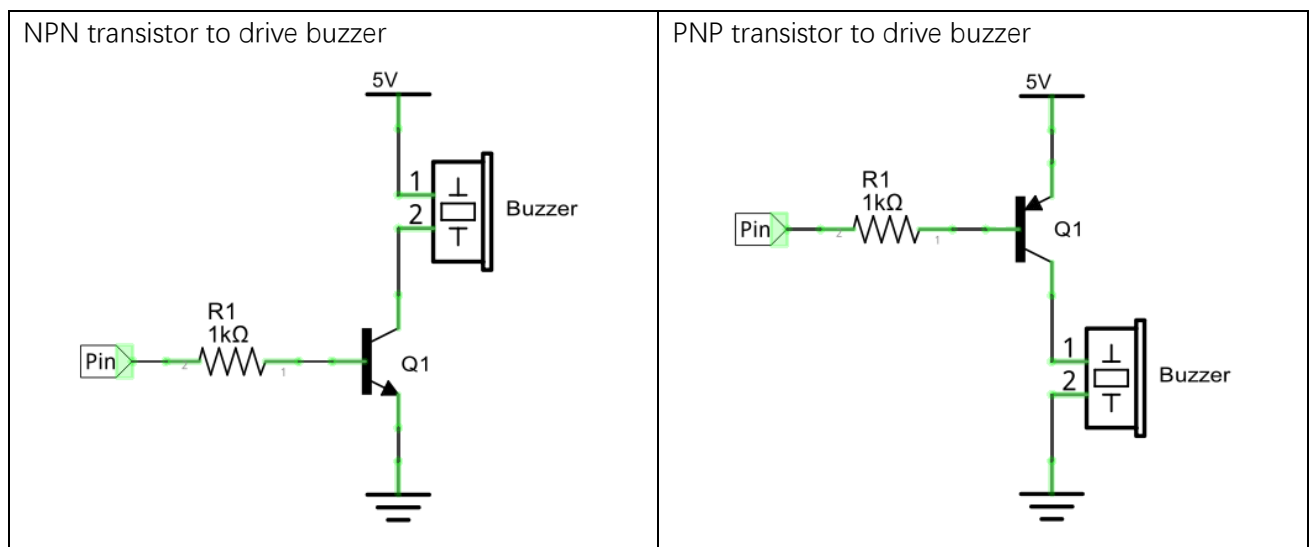


In our kit, the PNP transistor is marked with 8550, and the NPN transistor is marked with 8050.

Based on the transistor's characteristics, it is often used as a switch in digital circuits. As micro-controller's capacity to output current is very weak, we will use transistor to amplify current and drive large-current components.

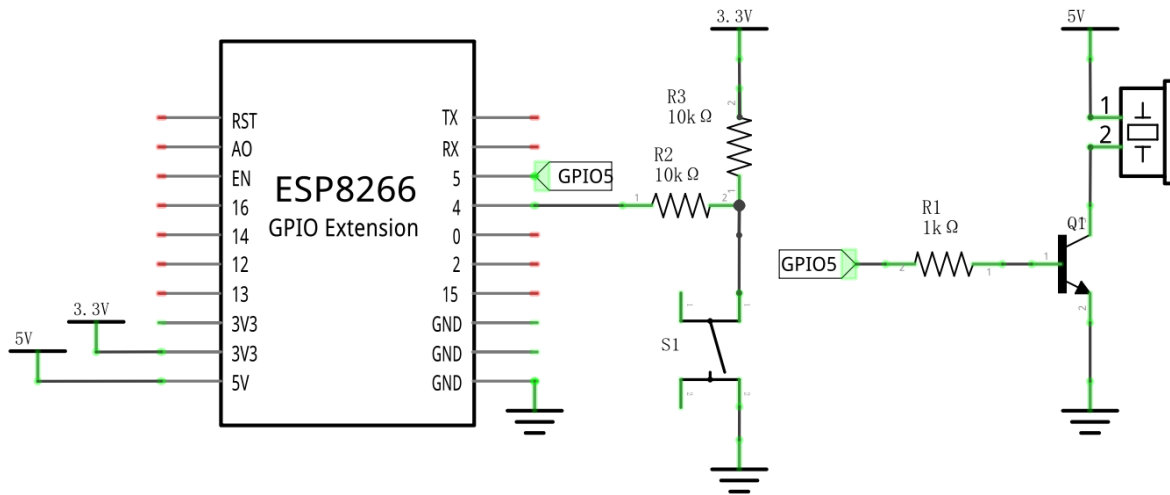
When use NPN transistor to drive buzzer, we often adopt the following method. If GPIO outputs high level, current will flow through R1, the transistor will get conducted, and the buzzer will sound. If GPIO outputs low level, no current flows through R1, the transistor will not be conducted, and buzzer will not sound.

When use PNP transistor to drive buzzer, we often adopt the following method. If GPIO outputs low level, current will flow through R1, the transistor will get conducted, and the buzzer will sound. If GPIO outputs high level, no current flows through R1, the transistor will not be conducted, and buzzer will not sound.

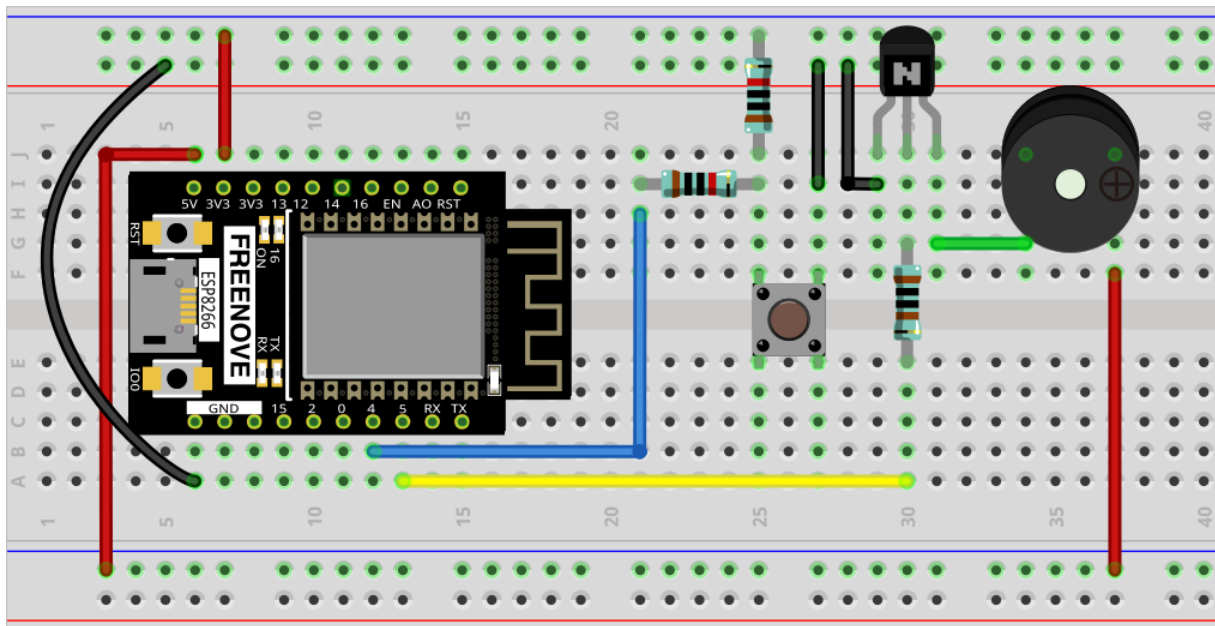


Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Note: in this circuit, the power supply for buzzer is 5V, and pull-up resistor of the button connected to the power 3.3V. The buzzer can work when connected to power 3.3V, but it will reduce the loudness.

Sketch

In this project, a buzzer will be controlled by a push button switch. When the button switch is pressed, the buzzer sounds and when the button is released, the buzzer stops. It is analogous to our earlier project that controlled a LED ON and OFF.

Upload following sketch:

Freenove_Basic_Starter_Kit_for_ESP8266\C\Sketches\Sketch_06.1_Doorbell

Sketch_06.1_Doorbell

```
Sketch_07.1_Doorbell | Arduino 1.8.19
File Edit Sketch Tools Help

Sketch_07.1_Doorbell

1 /*****
2  Filename   : Doorbell
3  Description: Control active buzzer by button.
4  Author    : www.freenove.com
5  Modification: 2022/05/10
6  *****/
7 #define PIN_BUZZER 5
8 #define PIN_BUTTON 4
9
10 void setup() {
11     pinMode(PIN_BUZZER, OUTPUT);
12     pinMode(PIN_BUTTON, INPUT);
13 }
14
15 void loop() {
16     if (digitalRead(PIN_BUTTON) == LOW) {
17         digitalWrite(PIN_BUZZER, HIGH);
18     } else {
19         digitalWrite(PIN_BUZZER, LOW);
20     }
21 }
```

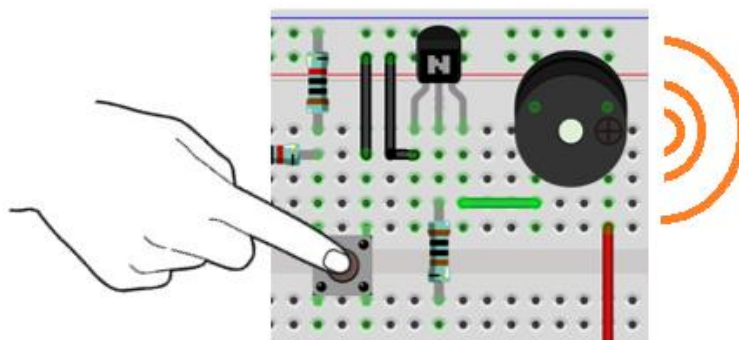
Done uploading.

Leaving...

Hard resetting via RTS pin...

stm32f103, Disabled, All SSL ciphers (most compatible), 32KB cache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA~1019KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4

Download the code to ESP8266, press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.



Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

The following is the program code:

```
1  #define PIN_BUZZER 5
2  #define PIN_BUTTON 4
3
4  void setup() {
5      pinMode(PIN_BUZZER, OUTPUT);
6      pinMode(PIN_BUTTON, INPUT);
7  }
8
9  void loop() {
10     if (digitalRead(PIN_BUTTON) == LOW) {
11         digitalWrite(PIN_BUZZER, HIGH);
12     }else{
13         digitalWrite(PIN_BUZZER, LOW);
14     }
15 }
```

The code is logically the same as using button to control LED.

Project 6.2 Alertor

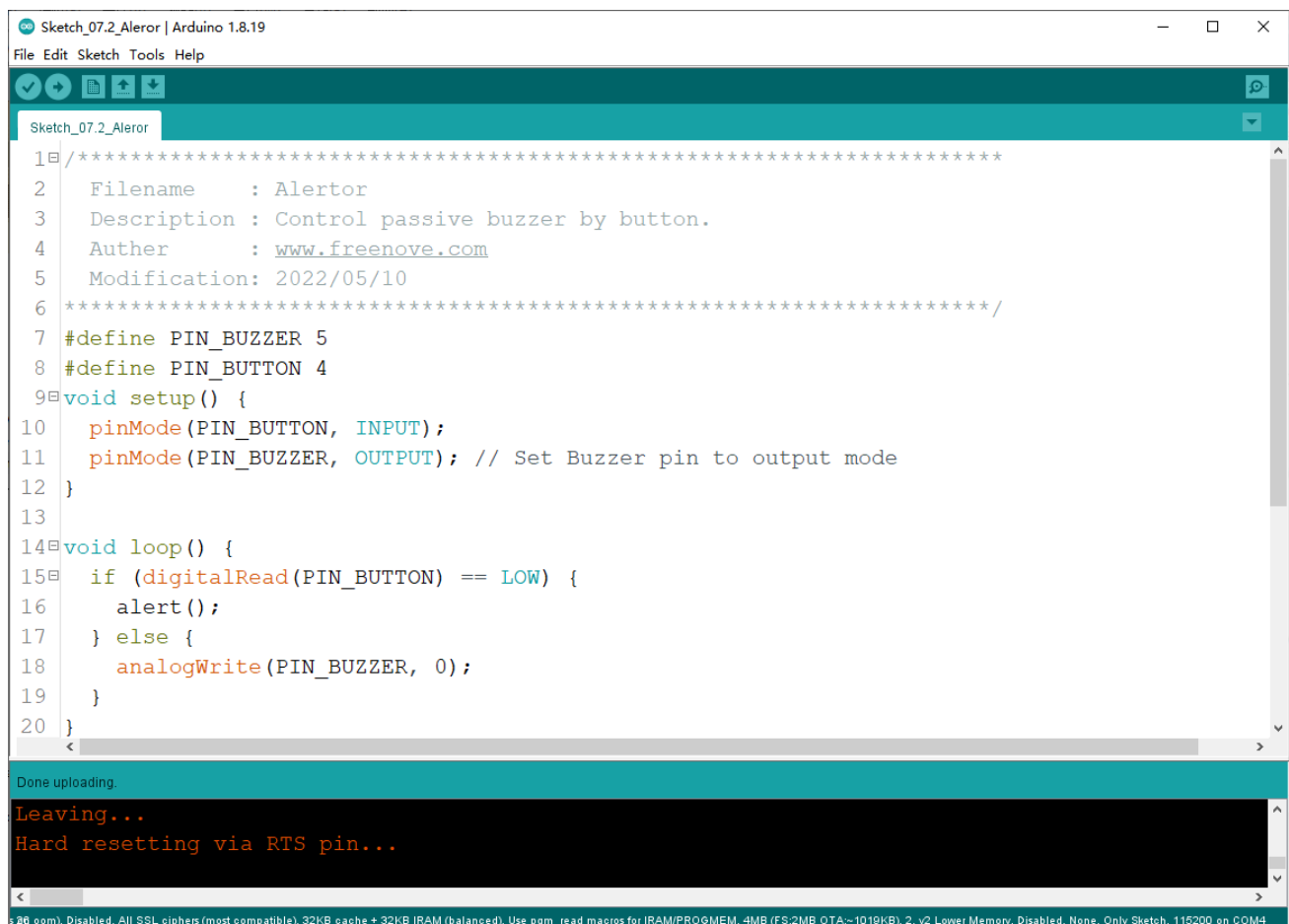
Next, we will use a passive buzzer to make an alarm.

Component list and the circuit is similar to the last section. In the Doorbell circuit only the **active buzzer** needs to be **replaced** with a **passive buzzer**.

Sketch

In this project, the buzzer alarm is controlled by the button. Press the button, then buzzer sounds. If you release the button, the buzzer will stop sounding. It is logically the same as using button to control LED, but in the control method, passive buzzer requires PWM of certain frequency to sound.

Sketch_06.2_Alertor



```
Sketch_07.2_Alertor | Arduino 1.8.19
File Edit Sketch Tools Help

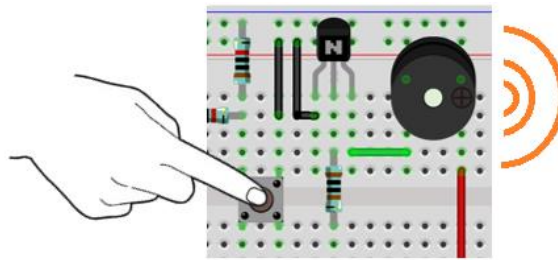
Sketch_07.2_Alertor

1 /*****
2  Filename      : Alertor
3  Description   : Control passive buzzer by button.
4  Author       : www.freenove.com
5  Modification : 2022/05/10
6  *****/
7 #define PIN_BUZZER 5
8 #define PIN_BUTTON 4
9 void setup() {
10  pinMode(PIN_BUTTON, INPUT);
11  pinMode(PIN_BUZZER, OUTPUT); // Set Buzzer pin to output mode
12 }
13
14 void loop() {
15  if (digitalRead(PIN_BUTTON) == LOW) {
16    alert();
17  } else {
18    analogWrite(PIN_BUZZER, 0);
19  }
20 }

Done uploading.
Leaving...
Hard resetting via RTS pin...

s 26 com), Disabled, All SSL ciphers (most compatible), 32KB cache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA~1019KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4
```

Download the code to ESP8266, press the button, then alarm sounds. And when the button is released, the alarm will stop sounding.



The following is the program code:

```

1  #define PIN_BUZZER 5
2  #define PIN_BUTTON 4
3  void setup() {
4      pinMode(PIN_BUTTON, INPUT);
5      pinMode(PIN_BUZZER, OUTPUT); // Set Buzzer pin to output mode
6      Serial.begin(115200);
7  }
8  void loop() {
9      if (digitalRead(PIN_BUTTON) == LOW) {
10         alert();
11     }
12     else {
13         analogWrite(PIN_BUZZER, 0);
14     }
15 }
16 void alert() {
17     float sinVal;           // Define a variable to save sine value
18     int toneVal;            // Define a variable to save sound frequency
19     for (int x = 0; x < 360; x++) { // X from 0 degree->360 degree
20         sinVal = sin(x * (PI / 180)); // Calculate the sine of x
21         toneVal = 500 + sinVal * 500; //Calculate sound frequency according to the sine of x
22         tone(PIN_BUZZER, toneVal);    // Output sound frequency to buzzerPin
23         delay(1);
24         noTone(PIN_BUZZER);
25         delay(1);
26         Serial.printf("Running %d \n", toneVal);
27     }
28 }

```

In the code, use one loop to control the sound frequency, varying according to sine curve in the range of 500 ± 500 .

```

19     for (int x = 0; x < 360; x++) { // X from 0 degree->360 degree
20         sinVal = sin(x * (PI / 180)); // Calculate the sine of x
21         toneVal = 500 + sinVal * 500; //Calculate sound frequency according to the sine of x

```

```
22     tone(PIN_BUZZER, toneVal);           // Output sound frequency to buzzerPin
23     delay(1);
24     noTone(PIN_BUZZER);
25     delay(1);
26     Serial.printf("Running  %d \n", toneVal);
27 }
```

The parameter of `sin()` function is radian, so we need convert unit of π from angle to radian first .

Reference

`tone(pin, frequency)`

Generates a square wave of the specified frequency (and 50% duty cycle) on a pin.

Verify and upload the code, passive buzzer starts making a warning sound.

You can try using PNP transistor to complete the project of this chapter again.

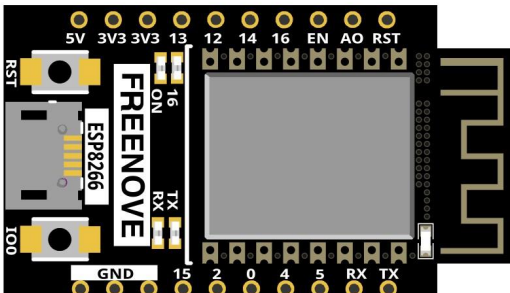
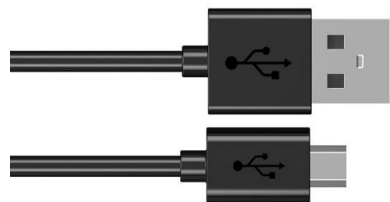
Chapter 7 Serial Communication

Serial Communication is a means of communication between different devices/devices. This section describes ESP8266's Serial Communication.

Project 7.1 Serial Print

This project uses ESP8266's serial communicator to send data to the computer and print it on the serial monitor.

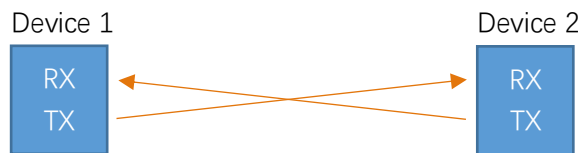
Component List

ESP8266 x1  A black ESP8266 module with a white label that reads 'FREENOVE' and 'ESP8266'. It has various pins labeled: 5V, 3V3, 3V3, 13, 12, 14, 16, EN, AO, RST, RST, 100, GND, 15, 2, 0, 4, 5, RX, TX. There is also a small switch labeled 'ON'.	USB cable  A black USB cable with a standard USB-A connector on one end and a USB-B connector on the other.
---	---

Related knowledge

Serial communication

Serial communication generally refers to the Universal Asynchronous Receiver/Transmitter (UART), which is commonly used in electronic circuit communication. It has two communication lines, one is responsible for sending data (TX line) and the other for receiving data (RX line). The serial communication connections of two devices is as follows:



Before serial communication starts, the baud rate of both sides must be the same. Communication between devices can work only if the same baud rate is used. The baud rates commonly used is 9600 and 115200.

Serial port on ESP8266

Freenove ESP8266 has integrated USB to serial transfer, so it could communicate with computer connecting to USB cable.

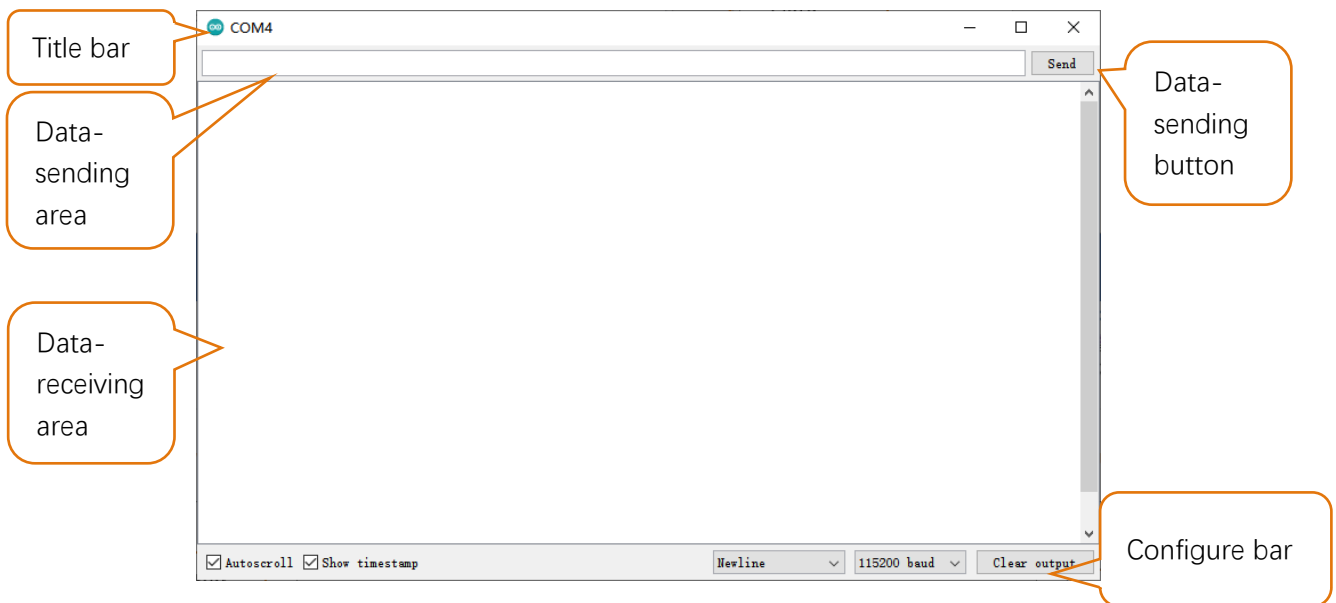


Arduino Software also uploads code to Freenove ESP8266 through the serial connection.

Your computer identifies serial devices connecting to it as COMx. We can use the Serial Monitor window of Arduino Software to communicate with Freenove ESP8266, connect Freenove ESP8266 to computer through the USB cable, choose the correct device, and then click the Serial Monitor icon to open the Serial Monitor window.

Interface of serial monitor window is as follows. If you can't open it, make sure Freenove ESP8266 has been connected to the computer, and choose the right serial port in the menu bar "Tools".





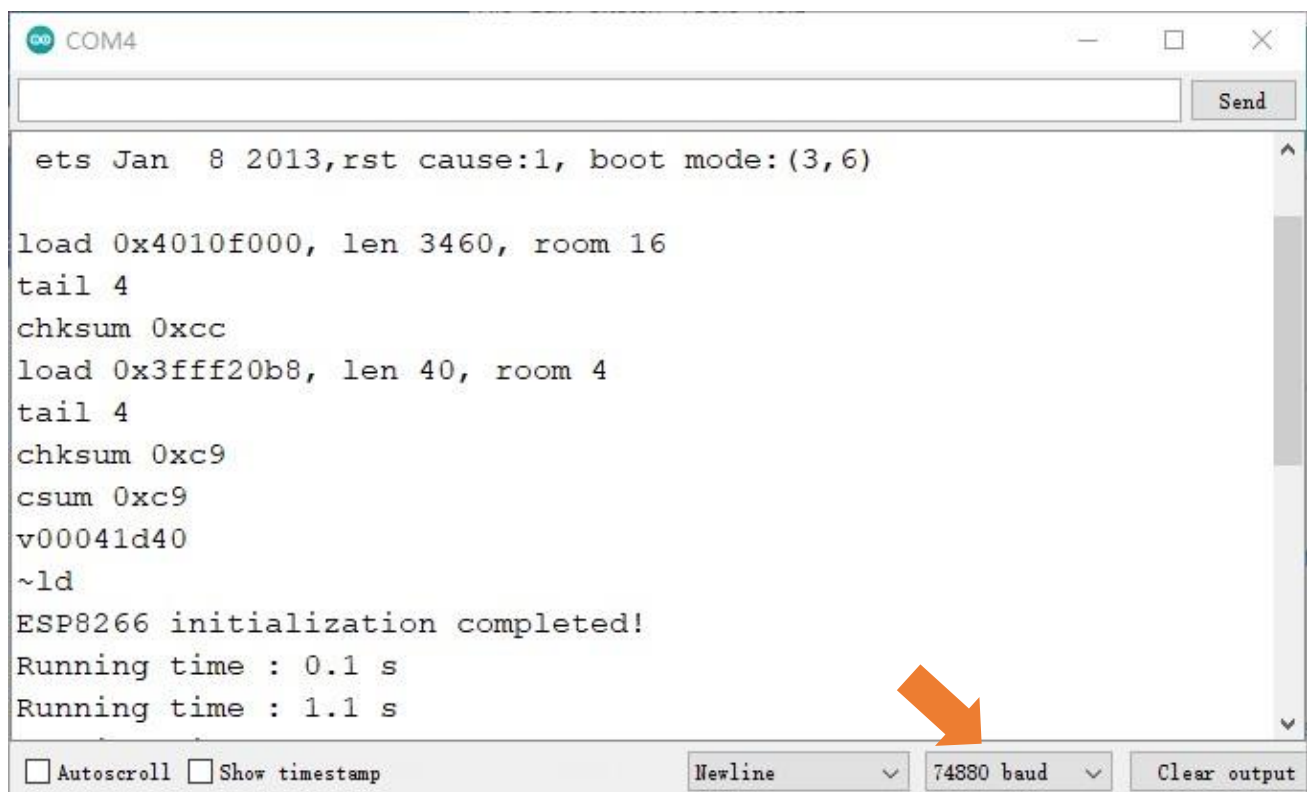
When the ESP8266 is powered on, the default baud rate is 74880. The default communication and serial port in the ESP8266 firmware is 115200. So if you set the serial port to 74880, this time can be displayed normally. If the baud rate is not 74880, some garbled characters will appear. Please do not worry.

Here, we use The Arduino IDE serial port tool for output and display. The details are as follows:

```
void setup() {
  Serial.begin(74880);
  Serial.println("ESP8266 initialization completed!");
}

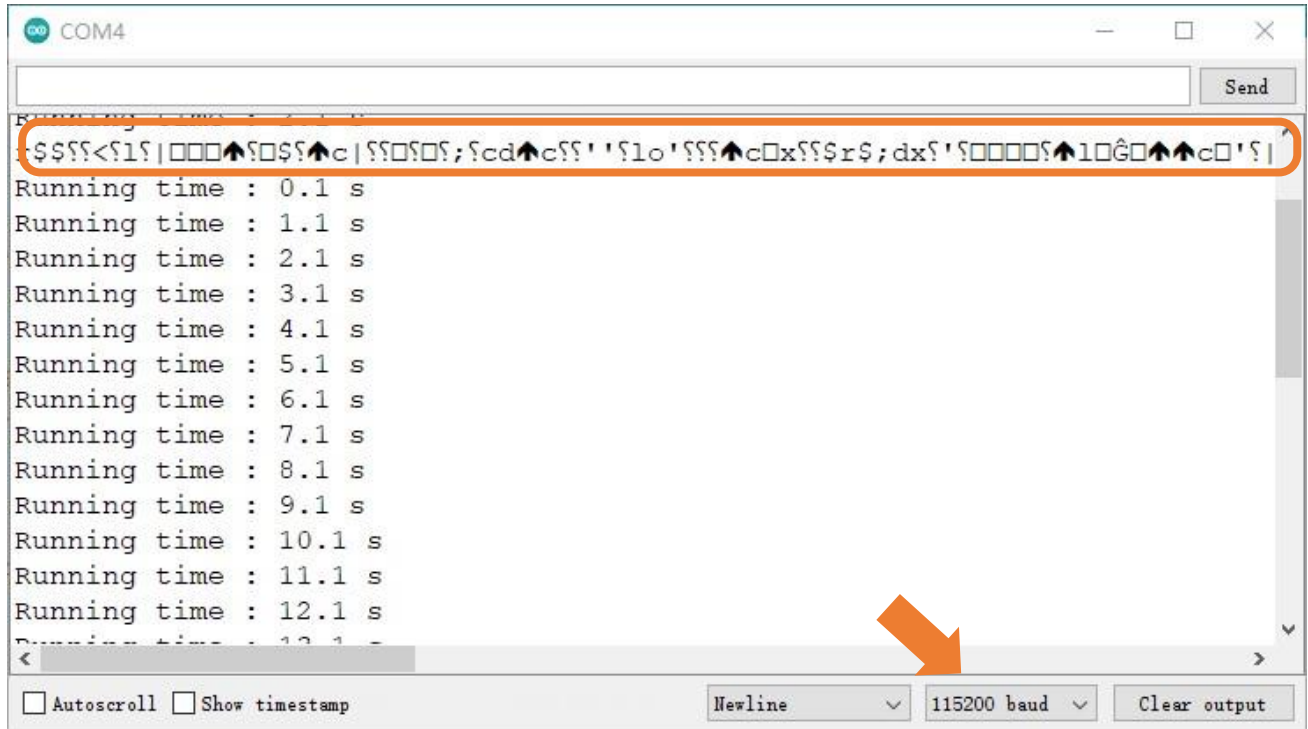
void loop() {
  Serial.printf("Running time : %.1f s\n", millis() / 1000.0f);
  delay(1000);
}
```

An orange arrow points to the `Serial.println("ESP8266 initialization completed!");` line in the code.



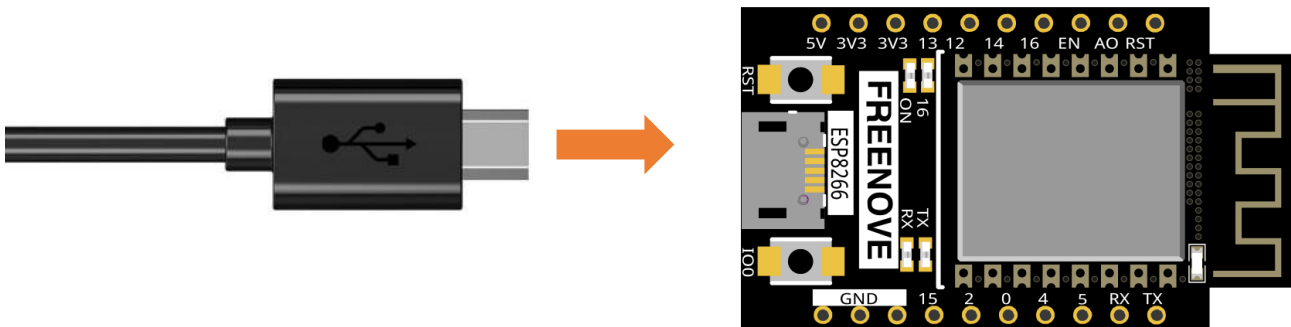
Garbled characters are displayed when you set the baud rate to another value as follows:

```
void setup() {  
  Serial.begin(115200);  
  Serial.println("ESP8266 initialization completed!");  
}  
  
void loop() {  
  Serial.printf("Running time : %.1f s\n", millis() / 1000.0f);  
  delay(1000);  
}
```



Circuit

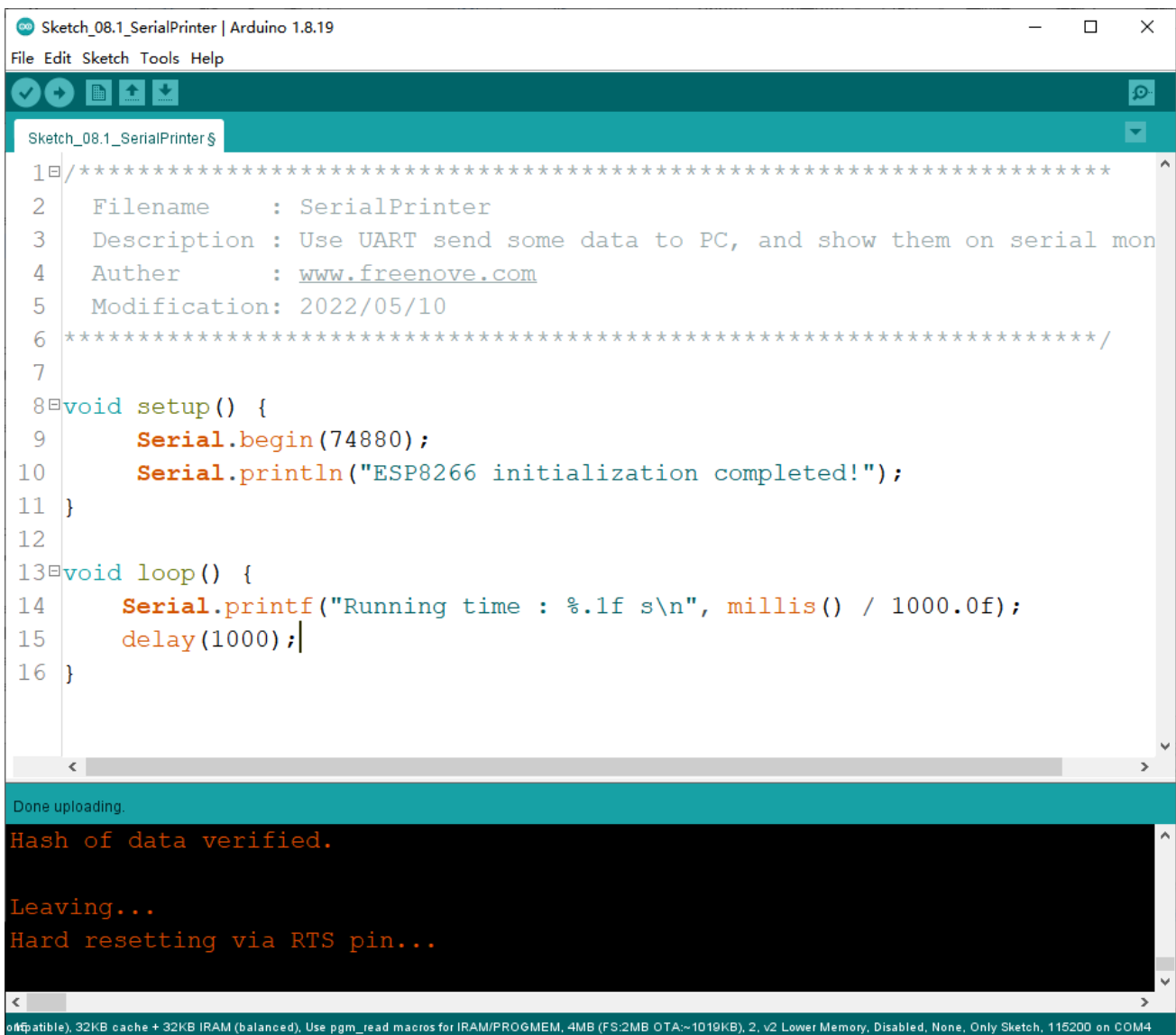
Connect Freenove ESP8266 to the computer with USB cable.



Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

Sketch

Sketch_07.1_SerialPrinter



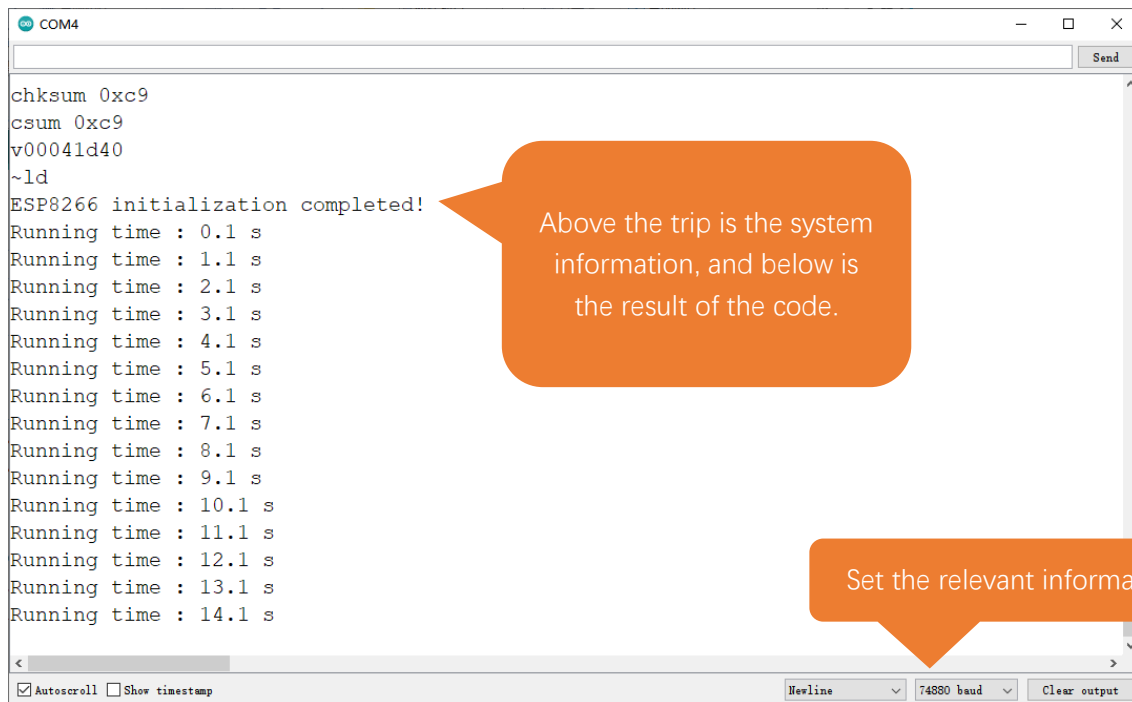
The screenshot shows the Arduino IDE interface. The top window displays the sketch code for 'Sketch_08.1_SerialPrinter'. The code includes a header section with metadata and two main functions: setup() and loop(). The setup() function initializes the serial port at 74880 baud and prints a completion message. The loop() function prints the running time in seconds every 1000 milliseconds. Below the code editor, a status bar indicates 'Done uploading.' and a message box shows 'Hash of data verified.', 'Leaving...', and 'Hard resetting via RTS pin...'. At the very bottom, a small text bar provides hardware specifications: 'compatible), 32KB cache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA~1019KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4'.

```
1 /*****  
2  Filename      : SerialPrinter  
3  Description   : Use UART send some data to PC, and show them on serial mon  
4  Author       : www.freenove.com  
5  Modification  : 2022/05/10  
6  *****/  
7  
8 void setup() {  
9     Serial.begin(74880);  
10    Serial.println("ESP8266 initialization completed!");  
11 }  
12  
13 void loop() {  
14     Serial.printf("Running time : %.1f s\n", millis() / 1000.0f);  
15     delay(1000);  
16 }
```

Done uploading.
Hash of data verified.
Leaving...
Hard resetting via RTS pin...

compatible), 32KB cache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA~1019KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4

Download the code to ESP8266, open the serial port monitor, set the baud rate to 74880, and press the reset button. As shown in the following figure:



As shown in the image above, "ESP8266 initialization completed! " The previous is the printing message when the system is started, it uses the baud rate of 120,000, which is incorrect, so the garbled code is displayed. The user program is then printed at a baud rate of 74880.

For more information, click [here](#).

The following is the program code:

```

1 void setup() {
2     Serial.begin(74880);
3     Serial.println("ESP8266 initialization completed! ");
4 }
5
6 void loop() {
7     Serial.printf("Running time : %.1f s\n", millis() / 1000.0f);
8     delay(1000);
9 }

```

Reference

```
void begin(unsigned long baud, uint32_t config=SERIAL_8N1, int8_t rxPin=-1,
           int8_t txPin=-1, bool invert=false, unsigned long timeout_ms = 20000UL);
```

Initializes the serial port. Parameter baud is baud rate, other parameters generally use the default value.

```
size_t println( arg );
```

Print to the serial port and wrap. The parameter **arg** can be a number, a character, a string, an array of characters, etc.

```
size_t printf(const char * format, ...) __attribute__((format (printf, 2, 3)));
```

Print formatted content to the serial port in the same way as print in standard C.

```
unsigned long millis();
```

Returns the number of milliseconds since the current system was booted.

Any concerns? ✉ support@freenove.com

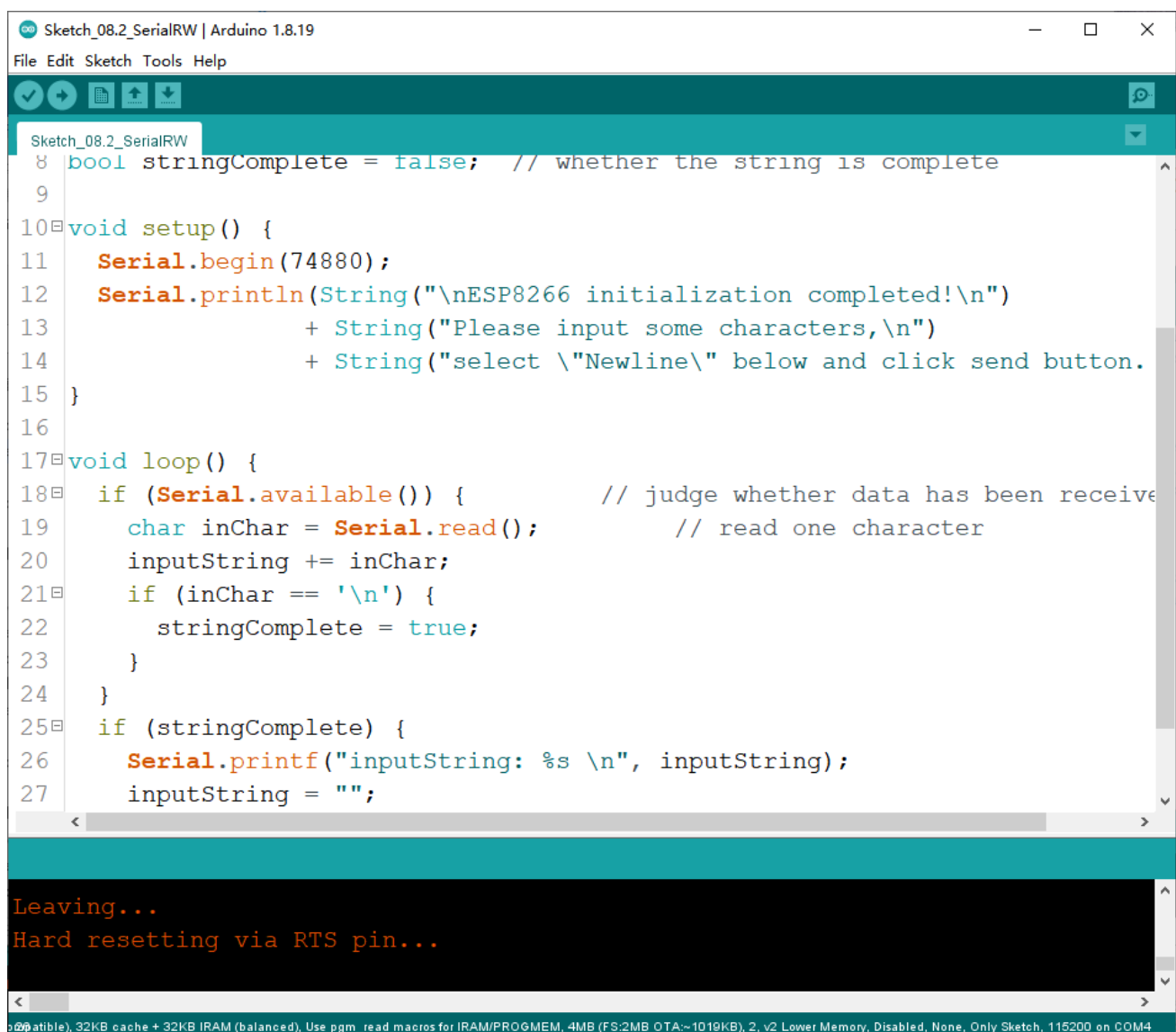
Project 7.2 Serial Read and Write

From last section, we use serial port on Freenove ESP8266 to send data to a computer, now we will use that to receive data from computer.

Component and circuit are the same as in the previous project.

Sketch

Sketch_07.2_SerialRW



```

Sketch_08.2_SerialRW | Arduino 1.8.19
File Edit Sketch Tools Help
Sketch_08.2_SerialRW
8 bool stringComplete = false; // whether the string is complete
9
10 void setup() {
11     Serial.begin(74880);
12     Serial.println(String("\nESP8266 initialization completed!\n")
13         + String("Please input some characters,\n")
14         + String("select \"Newline\" below and click send button.
15 }
16
17 void loop() {
18     if (Serial.available()) { // judge whether data has been received
19         char inChar = Serial.read(); // read one character
20         inputString += inChar;
21         if (inChar == '\n') {
22             stringComplete = true;
23         }
24     }
25     if (stringComplete) {
26         Serial.printf("inputString: %s \n", inputString);
27         inputString = "";

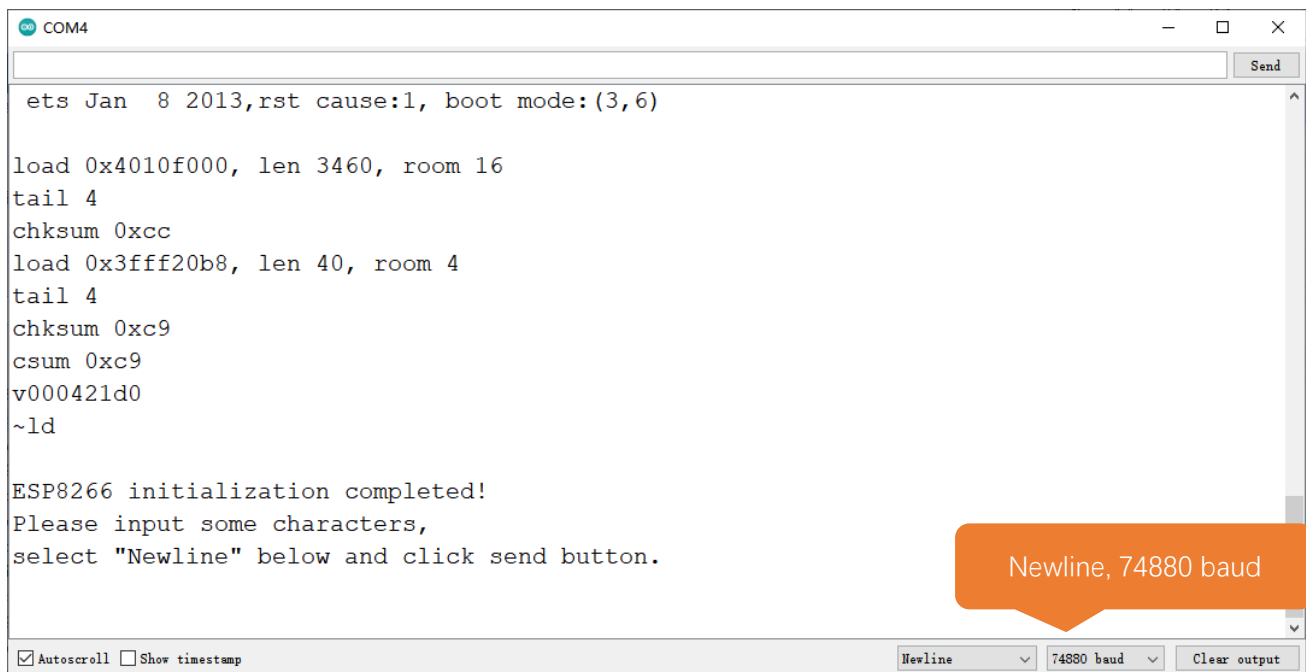
```

Leaving...

Hard resetting via RTS pin...

Compatible, 32KB cache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA~1019KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4

Download the code to ESP8266, open the serial monitor, and set the bottom to Newline, 74880. As shown in the following figure:



```
ets Jan 8 2013,rst cause:1, boot mode:(3,6)

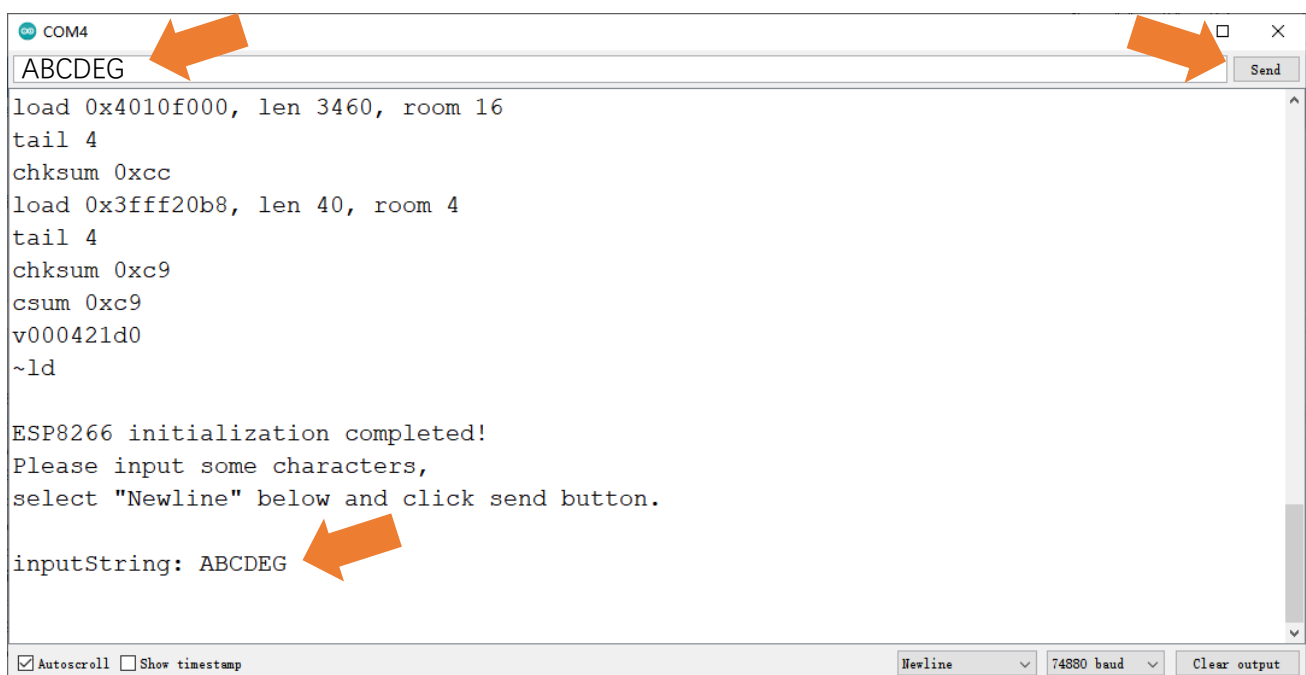
load 0x4010f000, len 3460, room 16
tail 4
chksum 0xcc
load 0x3fff20b8, len 40, room 4
tail 4
chksum 0xc9
csum 0xc9
v000421d0
~ld

ESP8266 initialization completed!
Please input some characters,
select "Newline" below and click send button.
```

Newline, 74880 baud

☒ Autoscroll ☐ Show timestamp Newline 74880 baud Clear output

Then type characters like 'ABCDEG' into the data sent at the top and click the Send button to print out the data ESP8266 receives.



```
load 0x4010f000, len 3460, room 16
tail 4
chksum 0xcc
load 0x3fff20b8, len 40, room 4
tail 4
chksum 0xc9
csum 0xc9
v000421d0
~ld

ESP8266 initialization completed!
Please input some characters,
select "Newline" below and click send button.

inputString: ABCDEG
```

☒ Autoscroll ☐ Show timestamp Newline 74880 baud Clear output

The following is the program code:

```

1  String inputString = "";      //a String to hold incoming data
2  bool stringComplete = false; // whether the string is complete
3
4  void setup() {
5      Serial.begin(74880);
6      Serial.println(String("\nESP8266 initialization completed! \n")
7          + String("Please input some characters, \n")
8          + String("select \"Newline\" below and click send button. \n"));
9  }
10
11 void loop() {
12     if (Serial.available()) {      // judge whether data has been received
13         char inChar = Serial.read(); // read one character
14         inputString += inChar;
15         if (inChar == '\n') {
16             stringComplete = true;
17         }
18     }
19     if (stringComplete) {
20         Serial.printf("inputString: %s \n", inputString);
21         inputString = "";
22         stringComplete = false;
23     }
24 }

```

In loop(), determine whether the serial port has data, if so, read and save the data, and if the newline character is read, print out all the data that has been read.

Reference

String();

Constructs an instance of the String class.

For more information, please visit

<https://www.arduino.cc/reference/en/language/variables/data-types/stringobject/>

int available(void);

Get the number of bytes (characters) available for reading from the serial port. This is data that's already arrived and stored in the serial receive buffer.

Serial.read();

Reads incoming serial data.

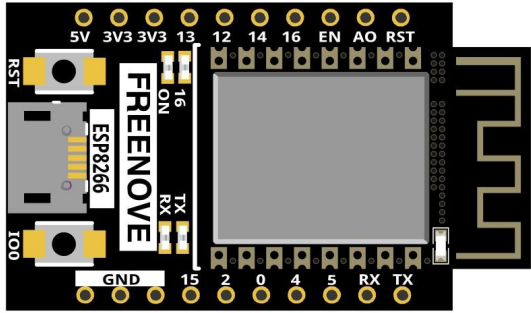
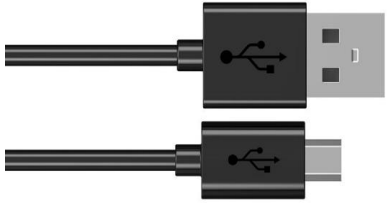
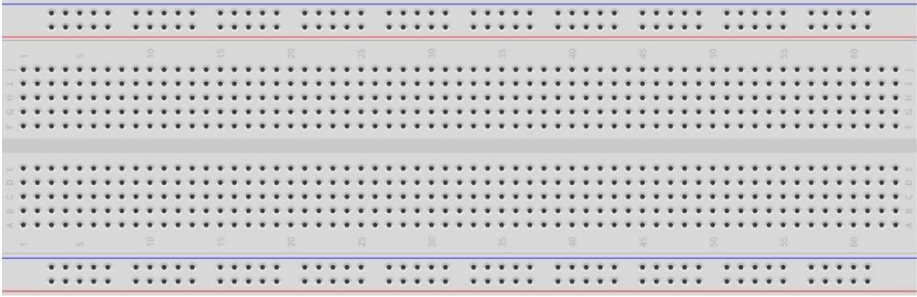
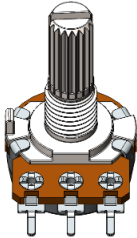
Chapter 8 ADC Converter

We have learned how to control the brightness of LED through PWM and understood that PWM is not the real analog before. In this chapter, we will learn how to read analog, convert it into digital. That is, ADC.

Project 8.1 Read the Voltage of Potentiometer

In this project, ADC is used to convert analog signals into digital signals. Control chip on the control board has integrated this function. Now let us try to use this function to convert analog signals into digital signals.

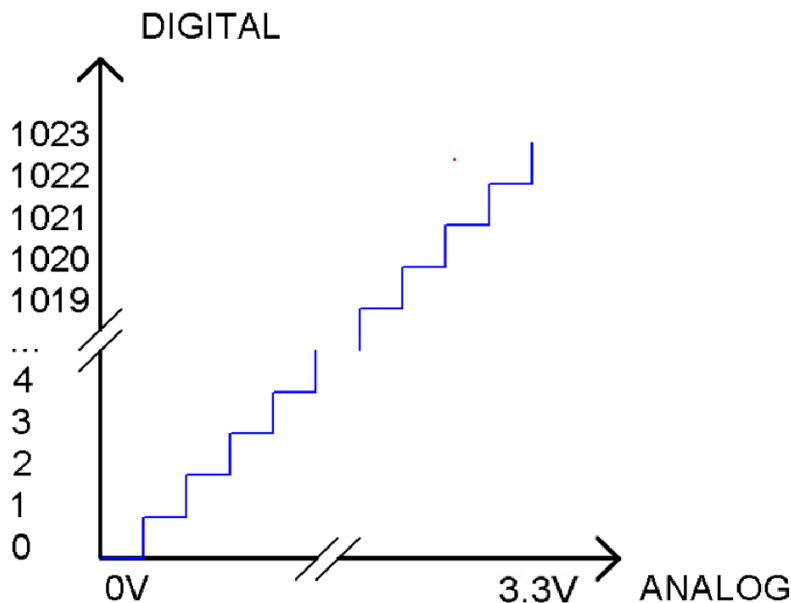
Component List

ESP8266 x1 	USB cable 
Breadboard x1 	
Rotary potentiometer x1 	Jumper wire M/M x3 

Related knowledge

ADC

An ADC is an electronic integrated circuit used to convert analog signals such as voltages to digital or binary form consisting of 1s and 0s. The range of our ADC on ESP8266 is 10 bits, that means the resolution is $2^{10}=1024$, and it represents a range (at 3.3V) will be divided equally to 1024 parts. The range of analog values corresponds to ADC values. So the more bits the ADC has, the denser the partition of analog will be and the greater the precision of the resulting conversion.



Subsection 1: the analog in range of $0V \sim 3.3/1023 V$ corresponds to digital 0;

Subsection 2: the analog in range of $3.3/1023 V \sim 2 \times 3.3 / 1023 V$ corresponds to digital 1;

...

The following analog will be divided accordingly.

The conversion formula is as follows:

$$ADC\ Value = \frac{Analog\ Voltage}{3.3} * 1023$$

ADC on ESP8266

ESP8266 has one digital analog converters with successive approximations of 10-bit accuracy, and a total of 1 pins can be used to measure analog signals. GPIO analog pin definition are shown in the following table. Note that the input voltage on the ADC pins of the ESP8266 module must be between 0V and 1.0V. For the ESP8266 development board designed by us, its input voltage range has been sampled by resistors. The ADC input voltage of the development board is 0V to 3.3V.

Do not exceed this voltage range when you use the ADC function. Exceeding this voltage range can cause permanent damage to your hardware!

Pin number in ESP8266

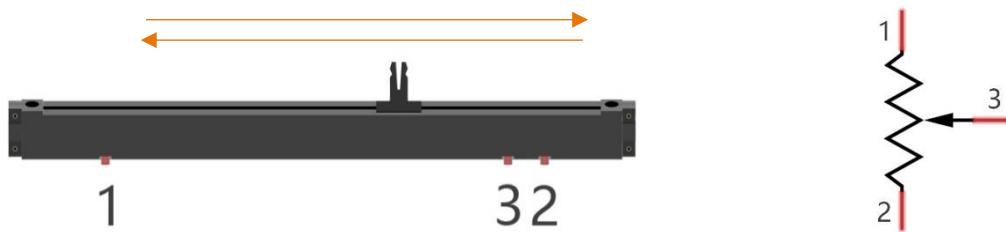
A0

The analog pin number is also defined in ESP8266's code base. For example, you can use A0 in your code.

Component knowledge

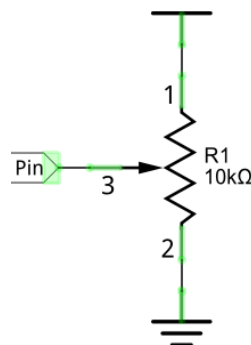
Potentiometer

A potentiometer is a three-terminal resistor. Unlike the resistors that we have used thus far in our project which have a fixed resistance value, the resistance value of a potentiometer can be adjusted. A potentiometer is often made up by a resistive substance (a wire or carbon element) and movable contact brush. When the brush moves along the resistor element, there will be a change in the resistance of the potentiometer's output side (3) (or change in the voltage of the circuit that is a part). The illustration below represents a linear sliding potentiometer and its electronic symbol on the right.



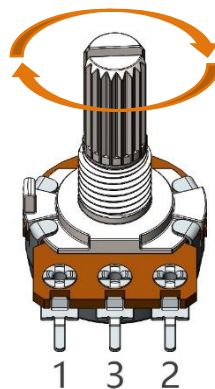
What between potentiometer pin 1 and pin 2 is the resistor body, and pins 3 is connected to brush. When brush moves from pin 1 to pin 2, the resistance between pin 1 and pin 3 will increase up to body resistance linearly, and the resistance between pin 2 and pin 3 will decrease down to 0 linearly.

In the circuit. The both sides of resistance body are often connected to the positive and negative electrode of the power. When you slide the brush pin 3, you can get a certain voltage in the range of the power supply.



Rotary potentiometer

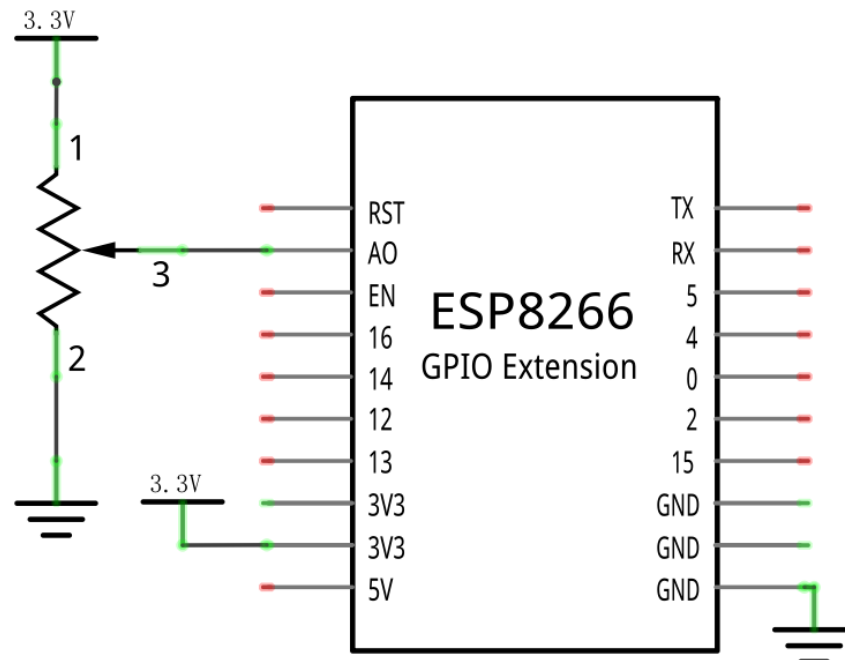
Rotary potentiometer and linear potentiometer have similar function; their only difference is: the resistance is adjusted by rotating the potentiometer.



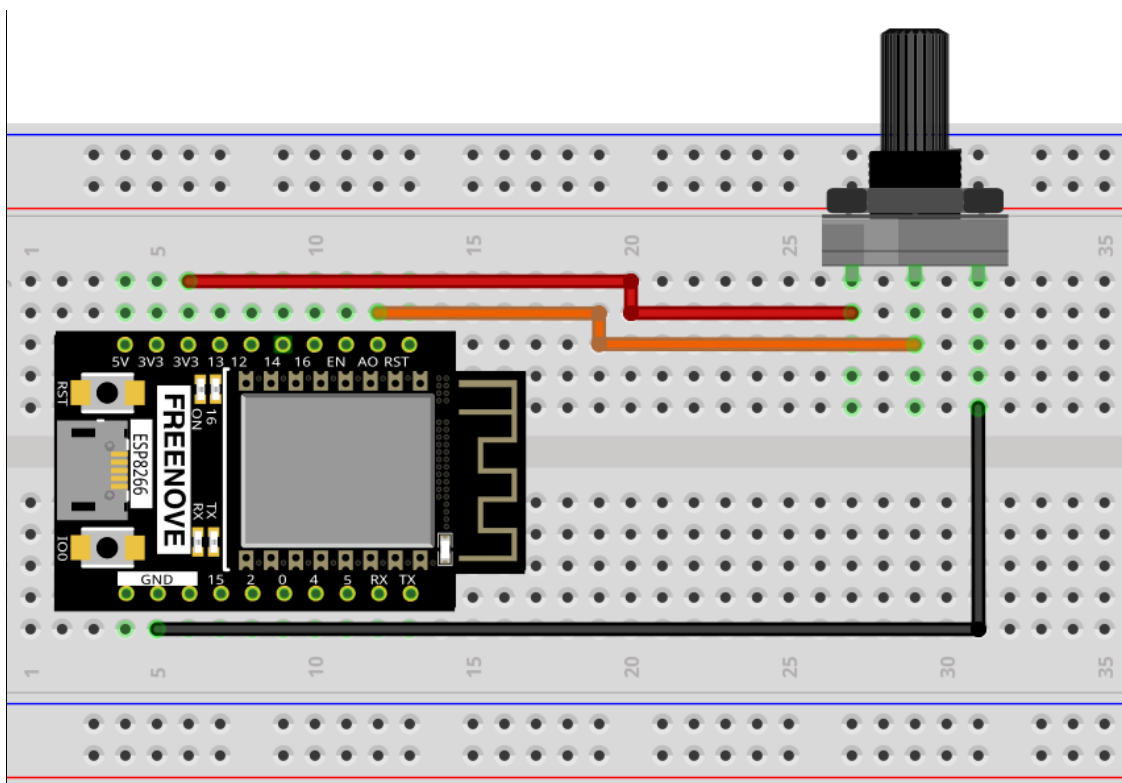
Circuit

Use pin A0 on the control board to detect the voltage of rotary potentiometer.

Schematic diagram

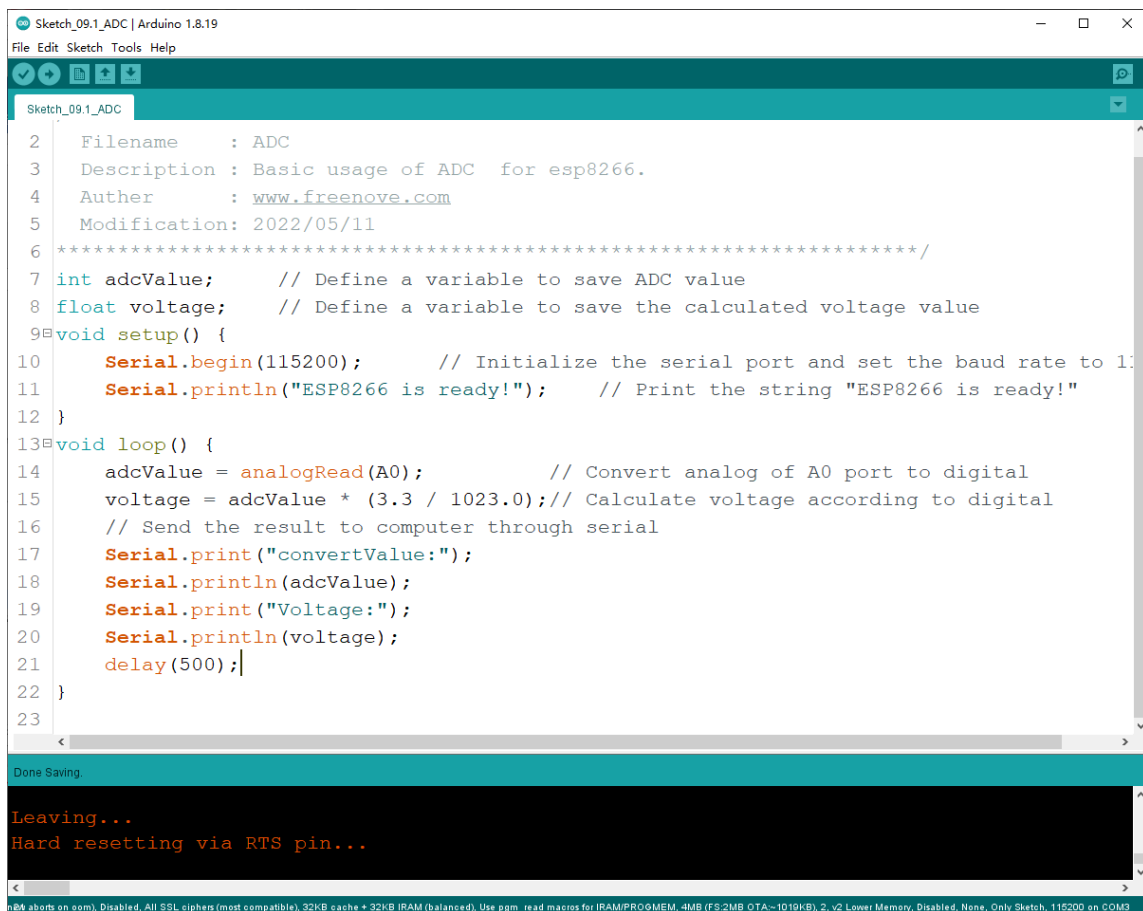


Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Sketch

Sketch_08.1_ADC



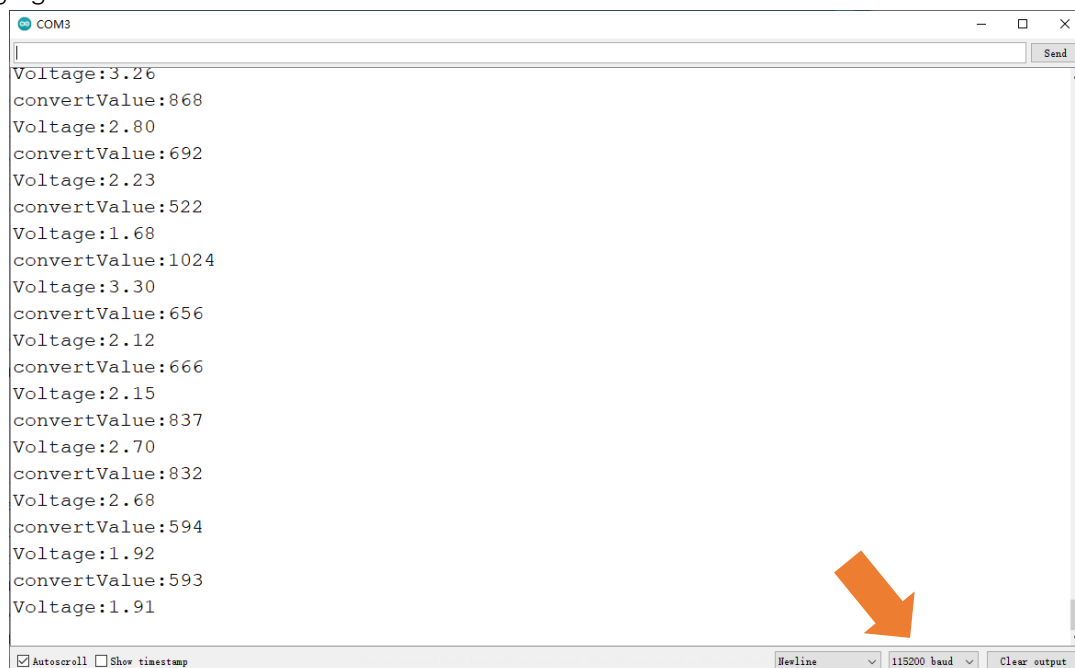
```
Sketch_08.1_ADC | Arduino 1.8.19
File Edit Sketch Tools Help

Sketch_08.1_ADC
2  Filename      : ADC
3  Description   : Basic usage of ADC for esp8266.
4  Author       : www.freenove.com
5  Modification  : 2022/05/11
6  *****/
7  int adcValue;    // Define a variable to save ADC value
8  float voltage;   // Define a variable to save the calculated voltage value
9  void setup() {
10     Serial.begin(115200);    // Initialize the serial port and set the baud rate to 115200
11     Serial.println("ESP8266 is ready!");    // Print the string "ESP8266 is ready!"
12 }
13 void loop() {
14     adcValue = analogRead(A0);    // Convert analog of A0 port to digital
15     voltage = adcValue * (3.3 / 1023.0); // Calculate voltage according to digital
16     // Send the result to computer through serial
17     Serial.print("convertValue:");
18     Serial.println(adcValue);
19     Serial.print("Voltage:");
20     Serial.println(voltage);
21     delay(500);
22 }
23

Done Saving.
Leaving...
Hard resetting via RTS pin...

n86 aborts on com). Disabled, All SSL cipher (most compatible), 32KB cache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA~1019KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM3
```

Download the code to ESP8266, open the serial monitor, and set the baud rate to 115200. As shown in the following figure:



```
COM3
Voltage:3.26
convertValue:868
Voltage:2.80
convertValue:692
Voltage:2.23
convertValue:522
Voltage:1.68
convertValue:1024
Voltage:3.30
convertValue:656
Voltage:2.12
convertValue:666
Voltage:2.15
convertValue:837
Voltage:2.70
convertValue:832
Voltage:2.68
convertValue:594
Voltage:1.92
convertValue:593
Voltage:1.91

Autoscroll Show timestamp Newline 115200 baud Clear output
```

Any concerns? support@freenove.com

From the code, we get the ADC value of pin A0, then convert it into voltage and sent to the serial port. Verify and upload the code, open the Serial Monitor, and then you will see the original ADC value and converted voltage sent from control board.

Turn the rotary potentiometer shaft, and you can see the voltage change.

The following is the code:

```
1  int adcValue;      // Define a variable to save ADC value
2  float voltage;     // Define a variable to save the calculated voltage value
3  void setup() {
4      Serial.begin(115200);    // Initialize the serial port and set the baud rate to 115200
5      Serial.println("ESP8266 is ready!");    // Print the string "ESP8266 is ready!"
6  }
7  void loop() {
8      adcValue = analogRead(A0);    // Convert analog of A0 port to digital
9      voltage = adcValue * (3.3 / 1023.0); // Calculate voltage according to digital
10     // Send the result to computer through serial
11     Serial.print("convertValue:");
12     Serial.println(adcValue);
13     Serial.print("Voltage:");
14     Serial.println(voltage);
15     delay(500);
16 }
```

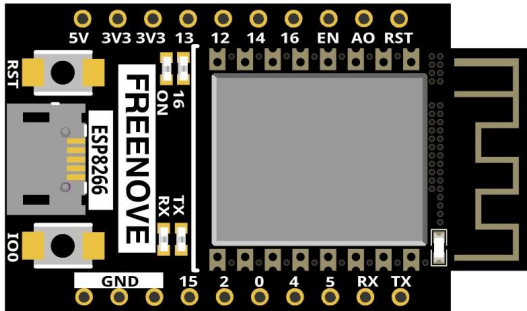
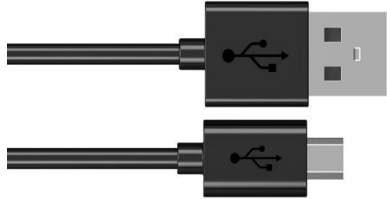
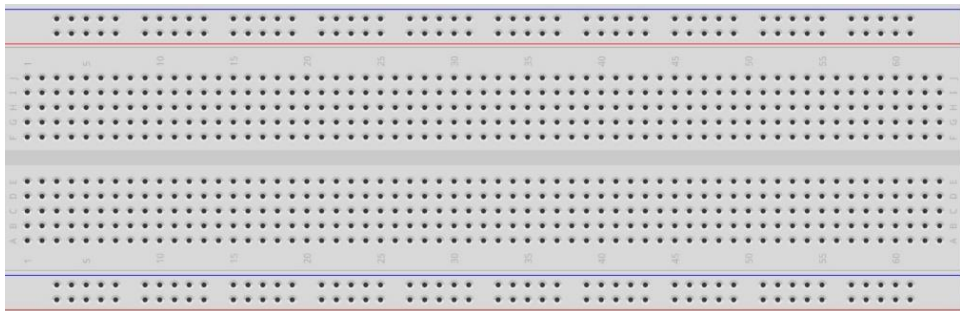
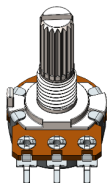
Chapter 9 Potentiometer & LED

In the previous section, we have finished reading ADC value and converting it into voltage. Now, we will try to use potentiometer to control the brightness of LED.

Project 9.1 Soft Light

In this project, we will make a soft light. We will use an ADC Module to read ADC values of a potentiometer and map it to duty cycle of the PWM used to control the brightness of a LED. Then you can change the brightness of a LED by adjusting the potentiometer.

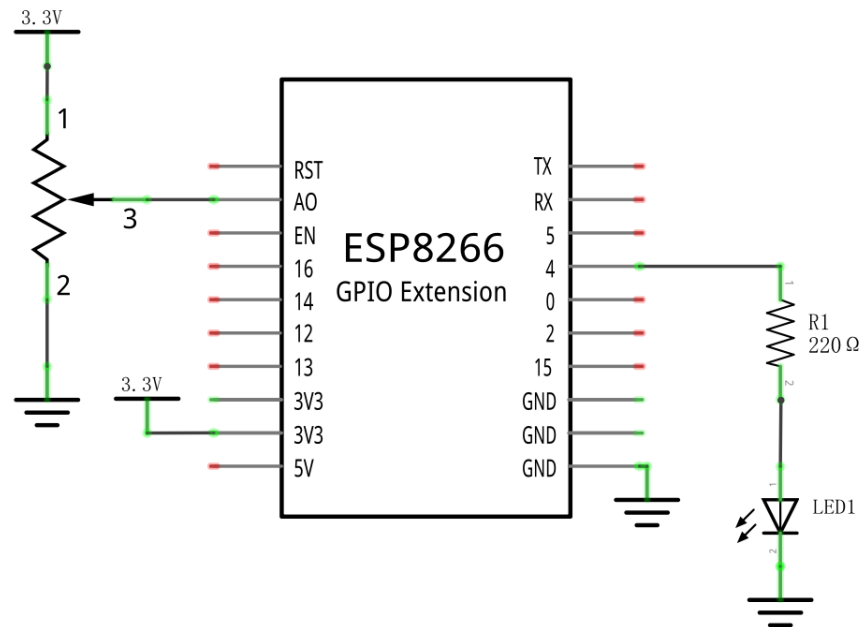
Component List

ESP8266 x1 	USB cable 		
Breadboard x1 			
Rotary potentiometer x1 	Resistor 220Ω x1 	LED x1 	Jumper wire M/M x8 

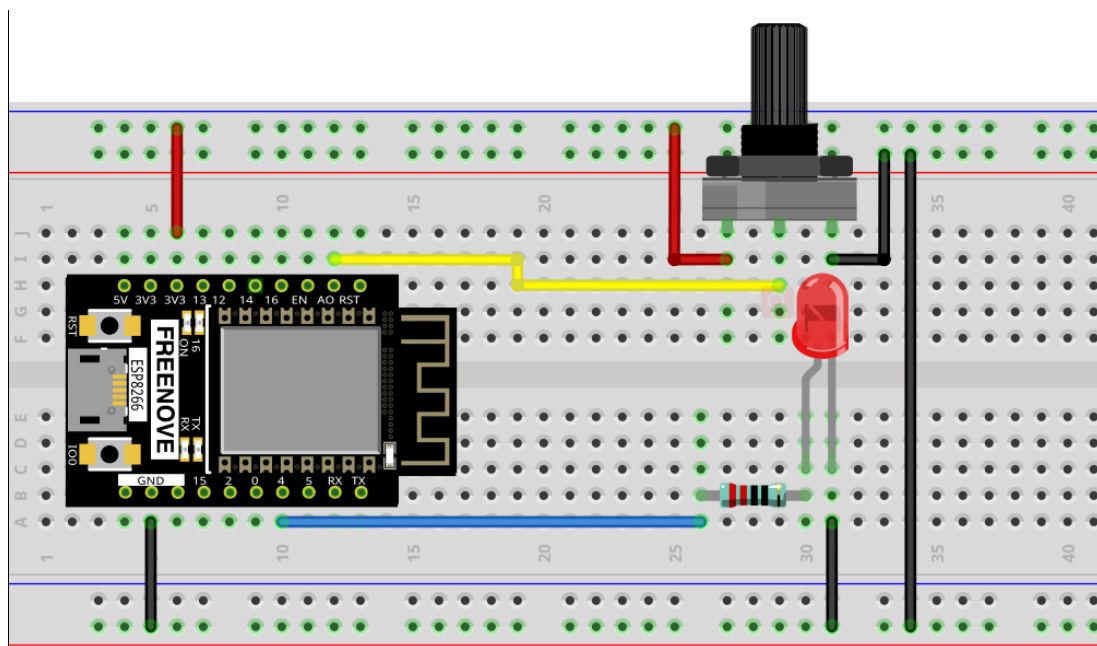
Circuit

Use pin A0 on control board to detect the voltage of rotary potentiometer, and use pin 4 to control one LED.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Sketch

Sketch_09.1_Softlight



```

Sketch_10.1_SoftLight | Arduino 1.8.19
File Edit Sketch Tools Help

Sketch_10.1_SoftLight

1 //*****
2   Filename      : SoftLight
3   Description   : Controlling the brightness of LED by potentiometer.
4   Author       : www.freenove.com
5   Modification  : 2022/05/11
6   *****/
7 int adcValue;    // Define a variable to save the ADC value
8 int ledPin = 4;  // Use D9 on Freenove UNO to control the LED
9
10 void setup() {
11     pinMode(ledPin, OUTPUT);          // Initialize the LED pin as an output
12 }
13 void loop() {
14     adcValue = analogRead(A0);        // Convert the analog of A0 port to digital
15     // Map analog to the 0-255 range, and works as PWM duty cycle of ledPin port
16     analogWrite(ledPin, map(adcValue, 0, 1024, 0, 255));
17 }
18

Done Saving.

nd aborts on com), Disabled, All SSL ciphers (most compatible), 32KB cache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA~1019KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM3

```

Download the code to ESP8266, by turning the adjustable resistor to change the input voltage of GPIO4, ESP8266 changes the output voltage of GPIO4 according to this voltage value, thus changing the brightness of the LED.

The following is the code:

```

1 int adcValue;    // Define a variable to save the ADC value
2 int ledPin = 4;
3 void setup() {
4     pinMode(ledPin, OUTPUT);          // Initialize the LED pin as an output
5 }
6 void loop() {
7     adcValue = analogRead(A0);        // Convert the analog of A0 port to digital
8     // Map analog to the 0-255 range, and works as PWM duty cycle of ledPin port
9     analogWrite(ledPin, map(adcValue, 0, 1024, 0, 255));
10 }

```

In the code, we get the ADC value of pin A0 and map it to PWM duty cycle of LED pin port. According to

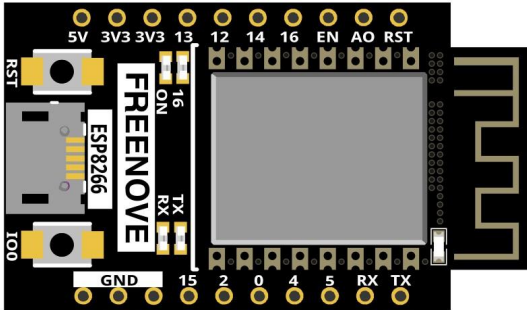
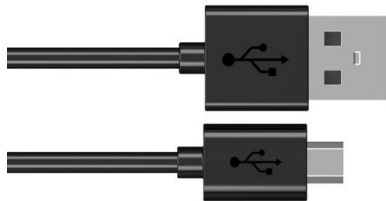
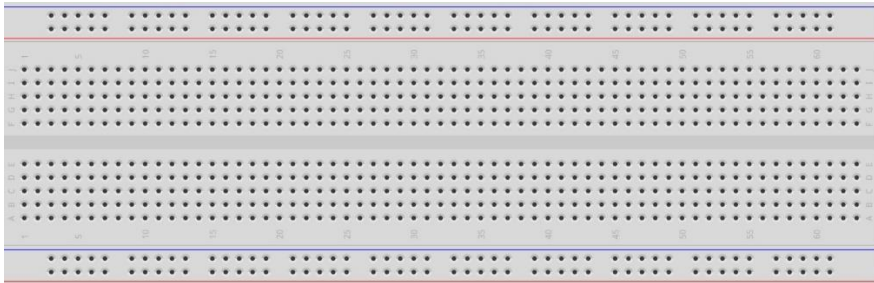
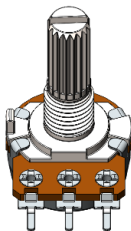
Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

different LED brightness, we can see the changes of voltage easily.

Project 9.2 Color Light

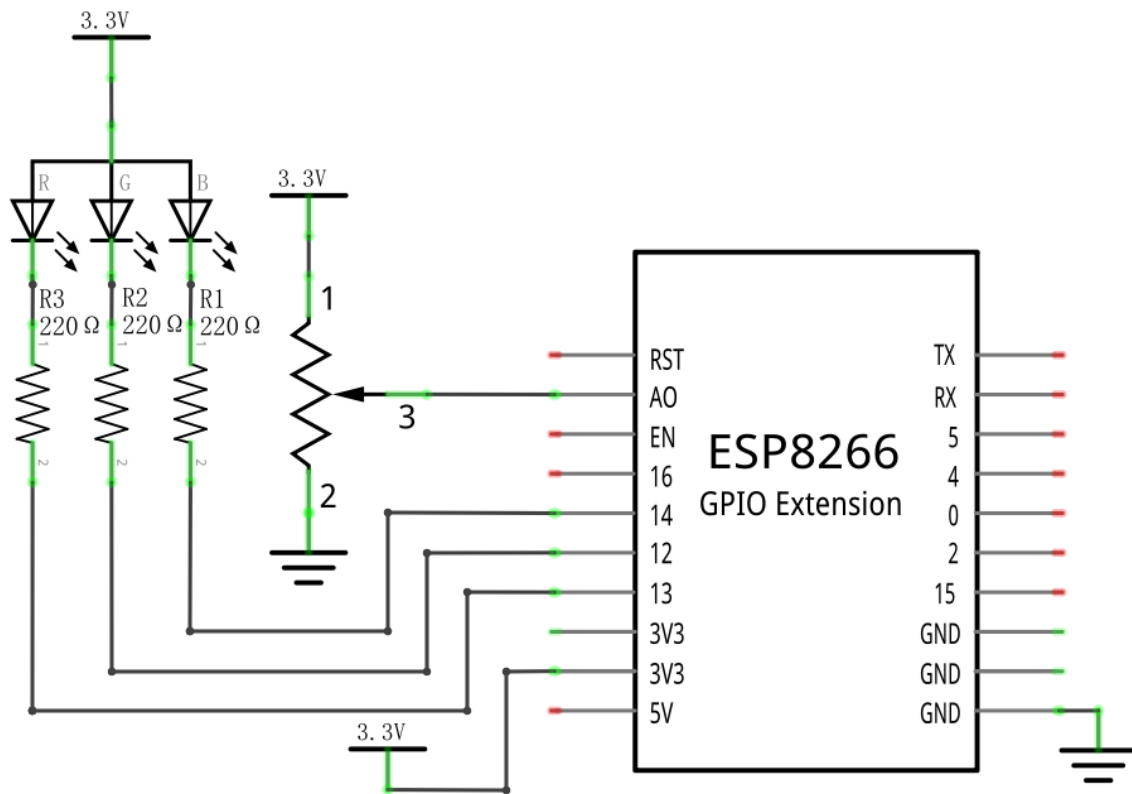
In this project, a potentiometer is used to control the RGB LED. The RGB LED is bright red when the potentiometer is near the midpoint, green when the potentiometer rotates to the "left" and blue when the potentiometer rotates to the "right".

Component List

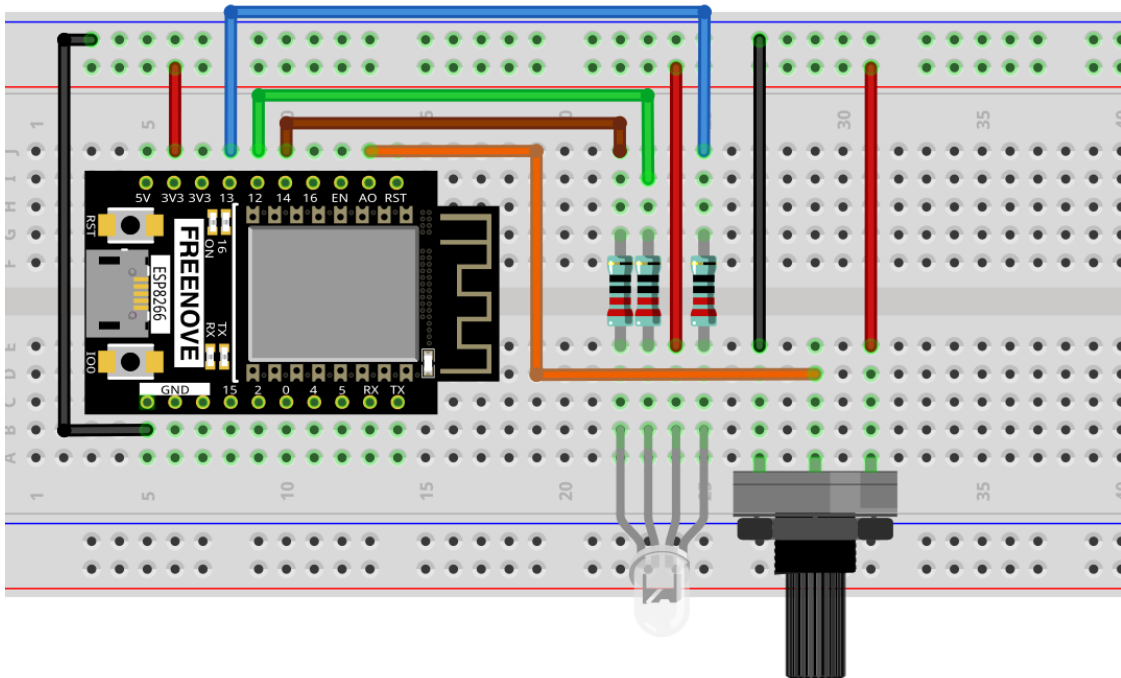
ESP8266 x1 	USB cable 		
Breadboard x1 			
Rotary potentiometer x1 	Resistor 220Ω x3 	RGBLED x1 	Jumper wire M/M x9 

Circuit

Schematic diagram

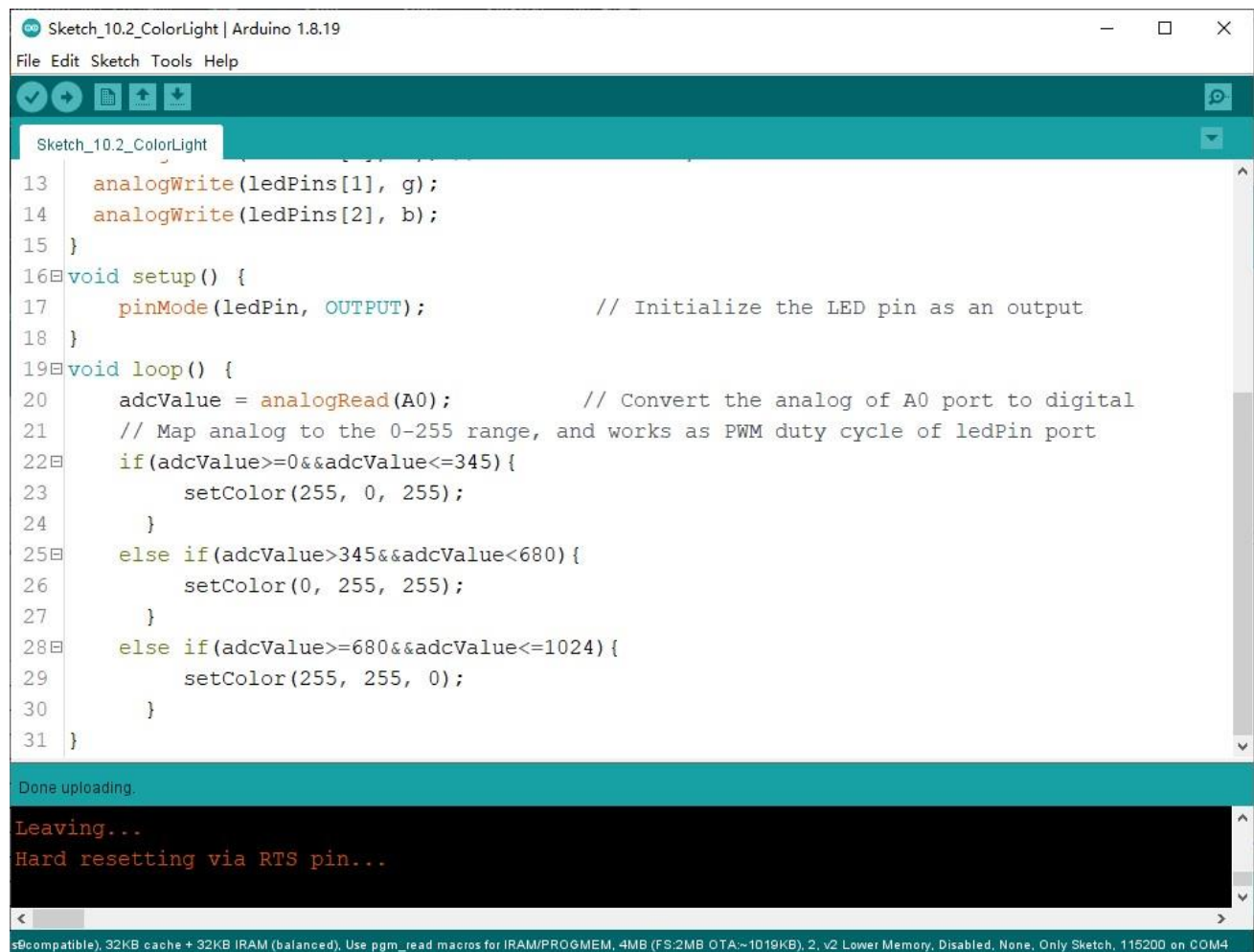


Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Sketch

Sketch_09.2_ColorLight



```
Sketch_10.2_ColorLight | Arduino 1.8.19
File Edit Sketch Tools Help

Sketch_10.2_ColorLight

13  analogWrite(ledPins[1], g);
14  analogWrite(ledPins[2], b);
15  }
16  void setup() {
17      pinMode(ledPin, OUTPUT);          // Initialize the LED pin as an output
18  }
19  void loop() {
20      adcValue = analogRead(A0);         // Convert the analog of A0 port to digital
21      // Map analog to the 0-255 range, and works as PWM duty cycle of ledPin port
22      if(adcValue>=0&&adcValue<=345){
23          setColor(255, 0, 255);
24      }
25      else if(adcValue>345&&adcValue<680){
26          setColor(0, 255, 255);
27      }
28      else if(adcValue>=680&&adcValue<=1024){
29          setColor(255, 255, 0);
30      }
31  }

Done uploading.

Leaving...
Hard resetting via RTS pin...

s9compatible), 32KB cache + 32KB IRAM (balanced). Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA~1019KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4
```

Download the code to ESP8266, rotate the potentiometers, then the color of RGB LED will change.

If you have any concerns, please contact us via: support@freenove.com

The following is the program code:

```
1  int adcValue;      // Define a variable to save the ADC value
2  int ledPin = 4;
3  const byte ledPins[] = {13, 14, 12};    //define red, green, blue led pins
4  int red, green, blue;
5  void setColor(byte r, byte g, byte b) {
6      analogWrite(ledPins[0], r); //Common anode LED, low level to turn on the led.
7      analogWrite(ledPins[1], g);
8      analogWrite(ledPins[2], b);
9  }
10 void setup() {
11     pinMode(ledPin, OUTPUT);           // Initialize the LED pin as an output
12 }
13 void loop() {
14     adcValue = analogRead(A0);          // Convert the analog of A0 port to digital
15     // Map analog to the 0-255 range, and works as PWM duty cycle of ledPin port
16     if(adcValue>=0&&adcValue<=345) {
17         setColor(255, 0, 255);
18     }
19     else if(adcValue>345&&adcValue<680) {
20         setColor(0, 255, 255);
21     }
22     else if(adcValue>=680&&adcValue<=1024) {
23         setColor(255, 255, 0);
24     }
25 }
```

In the code, you can read the potentiometer ADC value, judge the range of ADC value, to control the RGB LED color.

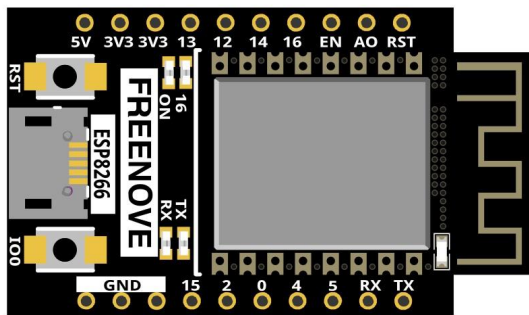
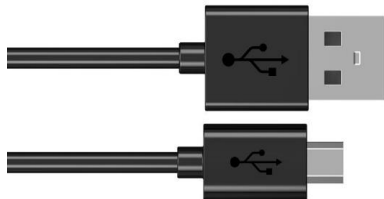
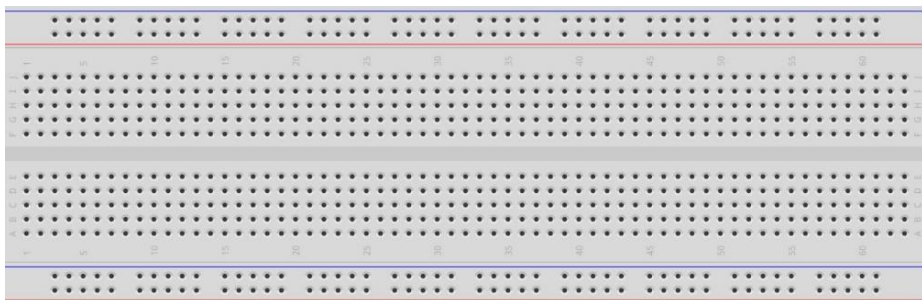
Chapter 10 Photoresistor & LED

In this chapter, we will learn how to use a photoresistor.

Project 10.1 NightLamp

A photoresistor is very sensitive to the amount of light present. We can take advantage of the characteristic to make a nightlight with the following function: when the ambient light is less (darker environment) the LED will automatically become brighter to compensate and when the ambient light is greater (brighter environment) the LED will automatically dim to compensate.

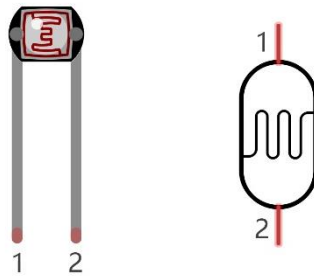
Component List

ESP8266 x1		USB cable	
			
Breadboard x1			
			
Photoresistor x1	Resistor		LED x1
	220Ω x1	10KΩ x1	
			
	Jumper wire M/M x7		
			

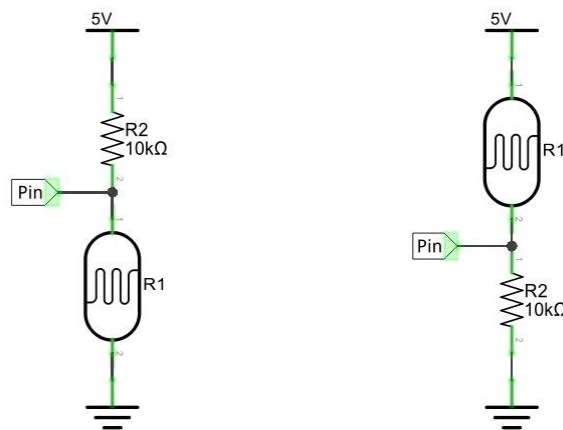
Component knowledge

Photoresistor

A photoresistor is simply a light sensitive resistor. It is an active component that decreases resistance with respect to receiving luminosity (light) on the component's light sensitive surface. A photoresistor's resistance value will change in proportion to the ambient light detected. With this characteristic, we can use a photoresistor to detect light intensity. The photoresistor and its electronic symbol are as follows.



The circuit below is used to detect the change of a photoresistor's resistance value:

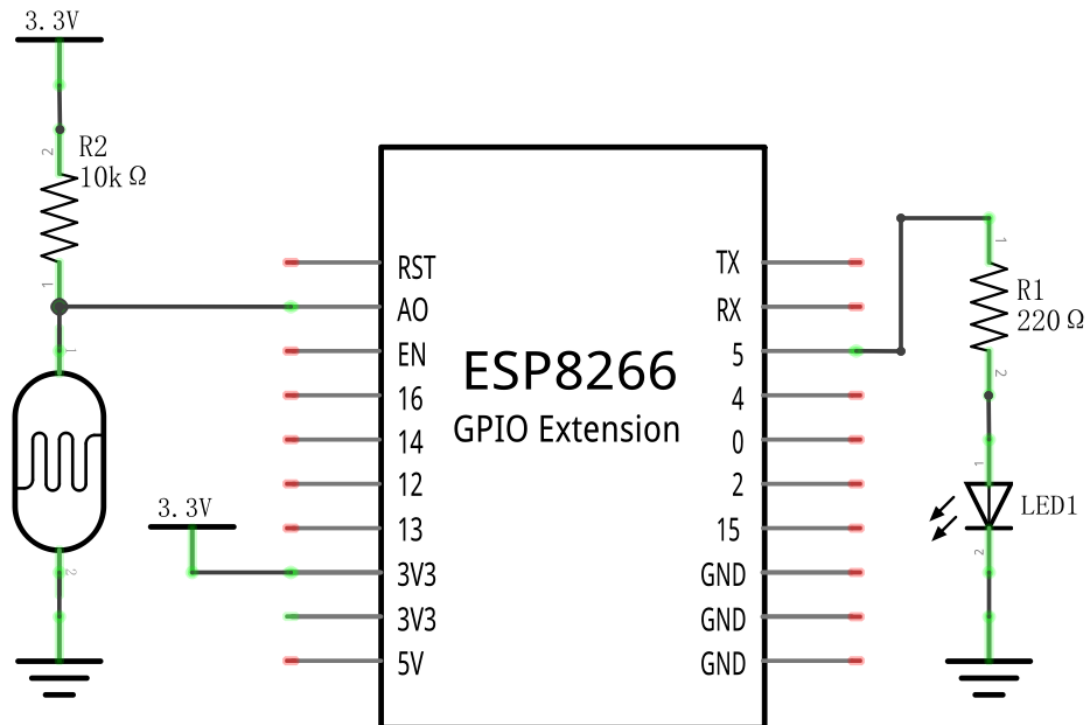


In the above circuit, when a photoresistor's resistance value changes due to a change in light intensity, the voltage between the photoresistor and resistor R1 will also change. Therefore, the intensity of the light can be obtained by measuring this voltage.

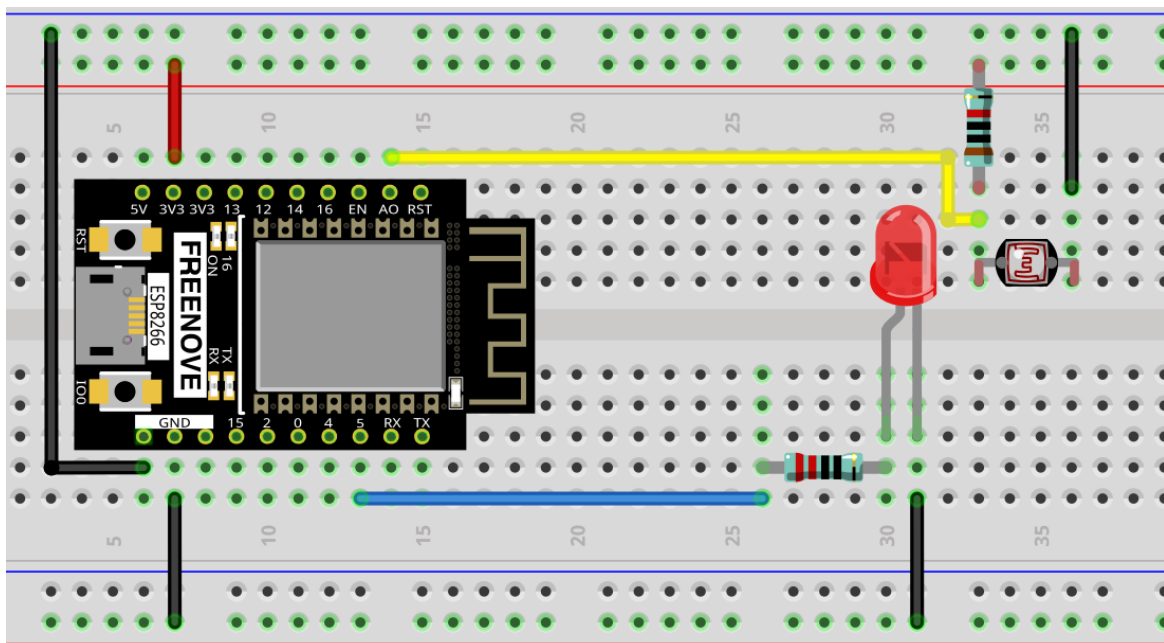
Circuit

The circuit of this project is similar to project Soft Light. The only difference is that the input signal is changed from a potentiometer to a combination of a photoresistor and a resistor.

Schematic diagram



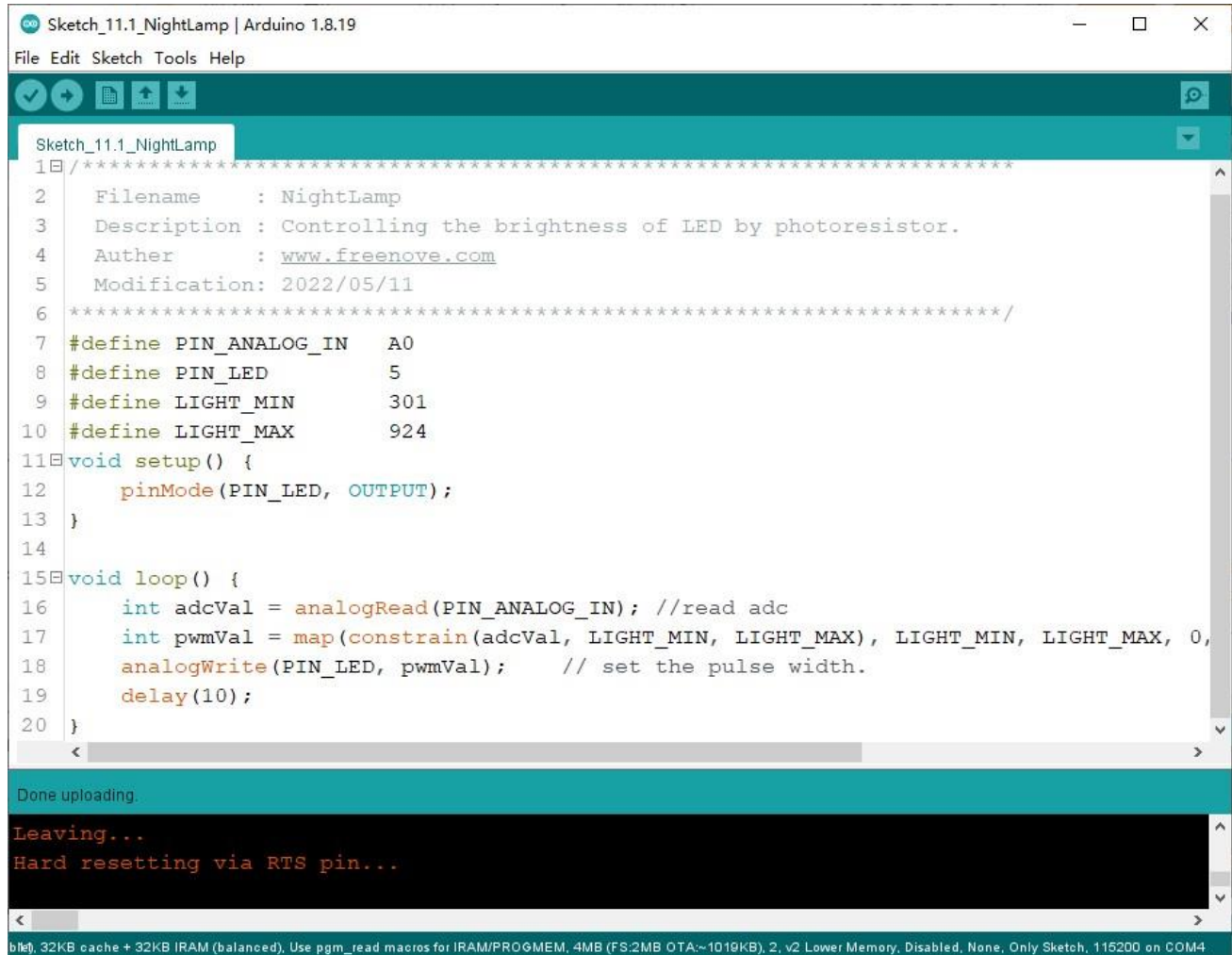
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Sketch

The circuit used is similar to the project Soft Light. The only difference is that the input signal of the A0 pin of ADC changes from a potentiometer to a combination of a photoresistor and a resistor.

Sketch_10.1_Nightlamp



```
Sketch_11.1_NightLamp | Arduino 1.8.19
File Edit Sketch Tools Help

Sketch_11.1_NightLamp
1 /*****
2  Filename   : NightLamp
3  Description : Controlling the brightness of LED by photoresistor.
4  Author    : www.freenove.com
5  Modification: 2022/05/11
6  *****/
7 #define PIN_ANALOG_IN  A0
8 #define PIN_LED        5
9 #define LIGHT_MIN      301
10 #define LIGHT_MAX      924
11 void setup() {
12     pinMode(PIN_LED, OUTPUT);
13 }
14
15 void loop() {
16     int adcVal = analogRead(PIN_ANALOG_IN); //read adc
17     int pwmVal = map(constrain(adcVal, LIGHT_MIN, LIGHT_MAX), LIGHT_MIN, LIGHT_MAX, 0,
18     analogWrite(PIN_LED, pwmVal); // set the pulse width.
19     delay(10);
20 }

Done uploading.
Leaving...
Hard resetting via RTS pin...

blat, 32KB cache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA:~1019KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4
```

Download the code to ESP8266, if you cover the photoresistor or increase the light shining on it, the brightness of the LED changes accordingly.

If you have any concerns, please contact us via: support@freenove.com

Any concerns? ✉ support@freenove.com

The following is the program code:

```
1  #define PIN_ANALOG_IN  A0
2  #define PIN_LED        5
3  #define LIGHT_MIN      301
4  #define LIGHT_MAX      924
5  void setup() {
6      pinMode(PIN_LED, OUTPUT);
7  }
8
9  void loop() {
10     int adcVal = analogRead(PIN_ANALOG_IN); //read adc
11     int pwmVal = map(constrain(adcVal, LIGHT_MIN, LIGHT_MAX), LIGHT_MIN, LIGHT_MAX, 0, 1023);
12     // adcVal re-map to pwmVal
13     analogWrite(PIN_LED, pwmVal);    // set the pulse width.
14     delay(10);
15 }
```

Reference

constrain(amt, low, high)

```
#define constrain(amt, low, high) ((amt)<(low)? (low):((amt)>(high)? (high):(amt)))
```

Constrain the value amt between low and high.

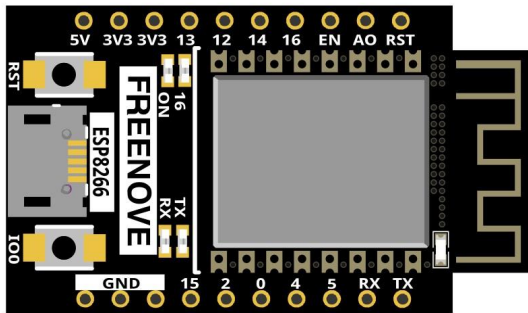
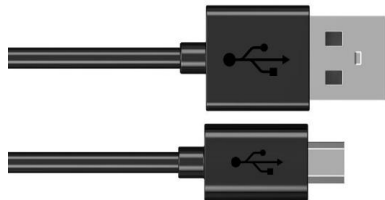
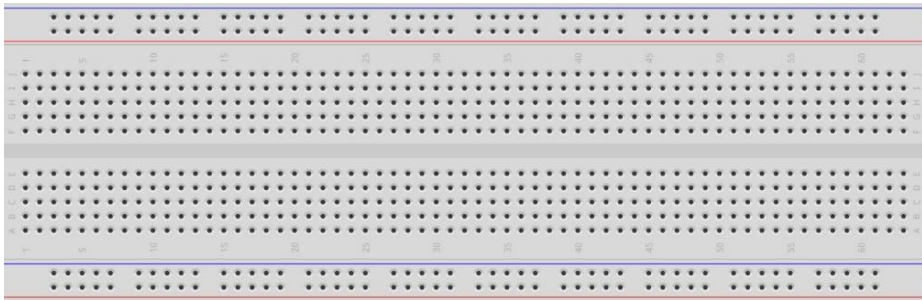
Chapter 11 Thermistor

In this chapter, we will learn about thermistors which are another kind of resistor

Project 11.1 Thermometer

A thermistor is a type of resistor whose resistance value is dependent on temperature and changes in temperature. Therefore, we can take advantage of this characteristic to make a thermometer.

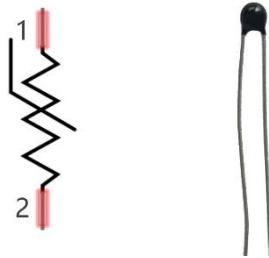
Component List

ESP8266 x1 	USB cable 	
Breadboard x1 		
Thermistor x1 	Resistor 10kΩ x1 	Jumper wire M/M x4 

Component knowledge

Thermistor

A thermistor is a temperature sensitive resistor. When it senses a change in temperature, the resistance of the thermistor will change. We can take advantage of this characteristic by using a thermistor to detect temperature intensity. A thermistor and its electronic symbol are shown below.



The relationship between resistance value and temperature of a thermistor is:

$$R_t = R * \text{EXP} \left[B * \left(\frac{1}{T_2} - \frac{1}{T_1} \right) \right]$$

Where:

R_t is the thermistor resistance under T_2 temperature;

R is the nominal resistance of thermistor under T_1 temperature;

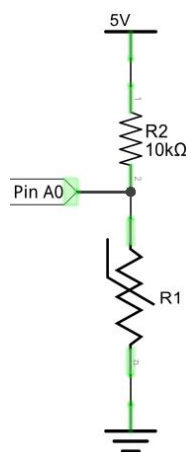
$\text{EXP}[n]$ is n th power of E ;

B is for thermal index;

T_1, T_2 is Kelvin temperature (absolute temperature). Kelvin temperature = 273.15 + Celsius temperature.

For the parameters of the thermistor, we use: $B=3950$, $R=10k$, $T_1=25$.

The circuit connection method of the thermistor is similar to photoresistor, as the following:



We can use the value measured by the ADC converter to obtain the resistance value of thermistor, and then we can use the formula to obtain the temperature value.

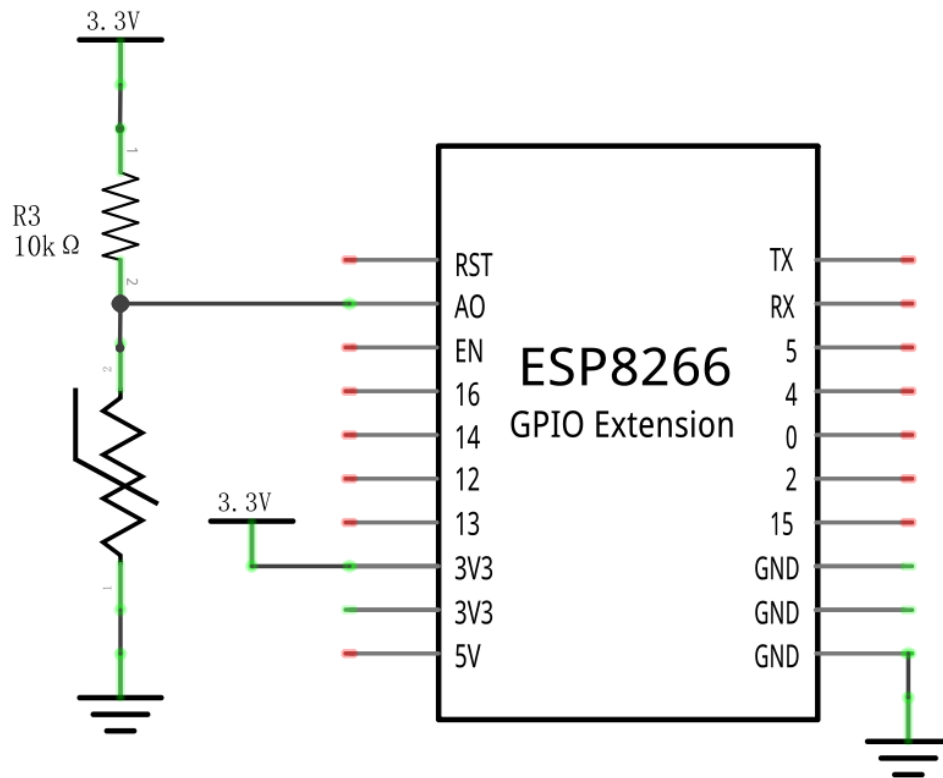
Therefore, the temperature formula can be derived as:

$$T_2 = 1 / \left(\frac{1}{T_1} + \ln \left(\frac{R_t}{R} \right) / B \right)$$

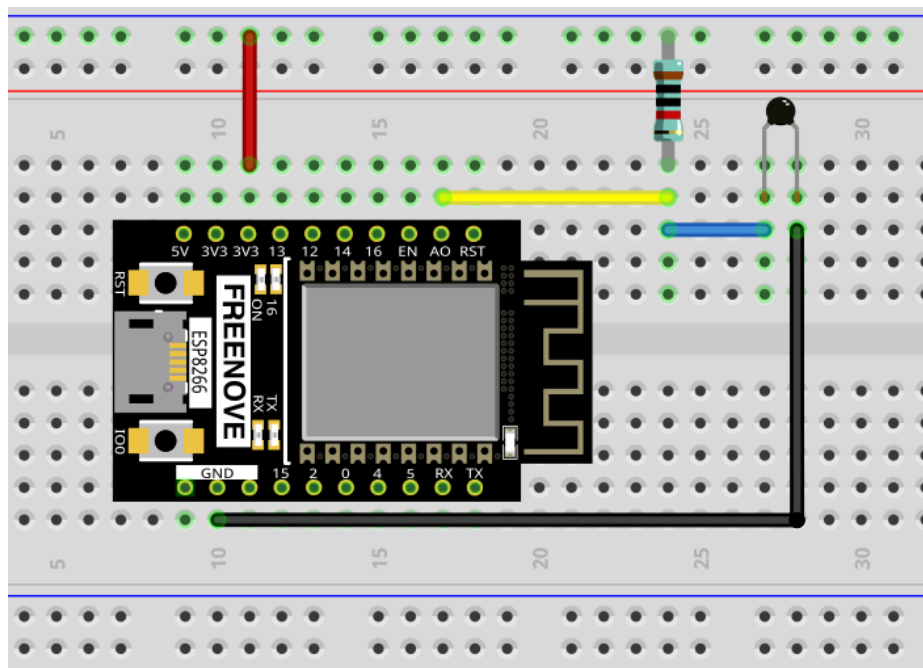
Circuit

The circuit of this project is similar to the one in the last chapter. The only difference is that the photoresistor is replaced by the thermistor.

Schematic diagram



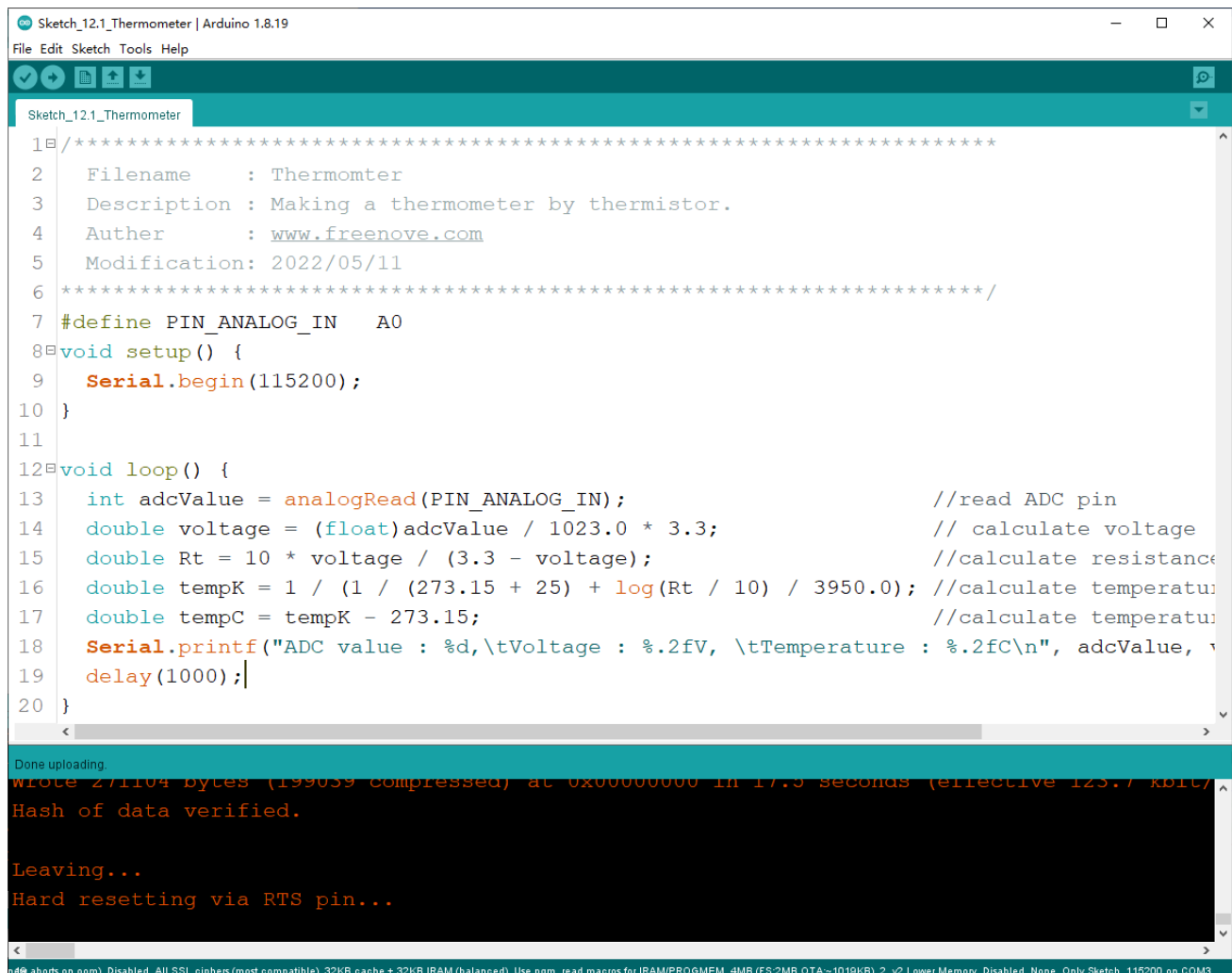
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

Sketch

Sketch_11.1_Thermometer



```

Sketch_12.1_Thermometer | Arduino 1.8.19
File Edit Sketch Tools Help

Sketch_12.1_Thermometer

1 //*****
2   Filename      : Thermomter
3   Description   : Making a thermometer by thermistor.
4   Auther       : www.freenove.com
5   Modification  : 2022/05/11
6   *****/
7 #define PIN_ANALOG_IN  A0
8 void setup() {
9     Serial.begin(115200);
10 }
11
12 void loop() {
13     int adcValue = analogRead(PIN_ANALOG_IN);           //read ADC pin
14     double voltage = (float)adcValue / 1023.0 * 3.3;    // calculate voltage
15     double Rt = 10 * voltage / (3.3 - voltage);         //calculate resistance
16     double tempK = 1 / (1 / (273.15 + 25) + log(Rt / 10) / 3950.0); //calculate temperature
17     double tempC = tempK - 273.15;                     //calculate temperature
18     Serial.printf("ADC value : %d,\tVoltage : %.2fV, \tTemperature : %.2fC\n", adcValue, voltage, tempC);
19     delay(1000);
20 }

Done uploading.
Wrote 271104 bytes (199039 compressed) at 0x00000000 in 17.3 seconds (effective 123.7 Kbit/s)
Hash of data verified.

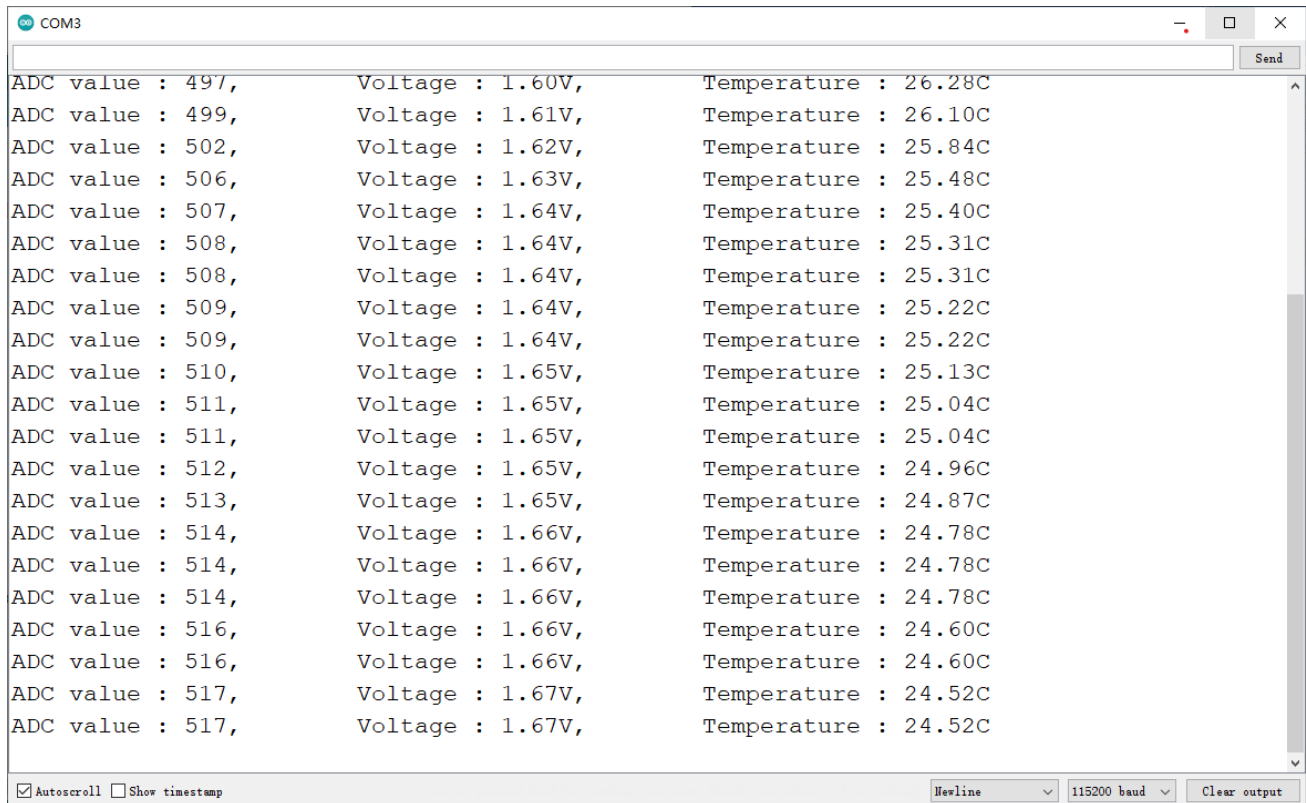
Leaving...
Hard resetting via RTS pin...

avr: aborts on com). Disabled, All SSL ciphers (most compatible), 32KB cache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA~1019KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM3

```

Download the code to ESP8266, the terminal window will display the current ADC value, voltage value and temperature value. Try to “pinch” the thermistor (without touching the leads) with your index finger and thumb for a brief time, you should see that the temperature value increases.

If you have any concerns, please contact us via: support@freenove.com



ADC value	Voltage	Temperature
497,	1.60V,	26.28C
499,	1.61V,	26.10C
502,	1.62V,	25.84C
506,	1.63V,	25.48C
507,	1.64V,	25.40C
508,	1.64V,	25.31C
508,	1.64V,	25.31C
509,	1.64V,	25.22C
509,	1.64V,	25.22C
510,	1.65V,	25.13C
511,	1.65V,	25.04C
511,	1.65V,	25.04C
512,	1.65V,	24.96C
513,	1.65V,	24.87C
514,	1.66V,	24.78C
514,	1.66V,	24.78C
514,	1.66V,	24.78C
516,	1.66V,	24.60C
516,	1.66V,	24.60C
517,	1.67V,	24.52C
517,	1.67V,	24.52C

The following is the code:

```
1  #define PIN_ANALOG_IN  A0
2  void setup() {
3      Serial.begin(115200);
4  }
5
6  void loop() {
7      int adcValue = analogRead(PIN_ANALOG_IN);           //read ADC pin
8      double voltage = (float)adcValue / 1023.0 * 3.3;      // calculate voltage
9      double Rt = 10 * voltage / (3.3 - voltage);          //calculate resistance value of thermistor
10     double tempK = 1 / (1/(273.15 + 25) + log(Rt / 10)/3950.0); //calculate temperature (Kelvin)
11     double tempC = tempK - 273.15;                      //calculate temperature (Celsius)
12     Serial.printf("ADC value : %d, \tVoltage : %.2fV, \tTemperature : %.2fC\n", adcValue,
13                   voltage, tempC);
14     delay(1000);
15 }
```

In the code, the ADC value of ADC module A0 port is read, and then calculates the voltage and the resistance of thermistor according to Ohms Law. Finally, it calculates the temperature sensed by the thermistor, according to the formula.

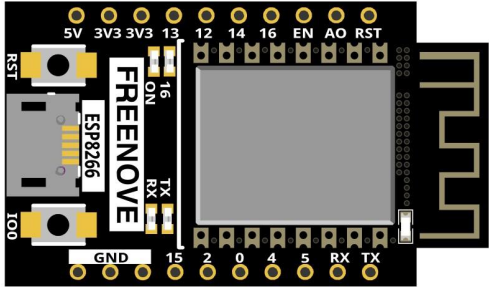
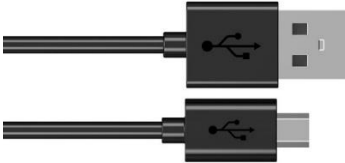
Chapter 12 WiFi Working Modes

In this chapter, we'll focus on the WiFi infrastructure for ESP8266.

ESP8266 has 3 different WiFi operating modes: station mode, AP mode and AP+station mode. All WiFi programming projects must be configured with WiFi operating mode before using WiFi, otherwise WiFi cannot be used.

Project 12.1 Station mode

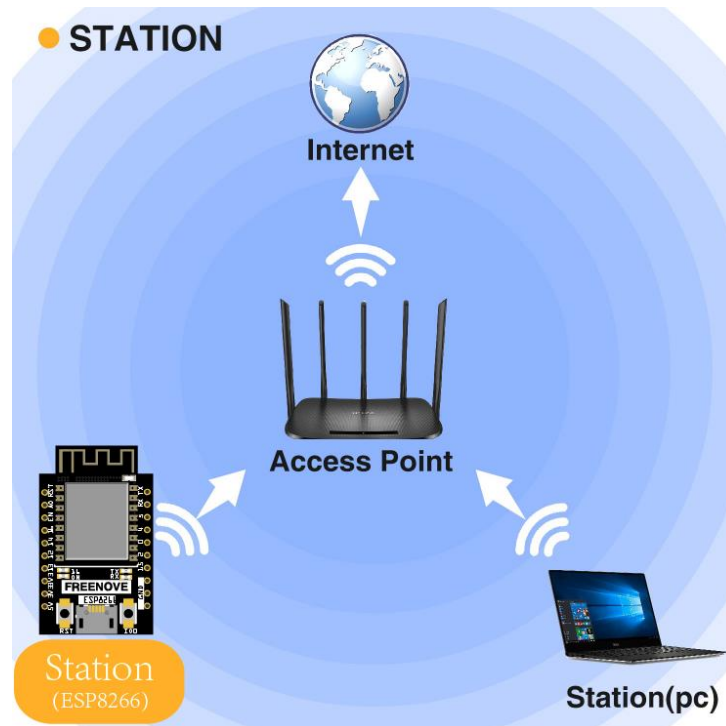
Component List

ESP8266 x1  A black ESP8266 module with a white label that says 'FREENOVE'. It has various pins labeled: 5V, 3V3, 3V3, 13, 12, 14, 16, EN, AO, RST, TX, RX, GND, 15, 2, 0, 4, 5, RX, TX. There is also a small antenna on the right side.	Micro USB Wire x1  A black Micro USB cable with a standard USB-A connector on one end and a Micro USB-B connector on the other.
--	---

Component knowledge

Station mode

When ESP8266 selects Station mode, it acts as a WiFi client. It can connect to the router network and communicate with other devices on the router via WiFi connection. As shown below, the PC is connected to the router, and if ESP8266 wants to communicate with the PC, it needs to be connected to the router.



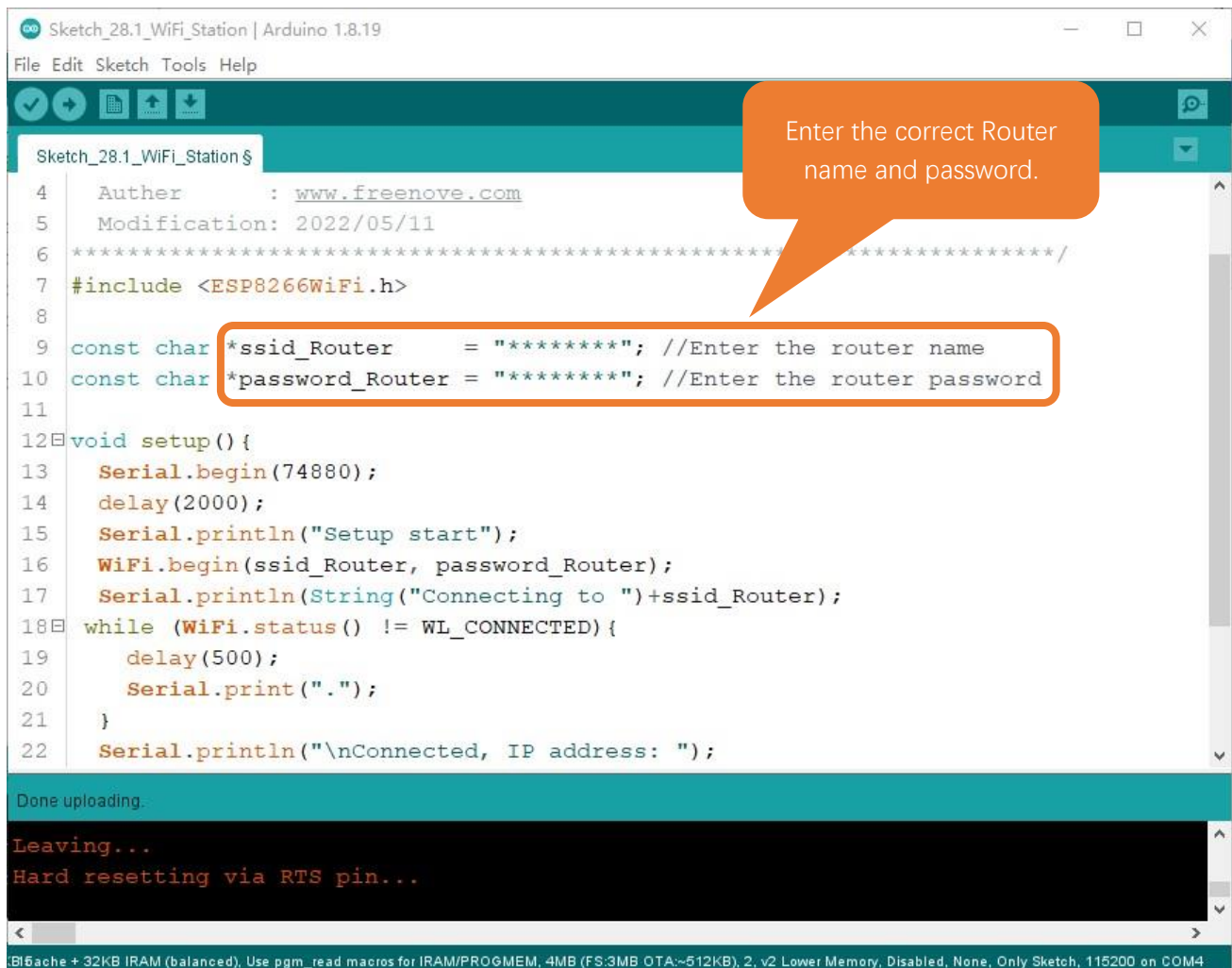
Circuit

Connect Freenove ESP8266 to the computer using the USB cable.



Sketch

Sketch_12.1_Station_mode



```

Sketch_28.1_WiFi_Station | Arduino 1.8.19
File Edit Sketch Tools Help

Sketch_28.1_WiFi_Station $
4   Author      : www.freenove.com
5   Modification: 2022/05/11
6   *****/
7   #include <ESP8266WiFi.h>
8
9   const char *ssid_Router    = "*****"; //Enter the router name
10  const char *password_Router = "*****"; //Enter the router password
11
12  void setup() {
13    Serial.begin(74880);
14    delay(2000);
15    Serial.println("Setup start");
16    WiFi.begin(ssid_Router, password_Router);
17    Serial.println(String("Connecting to ") + ssid_Router);
18    while (WiFi.status() != WL_CONNECTED) {
19      delay(500);
20      Serial.print(".");
21    }
22    Serial.println("\nConnected, IP address: ");
  
```

Done uploading.

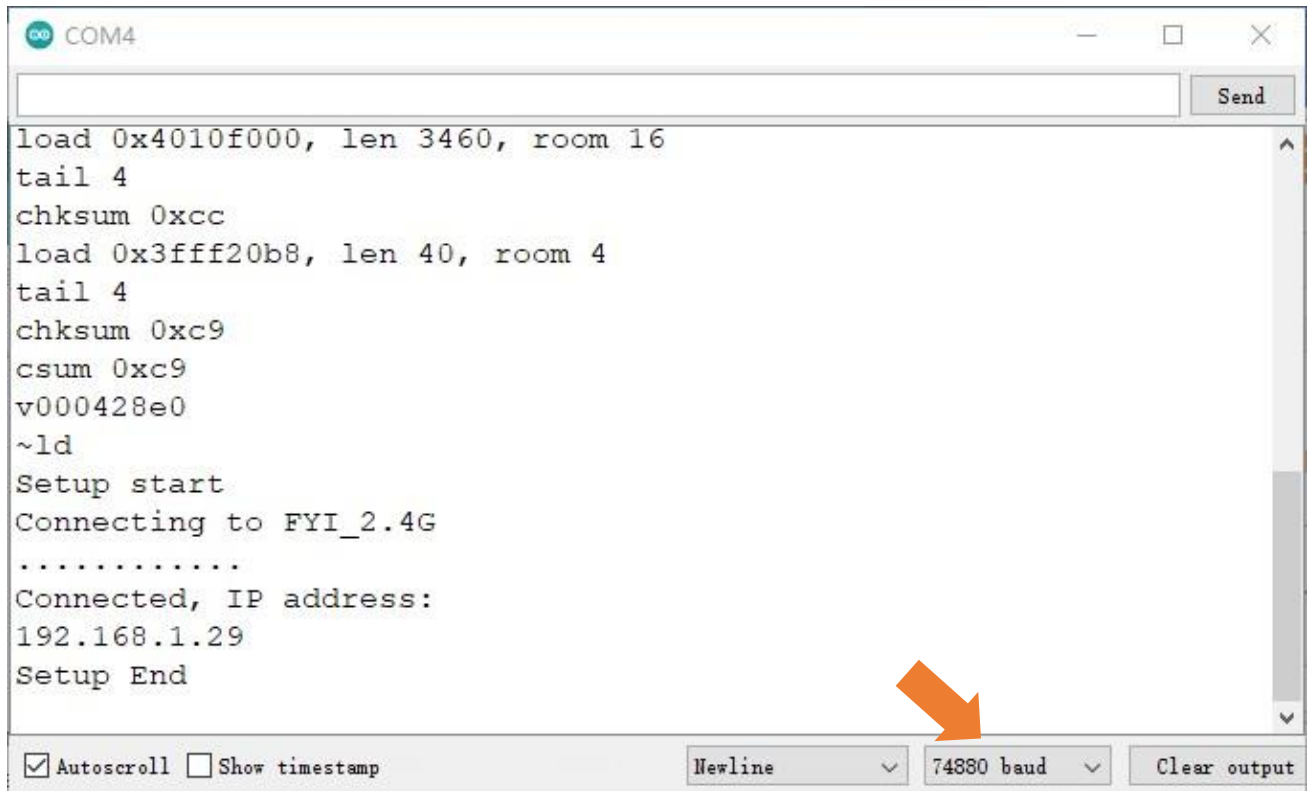
Leaving...

Hard resetting via RTS pin...

BISScache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:3MB OTA:~512KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4

Because the names and passwords of routers in various places are different, before the Sketch runs, users need to enter the correct router's name and password in the box as shown in the illustration above.

After making sure the router name and password are entered correctly, compile and upload codes to ESP8266, open serial monitor and set baud rate to 74880. And then it will display as follows:



```

load 0x4010f000, len 3460, room 16
tail 4
chksum 0xcc
load 0x3fff20b8, len 40, room 4
tail 4
chksum 0xc9
csum 0xc9
v000428e0
~ld
Setup start
Connecting to FYI_2.4G
.....
Connected, IP address:
192.168.1.29
Setup End

```

☒ Autoscroll
 ☐ Show timestamp
 Newline
 74880 baud
 Clear output

When ESP8266 successfully connects to "ssid_Router", serial monitor will print out the IP address assigned to ESP8266 by the router.

The following is the program code:

```

1  #include <ESP8266WiFi.h>
2
3  const char *ssid_Router    = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password
5
6  void setup() {
7      Serial.begin(74880);
8      delay(2000);
9      Serial.println("Setup start");
10     WiFi.begin(ssid_Router, password_Router);
11     Serial.println(String("Connecting to ") + ssid_Router);
12     while (WiFi.status() != WL_CONNECTED) {
13         delay(500);
14         Serial.print(".");
15     }
16     Serial.println("\nConnected, IP address: ");
17     Serial.println(WiFi.localIP());
18     Serial.println("Setup End");
19 }
20
21 void loop() {

```

```
22 }
```

Include the WiFi Library header file of ESP8266.

```
1 #include <ESP8266WiFi.h>
```

Enter correct router name and password.

```
3 const char *ssid_Router = "*****"; //Enter the router name
```

```
4 const char *password_Router = "*****"; //Enter the router password
```

Set ESP8266 in Station mode and connect it to your router.

```
10 WiFi.begin(ssid_Router, password_Router);
```

Check whether ESP8266 has connected to router successfully every 0.5s.

```
12 while (WiFi.status() != WL_CONNECTED) {
13     delay(500);
14     Serial.print(".");
15 }
```

Serial monitor prints out the IP address assigned to ESP8266

```
17 Serial.println(WiFi.localIP());
```

Reference

Class Station

Every time when using WiFi, you need to include header file "ESP8266WiFi.h".

begin(ssid, password, channel, bssid, connect): ESP8266 is used as Station to connect hotspot.

ssid: WiFi hotspot name

password: WiFi hotspot password

channel: WiFi hotspot channel number; communicating through specified channel; optional parameter

bssid: mac address of WiFi hotspot, optional parameter

connect: boolean optional parameter, defaulting to true. If set as false, then ESP8266 won't connect WiFi.

config(local_ip, gateway, subnet, dns1, dns2): set static local IP address.

local_ip: station fixed IP address.

subnet: subnet mask

dns1, dns2: optional parameter. define IP address of domain name server

status: obtain the connection status of WiFi

local IP(): obtain IP address in Station mode

disconnect(): disconnect wifi

setAutoConnect(boolean): set automatic connection Every time ESP8266 is power on, it will connect WiFi automatically.

setAutoReconnect(boolean): set automatic reconnection Every time ESP8266 disconnects WiFi, it will reconnect to WiFi automatically.

Project 12.2 AP mode

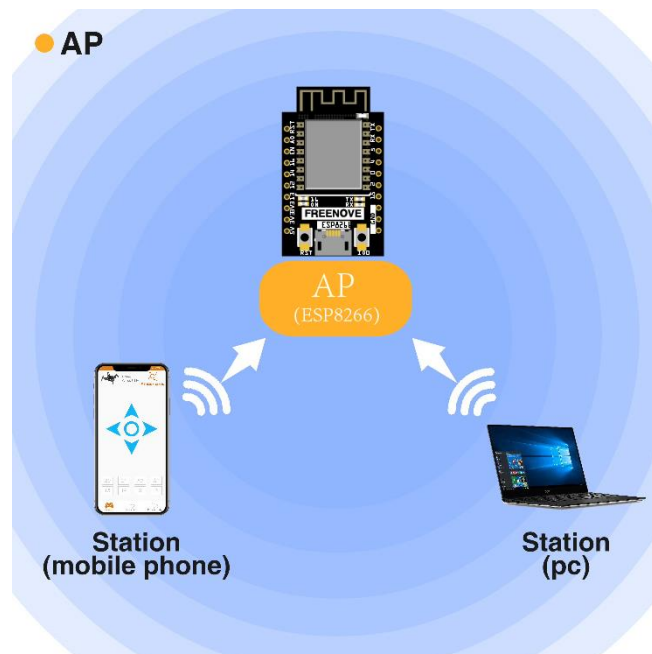
Component List & Circuit

Component List & Circuit are the same as in Section 28.1.

Component knowledge

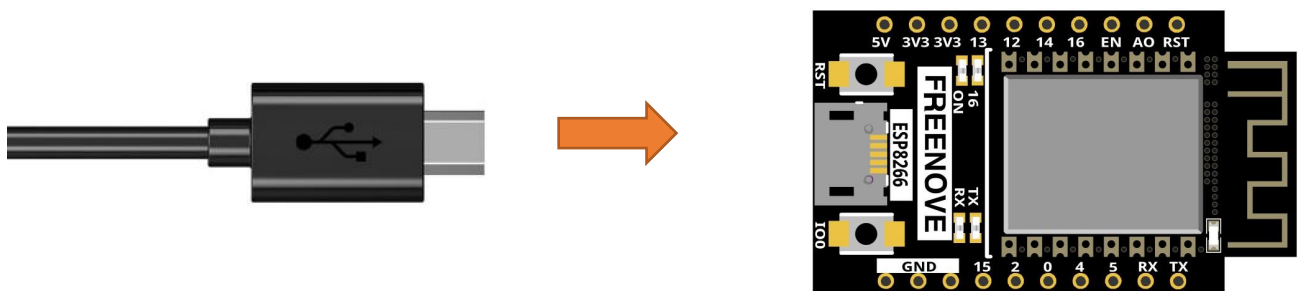
AP mode

When ESP8266 selects AP mode, it creates a hotspot network that is separate from the Internet and waits for other WiFi devices to connect. As shown in the figure below, ESP8266 is used as a hotspot. If a mobile phone or PC wants to communicate with ESP8266, it must be connected to the hotspot of ESP8266. Only after a connection is established with ESP8266 can they communicate.



Circuit

Connect Freenove ESP8266 to the computer using the USB cable.



Sketch

Sketch_12.2_AP_mode

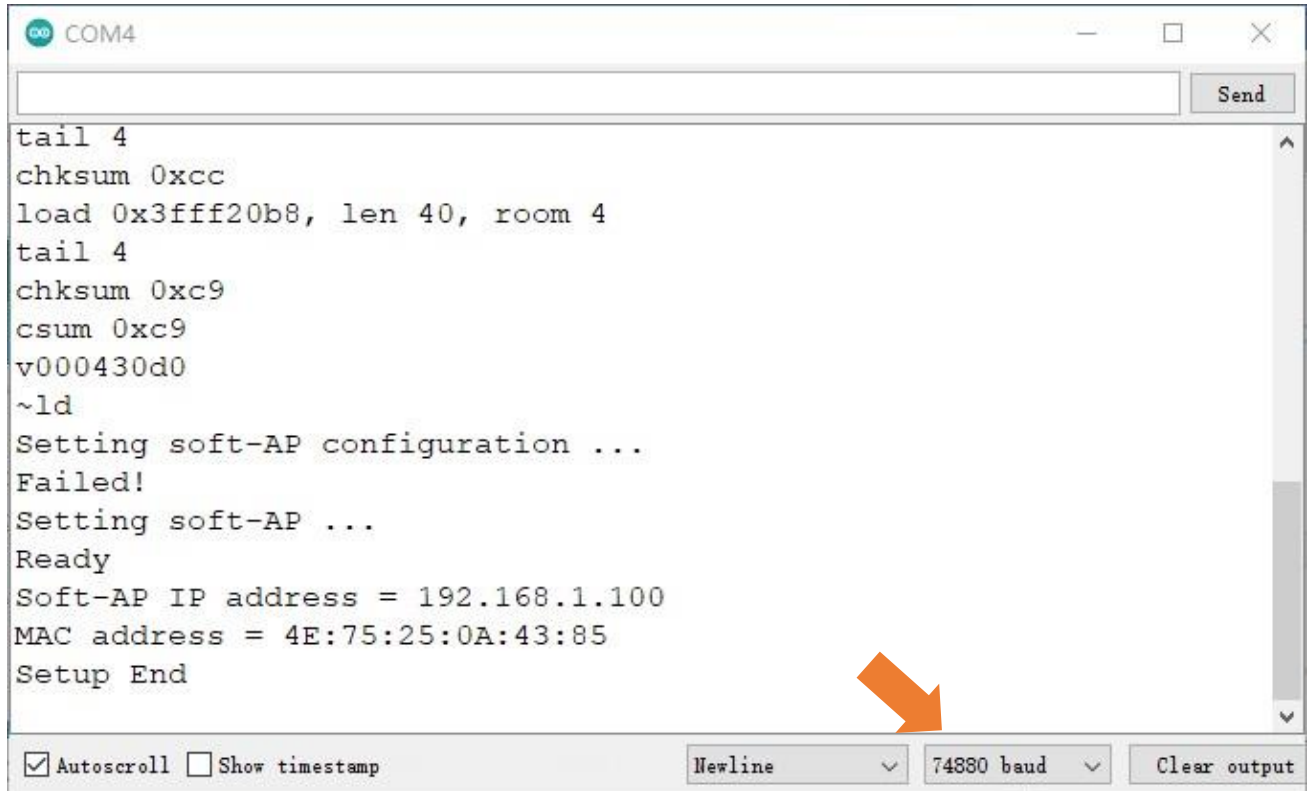


```
Sketch_28.2_WiFi_AP | Arduino 1.8.19
File Edit Sketch Tools Help

Sketch_28.2_WiFi_AP
1 /*****
2  Filename      : WiFi AP
3  Description   : Set ESP8
4  Auther       : www.free
5  Modification: 2022/05/
6  *****/
7 #include <ESP8266WiFi.h>
8
9 const char *ssid_AP    = "WiFi_Name"; //Enter the router name
10 const char *password_AP = "12345678"; //Enter the router password
11
12 IPAddress local_IP(192,168,1,100); //Set the IP address of ESP8266 itself
13 IPAddress gateway(192,168,1,10);   //Set the gateway of ESP8266 itself
14 IPAddress subnet(255,255,255,0);   //Set the subnet mask for ESP32 itself
15
16 void setup() {
17     Serial.begin(74880);
18     delay(2000);
19 }
20
21 Done uploading.
22 Leaving...
23 Hard resetting via RTS pin...
24
25 BIZache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:3MB OTA:~512KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4
```

Before the Sketch runs, you can make any changes to the AP name and password for ESP8266 in the box as shown in the illustration above. Of course, you can leave it alone by default.

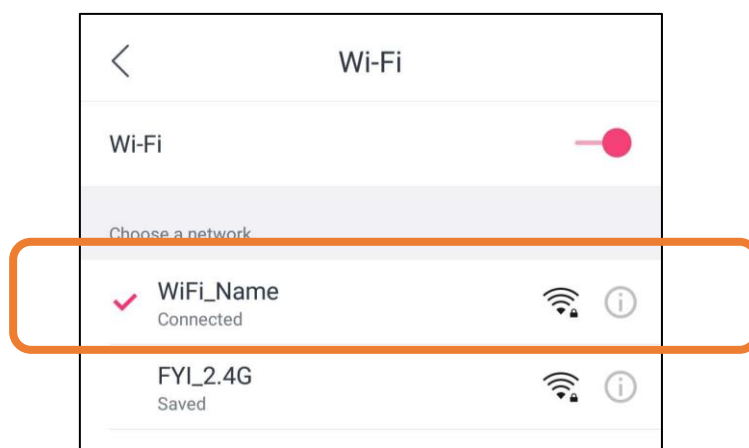
Compile and upload codes to ESP8266, open the serial monitor and set the baud rate to 74880. And then it will display as follows.



```
COM4
tail 4
checksum 0xcc
load 0x3fff20b8, len 40, room 4
tail 4
checksum 0xc9
csum 0xc9
v000430d0
~ld
Setting soft-AP configuration ...
Failed!
Setting soft-AP ...
Ready
Soft-AP IP address = 192.168.1.100
MAC address = 4E:75:25:0A:43:85
Setup End
```

Autoscroll Show timestamp Newline 74880 baud Clear output

When observing the print information of the serial monitor, turn on the WiFi scanning function of your phone, and you can see the ssid_AP on ESP8266, which is called "WiFi_Name" in this Sketch. You can enter the password "12345678" to connect it or change its AP name and password by modifying Sketch.



The following is the program code:

```

1  #include <ESP8266WiFi.h>
2
3  const char *ssid_AP      = "WiFi_Name"; //Enter the router name
4  const char *password_AP = "12345678"; //Enter the router password
5
6  IPAddress local_IP(192, 168, 1, 100); //Set the IP address of ESP8266 itself
7  IPAddress gateway(192, 168, 1, 10);   //Set the gateway of ESP8266 itself
8  IPAddress subnet(255, 255, 255, 0);  //Set the subnet mask for ESP8266 itself
9
10 void setup() {
11     Serial.begin(74880);
12     delay(2000);
13     Serial.println("Setting soft-AP configuration ... ");
14     WiFi.disconnect();
15     WiFi.mode(WIFI_AP);
16     Serial.println(WiFi.softAPConfig(local_IP, gateway, subnet) ? "Ready" : "Failed!");
17     Serial.println("Setting soft-AP ... ");
18     boolean result = WiFi.softAP(ssid_AP, password_AP);
19     if(result) {
20         Serial.println("Ready");
21         Serial.println(String("Soft-AP IP address = ") + WiFi.softAPIP().toString());
22         Serial.println(String("MAC address = ") + WiFi.softAPmacAddress().c_str());
23     } else {
24         Serial.println("Failed!");
25     }
26     Serial.println("Setup End");
27 }
28
29 void loop() {
30 }

```

Include WiFi Library header file of ESP8266.

```
1  #include <ESP8266WiFi.h>
```

Enter correct AP name and password.

```

3  const char *ssid_AP      = "WiFi_Name"; //Enter the router name
4  const char *password_AP = "12345678"; //Enter the router password

```

Set ESP8266 in AP mode.

```
15  WiFi.mode(WIFI_AP);
```

Configure IP address, gateway and subnet mask for ESP8266.

```
16  WiFi.softAPConfig(local_IP, gateway, subnet)
```

Turn on an AP in ESP8266, whose name is set by ssid_AP and password is set by password_AP.

```
18  WiFi.softAP(ssid_AP, password_AP);
```

Check whether the AP is turned on successfully. If yes, print out IP and MAC address of AP established by ESP8266. If no, print out the failure prompt.

```

19  if(result) {
20      Serial.println("Ready");
21      Serial.println(String("Soft-AP IP address = ") + WiFi.softAPIP().toString());
22      Serial.println(String("MAC address = ") + WiFi.softAPmacAddress().c_str());
23  } else {
24      Serial.println("Failed!");
25  }
26  Serial.println("Setup End");

```

Reference

Class AP

Every time when using WiFi, you need to include header file "ESP8266WiFi.h".

softAP(ssid, password, channel, ssid_hidden, max_connection):

ssid: WiFi hotspot name

password: WiFi hotspot password

channel: Number of WiFi connection channels, range 1-13. The default is 1.

ssid_hidden: Whether to hide WiFi name from scanning by other devices. The default is not hide.

max_connection: Maximum number of WiFi connected devices. The range is 1-4. The default is 4.

softAPConfig(local_ip, gateway, subnet): set static local IP address.

local_ip: station fixed IP address.

Gateway: gateway IP address

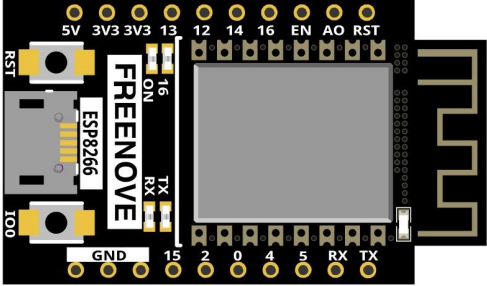
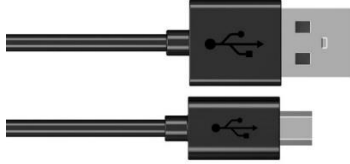
subnet: subnet mask

softAP(): obtain IP address in AP mode

softAPdisconnect (): disconnect AP mode.

Project 12.3 AP+Station mode

Component List

ESP8266 x1  A black ESP8266 module with a white label that reads 'FREENOVE' and 'ESP8266'. It features a USB port on the left, a micro-USB port on the right, and various pins labeled with numbers and functions like 5V, 3V3, GND, RX, TX, EN, AO, RST, and I/O.	Micro USB Wire x1  A black micro-USB cable with a standard USB-A connector on one end and a micro-USB-B connector on the other.
--	--

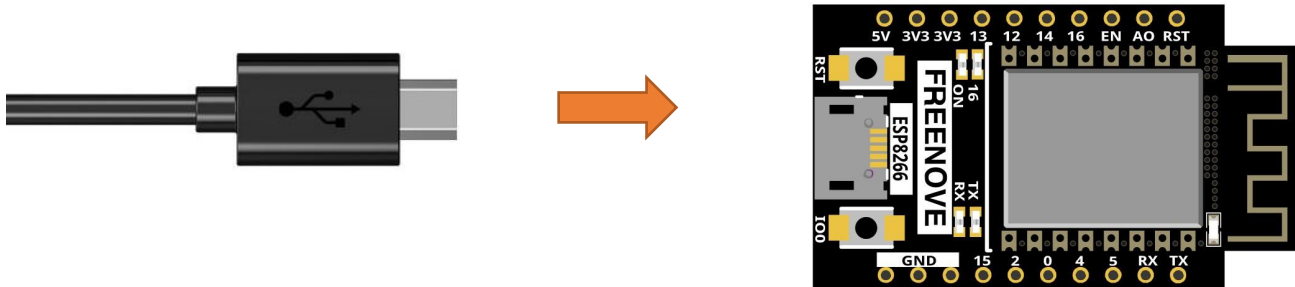
Component knowledge

AP+Station mode

In addition to AP mode and station mode, ESP8266 can also use AP mode and station mode at the same time. This mode contains the functions of the previous two modes. Turn on ESP8266's station mode, connect it to the router network, and it can communicate with the Internet via the router. At the same time, turn on its AP mode to create a hotspot network. Other WiFi devices can choose to connect to the router network or the hotspot network to communicate with ESP8266.

Circuit

Connect Freenove ESP8266 to the computer using the USB cable.



Sketch

Sketch_12.3_AP_Station_mode

```

Sketch_28.3_AP_Station | Arduino 1.8.19
File Edit Sketch Tools Help

Sketch_28.3_AP_Station $
7  #include <ESP8266WiFi.h>
8
9  const char *ssid_Router    = "*****"; //Enter the router name
10 const char *password_Router = "*****"; //Enter the router password
11 const char *ssid_AP        = "WiFi_Name"; //Enter the router name
12 const char *password_AP    = "12345678"; //Enter the router password
13
14 void setup() {
15   Serial.begin(74880);
16   Serial.println("Setting soft-AP configuration ... ");
17   WiFi.disconnect();
18   WiFi.mode(WIFI_AP);
19   Serial.println("Setting soft-AP ... ");
20   boolean result = WiFi.softAP(ssid_AP, password_AP);
21   if(result) {
22     Serial.println("Ready");
23     Serial.println(String("Soft-AP IP address = ") + WiFi.softAPIP().toString());
24     Serial.println(String("MAC address = ") + WiFi.softAPmacAddress().c_str());
25   }
26 }

Done uploading.

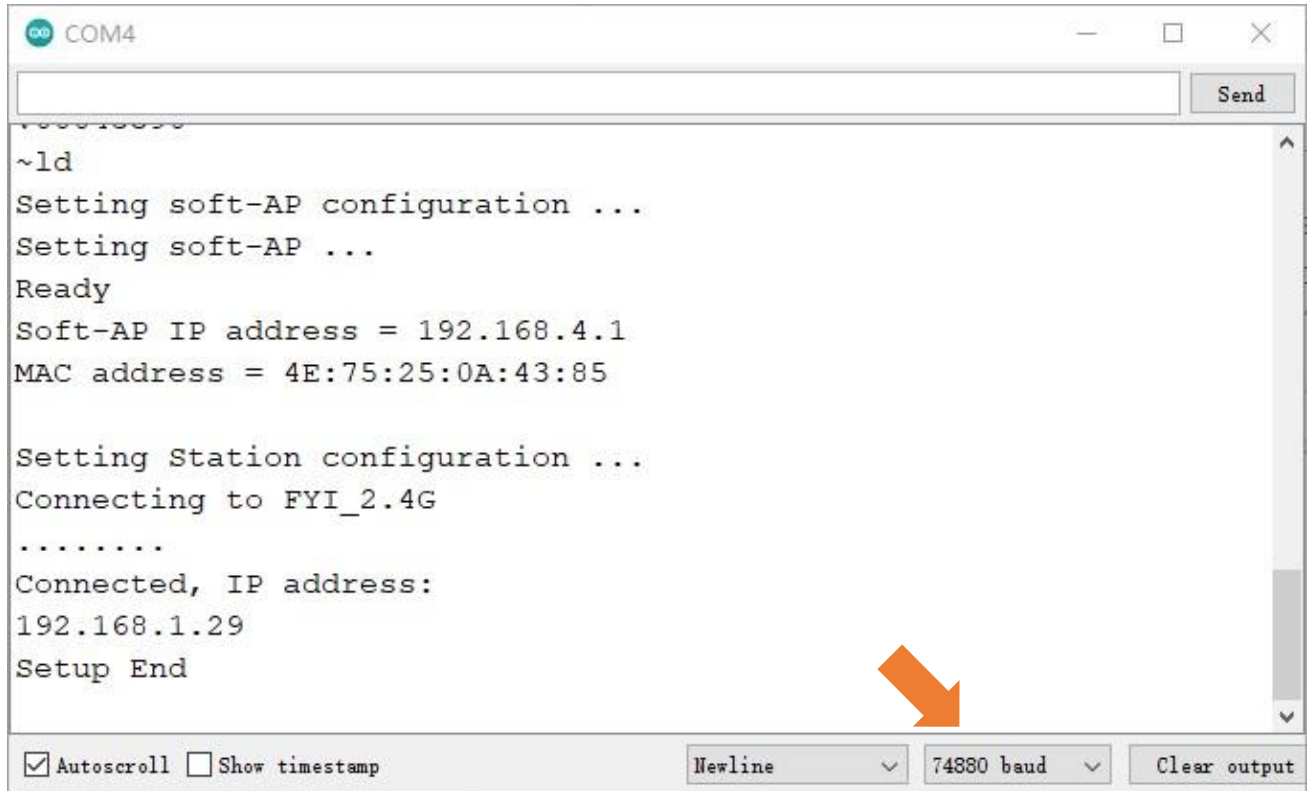
Leaving...
Hard resetting via RTS pin...

Cache + 32KB IRAM (balanced), Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:3MB OTA:~512KB), 2, v2 Lower Memory, Disabled, None, Only Sketch, 115200 on COM4

```

It is analogous to Project 12.1 and Project 12.2. Before running the Sketch, you need to modify `ssid_Router`, `password_Router`, `ssid_AP` and `password_AP` shown in the box of the illustration above.

After making sure that Sketch is modified correctly, compile and upload codes to ESP8266, open serial monitor and set baud rate to 74880. And then it will display as follows:



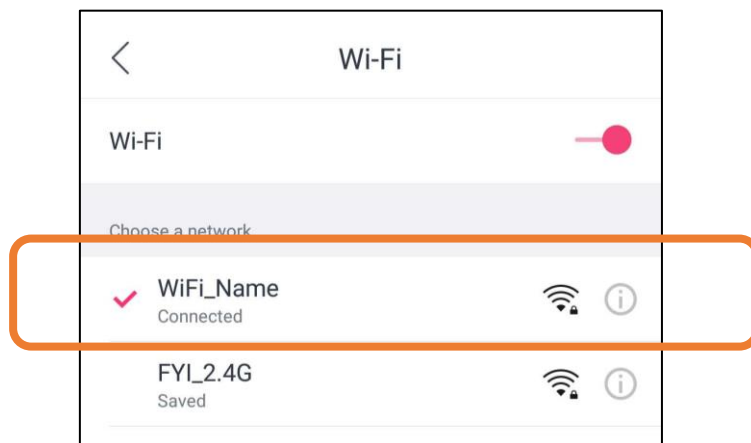
```

COM4
~ld
Setting soft-AP configuration ...
Setting soft-AP ...
Ready
Soft-AP IP address = 192.168.4.1
MAC address = 4E:75:25:0A:43:85

Setting Station configuration ...
Connecting to FYI_2.4G
.....
Connected, IP address:
192.168.1.29
Setup End
  
```

Autoscroll ☐ Show timestamp Newline 74880 baud Clear output

When observing the print information of the serial monitor, turn on the WiFi scanning function of your phone, and you can see the ssid_AP on ESP8266.



The following is the program code:

```

1  #include <ESP8266WiFi.h>
2
3  const char *ssid_Router    = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password
5  const char *ssid_AP        = "WiFi_Name"; //Enter the AP name
6  const char *password_AP    = "12345678"; //Enter the AP password
7
  
```

Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

```
8 void setup() {
9   Serial.begin(74880);
10  Serial.println("Setting soft-AP configuration ... ");
11  WiFi.disconnect();
12  WiFi.mode(WIFI_AP);
13  Serial.println("Setting soft-AP ... ");
14  boolean result = WiFi.softAP(ssid_AP, password_AP);
15  if(result) {
16    Serial.println("Ready");
17    Serial.println(String("Soft-AP IP address = ") + WiFi.softAPIP().toString());
18    Serial.println(String("MAC address = ") + WiFi.softAPmacAddress().c_str());
19  } else {
20    Serial.println("Failed!");
21  }
22
23  Serial.println("\nSetting Station configuration ... ");
24  WiFi.begin(ssid_Router, password_Router);
25  Serial.println(String("Connecting to ") + ssid_Router);
26  while (WiFi.status() != WL_CONNECTED) {
27    delay(500);
28    Serial.print(".");
29  }
30  Serial.println("\nConnected, IP address: ");
31  Serial.println(WiFi.localIP());
32  Serial.println("Setup End");
33 }
34
35 void loop() {
36 }
```

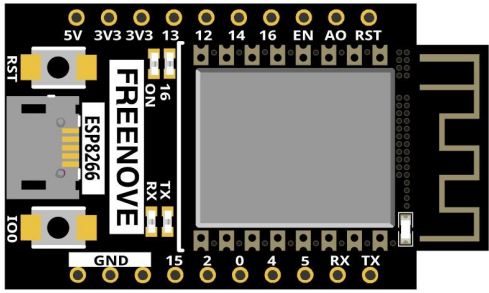
Chapter 13 TCP/IP

In this chapter, we will introduce how ESP8266 implements network communications based on TCP/IP protocol. There are two roles in TCP/IP communication, namely Server and Client, which will be implemented respectively with two projects in this chapter.

Project 13.1 As Client

In this section, ESP8266 is used as Client to connect Server on the same LAN and communicate with it.

Component List

ESP8266 x1  The image shows an ESP8266 module with a black PCB. It features a USB Type-C port on the left, a micro-USB port in the center, and a pin header on the right. The module is labeled 'ESP8266' and 'FREENOVE'. Various pins are labeled with numbers and functions like 5V, 3V3, GND, TX, RX, EN, AO, RST, and IO.	Micro USB Wire x1  The image shows a black micro-USB cable with a standard USB-A connector on one end and a micro-USB-B connector on the other.
---	--

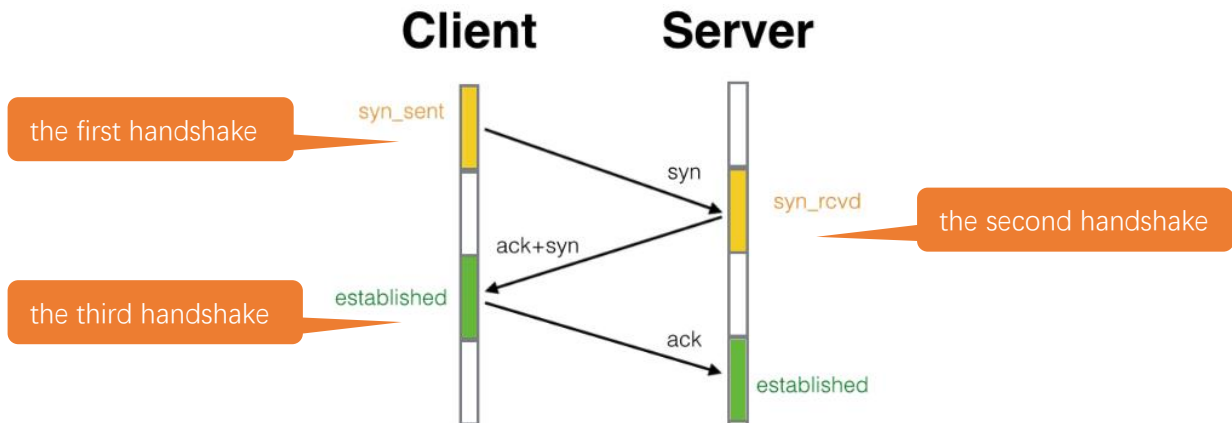
Component knowledge

TCP connection

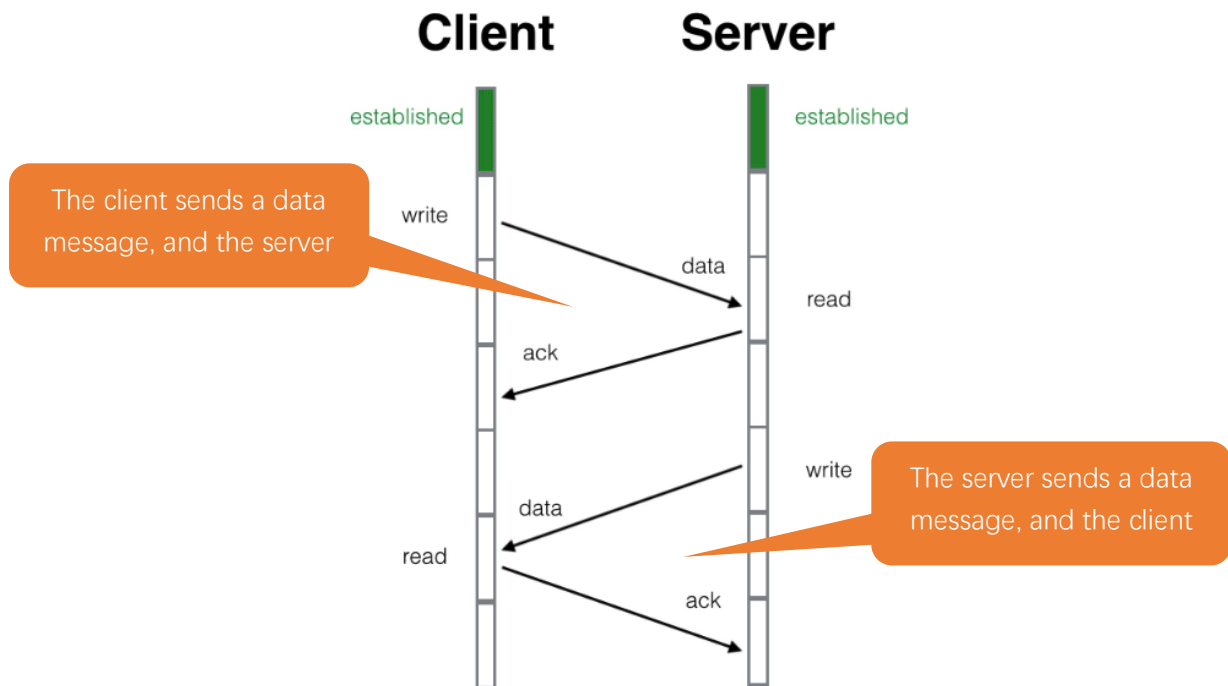
Before transmitting data, TCP needs to establish a logical connection between the sending end and the receiving end. It provides reliable and error-free data transmission between the two computers. In the TCP connection, the client and the server must be clarified. The client sends a connection request to the server, and each time such a request is proposed, a "three-times handshake" is required.

Three-times handshake: In the TCP protocol, during the preparation phase of sending data, the client and the server interact three times to ensure the reliability of the connection, which is called "three-times handshake". The first handshake, the client sends a connection request to the server and waits for the server to confirm. The second handshake, the server sends a response back to the client informing that it has received the connection request.

The third handshake, the client sends a confirmation message to the server again to confirm the connection.



TCP is a connection-oriented, low-level transmission control protocol. After TCP establishes a connection, the client and server can send and receive messages to each other, and the connection will always exist as long as the client or server does not initiate disconnection. Each time one party sends a message, the other party will reply with an ack signal.



Install Processing

In this tutorial, we use Processing to build a simple TCP/IP communication platform.

If you've not installed Processing, you can download it by clicking <https://processing.org/download/>. You can choose an appropriate version to download according to your PC system.

Processing p5.js Processing.py Processing for Android Processing for Pi Processing Foundation

Processing

Download Processing. Processing is available for Linux, Mac OS X, and Windows. Select your choice to download the software below.

3.5.4 (17 January 2020)

[Windows 64-bit](#) [Linux 64-bit](#) [Mac OS X](#)

[Windows 32-bit](#)

» [Github](#)
 » [Report Bugs](#)
 » [Wiki](#)
 » [Supported Platforms](#)

Read about the [changes in 3.0](#). The [list of revisions](#) covers the differences between releases in detail.

Unzip the downloaded file to your computer. Click "processing.exe" as the figure below to run this software.

	core	2020/1/17 12:16
	java	2020/1/17 12:17
	lib	2020/1/17 12:16
	modes	2020/1/17 12:16
	tools	2020/1/17 12:16
	processing.exe	2020/1/17 12:16
	processing-java.exe	2020/1/17 12:16
	revisions.txt	2020/1/17 12:16

Use Server mode for communication

Open the "Freenove_Basic_Starter_Kit_for_ESP8266\C\Sketches\Sketch_13.1_WiFiClient\sketchWiFi\sketchWiFi.pde", and click "Run".

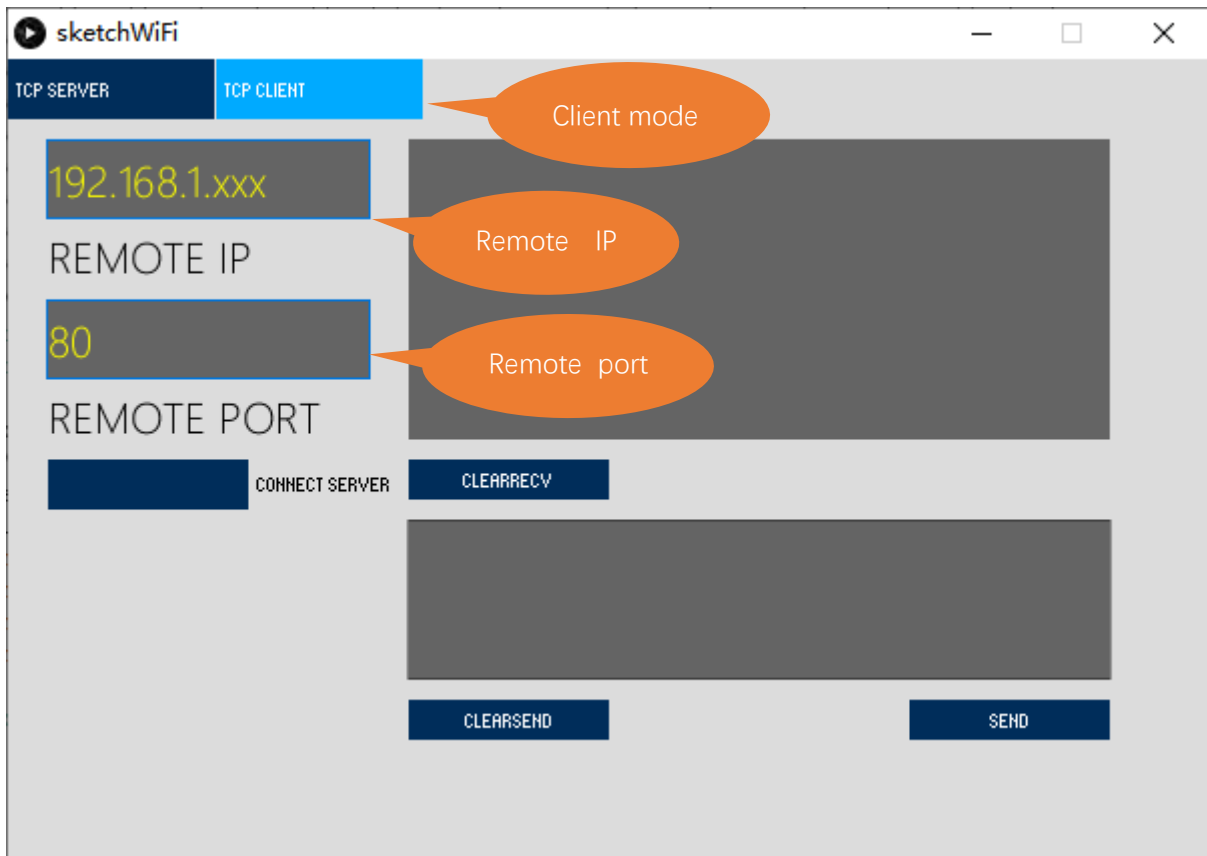


The new pop-up interface is as follows. If ESP8266 is used as client, select TCP SERVER mode for sketchWiFi.



When sketchWiFi selects TCP SERVER mode, ESP8266 Sketch needs to be changed according to sketchWiFi's displaying of LOCAL IP or LOCAL PORT.

If ESP8266 serves as server, select TCP CLIENT mode for sketchWiFi.



When sketchWiFi selects TCP CLIENT mode, the LOCAL IP and LOCAL PORT of sketchWiFi need to be changed according to the IP address and port number printed by the serial monitor.

Mode selection: select **Server mode/Client mode**.

IP address: In server mode, this option does not need to be filled in, and the computer will automatically obtain the IP address.

In client mode, fill in the remote IP address to be connected.

Port number: In server mode, fill in a port number for client devices to make an access connection.

In client mode, fill in port number given by the Server devices to make an access connection.

Start button: In server mode, push the button, then the computer will serve as server and open a port number for client to make access connection. During this period, the computer will keep monitoring.

In client mode, before pushing the button, please make sure the server is on, remote IP address and remote port number is correct; push the button, and the computer will make access connection to the remote port number of the remote IP as a client.

clear receive: clear out the content in the receiving text box

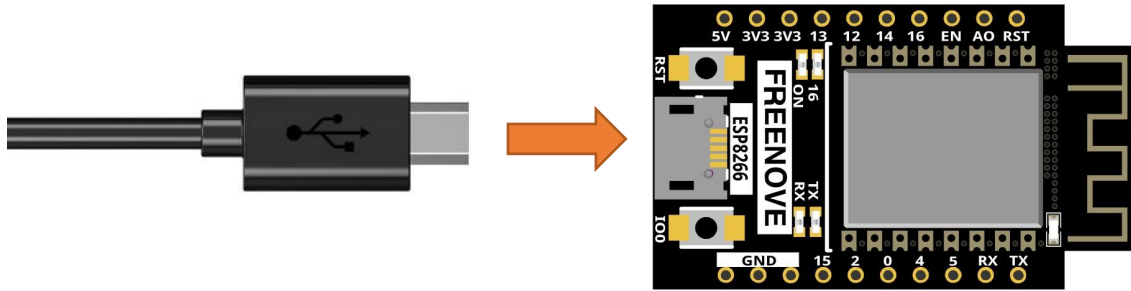
clear send: clear out the content in the sending text box

Sending button: push the sending button, the computer will send the content in the text box to others.

Circuit

Connect Freenove ESP8266 to the computer using USB cable.

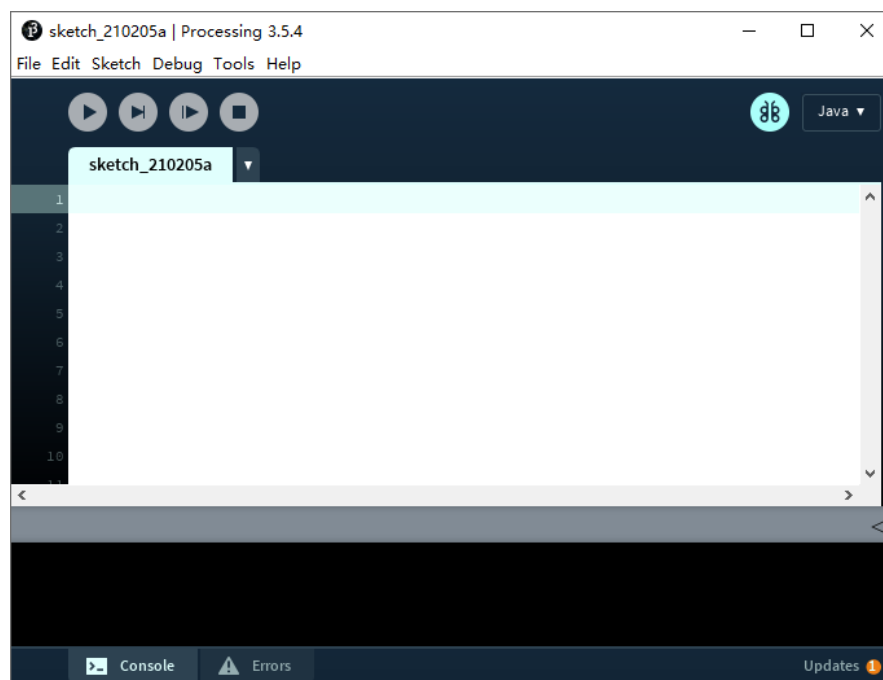
Any concerns? ✉ support@freenove.com



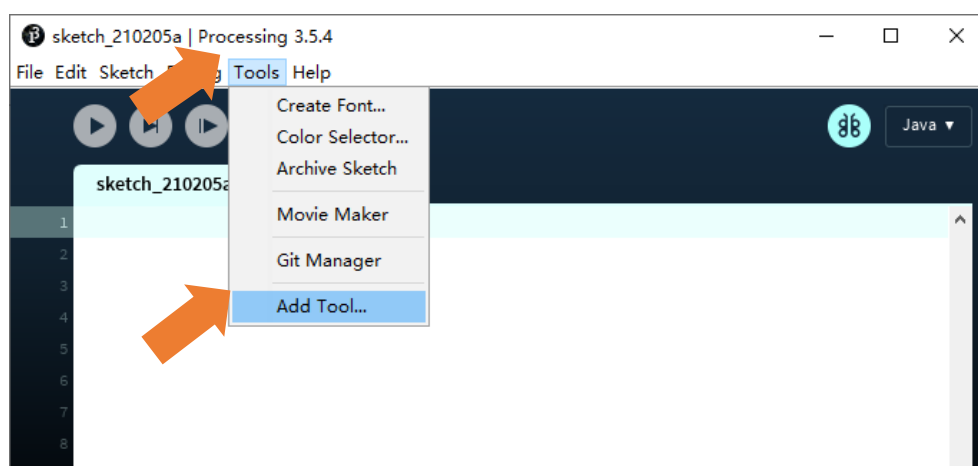
Sketch

If you have not installed “ControlP5”, please follow the following steps to continue the installation, if you have installed, please skip this section.

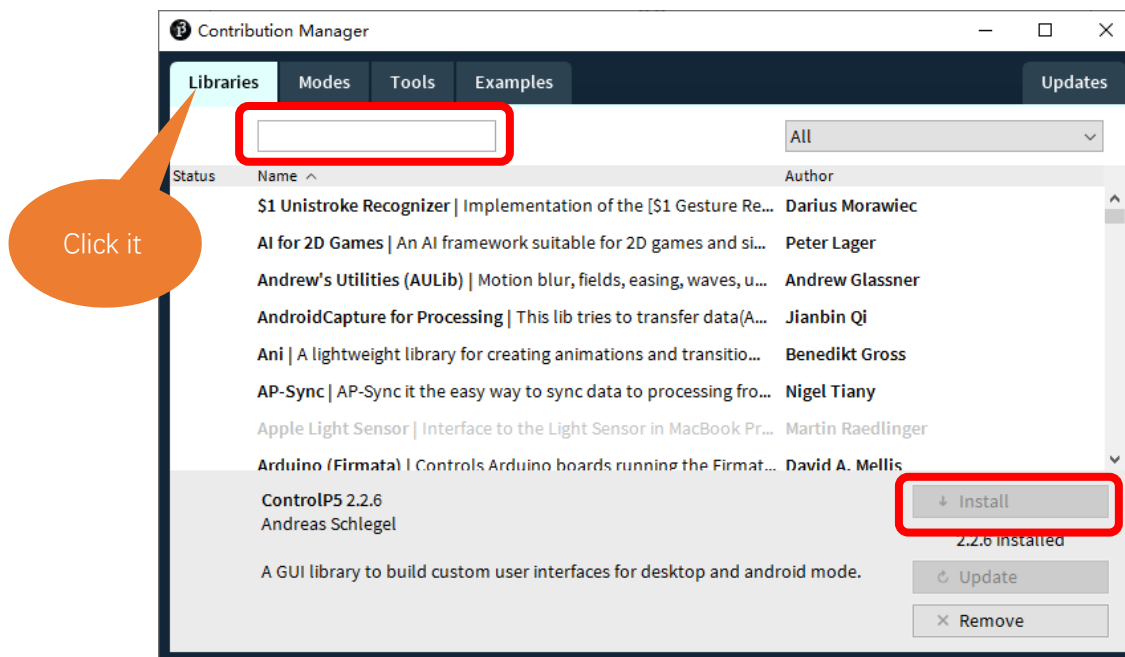
Open Processing.



Click Add Tool under Tools.

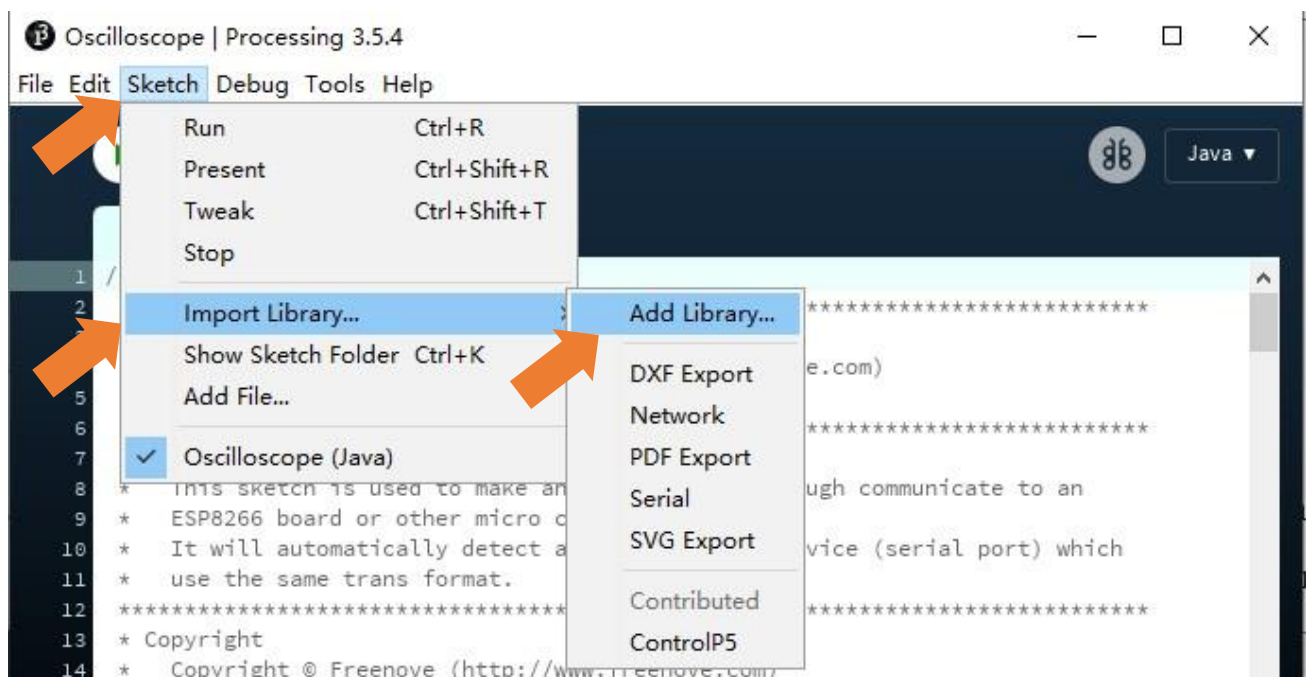


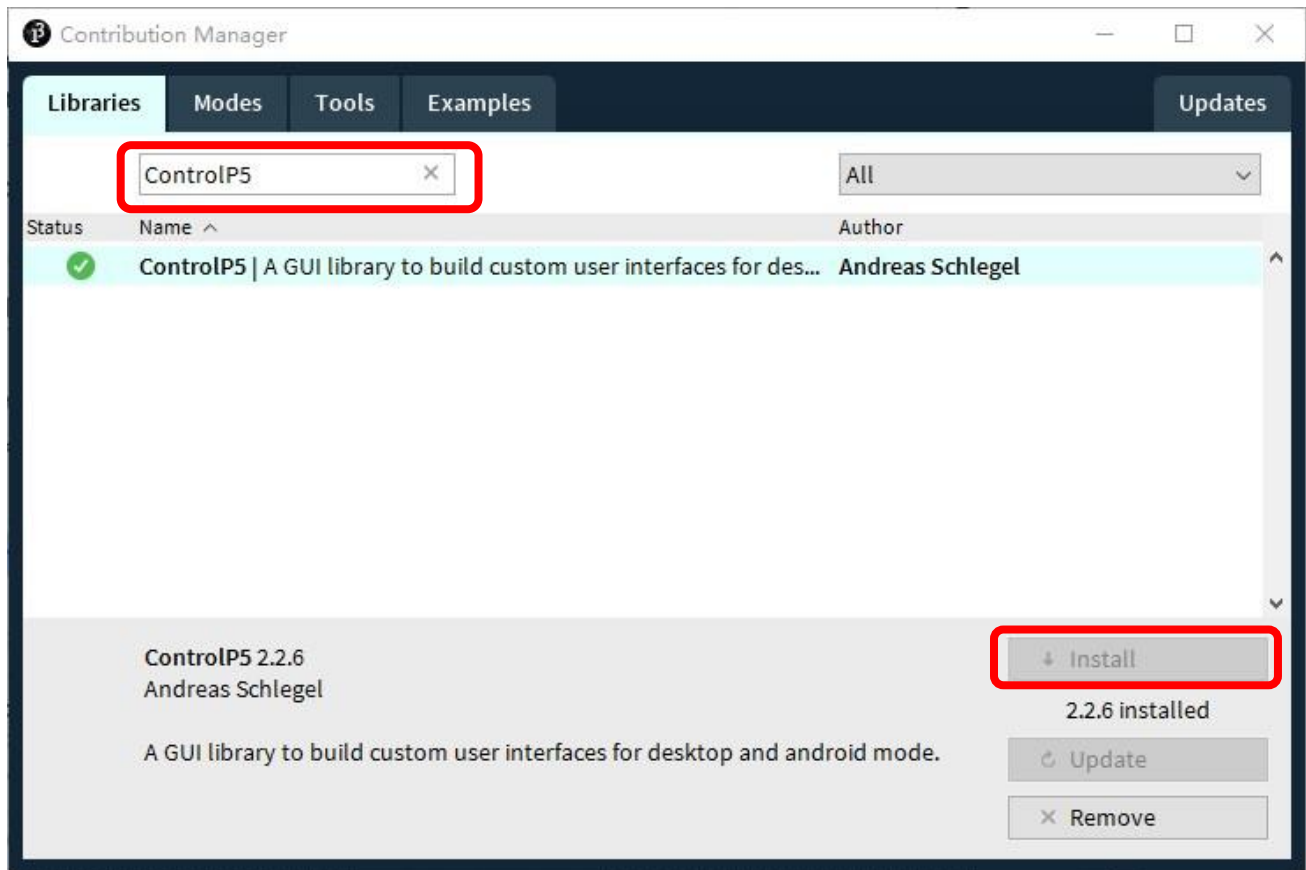
Select Libraries in the pop-up window.



Input "ControlP5" in the searching box, and then select the option as below. Click "Install" and wait for the installation to finish.

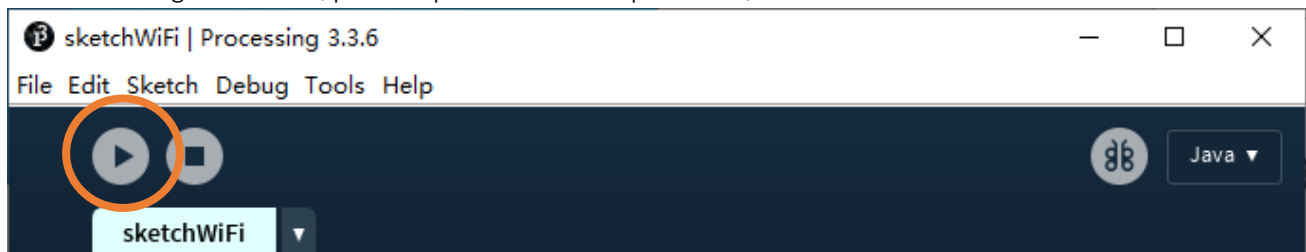
You can also click Add Library under 'Import Library' under 'Sketch'.



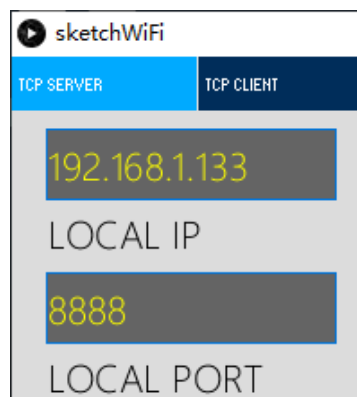


Sketch_13.1_As_Client

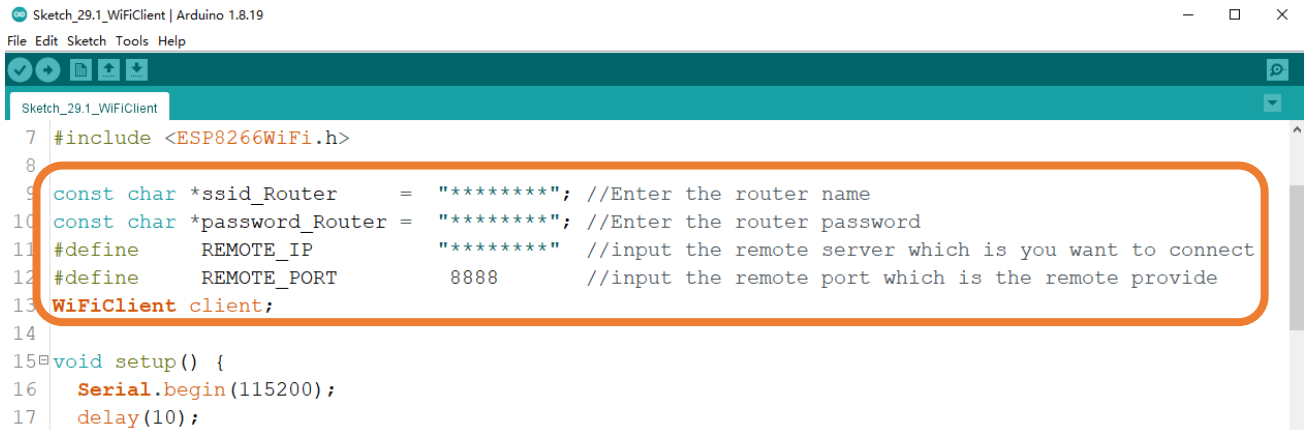
Before running the Sketch, please open "sketchWiFi.pde." first, and click "Run".



The newly pop up window will use the computer's IP address by default and open a data monitor port.



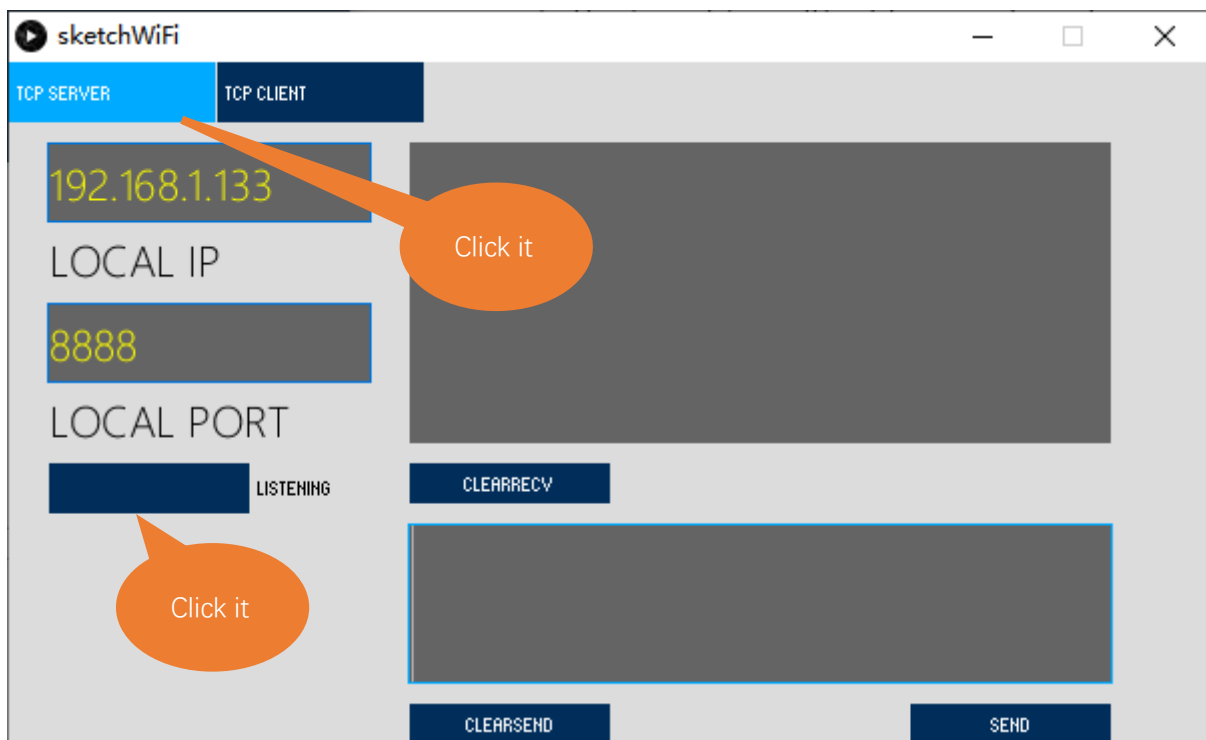
Next, open Sketch_13.1_WiFiClient.ino. Before running it, please change the following information based on "LOCAL IP" and "LOCAL PORT" in the figure above.



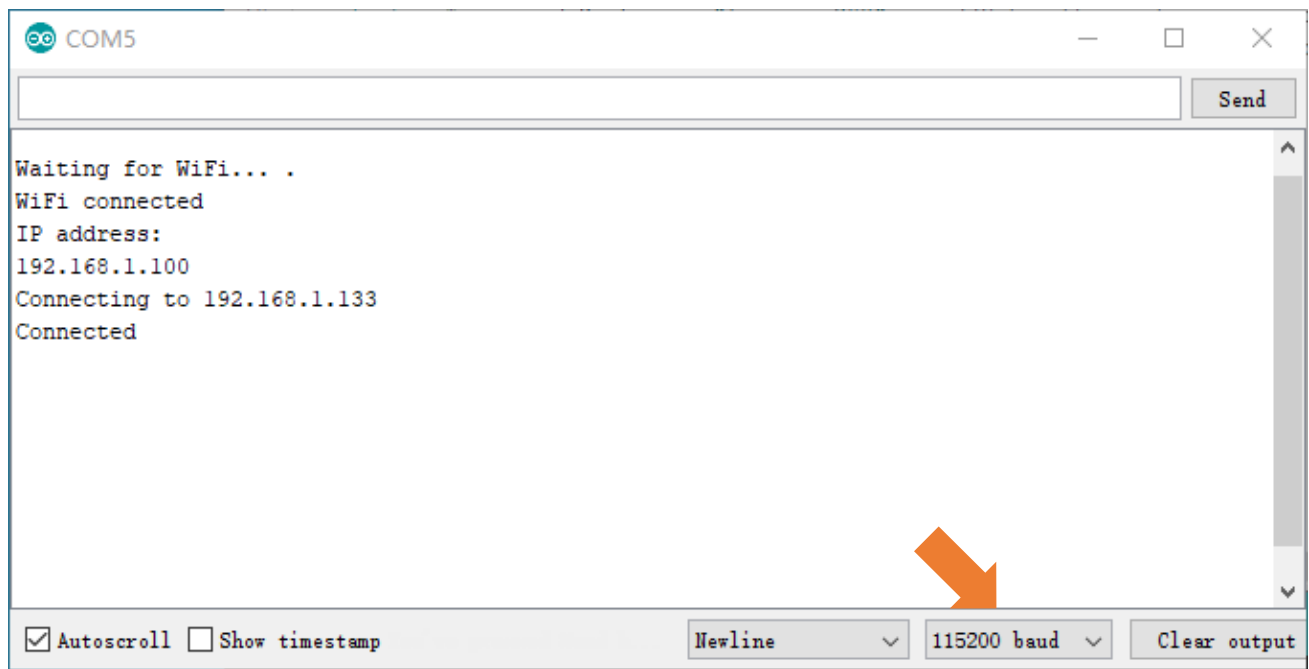
```
7 #include <ESP8266WiFi.h>
8
9 const char *ssid_Router    = "*****"; //Enter the router name
10 const char *password_Router = "*****"; //Enter the router password
11 #define    REMOTE_IP        "*****" //input the remote server which is you want to connect
12 #define    REMOTE_PORT      8888 //input the remote port which is the remote provide
13 WiFiClient client;
14
15 void setup() {
16     Serial.begin(115200);
17     delay(10);
```

REMOTE_IP needs to be filled in according to the interface of sketchWiFi.pde. Taking this tutorial as an example, its REMOTE_IP is "192.168.1.133". Generally, by default, the ports do not need to change its value.

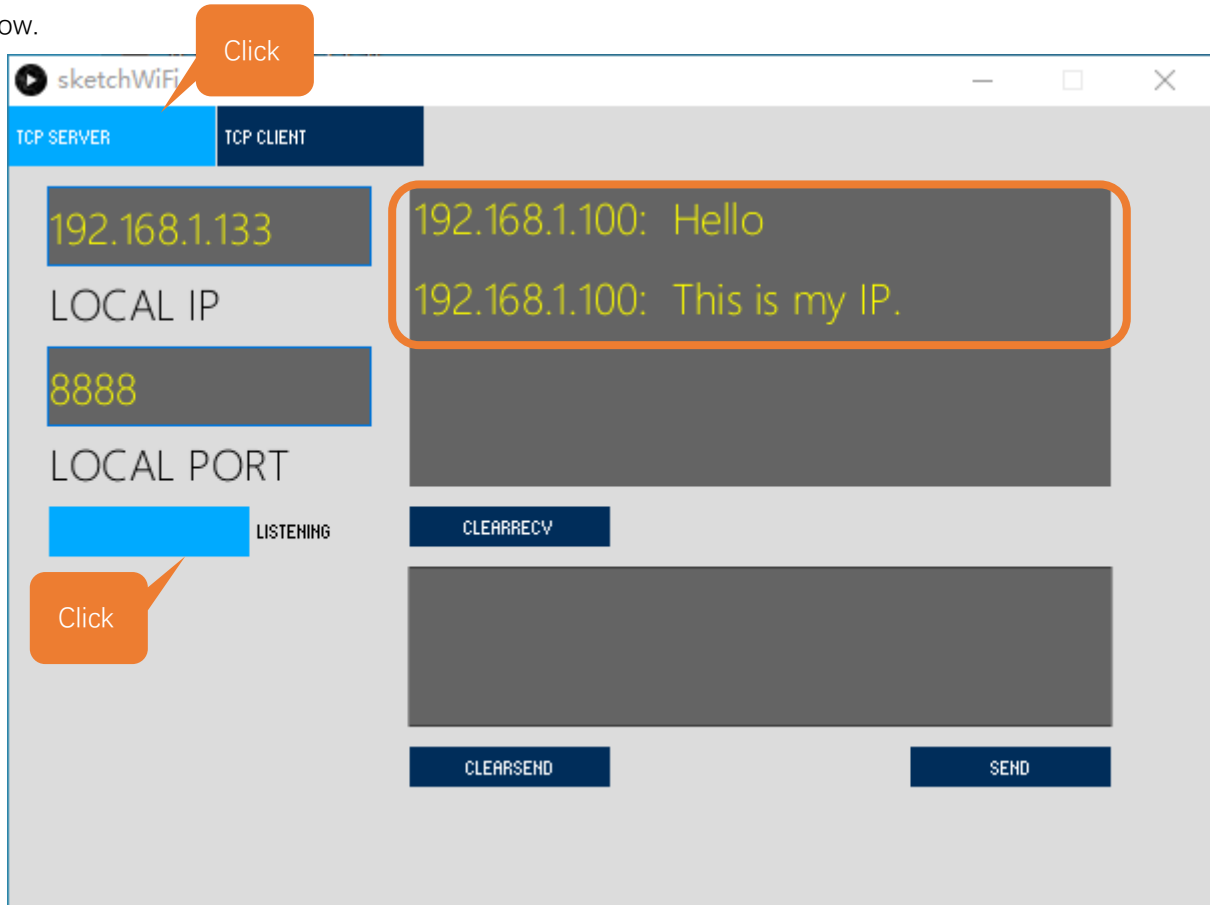
Click LISTENING, turn on TCP SERVER's data listening function and wait for ESP8266 to connect.



Compile and upload code to ESP8266, open the serial monitor and set the baud rate to 115200. ESP8266 connects router, obtains IP address and sends access request to server IP address on the same LAN till the connection is successful. When connect successfully, ESP8266 can send messages to server.



ESP8266 connects with TCP SERVER, and TCP SERVER receives messages from ESP8266, as shown in the figure below.



The following is the program code:

```
1  #include <ESP8266WiFi.h>
2
3  const char *ssid_Router    = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password
5  #define REMOTE_IP          "*****" //input the remote server which is you want to connect
6  #define REMOTE_PORT        8888     //input the remote port which is the remote provide
7  WiFiClient client;
8
9  void setup() {
10     Serial.begin(115200);
11     delay(10);
12
13     WiFi.begin(ssid_Router, password_Router);
14     Serial.print("\nWaiting for WiFi... ");
15     while (WiFi.status() != WL_CONNECTED) {
16         Serial.print(".");
17         delay(500);
18     }
19     Serial.println("");
20     Serial.println("WiFi connected");
21     Serial.println("IP address: ");
22     Serial.println(WiFi.localIP());
23     delay(500);
24
25     Serial.print("Connecting to ");
26     Serial.println(REMOTE_IP);
27
28     while (!client.connect(REMOTE_IP, REMOTE_PORT)) {
29         Serial.println("Connection failed.");
30         Serial.println("Waiting a moment before retrying...");
31     }
32     Serial.println("Connected");
33     client.print("Hello\n");
34     client.print("This is my IP.\n");
35 }
36
37 void loop() {
38     if (client.available() > 0) {
39         delay(20);
40         //read back one line from the server
41         String line = client.readString();
42         Serial.println(REMOTE_IP + String(":") + line);
43     }
44 }
```

```

43  if (Serial.available() > 0) {
44      delay(20);
45      String line = Serial.readString();
46      client.print(line);
47  }
48  if (client.connected () == 0) {
49      client.stop();
50      WiFi.disconnect();
51  }
52  }

```

Add WiFi function header file.

```

1  #include <ESP8266WiFi.h>

```

Enter the actual router name, password, remote server IP address, and port number.

```

3  const char *ssid_Router    = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password
5  #define     REMOTE_IP      "*****" //input the remote server which is you want to connect
6  #define     REMOTE_PORT    8888     //input the remote port which is the remote provide

```

Apply for the method class of WiFiClient.

```

7  WiFiClient client;

```

Connect specified WiFi until it is successful. If the name and password of WiFi are correct but it still fails to connect, please push the reset key.

```

13  WiFi.begin(ssid_Router, password_Router);
14  Serial.print("\nWaiting for WiFi... ");
15  while (WiFi.status() != WL_CONNECTED) {
16      Serial.print(".");
17      delay(500);
18  }

```

Send connection request to remote server until connect successfully. When connect successfully, print out the connecting prompt on the serial monitor and send messages to remote server.

```

28  while (!client.connect(REMOTE_IP, REMOTE_PORT)) { //Connect to Server
29      Serial.println("Connection failed.");
30      Serial.println("Waiting a moment before retrying...");
31  }
32  Serial.println("Connected");
33  client.print("Hello\n");

```

When ESP8266 receive messages from servers, it will print them out via serial port; Users can also send messages to servers from serial port.

```

37  if (client.available() > 0) {
38      delay(20);
39      //read back one line from the server
40      String line = client.readString();
41      Serial.println(REMOTE_IP + String(":") + line);
42  }
43  if (Serial.available() > 0) {

```

```

44     delay(20);
45     String line = Serial.readString();
46     client.print(line);
47 }

```

If the server is disconnected, turn off WiFi of ESP8266.

```

48     if (client.connected () == false) {
49         client.stop();
50         WiFi.disconnect();
51     }

```

Reference

Class Client

Every time when using Client, you need to include header file "ESP8266WiFi.h"

connect(ip, port, timeout)/connect(*host, port, timeout): establish a TCP connection.

ip, *host: ip address of target server

port: port number of target server

timeout: connection timeout

connected(): judge whether client is connecting. If return value is 1, then connect successfully; If return value is 0, then fail to connect.

stop(): stop tcp connection

print(): send data to server connecting to client

available(): return to the number of bytes readable in receive buffer, if no, return to 0 or -1.

read(): read one byte of data in receive buffer

readString(): read string in receive buffer

Project 13.2 As Server

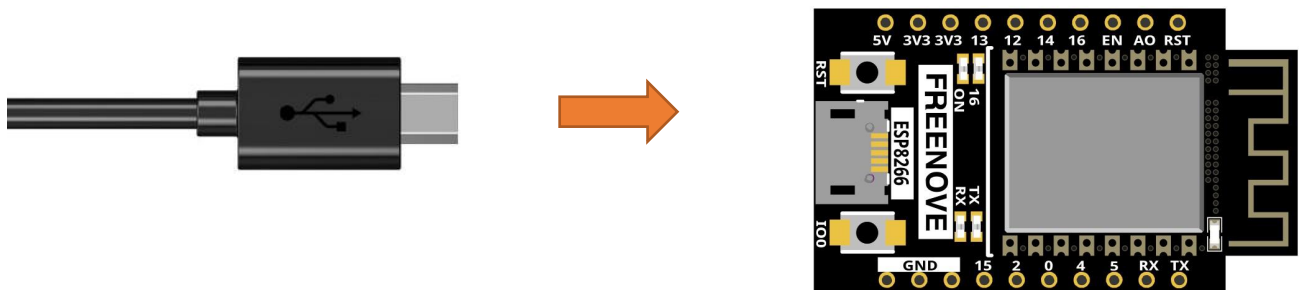
In this section, ESP8266 is used as a server to wait for the connection and communication of client on the same LAN.

Component List

ESP8266 x1	Micro USB Wire x1
------------	-------------------

Circuit

Connect Freenove ESP8266 to the computer using a USB cable.



Sketch

Before running Sketch, please modify the contents of the box below first.

Sketch_13.2_As_Server

Sketch_29.2_WiFiServer | Arduino 1.8.19

File Edit Sketch Tools Help

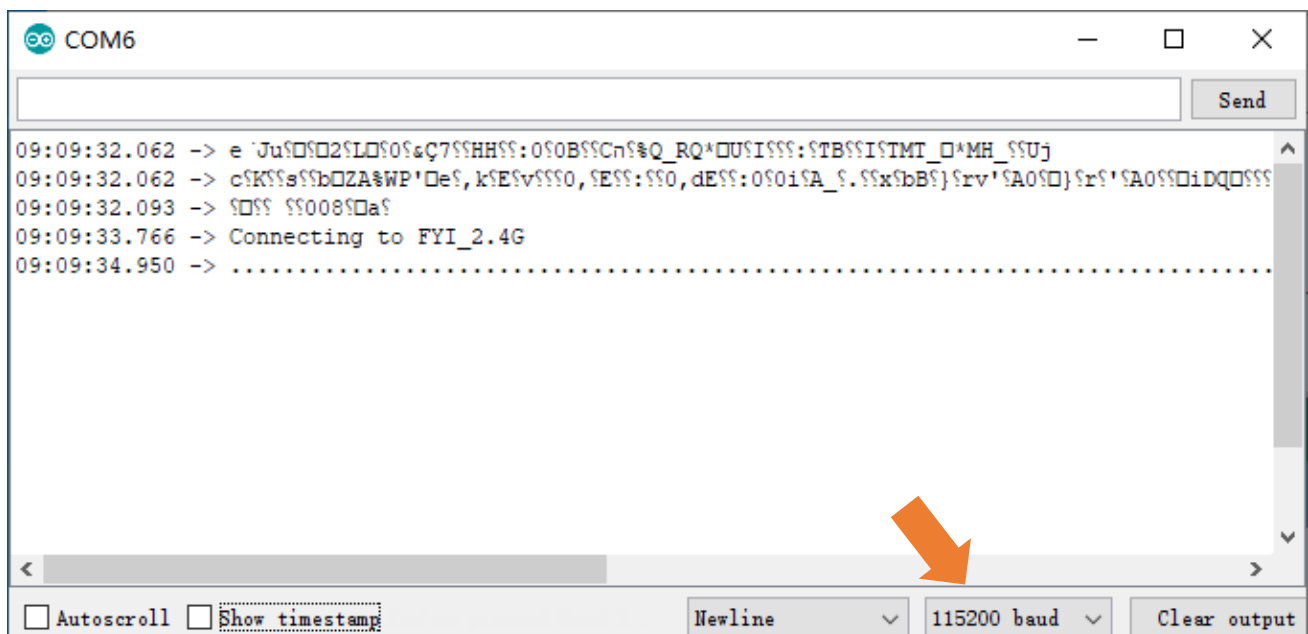
```

Sketch_29.2_WiFiServer$
7  *****/
8  #include <ESP8266WiFi.h>
9
10 #define port 80
11 const char *ssid_Router      = "*****"; //input your wifi name
12 const char *password_Router  = "*****"; //input your wifi passwords
13 WiFiServer server(port);
14
15 void setup()
16 {
17     Serial.begin(115200);

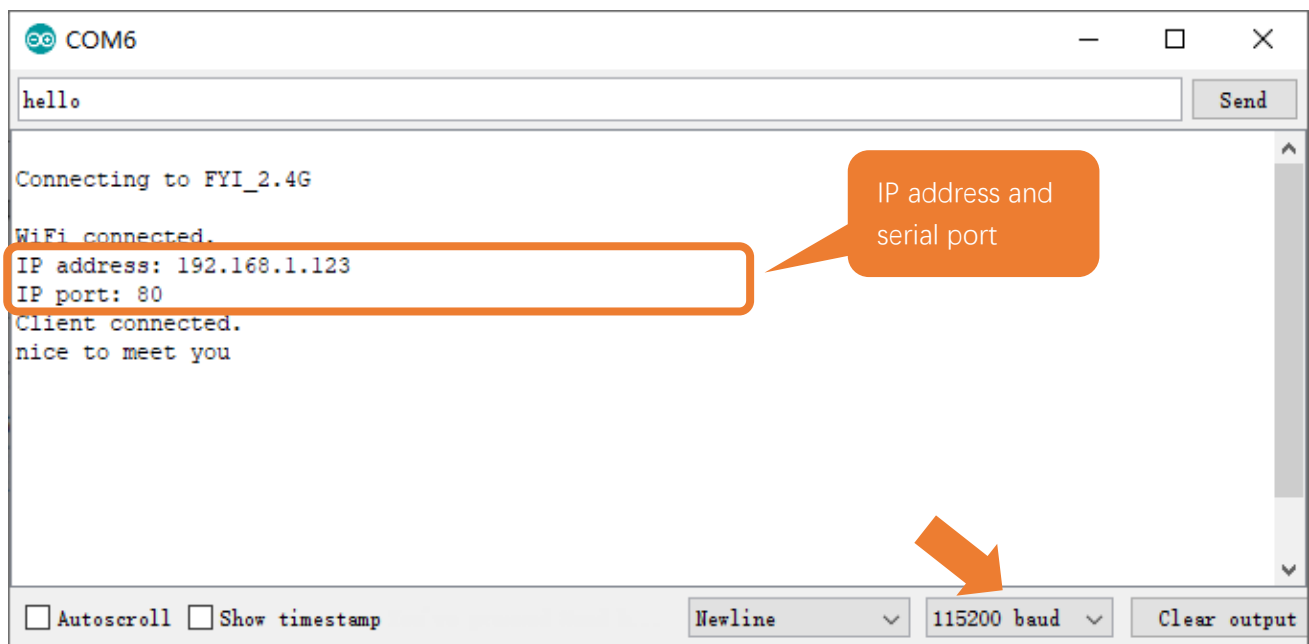
```

Compile and upload code to ESP8266 board, open the serial monitor and set the baud rate to 115200. Turn on server mode for ESP8266, waiting for the connection of other devices on the same LAN. Once a device connects to server successfully, they can send messages to each other.

If the ESP8266 fails to connect to router, press the reset button as shown below and wait for ESP8266 to run again.



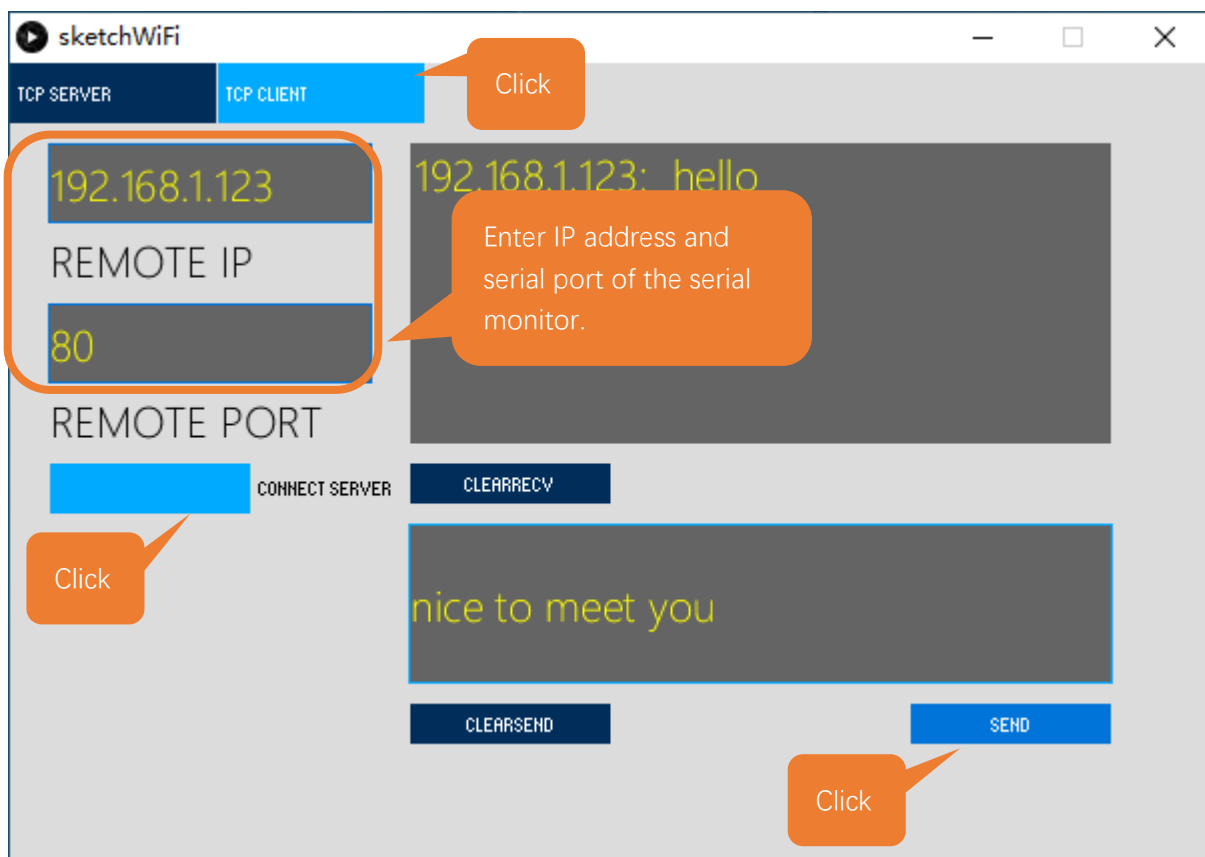
Serial Monitor



Processing:

Open the "Freenove_Basic_Starter_Kit_for_ESP8266\C\Sketches\Sketch_13.2_WiFiServer\sketchWiFi\sketchWiFi.pde".

Based on the messages printed by the serial monitor, enter correct IP address and serial port in Processing to establish connection and make communication.



The following is the program code:

```

1  #include <ESP8266WiFi.h>
2
3  #define port 80
4  const char *ssid_Router      = "*****"; //input your wifi name
5  const char *password_Router = "*****"; //input your wifi passwords
6  WiFiServer server(port);
7
8  void setup()
9  {
10     Serial.begin(115200);
11     Serial.printf("\nConnecting to ");
12     Serial.println(ssid_Router);
13     WiFi.disconnect();
14     WiFi.begin(ssid_Router, password_Router);
15     delay(1000);
16     while (WiFi.status() != WL_CONNECTED) {
17         delay(500);
18         Serial.print(".");
19     }
20     Serial.println("");
21     Serial.println("WiFi connected.");
22     Serial.print("IP address: ");
23     Serial.println(WiFi.localIP());
24     Serial.printf("IP port: %d\n",port);
25     server.begin(port);
26     WiFi.setAutoConnect(true);
27     WiFi.setAutoReconnect(true);
28 }
29
30 void loop() {
31     WiFiClient client = server.available(); // listen for incoming clients
32     if (client) { // if you get a client
33         Serial.println("Client connected.");
34         while (client.connected()) { // loop while the client's connected
35             if (client.available()) { // if there's bytes to read from the
client
36                 Serial.println(client.readStringUntil('\n')); // print it out the serial monitor
37                 while(client.read()>0); // clear the wifi receive area cache
38             }
39             if(Serial.available()){ // if there's bytes to read from the
serial monitor
40                 client.print(Serial.readStringUntil('\n')); // print it out the client.
41                 while(Serial.read()>0); // clear the wifi receive area cache

```



```

42     }
43     }
44     client.stop();                      // stop the client connecting.
45     Serial.println("Client Disconnected.");
46 }
47 }

```

Apply for method class of WiFiServer.

```

6  WiFiServer server(port);              //Apply for a Server object whose port number is 80

```

Connect specified WiFi until it is successful. If the name and password of WiFi are correct but it still fails to connect, please push the reset key.

```

13  WiFi.disconnect();
14  WiFi.begin(ssid_Router, password_Router);
15  delay(1000);
16  while (WiFi.status() != WL_CONNECTED) {
17      delay(500);
18      Serial.print(".");
19  }
20  Serial.println("");
21  Serial.println("WiFi connected.");

```

Print out the IP address and port number of ESP8266.

```

22  Serial.print("IP address: ");
23  Serial.println(WiFi.localIP());        //print out IP address of ESP8266
24  Serial.printf("IP port: %d\n",port);   //Print out ESP8266's port number

```

Turn on server mode of ESP8266, start automatic connection and turn on automatic reconnection.

```

25  server.begin();                       //Turn ON ESP8266 as Server mode
26  WiFi.setAutoConnect(true);
27  WiFi.setAutoReconnect(true);

```

When ESP8266 receive messages from servers, it will print them out via serial port; Users can also send messages to servers from serial port.

```

35  if (client.available()) {              // if there's bytes to read from the
client
36      Serial.println(client.readStringUntil('\n')); // print it out the serial monitor
37      while(client.read()>0);             // clear the wifi receive area cache
38  }
39  if(Serial.available()){                 // if there's bytes to read from the
serial monitor
40      client.print(Serial.readStringUntil('\n')); // print it out the client.
41      while(Serial.read()>0);             // clear the wifi receive area cache
42  }

```

Reference

Class Server

Every time use Server functionality, we need to include header file "ESP8266WiFi.h".

WiFiServer(uint16_t port=80, uint8_t max_clients=4): create a TCP Server.

port: ports of Server; range from 0 to 65535 with the default number as 80.

max_clients: maximum number of clients with default number as 4.

begin(port): start the TCP Server.

port: ports of Server; range from 0 to 65535 with the default number as 0.

setNoDelay(bool nodelay): whether to turn off the delay sending functionality.

nodelay: true stands for forbidden Nagle algorithm.

close(): close tcp connection.

stop(): stop tcp connection.

Chapter 14 Smart home

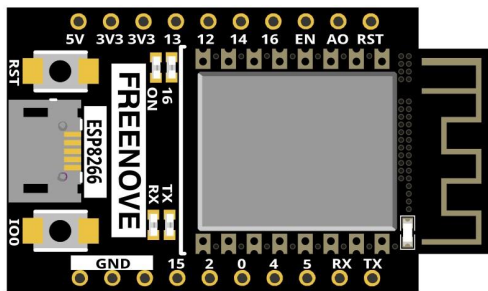
In this chapter, we will use ESP8266 to make a simple smart home. We will learn how to control LED lights through web pages.

Project 14.1 Control_LED_through_Web

In this project, we need to build a Web Service and then use ESP8266 to control the LED through the Web browser of the phone or PC. Through this example, you can remotely control the appliances in your home to achieve smart home.

Component List

ESP8266 x1



Micro USB Wire x1



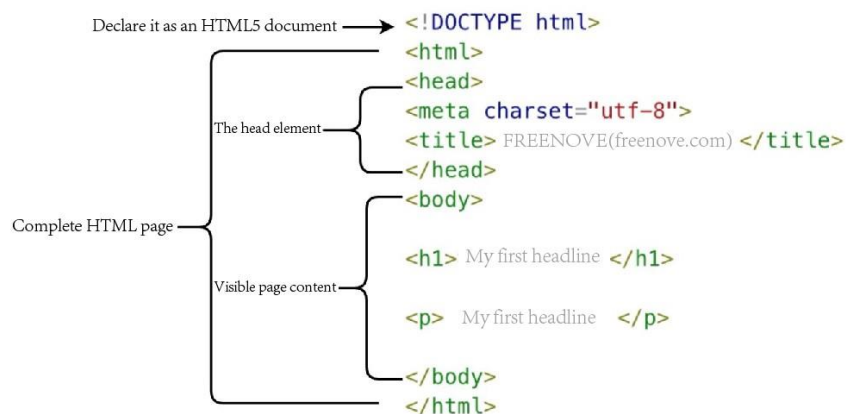
Component knowledge

HTML

HyperText Markup Language (HTML) is a standard Markup Language for creating web pages. It includes a set of tags that unify documents on the network and connect disparate Internet resources into a logical whole. HTML text is descriptive text composed of HTML commands that describe text, graphics, animations, sounds, tables, links, etc. The extension of the HTML file is HTML. Hyper Text is a way to organize information. It uses hyperlinks to associate words and charts in Text with other information media. These related information media may be in the same Text, other files, or files located on a remote computer. This way of organizing information connects the information resources distributed in different places, which is convenient for people to search and retrieve information.

The nature of the Web is hypertext Markup Language (HTML), which can be combined with other Web technologies (e.g., scripting languages, common gateway interfaces, components, etc.) to create powerful Web pages. Thus, HYPERtext Markup Language (HTML) is the foundation of World Wide Web (Web) programming, that is, the World Wide Web is based on hypertext. Hypertext Markup Language is called hypertext Markup language because the text contains so-called "hyperlink" points.

You can build your own WEB site using HTML, which runs on the browser and is parsed by the browser. Example analysis is shown in the figure below:



<!DOCTYPE html>:Declare it as an HTML5 document

<html>:Is the root element of an HTML page

<head>:Contains meta data for the document, such as `<meta charset="utf-8">`; Define the web page encoding format to UTF-8.

<title>:Notesthe title of the document

<body>:Contains visible page content

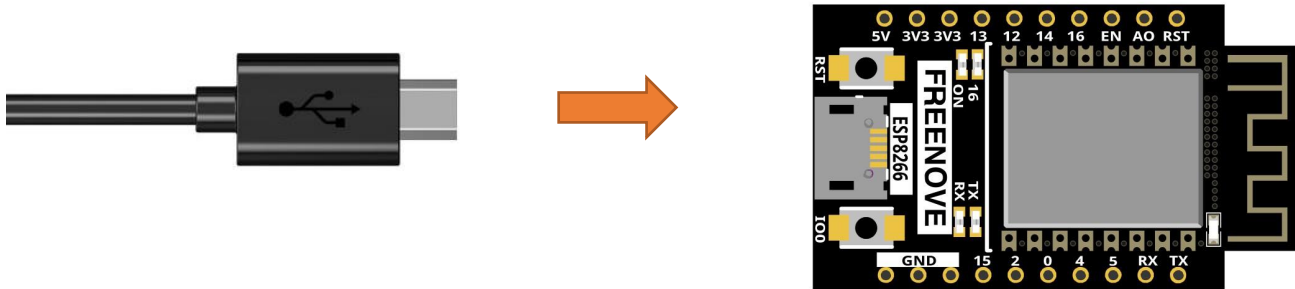
<h1>:Define a big heading

<p>:Define a paragraph

For more information, please visit: <https://developer.mozilla.org/en-US/docs/Web/HTML>

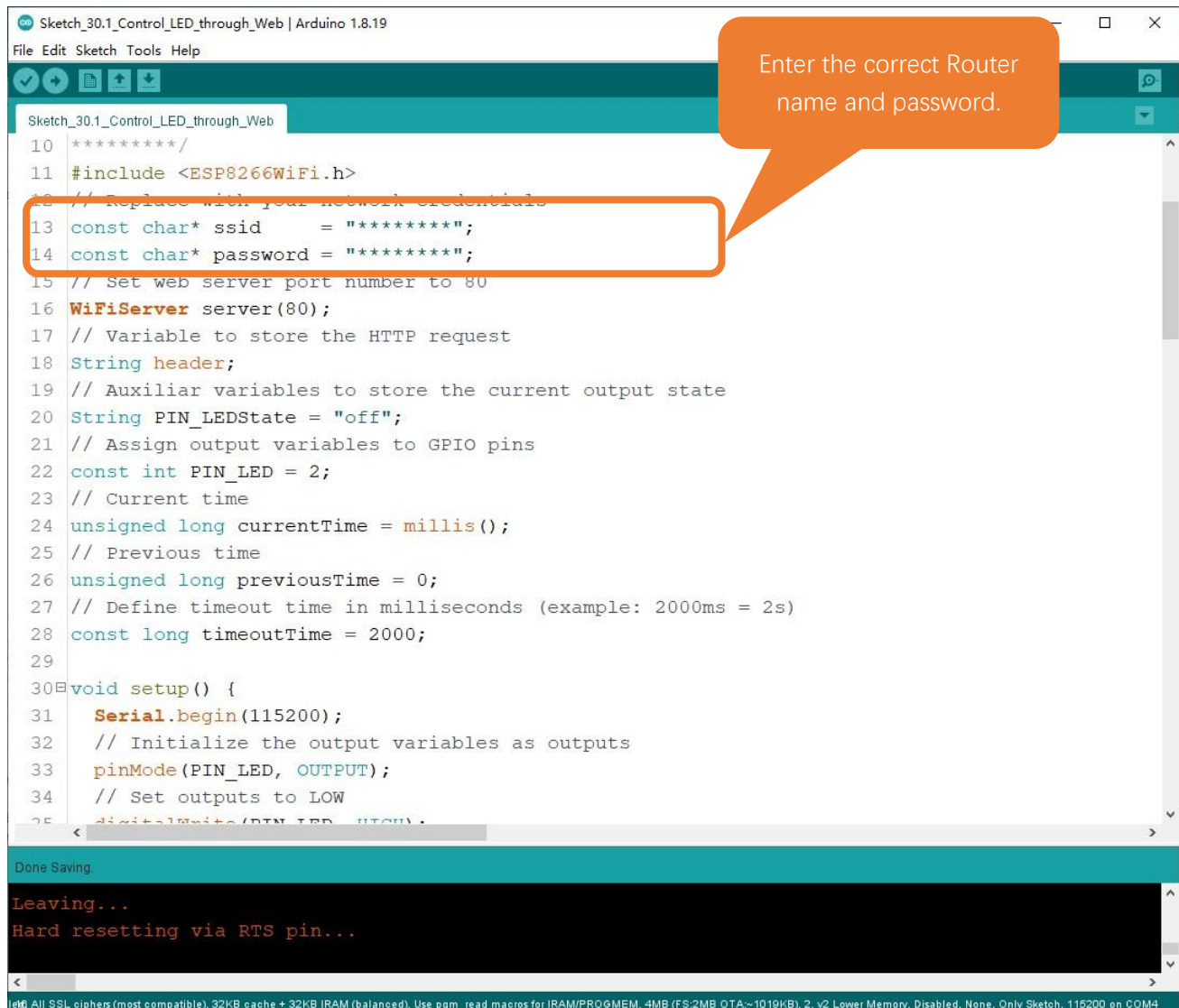
Circuit

Connect Freenove ESP8266 to the computer using a USB cable.



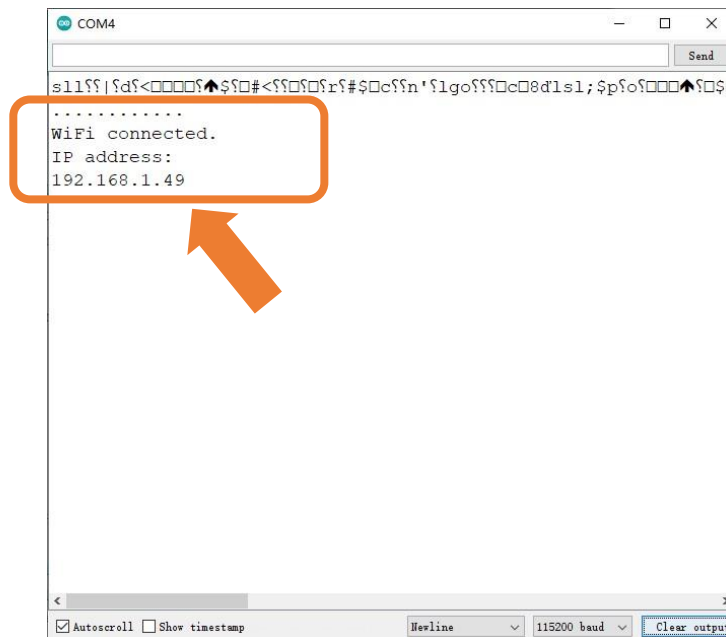
Sketch

Sketch_14.1_Control_LED_through_Web

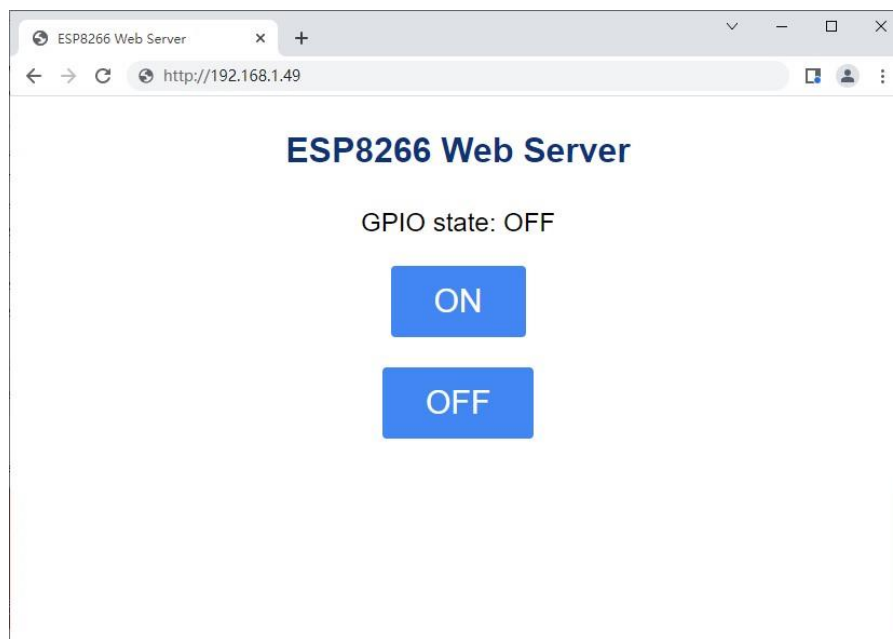


Any concerns?  support@freenove.com

Download the code to ESP8266, open the serial port monitor, set the baud rate to 115200 and you can use it to measure the distance between the ultrasonic module and the object. As shown in the following figure:



When ESP8266 successfully connects to “ssid_Router”, serial monitor will print out the IP address assigned to ESP8266 by the router. Access <http://192.168.1.49> in a computer browser on the LAN. As shown in the following figure:



You can click the corresponding button to control the LED on and off.

The following is the program code:

```
1 #include <ESP8266WiFi.h>
2 // Replace with your network credentials
3 const char* ssid      = "*****"; //Enter the router name
4 const char* password = "*****"; //Enter the router password
5 // Set web server port number to 80
```

```
6  WiFiServer server(80);
7  // Variable to store the HTTP request
8  String header;
9  // Auxiliar variables to store the current output state
10 String PIN_LEDState = "OFF";
11 // Assign output variables to GPIO pins
12 const int PIN_LED = 2;
13 // Current time
14 unsigned long currentTime = millis();
15 // Previous time
16 unsigned long previousTime = 0;
17 // Define timeout time in milliseconds (example: 2000ms = 2s)
18 const long timeoutTime = 2000;
19
20 void setup() {
21   Serial.begin(115200);
22   // Initialize the output variables as outputs
23   pinMode(PIN_LED, OUTPUT);
24   // Set outputs to LOW
25   digitalWrite(PIN_LED, HIGH);
26   // Connect to Wi-Fi network with SSID and password
27   Serial.print("Connecting to ");
28   Serial.println(ssid);
29   WiFi.begin(ssid, password);
30   while (WiFi.status() != WL_CONNECTED) {
31     delay(500);
32     Serial.print(".");
33   }
34   // Print local IP address and start web server
35   Serial.println("");
36   Serial.println("WiFi connected.");
37   Serial.println("IP address: ");
38   Serial.println(WiFi.localIP());
39   server.begin();
40 }
41 void loop() {
42   WiFiClient client = server.available(); // Listen for incoming clients
43   if (client) { // If a new client connects,
44     Serial.println("New Client."); // print a message out in the serial port
45     String currentLine = ""; // make a String to hold incoming data from the client
46     currentTime = millis();
47     previousTime = currentTime;
48     while (client.connected() && currentTime - previousTime <= timeoutTime) { // loop while
the client's connected
```

```

49     currentTime = millis();
50     if (client.available()) {           // if there's bytes to read from the client,
51         char c = client.read();         // read a byte, then
52         Serial.write(c);                // print it out the serial monitor
53         header += c;
54         if (c == '\n') {                // if the byte is a newline character
55             // if the current line is blank, you got two newline characters in a row.
56             // that's the end of the client HTTP request, so send a response:
57             if (currentLine.length() == 0) {
58                 // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
59                 // and a content-type so the client knows what's coming, then a blank line:
60                 client.println("HTTP/1.1 200 OK");
61                 client.println("Content-type:text/html");
62                 client.println("Connection: close");
63                 client.println();
64                 // turns the GPIOs on and off
65                 if (header.indexOf("GET /5/ON") >= 0) {
66                     Serial.println("GPIO 5 ON");
67                     PIN_LEDState = "ON";
68                     digitalWrite(PIN_LED, LOW);
69                 } else if (header.indexOf("GET /5/OFF") >= 0) {
70                     Serial.println("GPIO 5 OFF");
71                     PIN_LEDState = "OFF";
72                     digitalWrite(PIN_LED, HIGH);
73                 }
74                 // Display the HTML web page
75                 client.println("<!DOCTYPE html><html>");
76                 client.println("<head> <title>ESP8266 Web Server</title> <meta name=\"viewport\"
content=\"width=device-width, initial-scale=1\">");
77                 client.println("<link rel=\"icon\" href=\"data:,\">");
78                 // CSS to style the on/off buttons
79                 // Feel free to change the background-color and font-size attributes to fit your preferences
80                 client.println("<style>html {font-family: Helvetica; display:inline-block; margin:
0px auto; text-align: center;}");
81                 client.println(" h1{color: #0F3376; padding: 2vh;} p{font-size: 1.5rem;}");
82                 client.println(".button{background-color: #4286f4; display: inline-block; border:
none; border-radius: 4px; color: white; padding: 16px 40px;text-decoration: none; font-size:
30px; margin: 2px; cursor: pointer;}");
83                 client.println(".button2{background-color: #4286f4;display: inline-block; border:
none; border-radius: 4px; color: white; padding: 16px 40px;text-decoration: none; font-size:
30px; margin: 2px; cursor: pointer;}</style></head>");
84                 // Web Page Heading
85                 client.println("<body><h1>ESP8266 Web Server</h1>");
86                 client.println("<p>GPIO state: " + PIN_LEDState + "</p>");

```



```

87         client.println("<p><a href=\"/5/ON\"><button class=\"button
button2\">ON</button></a></p>");
88         client.println("<p><a href=\"/5/OFF\"><button class=\"button
button2\">OFF</button></a></p>");
89         client.println("</body></html>");
90         // The HTTP response ends with another blank line
91         client.println();
92         // Break out of the while loop
93         break;
94     } else { // if you got a newline, then clear currentLine
95         currentLine = "";
96     }
97     } else if (c != '\r') { // if you got anything else but a carriage return character,
98         currentLine += c;    // add it to the end of the currentLine
99     }
100 }
101 }
102 // Clear the header variable
103 header = "";
104 // Close the connection
105 client.stop();
106 Serial.println("Client disconnected.");
107 Serial.println("");
108 }
109 }

```

Include the WiFi Library header file of ESP8266.

```

1 #include <ESP8266WiFi.h>

```

Enter correct router name and password.

```

3 const char* ssid      = "*****"; //Enter the router name
4 const char* password = "*****"; //Enter the router password

```

Set ESP8266 in Station mode and connect it to your router.

```

29 WiFi.begin(ssid, password);

```

Check whether ESP8266 has connected to router successfully every 0.5s.

```

30 while (WiFi.status() != WL_CONNECTED) {
31     delay(500);
32     Serial.print(".");
33 }

```

Serial monitor prints out the IP address assigned to ESP8266.

```

38 Serial.println(WiFi.localIP());

```

Click the button on the web page to control the LED light on and off.

```

65 if (header.indexOf("GET /5/ON") >= 0) {
66     Serial.println("GPIO 5 ON");
67     PIN_LEDState = "ON";
68     digitalWrite(PIN_LED, LOW);

```

```
69         } else if (header.indexOf("GET /5/OFF") >= 0) {  
70             Serial.println("GPIO 5 OFF");  
71             PIN_LEDState = "OFF";  
72             digitalWrite(PIN_LED, HIGH);  
73         }
```

What's next?

Thanks for your reading. This tutorial is all over here. If you find any mistakes, omissions or you have other ideas and questions about contents of this tutorial or the kit and etc., please feel free to contact us:

support@freenove.com

We will check and correct it as soon as possible.

If you want learn more about ESP8266, you view our ultimate tutorial:

https://github.com/Freenove/Freenove_Basic_Starter_Kit_for_ESP8266/archive/master.zip

If you want to learn more about Arduino, Raspberry Pi, smart cars, robots and other interesting products in science and technology, please continue to focus on our website. We will continue to launch cost-effective, innovative and exciting products.

<http://www.freenove.com/>

End of the Tutorial

Thank you again for choosing Freenove products.